

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 16.09.2026, 11:18:21
Файлов исходного кода: 113
Суммарный объём кода: 2.02 MB
Глав/билетов в пособии: 37

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. 6. Графы. Представление графов, DFS, BFS [id: graph-basics]
7. 7. Графы. Планарные. Покраска [id: graph-planarity]
8. 7. Графы. Планарные. Покраска [id: planarity-euler]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 11. Графы. Мосты [id: bridges-code]
14. 12. Графы. Точки сочленения [id: graph-articulation-points]
15. 13. Графы. Эйлеров цикл [id: graph-euler]
16. 13. Графы. Эйлеров цикл [id: euler-path-vs-cycle]
17. Билет 14. Дейкстра [id: dijkstra]
18. Билет 15. Форд-Беллман [id: bellman-ford]
19. Билет 16. Флойд-Уоршелл [id: floyd]
20. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
21. 17. Графы. Поиск кратчайшего пути. Ускорения [id: shortest-path-accelerations]
22. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
23. 18–20. Графы. Остовное дерево: Краскал, Прима, Борувка
24. 19. Графы. Остовное дерево. Прима [id: mst-prim]
25. 20. Графы. Остовное дерево. Борувка [id: mst-kruskal]

- 17. src/data/additionalTickets.ts
- 18. src/data/content_temp.ts
- 19. src/data/content.ts
- 20. src/data/debugRunner.ts
- 21. src/data/glossary.ts
- 22. src/data/graphContent.ts
- 23. src/data/quizzes.ts
- 24. src/data/vizStepBus.ts
- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_segtree.ts
- 27. src/data/tickets/ticket14_20.ts
- 28. src/data/tickets/ticket18_20_mst.ts
- 29. src/data/tickets/ticket21_24.ts
- 30. src/data/tickets/ticket25_cheatsheet.ts
- 31. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 32. src/components/ArticulationPointsViz.tsx
- 33. src/components/BellmanFordViz.tsx
- 34. src/components/BfsViz.tsx
- 35. src/components/ChapterImage.tsx
- 36. src/components/ChapterNav.tsx
- 37. src/components/ChapterQuiz.tsx
- 38. src/components/ChapterView.tsx
- 39. src/components/CheatSheetViz.tsx
- 40. src/components/ComplexityViz.tsx
- 41. src/components/DfsBridgesSimulator.tsx
- 42. src/components/DijkstraViz.tsx
- 43. src/components/DPVisualizer.tsx
- 44. src/components/EulerSimulator.tsx
- 45. src/components/FloydViz.tsx
- 46. src/components/GraphComponentsViz.tsx
- 47. src/components/GraphTraversalViz.tsx
- 48. src/components/HeapViz.tsx
- 49. src/components/hints/demoKit.tsx
- 50. src/components/hints/demosExtra.tsx

HTML-разметка (layout)

87. src/layout/footer.html

88. src/layout/header.html

Утилиты

89. src/utils/cn.ts

90. src/utils/exportHtml.ts

91. src/utils/ticketLinks.ts

Скрипты сборки и экспорта

92. scripts/audit-coverage.mjs

93. scripts/audit-terms.mjs

94. scripts/export/build-guide-pdf.mjs

95. scripts/export/build-pdf.mjs

96. scripts/export/capture-viz-shots.mjs

97. scripts/export/collect-code.mjs

98. scripts/export/common.mjs

99. scripts/export/html-to-blocks.mjs

100. scripts/export/render-pages.mjs

101. scripts/verify/browser-checks.tsx

102. scripts/verify/check-links.mjs

103. scripts/verify/check-reference-code.mjs

104. scripts/verify/dom-env.mjs

105. scripts/verify/live-matrix.test.mjs

106. scripts/verify/probe-compiler.tsx

107. scripts/verify/README.md

108. scripts/verify/run-browser-checks.mjs

109. scripts/verify/run-probe.mjs

CI / автоматизация

110. .github/workflows/deploy-pages.yml

111. .github/workflows/rebuild-pdf.yml

112. .github/workflows/verify.yml

Прочее

113. .ai_skills/universal_guide_authoring/SKILL.md

```
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: true

jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
```

```
+ id-token: write
+
+concurrency:
+  group: pages
+  cancel-in-progress: true
+
+jobs:
+  build:
+    name: Build static site
+    runs-on: ubuntu-latest
+    steps:
+      - name: Checkout
+        uses: actions/checkout@v6
+
+      - name: Setup Node.js
+        uses: actions/setup-node@v7
+        with:
+          node-version: 24
+          cache: npm
+
+      - name: Install dependencies
+        run: npm ci --no-audit --no-fund
+
+      - name: Build for the repository subpath
+        env:
+          VITE_BASE: /algv0/
+        run: npm run build
+
+      - name: Make GitHub Pages fallback
+        run: cp dist/index.html dist/404.html
+
+      - name: Upload Pages artifact
+        uses: actions/upload-pages-artifact@v5
+        with:
+          path: ./dist
+
+  deploy:
+    name: Publish to GitHub Pages
```

```
  inputs:
    density:
      description: "DPI растеризации страниц (по умолчанию)"
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot]')"

    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommends imagemagick fonts-dejavu-core
          # На Ubuntu политика ImageMagick иногда запрещает доступ к сетевым ресурсам
```

- name: Upload artifacts (PDF + TXT)
uses: actions/upload-artifact@v7
with:
 name: full-code-export
 path: |
 public/export/full_code_all_pages.pdf
 public/export/full_code_all_pages.txt
 if-no-files-found: error
 retention-days: 30
- name: Upload artifacts (PNG-страницы)
uses: actions/upload-artifact@v7
with:
 name: full-code-pages-png
 path: tmp/export-pages/*.png
 if-no-files-found: error
 retention-days: 7
- name: Commit rebuilt export back to the branch
run: |
 git config user.name "github-actions[bot]"
 git config user.email "41898282+github-action@users.noreply.github.com"
 git add -- public/export/full_code_all_pages.pdf
 if git diff --cached --quiet; then
 echo "Экспорт не изменился — коммит не нужен"
 exit 0
 fi
 git commit -m "chore(export): пересборка full-code-export"
 git push origin "HEAD:\${GITHUB_REF_NAME}"

....

Patch относительно `main`

````diff

diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml  
index bdc3c33..ff49c0c 100644

--- a/.github/workflows/rebuild-pdf.yml

+++ b/.github/workflows/rebuild-pdf.yml

## `index.html`

### Полное содержимое после изменений

```
````html
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100"
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
    <link rel="icon" type="image/svg+xml" href="%BASE_URL" />
    <title> Дискретка для тупых – Интерактивный гид по
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen"
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
````
```

### Patch относительно `main`

```
````diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
+  <link rel="icon" type="image/svg+xml" href="%BASE_URL" />
    <title> Дискретка для тупых – Интерактивный гид по
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen"
````
```



```

import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/Simu
import { SimulatorHub } from "../components/SimulatorHub
import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeTab, setActiveTab] = useState<"guide" |
 const [activeChapterId, setActiveChapterId] = useStat
 const [modalVizId, setModalVizId] = useState<string |

 const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
 const activeChapter = chapters[index] ?? chapters[0];
 const prev = index > 0 ? chapters[index - 1] : undefi
 const next = index < chapters.length - 1 ? chapters[in

 const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
 }, []);

 // Стрелки ← → листают темы (как на обучающих сайтах)
 useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLInputElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id)
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown",
 }, [goTo, next, prev]);

 const progress = useMemo(() => ((index + 1) / chapter

```

```

 style={{ width: `${progress}%` }}
 />
 </div>

 <ChapterView
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 />
 </main>
</div>
</>
) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={(id) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
) : (
 <PythonCompiler />
)
}

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>
);

```

```
..... <SimulatorModal vizId={modalVizId} onClose={() => {
.....
```

```
`src/components/Navbar.tsx`
```

```
Полное содержимое после изменений
```

```
````tsx
```

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, C
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
  activeTab: 'guide' | 'simulator' | 'compiler';
  setActiveTab: (tab: 'guide' | 'simulator' | 'compiler
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeT
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };
};
```

```
  return (
    <header className="bg-slate-900 border-b border-sla
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg
        ap-3 lg:gap-0">
          <div className="flex items-center gap-3 w-full
            <div className="bg-gradient-to-tr from-indigo
              k-0">
```

```

<button
  id="tab-compiler-btn"
  onClick={() => setActiveTab('compiler')}
  className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
    activeTab === 'compiler'
      ? 'bg-emerald-600 text-white shadow-lg
      : 'text-slate-400 hover:text-white'
  }`}
>
  <Terminal className="w-4 h-4" /> Python
</button>
<a
  id="download-pdf-btn"
  href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
  target="_blank"
  rel="noreferrer"
  className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
  title="Скачать полный PDF сборник всех стра
>
  <FileDown className="w-4 h-4" /> Скачать PD
</a>
<a
  id="download-txt-btn"
  href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
  target="_blank"
  rel="noreferrer"
  className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
  title="Скачать полный TXT файл со всем кодом
>
  <FileText className="w-4 h-4" /> TXT
</a>
<button
  id="download-html-btn"

```

```

        >
          <Cpu className="w-4 h-4" /> Тренажёры
        </button>
+      <button
+        id="tab-compiler-btn"
+        onClick={() => setActiveTab('compiler')}
+        className={`flex-1 lg:flex-none flex items
duration-200 whitespace-nowrap ${
+          activeTab === 'compiler'
+            ? 'bg-emerald-600 text-white shadow-lg
+            : 'text-slate-400 hover:text-white'
+        }`>
+      >
+      <Terminal className="w-4 h-4" /> Python
+    </button>
    <a
      id="download-pdf-btn"
-      href="/export/full_code_all_pages.pdf"
+      href={`${import.meta.env.BASE_URL}export/f
download="full_code_all_pages.pdf"
      target="_blank"
      rel="noreferrer"
@@ -73,7 +84,7 @@ export const Navbar: React.FC<NavbarP
    </a>
    <a
      id="download-txt-btn"
-      href="/export/full_code_all_pages.txt"
+      href={`${import.meta.env.BASE_URL}export/f
download="full_code_all_pages.txt"
      target="_blank"
      rel="noreferrer"
    </a>
  </div>
}

## `src/components/PythonCompiler.tsx`

### Полное содержимое после изменений

```tsx

```

```

 return String(error);
}

export function PythonCompiler() {
 const [code, setCode] = useState(STARTER_CODE);
 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Нажмите «Запустить»");
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(false);
 const [runtimeMessage, setRuntimeMessage] = useState("");

 const runCode = useCallback(async () => {
 if (running) return;

 setRunning(true);
 setOutput("");
 setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не выполнила");

 const stdout: string[] = [];
 const stderr: string[] = [];
 const inputLines = stdin.split(/\r?\n/);

 try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 setRuntimeMessage("Python готов — код выполняется");

 runtime.setStdout({ batched: (message) => stdout.push(message) });
 runtime.setStderr({ batched: (message) => stderr.push(message) });
 runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
 await runtime.runPythonAsync(code);

 const text = [...stdout, ...stderr].join("");
 setOutput(text || "Готово: программа ничего не выполнила");
 } catch (error) {
 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
 setRuntimeMessage("Не удалось выполнить код — произошла ошибка");
 }
 }, [code, stdin, runtimeReady]);

 return {
 code,
 setCode,
 stdin,
 setStdin,
 output,
 setOutput,
 running,
 setRunning,
 runtimeReady,
 setRuntimeReady,
 runtimeMessage,
 setRuntimeMessage,
 runCode
 };
}

```

```

<div className="flex items-center gap-2">

</div>
<div className="flex items-center gap-2">
 <button
 type="button"
 onClick={reset}
 className="inline-flex items-center gap-2 text-slate-800 transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" />
 </button>
 <button
 type="button"
 onClick={() => void runCode()}
 disabled={running}
 className="inline-flex items-center gap-2 text-white text-xs font-bold transition-colors"
 >
 {running ? <Loader2 className="w-3.5 h-3.5" /> : <RotateCcw className="w-3.5 h-3.5" />}
 {running ? "Запускаю..." : "Запустить"}
 </button>
</div>
</div>

<textarea
 value={code}
 onChange={(event) => setCode(event.target.value)}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && event.key === 'Enter') {
 event.preventDefault();
 void runCode();
 }
 }}
 spellCheck={false}
 aria-label="Код Python"
/>

```

```

 </div>

 <div className="mt-5 grid md:grid-cols-3 gap-3">
 <div className="flex gap-2 rounded-xl border">
 <Info className="w-4 h-4 shrink-0 text-indigo">
 Ctrl или Cmd + Enter запускает код. В
 </div>
 <div className="flex gap-2 rounded-xl border">
 <Info className="w-4 h-4 shrink-0 text-amber">
 Первый запуск скачивает примерно неск
 </div>
 <div className="flex gap-2 rounded-xl border">
 <Info className="w-4 h-4 shrink-0 text-emerg
 Код выполняется в браузере и не отпра
 </div>
 </div>
 </div>
 </main>
);
}
....

```

### Patch относительно `main`

```

````diff
diff --git a/src/components/PythonCompiler.tsx b/src/com
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, PL
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/py
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/py
+

```



```

-----
+ const [running, setRunning] = useState(false);
+ const [runtimeReady, setRuntimeReady] = useState(false);
+ const [runtimeMessage, setRuntimeMessage] = useState('');
+
+ const runCode = useCallback(async () => {
+   if (running) return;
+
+   setRunning(true);
+   setOutput('');
+   setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не выполняет");
+
+   const stdout: string[] = [];
+   const stderr: string[] = [];
+   const inputLines = stdin.split(/\r?\n/);
+
+   try {
+     const runtime = await getPythonRuntime();
+     setRuntimeReady(true);
+     setRuntimeMessage("Python готов – код выполняется");
+
+     runtime.setStdout({ batched: (message) => stdout.push(message) });
+     runtime.setStderr({ batched: (message) => stderr.push(message) });
+     runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
+     await runtime.runPythonAsync(code);
+
+     const text = [...stdout, ...stderr].join("");
+     setOutput(text || "Готово: программа ничего не выполняет");
+   } catch (error) {
+     const captured = [...stdout, ...stderr].join("");
+     setOutput(`${captured}${captured && !captured.endsWith('\n')} `);
+     setRuntimeMessage("Не удалось выполнить код – программа завершилась с ошибкой");
+   } finally {
+     setRunning(false);
+   }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+   setCode(STARTER_CODE);
+   setRuntimeReady(false);
+   setRuntimeMessage('');
+   setOutput('');
+   setRunning(false);
+ }

```

```

+         type="button"
+         onClick={reset}
+         className="inline-flex items-center
late-800 transition-colors"
+     >
+         <RotateCcw className="w-3.5 h-3.5" />
+     </button>
+     <button
+         type="button"
+         onClick={() => void runCode()}
+         disabled={running}
+         className="inline-flex items-center
-60 text-white text-xs font-bold transition-colors"
+     >
+         {running ? <Loader2 className="w-3.5
+         {running ? "Запускаю..." : "Запустить"}
+     </button>
+ </div>
+ </div>
+
+ <textarea
+     value={code}
+     onChange={(event) => setCode(event.target
+     onKeyDown={(event) => {
+         if ((event.ctrlKey || event.metaKey) &
+         event.preventDefault();
+         void runCode();
+     }
+     }}
+     spellCheck={false}
+     aria-label="Код Python"
+     className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+     placeholder="Напишите Python-код..."
+ </>
+
+ <div className="border-t border-slate-800
+     <label className="block px-4 pt-3 text-

```

Универсальное пособие • полный исходный код

```
-----
+         <div className="flex gap-2 rounded-xl border
+           <Info className="w-4 h-4 shrink-0 text-amb
+           <span>Первый запуск скачивает примерно нес
+         </div>
+         <div className="flex gap-2 rounded-xl border
+           <Info className="w-4 h-4 shrink-0 text-еме
+           <span>Код выполняется в браузере и не отпр
+         </div>
+       </div>
+     </div>
+   </main>
+ );
+}
+....
```

`vite.config.ts`

Полное содержимое после изменений

```
````typescript
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локальн
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
```

---

ФАЙЛ 2/113: AGENTS.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7

---

## # AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

**\*\*CRITICAL DIRECTIVE\*\***:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, strictly required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML tags.

---

ФАЙЛ 3/113: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2

---

## # Gemini App Canvas Instructions

Ты — AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas).  
При запросе пользователя создать или сгенерировать новую страницу, ты должен **\*\*строго\*\*** придерживаться готовых шаблонов разметки, предоставленных ниже.

### ## Правила для малых страниц (Билетов / Глав)

- \*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML-теги для форматирования.
- \*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна быть сгенерирована в формате `{ "id": "...", "content": ... }` для вставки в ``src/data/content.json``.
- \*\*Строгая структура контента:\*\***

```

 "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
]
}
```

ФАЙЛ 6/113: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 54 | РАЗМЕР: 2

```

{
 "name": "algv0-exam-guide",
 "private": true,
 "version": "0.0.0",
 "type": "module",
 "scripts": {
 "dev": "vite",
 "build": "vite build",
 "preview": "vite preview",
 "audit:content": "node scripts/audit-coverage.mjs",
 "audit:terms": "node scripts/audit-terms.mjs",
 "export:txt": "node scripts/export/collect-code.mjs",
 "export:png": "node scripts/export/render-pages.mjs",
 "export:pdf": "node scripts/export/build-pdf.mjs",
 "export": "npm run export:txt && npm run export:png",
 "export:guide": "node scripts/export/build-guide-pdf.mjs",
 "export:shots": "node scripts/export/capture-viz-shots.mjs",
 "verify:unit": "node scripts/verify/live-matrix.test.mjs",
 "verify:code": "node scripts/verify/check-reference.mjs",
 "verify:links": "node scripts/verify/check-links.mjs",
 "verify:dom": "node scripts/verify/run-browser-check.mjs",
 "verify:probe": "node scripts/verify/run-probe.mjs",
 "verify": "npm run verify:unit && npm run verify:links",
 },
 "dependencies": {
 "clsx": "2.1.1",
 "lucide-react": "^1.21.0",
 "motion": "^12.40.0",
 },
}
```

```
```bash
npm install
npm run dev          # предпросмотр на http://0.0.0.0:5
npm run build        # сборка в dist/ (single-file)
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **\*\*весь\*\*** исходный код репозитория в о

| Команда              | Шаг       | Результат                               |
|----------------------|-----------|-----------------------------------------|
| `npm run export:txt` | код → txt | `public/export/full_code_all_pages.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/`                     |
| `npm run export:pdf` | png → pdf | `public/export/full_code_all_pages.pdf` |
| `npm run export`     | всё сразу | txt + png + pdf                         |

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash
sudo apt-get install -y imagemagick fonts-dejavu-core
```
```

Настройки через переменные окружения:

- \* `EXPORT\_DENSITY=200` – DPI растеризации (по умолчанию 300)
- \* `EXPORT\_GRAYSCALE=1` – оттенки серого вместо 1-битного

### Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается **\*\*на каж**  
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док  
и коммитит обновлённые `public/export/full\_code\_all\_pages`  
(с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code`

Если вы хотите парсить HTML конспект внешними инструментами, поддерживаемые теги и классы.

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `<h2>` и `<h3>`.
- \* Подзаголовки: Используют тег `<h3>`.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `<p>`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика: Используют тег `<p>`.

## ## Математические формулы

В приложении используется MathJax.

- \* В самом исходном коде страницы (и внутри тегов) формулы заключаются в `math` тег.
- \* Таким образом, при извлечении текстового содержимого формулы остаются в валидном Markdown-формате латеха (`math` тег).

## ## Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторные изображения.

- \* Они помещены внутрь оберток с классом `<div class="img">`.
- \* Все отрисовки находятся внутри векторного тега `<svg>`.
- \* Подпись к картинке, если она есть, идет сразу после `<img>`.

## ## Списки

- \* Стандартные списки оформлены через `<ul class="list-n">`.

## ## Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML документа.

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`.
2. Элементы `.analogy-badge` получают выделение маркера `!analogy`.
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `<li>` получают маркер `-`.
5. Результат извлекается в виде текста и чистится от лишнего.

Дополнительно – «осиротевшие» визуализаторы без глав (к

- \* `stack-dfs` → `StackViz.tsx` – \*\*Стек и DFS\*\*
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – \*\*Очередь и BFS
- \* `heap-beam-search` → `HeapViz.tsx` – \*\*Куча и лучевой поиск
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` – \*\*Деревья сегментов
- \* `dynamic-programming` → `DPVisualizer.tsx` – \*\*Динамическое программирование

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухая математика»)

| Глава                                      | id                    | Комментарий             |
|--------------------------------------------|-----------------------|-------------------------|
| 15. Планарность: K5, K3,3 и формула Эйлера | `planarity-euler`     | т «на пальцах».         |
| 16. Флойд (Флойд–Уоршелл)                  | `floyd`               | Есть матричный алгоритм |
| 17. Алгоритм Джонсона                      | `johnson-algo`        | Перевзвешивание рёбер   |
| Обзор: Кратчайшие пути и Графы             | `intro`               | Вводная страница        |
| Эйлер: Путь ≠ Цикл                         | `euler-path-vs-cycle` | Ключевая теорема        |

## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации (сложные алгоритмы)

> Замечание: «5. Двумерная матрица префиксных сумм» полна аналогий

| Глава                                | id                   | Чего не хватает              |
|--------------------------------------|----------------------|------------------------------|
| 3. Декартово дерево (Treap)          | `treap`              | Нет ни визуализации, ни кода |
| 8. Графы. Компоненты связности       | `graph-components`   | Нет кода                     |
| 24. Классы сложности, сведение задач | `complexity-classes` | Нет кода                     |

## Сводная таблица по всем главам



4. **\*\*Решить судьбу `alg-map`\*\*** – раскрыть или удалить.

ФАЙЛ 9/113: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
 "types": ["node"],

 /* Bundler mode */
 "moduleResolution": "bundler",
 "allowImportingTsExtensions": true,
 "resolveJsonModule": true,
 "isolatedModules": true,
 "noEmit": true,
 "jsx": "react-jsx",

 /* Path mapping */
 "baseUrl": ".",
 "paths": {
 "@/*": ["src/*"]
 },

 /* Linting */
 "strict": true,
 "noUnusedLocals": true,
 "noUnusedParameters": true,
 "noFallthroughCasesInSwitch": true
 },
 "include": ["src", "vite.config.ts"]
}
```

```
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

---

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

---

---

ФАЙЛ 11/113: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 218 | РАЗМЕР: 11.3 КБ

---

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import { PanelLeftClose, PanelLeftOpen } from "lucide-react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { LiteGuide } from "../components/LiteGuide";
import { PythonCompiler } from "../components/PythonCompiler";
```

```
/**
```

```
 * «Лёгкий» режим для печати и внешних парсеров: `?lite`
 * или `?lite=1&topic=mst` (одна тема). Без навигации, без поиска.
 */
```

```
function readLiteMode(): { lite: boolean; topic?: string } {
 if (typeof window === "undefined") return { lite: false };
 const q = new URLSearchParams(window.location.search);
 const lite = q.has("lite") || q.get("view") === "lite";
 return { lite, topic: q.get("topic") ?? undefined };
}
```

```

 return typeof window !== "undefined" && window.innerWidth > 0;
 });
 const toggleSidebar = useCallback(() => {
 setSidebarMode((m) => {
 const next = m === "wide" ? "compact" : m === "compact" ? "wide" : m;
 window.localStorage.setItem("sidebarMode", next);
 return next;
 });
 }, []);
 const sidebarCollapsed = sidebarMode === "hidden";

 // Десктопный отступ контента = ширине панели компиля
 const contentRef = useRef<HTMLDivElement>(null);
 useEffect(() => {
 const apply = () => {
 const el = contentRef.current;
 if (!el) return;
 el.style.paddingRight = compilerOpen && window.innerWidth > 0 ? compilerWidth : 0;
 };
 apply();
 window.addEventListener("resize", apply);
 return () => window.removeEventListener("resize", apply);
 }, [compilerOpen, compilerWidth]);

 const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
 const activeChapter = chapters[index] ?? chapters[0];
 const prev = index > 0 ? chapters[index - 1] : undefined;
 const next = index < chapters.length - 1 ? chapters[index + 1] : undefined;

 const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
 }, []);

 // Открыть компилятор и подогнать визуализацию вверх:

```

```

 total={chapters.length}
 />

<div className="flex flex-col lg:flex-row max-w-
 /* Сайдбар только на десктопе — на мобильном
 <div
 id="sidebar-root"
 className={`hidden lg:block shrink-0 border
 -all duration-200 print:hidden ${sidebarCol
 " : "lg:w-56"}`}>
 >
 {!sidebarCollapsed && (
 <Sidebar selectedChapterId={activeChapterId}
)}
 </div>
 /* Вкладка-переключатель содержания у левого
 <button
 type="button"
 onClick={toggleSidebar}
 className="hidden lg:flex items-center gap-
 border-slate-700 bg-slate-900/90 px-1 py-3
 aria-label={sidebarMode === "wide" ? "Сжать
 ание"}
 title={sidebarMode === "wide" ? "Сжать соде
 }
 >
 {sidebarCollapsed ? <PanelLeftOpen className=
 <span className="text-[10px] font-bold [wri
 </button>

<main id="main-content" className="flex-1 min
 /* Шапка главы с быстрыми переходами */
 <div className="flex items-center justify-b
 <div className="min-w-0">
 <div className="text-[10px] uppercase t
 {activeChapter.category || "Универсал
 </div>
 <h2 className="text-base sm:text-xl fon

```

```
 width={compilerWidth}
 height={compilerHeight}
 onWidthChange={setCompilerWidth}
 onHeightChange={setCompilerHeight}
 onClose={() => setCompilerOpen(false)}
 />
)}
 </div>
);
}
```

---

ФАЙЛ 12/113: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 129 | РАЗМЕР: 5.1 K

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, text";

/** Числовые константы TreeWalker: глобального `NodeFile
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

const escapeRe = (s: string) => s.replace(/[\.*\+\?\^\$\{\}\(\)]/
```

```

for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createElement("span"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("span"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

```

```
@import "tailwindcss";

@layer utilities {
 .neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
 }
 .neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
 }
 .neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
 }
 .neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
 }
 .no-scrollbar::-webkit-scrollbar {
 display: none;
 }
 .no-scrollbar {
 scrollbar-width: none;
 }
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
 }
}

.animate-pulse-gold {
```

```
.term-hint:hover,
.term-hint:focus-visible {
 background: rgba(99, 102, 241, 0.22);
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Ссылка на другой билет («...берите Форда–Беллмана (билет
.ticket-link {
 color: #a5b4fc;
 border-bottom: 1px dotted rgba(165, 180, 252, 0.7);
 text-decoration: none;
 border-radius: 3px;
 padding: 0 1px;
 cursor: pointer;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.ticket-link:hover,
.ticket-link:focus-visible {
 background: rgba(99, 102, 241, 0.25);
 color: #fff;
 outline: none;
}

/* Контент главы приходит готовым HTML — чиним его под
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}
```



```
}
.chapter-body :where(h2) {
 font-size: 1.25rem !important;
 line-height: 1.3 !important;
}
.chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
}
.chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
}
.chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
}
.chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
}
.chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
}
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}
```

```
 display: none !important;
}

main {
 padding: 0 !important;
 max-width: none !important;
}

/* текст темы – чёрным по белому, без теней и плашек
article.chapter-body .prose,
article.chapter-body .prose * {
 color: #111827 !important;
 background-color: transparent !important;
 background-image: none !important;
 box-shadow: none !important;
 text-shadow: none !important;
 border-color: #d1d5db !important;
}

article.chapter-body .prose pre,
article.chapter-body .prose code {
 background-color: #f3f4f6 !important;
 color: #111827 !important;
 font-size: 9pt;
}

article.chapter-body .prose table {
 border-collapse: collapse;
}

article.chapter-body .prose th,
article.chapter-body .prose td {
 border: 1px solid #9ca3af !important;
}

article.chapter-body .prose h2,
article.chapter-body .prose h3,
article.chapter-body .prose h4 {
```

---

ФАЙЛ 14/113: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/113: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
 category?: string;
}
```

---

ФАЙЛ 16/113: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
category: "Продвинутые структуры",
content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">

 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
] [k] </p>
 </div>

 <p class="text-slate-300 text-sm mb-4">Ячейки
 линии
 <i>
 . Любой отрезок покрывается двумя
 операция идемпотентна: min,
 , побитовые И и ИЛИ.</p>

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
 вет = op из четырёх блоков kx * ky, покрывае
 </div>

 <p class="text-slate-300 text-sm mb-4">Двумя
 по столбцам. Отсюда и цена: уровней становится
 s="font-mono text-emerald-300">log N

```

Работает ТОЛЬКО для суммы: менять элементы

```
</div>
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
```

Аналогия 2: Торт-пирамидка

```
</div>
<p class="text-slate-300 text-sm mb-4">
ия, зато красивый, как пирамидка: каждый из
сех блоков $2^{k_x} \times 2^{k_y}$), и ответ на запрос
ой операции нет: крем обратно в торт не запы-
```

```
<div class="rounded-lg bg-slate-950"
 <div><span class="text-slate-500"
n $0(1)$ </div>
```

```
 <div><span class="text-slate-500"
= 100 копий матрицы</div>
</div>
```

```
<p class="text-[12px] text-slate-400">
шает). Менять элементы тоже нельзя. Сумму ч
```

```
</div>
<p class="text-slate-400 text-sm mt-4">Поря
ерению) → 2D: построение (уровни
```

```
ос (прямоугольник раскладывается на чет
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-rose-400"
 <p class="text-slate-300 text-sm mb-4">Эти
 за <span class="font-mono text-emerald-300"
они решают разные задачи.</p>
```

```
<div class="bg-slate-800/70 p-5 rounded-lg"
 <p class="text-sm font-bold text-white"
 <p class="text-slate-300 text-sm mb-3">
цы, у разреженной таблицы – размер, умножен
<table class="w-full text-xs text-left">
```



```

 for (int c = 0; c + (1 << ky) - 1 < M; ++c)
 if (kx == 0 && ky == 0) st[r][c][0][0] = min(s, st[r][c][0][0]);
 else if (ky == 0) st[r][c][kx][0] = min(s, st[r][c][kx][0]);
 else st[r][c][kx][ky] = min(s, st[r][c][kx][ky]);
 }

```

```

// запрос минимума на прямоугольнике (R1,C1)..(R2,C2)
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);

```

жимаются к четырём углам запроса и вместе с ним возвращают минимальное значение.

**Сколько это памяти в числах**

Почему же так много памяти? Потому что у нас есть 1000 × 1000 уровней, а запросов очень много.

«Миф  $2k/2k+1$ » и «Поиск  $\neq$  сортировка»  
n) и ничего не сортирует, поэтому заменить  
ни точки сам, а судья-инварианты объяснит к

```
<div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">
 <p class="my-1 font-bold leading-none">

ext-2xl text-slate-300">Й
 </p>
 <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Split – таможня
 </div>
 <p class="text-slate-300 text-sm">S
х»». Спуск от корня: узел, который вместе
скрытия – досматривается только одна ве
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Merge – стыковка
 </div>
 <p class="text-slate-300 text-sm">M
аг – одно сравнение: чей у у корня больше,
сохраняет порядок x). Итого $O(h)$ сравнений
 </div>
</div>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
```



```

</details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код: универсальная вставка, merge
 </summary>
 <div class="p-5 text-sm text-slate-300
 <p class="text-xs text-amber-300"><
 </p>
 <pre class="bg-slate-950 p-4 rounded
def insert(root, node):
 if not root: return node
 L, R = split(root, node.key)
 return merge(merge(L, node), R)</pre>
 <p class="text-xs text-slate-400"><
 и элементы подаются на вход, будучи заранее
 p>
 <pre class="bg-slate-950 p-4 rounded
def merge(a, b):
 # все x в a < все
 if not a or not b: return a or b
 if a.pri > b.pri:
 a.right = merge(a.right, b); return a
 b.left = merge(a, b.left); return b

def erase(root, key):
 # удалить произвольн
 L, R = split(root, key)
 L_left, L_target = split(L, key - 1)
 return merge(L_left, R)</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "splay-tree",
 title: "4. Splay-дерево",

```

по факту

: ничего не хранит, дереву разрё

```
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Стопка бумаг

```
</div>
<p class="text-slate-300 text-sm">С
тировку не ведёшь, но через неделю самое ну
ерх».</p>
```

```
</div>
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Аналогия 2: Тропинка через га.

```
</div>
<p class="text-slate-300 text-sm">И,
рно вдвое. Первый проход дорогой, зато опла
ы.</p>
```

```
</div>
</div>
```

```
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Повор
Среднее поддерево β (между x и p) – единств
 x , и под p .</p>
```

```
<pre class="bg-slate-950 p-4 rounded-lg ove
ading-relaxed">
```

```

 / \ \
 x y
 / \ \
 α β
 / \ \
 x y
 / \ \
 α β

 x выше
 -----> α
 / \ \
 p r
 / \ \
 β γ

 p выше
 <-----
 / \ \
 p r
 / \ \
 β γ
 </pre>
```

```
<p class="text-slate-300 text-sm mt-3">Обхо
омать дерево поиска поворотами невозможно в
омент «отстёжки», а врезка «Механика одного
```

```
<h4 class="text-lg font-bold text-white mt-3">
```

```
<pre class="bg-slate-950 p-4 rounded-lg">
 амбук из 8 узлов 8→7→6→5→4→3→2→1, ищем 1 (ч
наивно (x, потом x): 3 × «два зига» + Zig → высота 8
Zig-Zig (p, потом x): 3 × Zig-Zig + Zig → высота 8
бамбук из 16 узлов: наивно 16 → 16, Zig-Zig 16 → 10<
```

```
 <p class="text-slate-300 text-sm mt-3">
де: <code class="bg-slate-950 px-2 py-0.5 r
s="bg-slate-950 px-2 py-0.5 rounded font-mon
изации не будет.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-6">Полн
м 1. Спуск 8 узлов, затем три Zig-Zig по
8 до 6 – линия перестала быть линией. Наивно
о перевёрнутой. Важно не врать себе: на кор
ван не на одном запросе, а в сумме. Следующ
по пути. Отсюда амортизация: дорогой за
дом потенциала: потенциал Φ = сумма по узла
ь его; отсюда же теорема о статической опти
ь запросов).</p>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
 <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечиваетс
 <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
ое <code class="bg-slate-950 px-2 py-0.5 ro
одиночная операция – до <code class="bg-sla
>O(n)</code>; n вставок подряд – амортизиро
-300 border border-slate-800">O(n log n)</c
```

```
<p class="text-slate-300 text-sm mt-4">Все
```

```

 rotate_up(x)
 else: # ZIG-ZAG: НИЖНЯЯ
 rotate_up(x)
 rotate_up(x)
 return x

def find(root, key): # поиск: спуск + s
 cur, last = root, None
 while cur:
 last = cur
 if key == cur.key: break
 cur = cur.left if key < cur.key else cur.right
 return splay(last) if last else None</pre>
 <p class="text-xs text-slate-400">В
 Одна строка – и вся амортизация.</p>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сумма подматрицы за O(1): 1D и 2D, дв
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>

 <div class="space-y-8">

 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 <td class="py-2 pr-3 font-mono">
 </td>
 </tr>
 </tbody>
</table>
<p class="text-slate-400 text-xs mt-2">
 ышли за границу» – поэтому в 2D почти всегда
</div>

<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-rose-300 t
 <p class="text-[11px] text-slate-500
 </div>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800
 <div class="absolute -top-3 left-4 b
 rder border-emerald-500 shadow-md">
 Аналогия 1: Сетка из вафельных
 </div>
 <p class="text-slate-300 text-sm mb
 о накопилось от угла (0,0) до меня». Хочешь
 у и верни дважды отрезанный угол. Вычитание

```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Тонкая: одна матрица
 </div>
 <p class="text-slate-300 text-sm mb-2">
 одной матрицы (плюс рамка из нулей). Никаких вычитаний, только сложение. Прямоугольника от угла. Всё остальное получается.
 </p>
</div>
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
 Не умеет: всё, что нельзя вычислить. Например, gcd вычитания не существует: это не запросы — это 1234567890 платит она за это памятью 1234567890
 </div>
</div>

<div class="bg-slate-800/70 p-5 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <p class="text-sm font-bold text-white">Таблица</p>
 <table class="w-full text-xs text-left" style="width: 100%; border-collapse: collapse;">
 <thead>
 <tr class="text-slate-500 border-b border-slate-700">
 <th class="py-1.5 pr-2 font-normal">№</th>
 <th class="py-1.5 pr-2 font-normal">Имя</th>
 <th class="py-1.5 pr-2 font-normal">Возраст</th>
 </tr>
 </thead>
 <tbody>
 <tr class="border-b border-slate-700">
 <td class="py-2 pr-3 font-normal">1</td>
 <td class="py-2 pr-3 font-normal">Иван</td>
 <td class="py-2 pr-3 font-normal">25</td>
 </tr>
 <tr class="border-b border-slate-700">
 <td class="py-2 pr-3 font-normal">2</td>
 <td class="py-2 pr-3 font-normal">Мария</td>
 <td class="py-2 pr-3 font-normal">30</td>
 </tr>
 </tbody>
 </table>

```

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код: 1D (тот же, что выполняет пане
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [3, 1, 4, 1, 5, 9, 2, 6] # исходный ма
n = len(a) # префиксов будет n + 1
P = [0] # P[0] = 0 – пустой преф
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)</code></pre>
 <p class="text-slate-300 text-sm">П
 - часть демонстрации: лента подсвечивает, из
 </div>
 </details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код: 2D – построение и запрос прямо
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4

S = [] # рамка из нуле
for rr in range(n + 1):
 S.append([0] + [0] * m)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i-1][j-1] + S[i-1][j] + S[i][j-1] -

r1, c1, r2, c2 = 1, 1, 2, 2 # прямоугольник
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1]
print("итого:", ans)</code></pre>

```

```

 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Покраска
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.</p>
 <p class="text-xl font-mono text-white">
 1. Если граф не планарный, то его нельзя раскрасить 4 цветами.
 2. Если граф планарный, то его можно раскрасить 4 цветами.
 </p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Границы не пересекаются в одной точке, поэтому можно раскрасить карты, чтобы они не сливались цветами. Всегда можно всего 4 цвета.
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",

```



```

 </div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая.",
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">В strongly connected component (SCC) графа — это максимальный подграф, в котором каждая вершина достижима из каждой другой вершины. Другими словами, в SCC для любых двух вершин u и v существует путь из u в v и из v в u.
 <p class="text-xl font-mono text-white">Алгоритм Косарая позволяет найти все SCC в орграфе, инвертированному в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением
 </div>
 <p class="text-slate-300 text-sm mb-4">Представьте себе карту города, где улицы имеют одностороннее движение. Если вы можете пройти от точки A к точке B по улицам, и наоборот, то A и B находятся в одной «сильно связной компоненте».</p>
 <p class="text-slate-300 text-sm mb-4">В реальных графах (например, в социальных сетях или в системах зависимостей) SCC помогают понять структуру взаимосвязей. Если граф состоит из нескольких SCC, это означает, что есть группы узлов, которые тесно связаны между собой, но слабо связаны с остальными частями графа.</p>
 <p class="text-slate-300 text-sm mb-4">В БХ направлениях по односторонним улицам. Как это выглядит на карте.</p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },

```

```

 description: "Вершины, удаление которых убивает свя
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 теи > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb-4">
 правильный роутер - и два сегмента офиса б
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-blue-300 italic"
 <p class="text-xl font-mono text-white"
d-длина.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под .
 </div>
 <p class="text-slate-300 text-sm mb
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площ
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-2">
 (или вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </p>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </h2>

 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </p>
 </div>
 </h3>
 <div class="grid grid-cols-1 gap-6">

```

```

 rder border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс
 </div>
 <p class="text-slate-300 text-sm mb
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
меня перепроверок!</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

 </div>
</section>`,
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description: "",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 24
 <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">Классы сложности</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-purple-400">Сведение задач</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">Пример: класс P (простые задачи) vs класс NP (нестрогие задачи).</p>
 <p class="text-base sm:text-lg font-monospace">
 P: класс decision-задач, решаемый за полиномиальное время.
 NP: класс decision-задач с сертификатом.
 </p>
 <p>).

 Сведение (Reduction) A ≤_p B: задача A сводится к задаче B.
 </p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Судоку (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">Пример: сортировка чисел. Класс NP – задача нахождения минимальную сетку (ответ), он БЫСТРО (за класс P) решается.</p>
 </div>
 </div>
 </div>
 </div>
`
 }
]

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"

 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-blue-300 italic"
 <div class="text-left font-mono text-sm"
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
 <p><strong class="text-white">Эйлер
 <div class="p-3 bg-slate-950 rounded"
 <p class="text-sm text-blue-300"
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </div>
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

• <span class="text-green-400 font-
шел-вошёл-вышел).

• <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap"
 <div class="bg-slate-800 p-6 rounded-lg bor"
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 Аналогия 1: Путь – из дома в бар,
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная)
 талый пришёл в бар (вторая нечётная
 Домой вернуться не смог, потому что
 </p>

```

```

 </div>
</section>`,
 },
 {
 id: "bridges-code",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Мосты через DFS Тарьяна: tin/low, усл
 category: "Графы. Продвинутые",
 content: `<section id="bridges-code" class="mb-20 s
 <div class="flex items-center mb-6">
 <span class="bg-emerald-600 text-white px-4 py-
 <h2 class="text-3xl font-bold text-white">Мосты
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-400">
 Мосты через DFS Тарьяна: tin/low, усл

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 ital">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">tin[v
 <p><strong class="text-white">low[v
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-400
 </div>
 </div>
 </div>
 </div>
 </div>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков <str
 Единственная ниточка. Порвётся – граф р
 </p>
 </div>
 </div>
 }
}

```



```

<pre class="text-xs font-mono text-
#include <vector>;
#include <algorithm>;

using namespace std;

int n; // число вершин
vector<vector<int>>> adj; // список смежност
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем
 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад
 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный в
 low[v] = min(low[v], low[to]);
 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
}
```

```

 <p><span class="text-emerald-400 font-
 <p><span class="text-rose-400 font-l
 </div>
</div>

<p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше – граф гарантированно
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 K5: Полный граф на 5 вершинах
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У K5: <code class="text-white">V=5,
-white">10 > 9</code> – БАХ, непланарен!
 Все 5 вершин соединены со всеми. На
 Теорема Куратовского: K5 и K3,3 – д
 Если внутри графа спрятано подра
 – плоской раскладки не будет.
 </p>
 </div>

<div class="bg-slate-900 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
 K3,3: 3 дома и 3 колодца
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Три дома хотят соединиться с тремя
 Нарисовать без пересечений невозможно
 У него <code class="text-white">V=6
xt-white">9 ≤ 12</code> – проходит, но он в
```

```
<div class="grid grid-cols-1 xl:grid-cols-2" style="border: 1px solid #000; padding: 10px;">
 <div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; margin-bottom: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 1px solid #000; padding: 5px;">
 Аналогия 1: Расписание экзаменов
```

```
<div class="overflow-x-auto my-6" style="border: 1px solid #000; padding: 10px;">
 <table class="w-full text-xs text-left" style="border-collapse: collapse;">
 <thead>
 <tr class="text-slate-500 border-b" style="border-bottom: 1px solid #000;">
 <th class="py-2 pr-3 font-b" style="text-align: left; padding: 5px 10px 5px 5px;">$\chi \leq \Delta + 1$
 <th class="py-2 pr-3 font-b" style="text-align: left; padding: 5px 10px 5px 5px;">$\chi \leq \Delta$
 <th class="py-2 font-bold" style="text-align: left; padding: 5px 10px 5px 5px;">$\chi \leq \Delta + 1$
 </tr>
 </thead>
 <tbody class="text-slate-300" style="border-top: 1px solid #000;">
 <tr class="border-b border-slate-800" style="border-bottom: 1px solid #000;">
 <td class="py-2 pr-3 font-mono" style="padding: 5px 10px 5px 5px;">$\chi \leq \Delta + 1$
 а свободен</td>
 </tr>
 <tr class="border-b border-slate-800" style="border-bottom: 1px solid #000;">
 <td class="py-2 pr-3 font-mono" style="padding: 5px 10px 5px 5px;">$\chi \leq \Delta$
 </td>
 </tr>
 </tbody>
 </table>
</div>
```

```

 веркой цветов соседей; проверка двудольности
border border-slate-800">0(V + E)</code>; т
-300 border border-slate-800">NP-трудно</co

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: жадная раскраска и проверка
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
я раскраска: не больше $\Delta + 1$ цвета
color = [-1] * n # -1 = ещё не покрашен
for v in order: # порядок влияет на чи
 used = set()
 for to in graph[v]:
 if color[to] != -1:
 used.add(color[to]) # цвета уже покрашены
 c = 0
 while c in used: # минимальный свободны
 c += 1
 color[v] = c
chi = max(color) + 1 # χ , который выдал жад

Проверка двудольности = раскраска в 2 цвета (BFS)
from collections import deque
col = [-1] * n
bipartite = True
for start in range(n):
 if col[start] != -1:
 continue
 col[start] = 0
 dq = deque([start])
 while dq:
 v = dq.popleft()
 for to in graph[v]:
 if col[to] == -1:
 col[to] = col[v] ^ 1 # второй цвет

```

```
<p class="text-slate-300 text-sm">
```

```
 Важно: Это работает то-
 еплятся к точкам сочленения. То есть,
 ег - это тупик со степенью 1).
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-
```

```
 <div class="bg-slate-800 p-6 rounded-lg border-
```

```
 <div class="absolute -top-3 left-4 bg-slate-800
 rder-rose-500 shadow-md">
```

```
 Аналогия: Главный перекресток
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 Представь город, где два района соединены
 её перекроют (удалят вершину) — районы будут
 g>хрупкость системы.
```

```
 </p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-
```

```
 <div class="absolute -top-3 left-4 bg-emerald-500
 er border-emerald-500 shadow-md">
```

```
 Телеком: Центральный свитч
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 У тебя несколько компьютеров в одной сети,
 абелем (мостом). Сами свитчи — это точки со-
```

```
 </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-
```

```
 <summary class="p-4 font-bold text-slate-400
 order-slate-700/50">
```

```
 Душный мод: Код и корень DFS (Скрыто)
```

```
</summary>
```

```

 <ul class="list-disc list-inside text-sm text-gray-500">
 <strong class="text-rose-400">Точки ориентированных графов. Они описываются с помощью матрицы смежности
 <strong class="text-emerald-400">Сжатие SCC ориентированных графов. Сжатие SCC даёт ориентированный граф, в котором вершины являются SCC
 Внимание: Нельзя применять алгоритмические структуры для разных типов графов.

 </div>
</div>
</section>`,
 },
];

```

ФАЙЛ 19/113: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 149 | РАЗМЕР: 1.1 КБ

```

import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { ticket1Segtree } from "../tickets/ticket1_segtree";
import { tickets18to20Mst } from "../tickets/ticket18_20_mst";
import { ticket25Cheatsheet } from "../tickets/ticket25_cheatsheet";
import { chapters as newChapters } from "../content_temp";

/**
 * Содержание пособия = билеты 1–24 из списка тем.
 *
 * Порядок массивов важен: при одинаковом id побеждает последняя
 * версия главы. `newChapters` (content_temp) перетирает старые.
 * поздние переписанные страницы (Эйлер, Мосты, Планарные графы).
 *
 */

```

```

 if (!toRemove.includes(c.id)) {
 dedup.set(c.id, c);
 }
 });

// Самые свежие версии страниц перетирают старые.
if (newChapters) {
 newChapters.forEach((c) => {
 if (!toRemove.includes(c.id)) dedup.set(c.id, c);
 });
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
 // Главы без номера билета (введение) стоят ДО билетов
 const numOf = (t: string) => {
 const m = t.match(/(\d+)/);
 return m ? parseInt(m[0], 10) : -1;
 };
 return numOf(a.title) - numOf(b.title);
});

export const chapters: Chapter[] = finalChapters;

/**
 * Билеты, закрытые каждой страницей (для шапки, экспор
 * Страница `mst`, например, закрывает сразу 18, 19 и 20
 */
export const chapterTopics: Record<string, number[]> =
 "segment-trees": [1],
 "sparse-table": [2],
 treap: [3],
 "splay-tree": [4],
 "prefix-sums-2d": [5],
 "graph-dfs-bfs": [6],
 "planarity-euler-formula": [7],
 "graph-components": [8],

```

```

const tickets = chapterTopics[chapterId] ?? [];
// Глав без билетов (введение) относим к первому разд
if (!tickets.length) return SECTIONS[0];
const first = Math.min(...tickets);
return SECTIONS.find((sec) => first >= sec.from && fi
}

export const activeVizIds = [
 "euler-path-vs-cycle", "bridges-code", "planarity-euler
 "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se
 "dijkstra", "bellman-ford", "floyd", "aho-corasick",
 "mst", "string-kmp", "string-z-func", "segment-trees", "s
 "dynamic-programming", "johnson-algo", "graph-articulat
 "complexity-classes", "treap", "splay-tree", "prefix-sum
 "graph-cheatsheet",
];

```

ФАЙЛ 20/113: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 212 | РАЗМ

```

/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

```

```

export type DebugValue = null | boolean | number | string

```

```

export interface DebugStep {
 /** Номер строки в коде пользователя (1-based). */
 line: number;
 /** Имя функции-фрейма ("<module>" – верхний уровень)
 func: string;
 /** Локальные переменные: имя → герг (усечённый), для
 locals: Record<string, string>;

```



```

__src = ${srcLiteral}
__steps = []
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False

class __TraceLimit(Exception):
 pass

def __safe(v):
 try:
 r = repr(v)
 except Exception:
 r = "<...>"
 return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
 """Небольшой JSON-снимок: UI использует его без парсера
 if v is None or isinstance(v, (bool, int, str)):
 return v
 if isinstance(v, float):
 return v if v == v and v not in (float("inf"), float("-inf")) else "..."
 if depth >= 4:
 return __safe(v)
 limit = 80 if depth < 2 else 32
 if isinstance(v, (list, tuple)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, (set, frozenset)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, dict):
 # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
 # предел 80 незаметно обрезал уровни 4x4 и 8x8
 dict_limit = 192 if depth == 0 else limit
 return {str(k): __snapshot(x, depth + 1) for k, x in list(v.items())}
 # deque, heap-подобные и другие итерируемые учебные
 if type(v).__name__ == "deque":
 return [__snapshot(x, depth + 1) for x in list(v)]

```

```

 loc, values = __capture(frame)
 # PEP 709: с 3.12 компрехеншны встроены во фрейм
 # на своей строке каждую итерацию. Склеиваем по
 if __steps and __steps[-1]["line"] == frame.f_lineno:
 __steps[-1]["locals"] = loc
 __steps[-1]["values"] = values
 else:
 __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
 if len(__steps) >= __CAP:
 # Нельзя просто отключить trace: бесконечный цикл
 # тогда продолжит работать вечно. Мягко прерываем
 __was_truncated = True
 raise __TraceLimit()
 # Финальные локалы программы (после последней строки)
 if event == "return" and frame.f_code.co_filename == __FILE__:
 loc, values = __capture(frame)
 __final.update(loc)
 __final_values.update(values)
 return __tracer

__err = ""
try:
 __sys.settrace(__tracer)
 __init_globals = {
 "__name__": "__main__",
 "A": [5, 8, 3, 12, 7, 2, 0, 0],
 "a": [5, 8, 3, 12, 7, 2, 0, 0],
 }
 exec(compile(__src, "<user-code>", "exec"), __init_globals)
except __TraceLimit:
 pass
except Exception:
 import traceback as __tb
 __err = __tb.format_exc(limit=3)
finally:
 __sys.settrace(None)

OUT = json.dumps({

```

```

const syncEntries = Object.entries(cur).filter(([k]) => !inSync.has(k))
const rest = Object.entries(cur)
 .filter(([k]) => !inSync.has(k))
 .sort(([a], [b]) => a.localeCompare(b));
return [...syncEntries, ...rest];
}
```

ФАЙЛ 21/113: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 639 | РАЗМЕР: 10.5 КБ

```

import type { DemoKind } from '../components/hints/MiniHints';

/**
 * Глоссарий «википедийных» подсказок.
 *
 * Любое слово из `labels`, встреченное в тексте главы,
 * в подчёркнутый термин: при наведении (или тапе на мобильном)
 * с объяснением на пальцах и мини-визуализацией.
 */
export interface GlossaryEntry {
 id: string;
 /** Варианты написания в тексте (регистр не важен). */
 labels: string[];
 title: string;
 /** Объяснение «на пальцах» – 1–2 фразы. */
 short: string;
 /** Строгая формулировка / что важно не забыть. */
 formal?: string;
 complexity?: string;
 /** Какая мини-анимация показывается в карточке. */
 demo?: DemoKind;
 /** id полного симулятора (см. vizRegistry) – кнопка «симулятор» */
 simulator?: string;
}
```

```

 },
 {
 id: "heap",
 labels: ["куча", "кучу", "кучи", "куче", "кучей", "куча"],
 title: "Куча (priority queue)",
 short:
 "Не отсортированный список, а «мешок, из которого достаём минимальный элемент»",
 formal:
 "Полное бинарное дерево в массиве: дети узла i лежат в узлах $2i$ и $2i+1$.",
 complexity: "вставка/извлечение $O(\log n)$, «посмотреть минимальный элемент» $O(1)$ ",
 demo: "heap",
 simulator: "heap-beam-search",
 },
 {
 id: "stack",
 labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
 title: "Стек (LIFO)",
 short: "Стопка тарелок: кладёшь сверху и берёшь сверху",
 formal: "push / pop / top за $O(1)$. На стеке живёт рекурсия",
 demo: "stack",
 simulator: "stack-dfs",
 },
 {
 id: "queue",
 labels: ["очередь", "очереди", "очередью", "FIFO"],
 title: "Очередь (FIFO)",
 short: "Очередь в столовой: кто первым встал, тот первым и ест",
 formal: "push_back / pop_front за $O(1)$. Основа BFS",
 demo: "queue",
 simulator: "queue-bfs",
 },
 {
 id: "dsu",
 labels: ["DSU", "СНМ", "система непересекающихся множеств"],
 title: "DSU — система непересекающихся множеств",
 short:
 "«Кто твой староста?» Каждый элемент знает своего старосту",
 formal:
 "Система непересекающихся множеств (DSU) — это структура данных, которая позволяет эффективно управлять группами элементов, которые не пересекаются. Она используется для решения задач, связанных с объединением и поиском принадлежности элементов к той же группе."
 }
]
}

```

```

 simulator: "aho-corasick",
},
{
 id: "suffix-link",
 labels: ["суффиксная ссылка", "суффиксные ссылки",
 title: "Суффиксная ссылка",
 short:
 "«Куда откатиться, если следующая буква не подошла",
 строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает",
 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "то",
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая с",
 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, л",
 complexity: "O(V + E)",
 demo: "toposort",
 simulator: "top-sort",
},
{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь",
 ,
 formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{

```

```

 labels: ["мост", "моста", "мосты", "мостов", "мостов"],
 title: "Мост",
 short:
 "Ребро, после удаления которого граф разваливается",
 formal: "Ребро (u, v) — мост $\iff \text{low}[v] > \text{tin}[u]$, где low — минимальный индекс вершины в поддереве, достижимом из v по ребрам, не являющимся ребром в родителя — нет).",
 complexity: "O(V + E) одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
{
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "точка сочленения"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её — и коллапсирует",
 formal: "Для не-корня: v — точка сочленения $\iff \exists$ ребро (u, v) такое, что $\text{low}[u] \geq \text{tin}[v]$ ",
 demo: "articulation",
 simulator: "graph-articulation",
},
{
 id: "mst",
 labels: [
 "остовное дерево", "остовного дерева", "остовное",
 "минимальное остовное дерево", "MST",
],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная длина была минимальной",
 formal: " $V-1$ ребро, никаких циклов. Краскал (жадно по возрастанию веса).",
 demo: "mst",
 simulator: "mst",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизированный"
]
}
```

```

 "Группы вершин, где из каждой можно дойти до каждой",
 formal: "Косарайю: DFS с сохранением порядка выхода",
 complexity: "O(V + E)",
 demo: "scc",
 simulator: "scc-kosaraju",
},
{
 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-функцию",
 "префикс", "префикса", "суффиксом", "суффикс", "суффиксов",
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начался в этой позиции и какой кусок.",
 formal:
 " $\pi[i]$ — длина наибольшего собственного префикса $s[0..i]$, который является суффиксом $s[0..i-1]$, возвращая указатель по тексту.",
 complexity: "O(n + m)",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "z-функция", "z-функции", "z-функцию"],
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начиная с этой позиции, совпадают с началом строки.",
 formal:
 " $z[i]$ = длина наибольшего общего префикса s и $s[i..n-1]$ ",
 complexity: "O(n)",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{

```

```

 id: "greedy",
 labels: [
 "жадный алгоритм", "жадный", "жадно", "жадная",
 "самое дешевое ребро", "самое дешёвое ребро", "са
],
 title: "Жадный алгоритм",
 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас",
 formal:
 "Корректность обычно доказывается «леммой о разре
 demo: "mst",
 simulator: "mst",
},
{
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова
 title: "Эйлеров путь и цикл",
 short:
 "Пройти по каждому ребру ровно один раз, не отрыв
 formal:
 "Связный граф: эйлеров цикл все степени чётные;
 simulator: "euler-path-vs-cycle",
},
{
 id: "planarity",
 labels: ["планарный", "планарность", "планарного",
 title: "Планарность",
 short:
 "Граф планарен, если его можно нарисовать на листе
 ,
 formal:
 " $V - E + F = 2$ для связного планарного графа. Кур
 ,
 simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [

```



```

],
 title: "Алгоритм Форда–Беллмана",
 short:
 "Тупой, но честный: V-1 раз проходим по ВСЕМ рёбрам",
 formal:
 "После k-й итерации гарантированно верны все кратчайшие пути, которые не содержат отрицательный цикл.",
 complexity: "O(V · E)",
 demo: "relax",
 simulator: "bellman-ford",
},
{
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд–Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл – это не «шаг», а «какие пересадки можно сделать, чтобы посадить пересадку в k?",
 formal:
 " $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$. Порядок вершин произвольный.",
 complexity: "O(V³) времени, O(V²) памяти",
 demo: "floyd",
 simulator: "floyd",
},
{
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на каждом шаге добавляем ребро, не создавая циклов",
 formal:
 "Множество посещённых + куча рёбер на границе. Добавляем ребро с минимальным весом, не создавая циклов.",

```

```

 formal:
 "Считаются одним проходом: пока есть непосещённая
complexity: "O(V + E)",
demo: "components",
simulator: "graph-dfs-bfs",
},
{
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "гра
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Горо
 formal:
 "G = (V, E). Ребро бывает ориентированным (улица
 demo: "graph",
 simulator: "graph-dfs-bfs",
},
{
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "с
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам ну
 порядковым номером.",
 formal:
 "sort + unique + binary search: значение → индекс
 ,
 complexity: "O(n log n)",
 demo: "coord-compress",
},
{
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова де
 title: "Treap = Tree + Heap",
 short:

```

```

 title: "Сведение задач",
 short:
 "«Я не умею решать A, зато умею B». Если A можно
 formal:
 "A $\leq_p$ B: есть полиномиальная функция, переводящая
 NP-полную задачу к новой.",
 demo: "np",
},
{
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива
 предподсчёты.",
 formal:
 "Суммы — префиксные суммы $O(1)$. Минимум без изменений
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "с"],
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: где
 (Z-функция) и где встречаются слова словаря (бор),
 formal:
 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функция
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "поворо"],
 title: "Поворот (rotation) в дереве",
 short:
 "Локальная перевеска двух узлов: ребёнок встает на

```

```
 formal: "Эквивалентно двойному повороту AVL-дерева",
 demo: "zig-zag",
 simulator: "splay-tree",
 },
];
```

```
/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {
```

```
/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
 e.labels.map((label) => ({ label, id: e.id }))
).sort((a, b) => b.label.length - a.label.length);
```

---

ФАЙЛ 22/113: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 520 | РАЗМЕР: 10.5 КБ

---

```
import { Chapter } from "../types";
```

```
export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Введение. Базовая база: куча, очередь, бинарное дерево",
 type: "html",
 category: "Введение",
 content: `<section id="intro-content" class="mb-12">
<div class="flex items-center mb-6 flex-wrap gap-2">
 Введение
 <h2 class="text-3xl font-bold text-white">Базовая база</h2>
</div>

<div class="bg-slate-900 border border-indigo-500/40 p-6">
 <p class="text-slate-200 text-sm leading-relaxed">
 Введение в базовую базу: куча, очередь, бинарное
 <b class="text-indigo-300">Базовая база — это структура данных, которая
 дерево как массив, куча, очередь и стек, бинарное</p>
 </div>
</section>`
 }
];
```

```

<tr class="hover:bg-slate-800/50">
 <td class="p-3 font-bold text-white">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 font-mono text-emerald-400">
</tr>
<tr class="hover:bg-slate-800/50 bg-slate-800/50">
 <td class="p-3 font-bold text-white">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 text-slate-500 italic">
</tr>
<tr class="hover:bg-slate-800/50">
 <td class="p-3 font-bold text-white">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 font-mono text-emerald-400">
</tr>
<tr class="hover:bg-slate-800/50 bg-slate-800/50">
 <td class="p-3 font-bold text-white">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 font-mono text-emerald-400">
</tr>
<tr class="hover:bg-slate-800/50">
 <td class="p-3 font-bold text-white">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">

```

```

<div class="space-y-8">

 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-indigo-400 mb
 <p class="text-slate-300 text-sm mb-4">Дерево н
 единицы, то родственников выдаёт простая ари
 чейках <span class="font-mono text-emerald-300
 n /
 ра.</p>
 <pre class="bg-slate-950 border border-slate-700
tree = [None, 40, 20, 60, 10, 30, 50, 70] # tree[0] н
корень n=1
дети 40: 2*1=2 → 20 и 2*1+1=3 → 60
дети 20: 2*2=4 → 10 и 2*2+1=5 → 30
родитель 30: 5 // 2 = 2 → 20</pre>
 <p class="text-slate-300 text-sm mb-4">Цена: пу
 > деревья — кучу (ниже) и дерево отрезков из
 emerald-300">2n+1 и <span class="font-m
 ">(n-1)//2.</p>
 <details class="bg-slate-900 border border-slate
 <summary class="cursor-pointer px-4 py-2 text
 <div class="px-4 pb-4 text-slate-300 text-sm
 На уровне <span class="font-mono text-emera
 /span> вершин, а перед ним суммарно <span c
 class="font-mono text-emerald-300">k
 lass="font-mono text-emerald-300">n
 0">2n и <span class="font-mono text-
 де, включая heapq.
 </div>
 </details>
 </div>

 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-emerald-400 mb
 <p class="text-slate-300 text-sm mb-4">Куча
 детей. Минимум всегда лежит в <span class="fo
 an>)), а вставка и извлечение минимума стоят <
 учи из готового массива (<span class="font-mo

```

```
<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-rose-400 mb-4"
 <p class="text-slate-300 text-sm mb-4">Писать с
 o text-emerald-300">O(n log n) в худшем
 анных данных. Внутри он ищет уже упорядоченные
 вставками на короткихранах. Сортировка ус
 <pre class="bg-slate-950 border border-slate-700
sorted([3, 1, 2]) # [1, 2, 3] – новый с
sorted([(1, "6"), (1, "a")], key=lambda t: t[0]) # ра
 <p class="text-slate-300 text-sm">Что лежит вну
 отвечают: слияние двух ранов длины k стоит O(
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-purple-400 mb
 <p class="text-slate-300 text-sm mb-4">AVL-д
 >каждой вершины выполнено условие баланса
 class="font-mono text-emerald-300">|h_L – h_R
 зывают фактором баланса; из условия следует в
 авка и удаление за O(log n) даже в худшем слу
 <pre class="bg-slate-950 border border-slate-700
Малый поворот (цепочка вправо-вправо стрелочками):
```

```

 z x
 L> x => L> y
 L> y z y
```

Зигзаг (вправо-влево): сначала малый поворот нижнего из  
потом малый поворот верхнего – две перерисовки стрелок:

```

 z z x
 L> x => L> x => L> y
 L> y L> y z y</pre>
```

```
<p class="text-slate-300 text-sm mb-4">Поворот
 родителя y среднего звена и перевесить одно
 отов вдоль пути вставки, и условие <span clas
 <details class="bg-slate-900 border border-slate
```

```

 рху вниз и слева направо.</p>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg border-1
 <div class="absolute -top-3 left-4 bg-slate-800
 border-emerald-500 shadow-md">
 Аналогия 2: Турнирная сетка
 </div>
 <p class="text-slate-300 text-sm">Куча
 ь, не нужно переигрывать все матчи. Снял по
 тавшихся.</p>
 </div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-1
 <h3 class="text-xl font-bold text-indigo-400 mb-2">
 <p class="text-slate-300 text-sm leading-relaxed">
 База закрыта — теперь ею пользуются остальные
 Кратчайшие пути живут в билетах 14–16 (<a data-goto="string"
 derline font-semibold cursor-pointer">Дейкс
 300 hover:text-white underline font-semibold text-indigo-300
 "text-indigo-300 hover:text-white underline font-semibold
 остовные деревья — в билетах 18–20 (<a data-goto="string"
 semibold cursor-pointer">Краскал, Прим, Бор
 обходы и связность — в билетах 6–13 (<a data-goto="string"
 white underline font-semibold cursor-pointer">КМП
 indigo-300 hover:text-white underline font-semibold
 dges-code" class="text-indigo-300 hover:text-white underline
 строки — в билетах 21–23 (<a data-goto="string"
 ont-semibold cursor-pointer">КМП, <a data-goto="string"
 xt-white underline font-semibold cursor-pointer">КМП
 text-indigo-300 hover:text-white underline font-semibold
 Начинать удобно с <a data-goto="graph-dfs-bfs"
 t-semibold cursor-pointer font-bold">билета
 </p>
</div>
</div>

```



## Ограничения

```

 </div>
 <p class="text-slate-300 text-sm mb-6">
 охождение улицы), Дейкстра сломается, так как не может
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Строгое доказательство (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p>Алгоритм использует приоритетную очередь (например,
 g V). Гарантирует корректность только для неориентированных
 </div>
 </details>

 <div id="slot-dijkstra-viz" class="mt-8"></div>
 </div>
</div>
</section>`
},
{
 id: "bellman-ford",
 title: "Билет 15. Форд-Беллман",
 type: "html",
 category: "Графы",
 content: `<section id="bellman-ford-content" class="flex items-center mb-6">
 <div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Форд-Беллман
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-rose-400">Ограничения

```

```

content: `<section id="floyd-content" class="mb-12
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 rounded
 <h2 class="text-3xl font-bold text-white">Флойд
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-indigo-400">
 Алгоритм Флойда-Уоршелла

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">Пример:
 <p class="text-xl font-mono text-white">
 1 2 3
 1 0 4 1
 2 3 0 4
 3 1 4 0
 </p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4
 border border-emerald-500 shadow-md">
 Аналогия 1: Автомагистрали
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Сколько времени долететь из Москвы в Париж, если мы сможем
 использовать промежуточные города?
 </p>
 <div id="slot-chapter-image-floyd" class="h-400px">

 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4
 border border-rose-500 shadow-md">
 Аналогия 2: Время работы
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Сколько времени потребуется, чтобы пройти все возможные
 пути между всеми городами?
 </p>
 <div class="text-slate-300 text-sm mb-4">
 Три вложенных цикла. Сложность $O(V^3)$. Если
 количество городов увеличится, время выполнения
 увеличится кубически.
 </div>
 </div>
 </div>
 </div>
`

```

```

 ("нити страховки") в самый длинный из известных
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2 border-rose-900
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представим, что мы
 нашли и "he", потому что оно спрятано внутри
 терминальные ссылки (term_link) – прямые мо
 <p>
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border border-rose-900
 <summary class="p-4 font-bold text-slate-400 outline-1
 pen:border-slate-700/50">
 Внутренняя структура автомата и код BFS (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <p>Каждая вершина имеет таблицу переходов <code>transitions
 ссылке <code>term_link</code> (ближайшая терминальная
 <div class="bg-black/50 p-4 rounded-lg font-monospace">
struct Node {
 map<char, int> go; // Предвычисленные переходы
 int link = 0; // Суффиксная ссылка: куда перейти
 int term_link = 0; // Терминальная ссылка: матёшка
 bool is_terminal = false;
 vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные link
void buildLinks() {
 queue<int> q;
 // Корни и их дети

```

```

</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-purple-400">
 <p class="text-slate-300 text-sm mb-4">Когда
 ут быть большими дробными числами (долгота
 ву от 0 до N, и уже по ним строим графы.</p>
 </div>
</div>
</section>`
 }
};

```

ФАЙЛ 23/113: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 200 | РАЗМ

```

export interface QuizQuestion {
 question: string;
 options: string[];
 correctIndex: number;
 explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
 "sparse-table": [
 {
 question: "Сколько памяти занимает двумерная разр
 options: [
 "N × M — столько же, сколько сама матрица",
 "N × M × log N × log M: на каждом уровне почти
 "N × M × 4 — по одной ячейке на каждый угол зап
],
 correctIndex: 1,
 explanation: "Уровней log N по строкам и log M по

```

```

{
 question: "Сколько памяти занимает двумерная преф
 options: [
 "N × M – как исходная матрица, плюс рамка из ну
 "N × M × log N × log M",
 "N + M: одна строка и один столбец"
],
 correctIndex: 0,
 explanation: "Сумма накапливается: каждая клетка
 ная здесь как раз разреженная таблица (билет 2)
},
{
 question: "Запрос суммы прямоугольника (r1,c1)...(r
 options: [
 "S[r2][c2] - S[r1][c2] - S[r2][c1] + S[r1][c1]"
 "S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[
 "S[r2+1][c2+1] - S[r1][c1]"
],
 correctIndex: 1,
 explanation: "С рамкой из нулей S[i][j] – это сум
 ариант без +1 даёт ответ со сдвигом на клетку –
},
{
 question: "Почему левый верхний угол в формуле пр
 options: [
 "Он входит и в левую, и в верхнюю полосу – его
 "Он всегда равен нулю, поэтому прибавление безоп
 "Так компенсируется рамка из нулей"
],
 correctIndex: 0,
 explanation: "Отрезая полосу слева и полосу сверх
 аланс: это и есть формула включений-исключений.
},
{
 question: "Можно ли тем же приёмом считать миниму
 options: [
 "Да, если заменить вычитание на min",
 "Нет: у минимума нет обратной операции, нужна р

```

```

 explanation: "Для эйлера цикла работает критерий

 ого критерия. Это NP-полная задача, которую не

 },

 {

 question: "Куда ведёт эйлеров путь, если нечётных

 options: [

 "Всё ещё можно пройти по всем рёбрам ровно раз",

 "Такого графа не существует",

 "Эйлерова пути нет – нужно удалять лишние рёбра",

],

 correctIndex: 2,

 explanation: "Если нечётных вершин больше 2, эйлер

 ровно 2 нечётные или 0."

 }

],

"bridges-code": [

 {

 question: "После DFS у нас low[u] = 4, а tin[v] =

 options: [

 "Да, так как low[u] (4) > tin[v] (2)",

 "Нет, low[u] должно быть меньше или равно",

 "Только если граф связный и неориентированный",

],

 correctIndex: 0,

 explanation: "Условие моста: low[u] > tin[v]. В н

 },

 {

 question: "Для чего нужен массив low[v]?",

 options: [

 "Для того чтобы было больше переменных в коде",

 "Чтобы знать минимальный tin предка, до которого

 "Чтобы хранить глубину рекурсии",

],

 correctIndex: 1,

 explanation: "low[v] хранит минимальное время вхо

 поиска мостов и точек сочленения."

 },

 {

```

```
],
 correctIndex: 0,
 explanation: "У K5: V=5, E=10, 3V - 6 = 9. 10 > 9"
 }
]
};
```

---

ФАЙЛ 24/113: src/data/vizStepBus.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРӨК: 292 | РАЗМ

---

```
import { createContext, useContext, useEffect, useState } from "react";
import type { DebugValue } from "../debugRunner";
```

```
/**
```

```
 * Прямая связь «компилятор ↔ открытая визуализация».
 *
 * Компилятор передаёт не «магический номер строки эталонного
 * выполненного Python-кода. Визуализация читает захардкоженные
 * (i, j, k, v, dist, P, st...) и подсвечивает соответствующие строки.
 * любой пользовательский код работает, если в нём исполняются те же
 */
```

```
export const VizChapterContext = createContext("");
```

```
const CHANNEL = "algo:viz-state";
```

```
export interface VizStateEventDetail {
```

```
 chapterId: string;
```

```
 /** Строка и функция реально выполненного пользователя */
```

```
 line?: number;
```

```
 func?: string;
```

```
 /** Машинно-читаемые globals + locals: числа остаются строками */
```

```
 variables?: Record<string, DebugValue>;
```

```
 /** Имена, изменённые выделенной строкой. */
```

```
 changed?: string[];
```

```

 };
 window.addEventListener(CHANNEL, handler);
 return () => window.removeEventListener(CHANNEL, handler);
 }, [chapterId]);

 return runtime;
}

export type VizStepSelector = (variables: Record<string, string>) => number;

/**
 * Подписка старых покадровых демонстраций. Если передать номер кадра
 * выбирается по именам/значениям переменных.
 */
export function useVizStepSync(
 currentStep: number,
 goToStep: (n: number) => void,
 maxStep: number,
 selectByVariables?: VizStepSelector
) {
 const chapterId = useContext(VizChapterContext);

 useEffect(() => {
 if (!chapterId) return;

 const apply = (detail?: VizStateEventDetail) => {
 if (!detail || detail.chapterId !== chapterId) return;
 // Без явного селектора компонент либо рисует useVizStepSync,
 // либо остаётся ручным. Номер Python-строки никак не связан с
 const requested =
 selectByVariables && detail.variables ? selectByVariables(detail.variables) : null;
 // null означает: компонент рисует runtime-переменные, но
 // не увидел знакомых захардкоженных имён.
 if (requested === null || !Number.isFinite(requested)) return;
 const target = Math.max(0, Math.min(Math.max(0, requested), maxStep));
 if (target !== currentStep) goToStep(target);
 };
 }, [chapterId, currentStep, goToStep, maxStep, selectByVariables]);
}

```



```

 rows: number;
 cols: number;
 /** Значение ячейки; null – таких данных в коде пользо
at: (i: number, j: number) => number | null;
 /** Подпись размера для бейджа связи: «5x5». */
 sizeLabel: string;
 /** true, если хотя бы одна ячейка – число. */
 hasNumbers: boolean;
}

const MAX_MATRIX_SIDE = 64;

const toCellNumber = (value: DebugValue | undefined): number | null {
 if (typeof value === "number" && Number.isFinite(value)) return value;
 // float("inf") снапшотится строкой "inf" – показываем
 if (typeof value === "string") {
 const t = value.trim().toLowerCase();
 if (t === "inf" || t === "+inf" || t === "infinity" ||
 t === "-inf" || t === "-infinity") return Number(t);
 const n = Number(t);
 return Number.isFinite(n) ? n : null;
 }
 return null;
};

/** Ключ-пара вида "{0, 3}" или "0, 3" → [0, 3]. */
const parsePairKey = (key: string): [number, number] | null {
 const m = key.match(/^(\s*(-?\d+)\s*,\s*(-?\d+)\s*)$/);
 if (!m) return null;
 return [Number(m[1]), Number(m[2])];
};

export function liveMatrix(value: DebugValue | undefined): Matrix {
 if (value === undefined || value === null) return null;

 const cells = new Map<string, number>();
 let rows = 0;
 let cols = 0;

```

```

 if (inner) {
 inner.slice(0, MAX_MATRIX_SIDE).forEach((v, j) => {
 continue;
 });
 }
 const innerRecord = vizRecord(cell);
 if (innerRecord) {
 for (const [k2, v2] of Object.entries(innerRecord)) {
 const j = Number(k2);
 if (Number.isInteger(j)) put(i, j, v2);
 }
 }
 }
}

if (rows === 0 || cols === 0) return null;
return {
 rows,
 cols,
 at: (i, j) => (i >= 0 && j >= 0 && i < rows && j < cols),
 sizeLabel: `${rows}x${cols}`,
 hasNumbers: cells.size > 0,
};
}

/** Первое имя из списка, которое реально есть в снимке
export function pickLiveVar(
 vars: Record<string, DebugValue> | undefined,
 names: string[]
): { name: string; value: DebugValue } | null {
 if (!vars) return null;
 for (const name of names) {
 if (Object.prototype.hasOwnProperty.call(vars, name)) {
 return { name, value: vars[name] };
 }
 }
 return null;
}

/** Индекс, ограниченный реальной размерностью данных по строкам
export const clampIndex = (value: number | null, size: number) => {
 if (value == null) return null;
 if (value < 0) return 0;
 if (value >= size) return size - 1;
 return value;
}

```



# --- тело алгоритма ---

class Node:

def \_\_init\_\_(s, x, y):

s.x = x; s.y = y; s.l = None; s.r = None

def build(pts):

"""Декартово дерево из отсортированных по x точек: """

pts = sorted(pts, key=lambda p: p[0])

stack = []

for x, y in pts:

cur = Node(x, y)

last = None

while stack and stack[-1].y < y: # поднимаемс

last = stack.pop()

cur.l = last

if stack:

stack[-1].r = cur

stack.append(cur)

return stack[0] if stack else None

def split(t, m):

"""(L, R): в L все x <= m, в R — x > m."""

global step

if t is None:

return (None, None)

step += 1

if t.x <= m:

a, b = split(t.r, m)

t.r = a

return (t, b)

a, b = split(t.l, m)

t.l = b

return (a, t)

def merge(a, b):

"""Слияние: все x в a меньше всех x в b."""

global step

```

variables: [
 { name: "points", role: "набор пар (x; y) – по не
 демо" },
 { name: "n", role: "число узлов = число пар", ran
 { name: "mid", role: "разрез для вкладки Split: в
 { name: "key", role: "какую точку удаляет вкладка
 { name: "step", role: "номер шага спуска/слияния
 { name: "height", role: "высота дерева: в среднем
 { name: "root", role: "корень – узел с максимальн
],
},
"splay-tree": {
 vizTitle: "Splay: Zig / Zig-Zig / Zig-Zag на узлах
 stepNote: "Шаг = одно применение случая: переменная
 rota.",
 code: `# Splay-дерево: три случая поворота на узлах
x, p, g = None, None, None # узел, родитель, дед (о
xk, pk, gk = 0, 0, 0 # их ключи-числа: 20 / 4
case = "zig" # какой случай сработал:
step = 0 # номер поворота
root = None # корень текущего дерева

--- тело алгоритма ---
class N:
 def __init__(s, k):
 s.k = k; s.l = None; s.r = None; s.p = None

def rot(x): # один поворот: поднять
 p = x.p
 g = p.p if p else None
 if p and p.l is x:
 p.l = x.r
 if x.r: x.r.p = p
 x.r = p
 elif p:
 p.r = x.l
 if x.l: x.l.p = p
 x.l = p

```

```
3) Zig-Zag: змейка 60 -> 20 -> 40 (40 – правый сын 20)
root = N(60); root.l = N(20); root.l.p = root; root.l.r.p
splay(root.l.r)`,
 variables: [
 { name: "case", role: "какой случай сработал: \"z
 a" },
 { name: "step", role: "номер применения случая –
 { name: "xk / x", role: "ключ узла, который подни
 { name: "pk / p", role: "ключ родителя x перед по
 { name: "gk / g", role: "ключ деда – нужен в Zig-
 { name: "root", role: "корень дерева после расшей
],
},
"sparse-table": {
 vizTitle: "Разрезанная таблица (1D и 2D)",
 stepNote: "Шаг = одна ячейка st[i][j] = min(st[i][j
 code: `a = [2, 3, 5, 62, 3, 21, 1, 4]
n, LOG = 8, 4
i, j = 0, 0
st = [] # сюда складываем строки
for row in a:
 st.append([row] + [None] * (LOG - 1))

for j in range(1, LOG):
 for i in range(n - (1 << j) + 1):
 st[i][j] = min(st[i][j - 1], st[i + (1 << (j -

for row in st:
 print(row) # готовая таблица: None т

запрос min на отрезке за O(1): отрезок покрывают ДВА
L, R = 1, 6
K = (R - L + 1).bit_length() - 1
print("min на [" + str(L) + ", " + str(R) + "] =",
 min(st[L][K], st[R - (1 << K) + 1][K]),
 "| блоки длины", 1 << K)`,
 variables: [
```

```

 if k <= 2:
 return 1
 print(f"считаем fib({k-1}) + fib({k-2})")
 r1 = fib(k - 1)
 r2 = fib(k - 2)
 memo[k] = r1 + r2
 return memo[k]

print("итог:", fib(n))`,
 variables: [
 { name: "n", role: "целевой номер числа Фибоначчи"},
 { name: "k", role: "аргумент текущего рекурсивного"},
 { name: "memo", role: "кэш уже посчитанных значений"},
 { name: "r1, r2", role: "ответы двух рекурсивных"},
],
},
"heap-beam-search": {
 vizTitle: "Куча (priority queue)",
 stepNote: "Шаг = просеивание: вставленный элемент в",
 code: `a = [7, 3, 5, 1] # куча-массивом
n = len(a)
i = n - 1 # всплываемый элемент

parent = (i - 1) // 2 # родитель
kids = (2 * i + 1, 2 * i + 2) # дети
print(f"i={i} parent={parent} kids={kids}")`,
 variables: [
 { name: "n", role: "текущее число элементов в куче"},
 { name: "i", role: "индекс элемента, который сейчас"},
 { name: "(i-1)//2", role: "индекс родителя вершины"},
 { name: "2*i+1, 2*i+2", role: "индексы левого и правого"},
],
},
"stack-dfs": {
 vizTitle: "Стек и обход в глубину",
 stepNote: "Шаг = push новой вершины на стек или pop",
 code: `st = [] # стек = рекурсия

```

```

 i += 1
print("тарелка:", top, "| первый из очереди:", first, "
 variables: [
 { name: "top / front", role: "верх стопки (стек)",
 { name: "i", role: "индекс в массиве кучи; родите
],
},
"graph-dfs-bfs": {
 vizTitle: "DFS и BFS на графе",
 stepNote: "Шаг = переход из текущей вершины v к неп
 code: `from collections import deque

adj = {"A": ["B", "C"], "B": ["D"], "C": [], "D": []}
visited, order = set(), []

stack = ["A"] # DFS: свой стек вместо рекурсии
while stack:
 v = stack.pop() # шаг: достали вершину с вершины
 if v in visited:
 continue
 visited.add(v); order.append(v)
 for u in adj[v]: # соседи v → на стек
 stack.append(u)
print("DFS:", order)

visited, order = set(), []
q = deque(["A"]) # BFS: очередь, обход по слоям
while q:
 v = q.popleft() # шаг: достали из начала очереди
 if v in visited:
 continue
 visited.add(v); order.append(v)
 for u in adj[v]: # соседи v → в конец очереди
 q.append(u)
print("BFS:", order)`
 variables: [
 { name: "v", role: "текущая вершина обхода", gang: 1,
 { name: "to / u", role: "очередной сосед вершины v",

```



```

 { name: "n", role: "число вершин; в демонстрации", range: "1 .. n" },
 { name: "adj", role: "списки смежности — тот же граф", range: "1 .. n" },
 { name: "comp", role: "разметка: номер компоненты", range: "1 .. comps" },
 { name: "used", role: "булев вариант разметки — для DFS", range: "bool" },
 { name: "comps", role: "счётчик найденных компонент", range: "1 .. comps" },
 { name: "c", role: "номер компоненты, которую кра...", range: "1 .. comps" },
 { name: "v", role: "вершина, где обход находится", range: "1 .. n" },
 { name: "stack", role: "фронт обхода: стек (DFS)", range: "1 .. n" },
 { name: "i / step", role: "номер шага — перематыва...", range: "1 .. n" },
],
},
"top-sort": {
 vizTitle: "Топологическая сортировка",
 stepNote: "Шаг = выход рекурсии из вершины v: она д...",
 code: `adj = {0: [1, 4], 1: [2, 3], 2: [], 3: [2], 4: []}
used, order = set(), []

def dfs(v):
 used.add(v)
 for to in adj[v]:
 # шаг: ребро v → to
 if to not in used:
 dfs(to)
 order.append(v)
 # выход из v!

for v in adj:
 if v not in used:
 dfs(v)

answer = order[::-1]
разворот — топологический
for i, v in enumerate(answer):
 print(f"позиция {i}: вершина {v}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range: "1 .. n" },
 { name: "to", role: "вершина, куда ведёт ребро из v", range: "1 .. n" },
 { name: "i", role: "перебор стартовых вершин во в...", range: "1 .. n" },
 { name: "order", role: "список выхода из рекурсии", range: "1 .. n" },
],
}
```

```

for i in range(len(order) - 1, -1, -1):
 v = order[i]
 if v in comp:
 continue
 stack = [v]
 comp[v] = c
 while stack:
 u = stack.pop()
 for to in adjT[u]:
 if to not in comp:
 comp[to] = c
 stack.append(to)
 print("SCC", c, "собрана из", v, ":", comp)
 c += 1

print("порядок выхода:", order, "| компонент:", c, "| р
variables: [
 { name: "adj / adjT", role: "граф и его транспози
 { name: "v", role: "текущая вершина DFS (в фазе 1
 { name: "to", role: "сосед по ребру: в фазе 1 пря
 { name: "used", role: "помеченные первым проходом
 { name: "order", role: "порядок ВЫХОДА из DFS пер
 { name: "i", role: "индекс прохода по order на вто
 { name: "stack", role: "стек DFS второго прохода:
 { name: "u", role: "вершина, снятая со стека во в
 { name: "comp", role: "номер компоненты сильной с
 { name: "c", role: "счётчик компонент: каждая нов
 { name: "n", role: "число вершин", range: "5" },
],
},
"graph-articulation": {
 vizTitle: "Точки сочленения",
 stepNote: "Шаг = обновление low[v] по ребру; вершин
 code: `tin, low, timer = {}, {}, 0

i = 0 # номер шага DFS
v, to = 0, 1 # текущее ребро DFS: v → to
tin[v] = tin.get(v, timer); low[v] = tin[v]

```

```

for i, (u, v) in enumerate(edges):
 adj[u].append((v, i))
 adj[v].append((u, i))
deg = {v: len(adj[v]) for v in range(n)} # степени вершин
odd = [v for v in range(n) if deg[v] % 2 == 1] # нечётные
start = odd[0] if odd else 0 # путь начинается
used = [False] * len(edges) # какие рёбра
path = [start] # маршрут: пос
last = start # где мы сейчас
stack = [start] # стек для отката
v = start
step = 0

--- тело алгоритма ---
while stack:
 v = stack[-1]
 moved = False
 for to, i in adj[v]:
 if not used[i]:
 used[i] = True
 stack.append(to)
 path.append(to)
 last = to
 step += 1
 moved = True
 break
 if not moved:
 stack.pop() # из v все рёбра

print("степени:", deg)
print("нечётных вершин:", len(odd), odd, "->", "цикл" if odd else "нет")
print("маршрут:", path, "| рёбер пройдено:", sum(used), "из", len(edges))
variables: [
 { name: "path", role: "маршрут — последовательность вершин"},
 { name: "last / v", role: "вершина, где обход находится"},
 { name: "used", role: "какие рёбра уже пройдены (по индексу)"},
 { name: "deg", role: "степени вершин: чётные всюду, нечётные — по"},
 { name: "odd", role: "список нечётных вершин — по"},
 { name: "stack", role: "стек отката алгоритма Иерархического"}
]

```

```

 { name: "u", role: "сосед вершины v, чей цвет уже"
 { name: "delta", role: "максимальная степень $\Delta =$ "
 { name: "chi", role: "сколько цветов реально пона"
 { name: "i / step", role: "номер шага – перематыва"
 { name: "V, E, F", role: "для вкладки «Планарности"
 9" },
],
},
dijkstra: {
 vizTitle: "Дейкстра",
 stepNote: "Шаг = старт, извлечение ближайшей вершины"
 code: `import heapq

g = {"A": [{"B", 4}, {"C", 2}], "B": [{"G", 4}, {"D", 5}
 , "E": [{"F", 1}], "F": []}
dist = {"A": 0}
prev = {}
done = set()
pq = [(0, "A")]
while pq:
 du, u = heapq.heappop(pq)
 if u in done:
 continue
 done.add(u)
 print(f"обрабатываем {u} (dist={du})")
 for v, w in g[u]:
 if v in done:
 continue
 print(f"смотрим ребро {u}→{v} (w={w})")
 if du + w < dist.get(v, float("inf")):
 dist[v] = du + w
 prev[v] = u
 heapq.heappush(pq, (dist[v], v))
print("итог:", dist)`
 variables: [
 { name: "u", role: "вершина с минимальным dist ср"
 { name: "v", role: "сосед вершины u, до которого"
 { name: "w", role: "вес ребра (u, v)", range: "по

```

```

 break

if cycle_edge:
 print("отрицательный цикл: улучшается ребро", cycle)
else:
 print("отрицательных циклов нет:", {k: dist[k] for
 variables: [
 { name: "n", role: "число вершин графа – ровно n-"},
 { name: "m", role: "число рёбер, по которым прохо"},
 { name: "i", role: "номер итерации (после i итера"},
 { name: "u, v, w", role: "текущее ребро: откуда,"},
 { name: "dist[v]", role: "расстояние; если обнов"},
],
},
floyd: {
 vizTitle: "Флойд–Уоршелл",
 stepNote: "Шаг = инициализация матрицы, смена проме",
 code: `INF = 999

d = [[0, 4, INF, 2, INF, INF, INF], [INF, 0, 3, INF, IN
 NF, INF, INF, 0, INF, 1], [INF, INF, INF, INF, INF, 0
n = 7
for k in range(n):
 print(f"посредник k={k}")
 for i in range(n):
 for j in range(n):
 if i != j and i != k and j != k and d[i][k]
 if d[i][k] + d[k][j] < d[i][j]:
 d[i][j] = d[i][k] + d[k][j]
print("итог: все пары посчитаны")`,
 variables: [
 { name: "n", role: "число вершин – размер квадрата"},
 { name: "k", role: "промежуточная вершина – ВНЕШНИ"},
 { name: "i", role: "начало пути – строка матрицы"},
 { name: "j", role: "конец пути – столбец матрицы"},
 { name: "dist[i][j]", role: "кратчайший путь i →"},
 { name: "ни", role: "ни"},
],
},

```

```

 x = parent[x]
 return x

for i, (w, a, b) in enumerate(edges):
 u, v = a, b
 ra, rb = find(a), find(b)
 if ra != rb: # разные компоненты
 parent[ra] = rb
 if rank[ra] == rank[rb]:
 rank[rb] += 1
 mst.append((w, a, b))
 total += w
print("вес остова:", total, "| рёбер:", len(mst))
print(mst)`
 variables: [
 { name: "n", role: "число вершин (A..I); в остов в"},
 { name: "edges", role: "список рёбер (вес, u, v)"},
 { name: "i", role: "номер текущего ребра в отсорти"},
 { name: "u, v, w", role: "концы и вес ребра, котор"},
 { name: "parent", role: "DSU: представитель множе"},
 { name: "rank", role: "высота дерева DSU – union f"},
 { name: "mst", role: "взятые рёбра остова – рисуе"},
 { name: "total", role: "вес остова; на этом графе"}
],
},
"string-kmp": {
 vizTitle: "Префикс-функция (КМП)",
 stepNote: "Шаг = обработка символа s[i]: откаты по",
 code: `s = "aabaabaaba"
n = len(s)
i, j = 1, 0
pi = [None] * n # пустые клетки: значения п
pi[0] = 0

for i in range(1, n): # шаг: считаем pi[i]
 j = pi[i - 1] # длина совпадения
 while j > 0 and s[i] != s[j]:
 j = pi[j - 1] # откат назад

```

```

"aho-corasick": {
 vizTitle: "Ахо-Корасик: бор, fail-ссылки и автомат"
 stepNote: "Шаг = один символ текста: state = go(state, ch) совпадают с картинкой «водопада».",
 code: `words = ["HERS", "HIS", "SHE", "HE"] # образцы
text = "USHERS" # текст, в котором ищем
n = len(text) # длина текста
state = 0 # текущее состояние
i = 0 # индекс текущего символа
c = "" # сам символ
nodes = 1 # сколько вершин в боре
queue = [] # очередь BFS, куда добавляем вершины
fail = {} # fail[v] = вершина, куда перейти по fail-ссылке
matches = [] # найденные вхождения

--- тело алгоритма ---
nxt = {} # nxt[(v, ch)] = u — переходы бора
term = {} # term[v] = слово, которое заканчивается в вершине v

def add_word(w):
 """Добавить образец в бор: недостающие вершины подготавливаем"""
 global nodes
 v = 0
 for ch in w:
 if (v, ch) not in nxt:
 nxt[(v, ch)] = nodes
 nodes += 1
 v = nxt[(v, ch)]
 term[v] = w

for w in words:
 add_word(w)

BFS по уровням: fail-ссылка ребенка = куда приведёт fail-ссылка родителя
fail[0] = 0
queue = sorted(v for (p, ch), v in nxt.items() if p == 0)
for v in queue:
 fail[v] = 0
 queue += sorted(v for (p, ch), v in nxt.items() if p == v)

```

```

 { name: "i / c", role: "индекс и сам символ текста"},
 { name: "nxt", role: "переходы бора: nxt[{v, ch}]"},
 { name: "fail", role: "fail[v] – вершина наибольшей длины"},
 { name: "term", role: "в каких вершинах заканчиваются слова"},
 { name: "queue", role: "очередь BFS, которой натягиваются"},
 { name: "matches", role: "найденные вхождения: (индекс, длина)"},
 { name: "nodes", role: "сколько вершин в боре (корень, листья)"},
],
},
"graph-cheatsheet": {
 vizTitle: "Быстрая шпаргалка: BFS – Прим – Декстер",
 stepNote: "Шаг = вычисление x и релаксация ребер из x",
 code: `# Быстрая шпаргалка: Декстер на едином шаге
edges = {
 0: [(1, 4), (2, 2)],
 1: [(0, 4), (2, 1), (3, 5)],
 2: [(0, 2), (1, 1), (3, 8), (4, 10)],
 3: [(1, 5), (2, 8), (4, 2)],
 4: [(2, 10), (3, 2)]
}
n = 5
start = 0

min_way = [float('inf')] * n
min_way[start] = 0
seen = set()

for _ in range(n):
 x = -1
 for i in range(n):
 if i not in seen:
 if x == -1 or min_way[i] < min_way[x]:
 x = i

 if x == -1 or min_way[x] == float('inf'):
 break

 seen.add(x)

```



```

step = 3
cert = [1, 0, 1, 0] # значения переменных x1...x4
k = len(cert) # проверяем все k бит
ok = all(bit in {0, 1} for bit in cert)
cls = "NP" # угадать + быстро проверить
print("сертификат:", cert, "| k =", k, "| принят:", ok,
 variables: [
 { name: "n", role: "размер входа – горизонтальная
 { name: "ops", role: "сколько операций выполнил а
 " },
 { name: "cls", role: "имя класса (P / NP / PSPACE
 { name: "step / i", role: "кадр схемы (0 вход → 1
 i 0...9" },
 { name: "k", role: "размер сертификата: сколько з
 { name: "cert", role: "сертификат – назначение пер
 { name: "ok", role: "вердикт верификатора: принят
],
 },
 intro: {
 // мнемодкарточки без шагов алгоритма
 variables: [],
 },

/* — Дополнительные демонстрации внутри страниц (вкл.

"planarity-euler-formula#greedy": {
 vizTitle: "Жадная покраска вершин ($\chi \leq \Delta + 1$)",
 stepNote: "Шаг = вершина v получает наименьший цвет
 code: `n = 8 # вершин в графе
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3,
delta = max(len(a) for a in adj) # максимальная степень
color = [0] * n # 0 = ещё не покрашена
chi = 0 # сколько цветов понадобилось
v = -1 # вершина, которую красим сейчас
i = 0 # номер шага

--- тело алгоритма ---
for v in range(n):

```

```

 v = queue.pop(0)
 for u in adj[v]:
 if color[u] == 0:
 color[u] = 3 - color[v] # противоположный
 queue.append(u)
 i += 1
 elif color[u] == color[v]:
 ok = False # нечётный
 print("конфликт на ребре", (v, u))
print("двудольный:", ok, "| разметка:", color)`
variables: [
 { name: "color", role: "разметка на две доли: 1 и 2"},
 { name: "v", role: "вершина, из которой BFS смотрит"},
 { name: "u", role: "сосед, которого красим (или с которого)"},
 { name: "queue", role: "очередь BFS", range: "≤ n"},
 { name: "ok", role: "вердикт: граф двудольный (не планарный)"},
 { name: "i / step", role: "номер кадра", range: "0..n-1"}
],
},
"planarity-euler-formula#edges": {
 vizTitle: "Покраска рёбер (Визинг: $\Delta \leq \chi' \leq \Delta + 1$)",
 stepNote: "Шаг = ребро e получает наименьший цвет, не использованный",
 code: `n = 8
edges = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 4), (3, 4)]
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj)
ecolor = [0] * len(edges) # цвет каждого ребра
inc = [dict() for _ in range(n)] # inc[v][цвет] = ребро
e = -1 # номер текущего ребра
v = u = -1 # его концы
chi_e = 0

--- тело алгоритма ---
for e, (a, b) in enumerate(edges):
 v, u = a, b
 used = set(inc[a].values()) | set(inc[b].values())
 c = 1
 while c in used:

```

```

while stack:
 v = stack.pop() # для BFS: v = stack.pop(0)
 i += 1
 for to in adj[v]:
 if comp[to] == -1:
 comp[to] = c
 used[to] = True
 stack.append(to)
print("компонент:", comps)
print("разметка comp:", comp)`,
 variables: [
 { name: "comp", role: "номер компоненты каждой вершины"},
 { name: "v", role: "текущая вершина обхода", group: "BFS"},
 { name: "stack", role: "стек обхода (для BFS разметки)"},
 { name: "comps / c", role: "счётчик компонент и номер компоненты"},
 { name: "i", role: "номер шага — перематывает кадры"},
],
},
"graph-components#dsu": {
 vizTitle: "Компоненты связности: DSU по рёбрам",
 stepNote: "Шаг = одно ребро (u, v): union склеивает компоненты",
 code: `
n = 10 # вершин в графе
edges = [(0, 1), (1, 2), (2, 3), (0, 3), (1, 3), (4, 5)]
parent = list(range(n)) # DSU: parent[v] = представитель
rank = [0] * n # высота дерева — по ней решаем
comps = n # сначала каждая вершина — о-собная
e = -1 # номер обрабатываемого ребра
u, v = -1, -1 # концы текущего ребра
step = 0 # номер шага — двигает показ

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

def union(a, b):

```

```

 for j in range(len(a) - 1):
 ops += 1
 if a[j] > a[j + 1]:
 a[j], a[j + 1] = a[j + 1], a[j]
print(cls, "отсортировано за", ops, "операций:", a)\,
 variables: [
 { name: "cls", role: "имя класса – P, NP, PSPACE"},
 { name: "n", role: "размер входа задачи, про которую"},
 { name: "ops", role: "сколько операций потребовалось"},
],
},
"complexity-classes#reduce": {
 vizTitle: "Сведение $A \leq_p B$ (3-SAT $\rightarrow$ КЛИКА)",
 stepNote: "Шаг = один кадр схемы сведения: вход $x \rightarrow$ ",
 code: `cls = "NP" # класс задачи A,
step = 0 # кадр сведения $A \leq_p B$: 0...6
n = 9 # размер входа x (литералов в дизъюнкте)
k = 3 # размер клики, которую ищем

--- тело алгоритма ---
Сведение 3-SAT $\leq_p$ КЛИКА: вершина = (номер дизъюнкта,
ребро = два литерала из РАЗНЫХ дизъюнктов, не противоречивые)
clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
V = [(i, l) for i, cl in enumerate(clauses) for l in cl]
E = [(a, b) for a in V for b in V if a[0] < b[0] and a[1] != b[1]]
for step in range(7):
 print("кадр", step, "| вершин:", len(V), "| рёбер:", len(E))
print("формула выполнима $\Leftrightarrow$ в графе есть клика размера k")
 variables: [
 { name: "step", role: "номер кадра сведения (0...6)"},
 { name: "cls", role: "класс задачи A, которую сводим к B"},
 { name: "n", role: "размер входа x; после сведения к B"},
 { name: "k", role: "размер клики, которую ищет алгоритм"},
],
},
"complexity-classes#verify": {
 vizTitle: "Верификатор 3-SAT: проверка сертификата",
 stepNote: "Шаг = один дизъюнкт: верификатор подставляет значения переменных",

```

## Универсальное пособие • полный исходный код

```

Экспоненциальный (полный перебор подмножеств) – раскол
for n in range(1, 15):
ops = 2 ** n
print("n =", n, "ops =", ops)`,
variables: [
{ name: "n", role: "размер входа – по нему строится"},
{ name: "ops", role: "число операций алгоритма на"},
{ name: "i", role: "внутренний счётчик цикла – то"},
],
},
"mst#kruskal": {
 vizTitle: "Остовное дерево: Краскал и DSU",
 stepNote: "Шаг = следующее ребро из отсортированных",
 code: `n = 9 # вершин: A..I – т
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10
 (12, "H", "I"), (13, "F", "I")]
edges.sort() # Краскал: рёбра по возраста
parent = {v: v for v in "ABCDEFGHI"} # DSU: представи
rank = {v: 0 for v in "ABCDEFGHI"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

for i, (w, a, b) in enumerate(edges):
 u, v = a, b
 ra, rb = find(a), find(b)
 if ra != rb: # разные компон
 parent[ra] = rb
 if rank[ra] == rank[rb]:
```

```

 if frm != x:
 mst.append((w, frm, x))
 total += w
 for to, wt in adj[x]:
 if to not in taken:
 heapq.heappush(heap, (wt, to, x))
print("вес остова:", total, "| вершин захвачено:", len(taken))
print(mst)`
variables: [
 { name: "start", role: "вершина, из которой растёт остова", weight: 0, type: "v"},
 { name: "heap", role: "куча рёбер-кандидатов (вес, вершина, вершина)", type: "e"},
 { name: "taken", role: "вершины, уже вошедшие в остова", type: "v"},
 { name: "u, v, w", role: "ребро, которое только что вошло в остова", type: "e"},
 { name: "mst / total", role: "рёбра остова и его вес", type: "e"}
],
},
"mst#boruvka": {
 vizTitle: "Остовное дерево: Борувка (фазы-«пятiletки»)",
 stepNote: "Шаг = одна фаза: каждая компонента ОДНОВЕРХОВА",
 code: `n = 9 # вершин: A..I
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "A", "C"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "D", "F"),
 (12, "H", "I"), (13, "F", "I")]
comp = {v: v for v in "ABCDEFGH I"} # DSU: корень компоненты
cheapest = {} # корень компоненты -> самое дешёвое ребро
mst = [] # рёбра остова
total = 0 # вес остова (= 51)
phase = 0 # номер фазы («пятiletки») — от 0 до 4
u, v, w = None, None, None # ребро-кандидат

--- тело алгоритма ---
def find(x):
 while comp[x] != x:
 comp[x] = comp[comp[x]]
 x = comp[x]
 return x

while len(mst) < n - 1:

```

```

--- тело алгоритма ---
База не отдельный уровень вычислений: блок 1x1 – это 1
Записываем её одним шагом, а не проигрываем 64 одноклеточных
Генератор остаётся отдельным фреймом и не конфликтует
st2.update(dict(({r, c, 0, 0}, A[r][c]) for r in range(9) for c in range(9)))

for k in range(1, 4):
 print(f"уровень k={k}: блоки {1 << k}x{1 << k}")
 size = 9 - (1 << k)
 half = 1 << (k - 1)
 for r in range(size):
 for c in range(size):
 st2[(r, c, k, k)] = min(st2[(r, c, k - 1, k - 1),
 (r + half, c, k - 1, k - 1)],
 st2[(r, c + half, k - 1, k - 1),
 (r, c + half, k - 1, k - 1)])
 variables: [
 { name: "r, c", role: "левый верхний угол блока, координаты"},
 { name: "k", role: "уровень: блоки 2^k x 2^k (шаг 2^k)"},
 { name: "st2[(r,c,k,k)]", role: "минимум квадратной области"},
],
},
"sparse-table#2d-query": {
 vizTitle: "Разреженная таблица 2D: запрос",
 stepNote: "Демонстрация кликабельна вручную: выбери область",
 code: `# st2[(r, c, kx, ky)] из вкладки «построение»
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({kx}, {ky})")
 variables: [
 { name: "r1, c1, r2, c2", role: "углы запрашиваемой области"},
 { name: "kx, ky", role: "уровень самого крупного блока"},
],
},
"prefix-sums-2d#2d": {
 vizTitle: "Префиксные суммы 2D (прямоугольники)",

```

```

 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle} .`
 : `# «${chapterTitle}»\n# демо на странице нет – св

 if (!sync.code) return head;
 return `${head}\n${sync.code}\n`;
 }

/**
 * Инициализация скелета: только объявление демо-данных
 * (без for/while/def/if/print). Именно это показывается
 * умолчанию – дальше пользователь пишет свой вариант,
 * заменяет редактор полным референсным кодом.
 */
function extractInit(code: string): string {
 const keep: string[] = [];
 for (const line of code.split("\n")) {
 const t = line.trim();
 if (t === "# --- тело алгоритма ---" || /^(for|while|
 keep.push(line);
 }
 // убрать пустые строки с конца
 while (keep.length > 0 && keep[keep.length - 1].trim()
 return keep.join("\n");
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО иници
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, ch
 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#
 : `# «${chapterTitle}»\n# демо на странице нет – св

```



```

<section id="segment-trees" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
ree[2v+1]), где $m = (l + r) / 2$. Запрос и отклик
для итеративного bottom-up без lazy).</p>
 </div>

 <p class="text-slate-300 text-sm mb-4">Массив
 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99]
 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99]
 – это одиночные элементы массива. В узле левое и правое
 количество единиц, И/ИЛИ – любая операция, которую можно
 >).</p>

 <p class="text-slate-300 text-sm mb-4">Отсюда
 а каждом уровне встречается один из трёх случаев:
 не пересекается – возвращаем нейтральный элемент,
 склеиваем. Частичных пересечений на уровне нет.
 одного элемента – спуск к листу и пересчёт.

 <div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-slate-800">
 <p class="text-[10px] uppercase tracking-wider">
 <p class="font-mono text-emerald-300">
 </div>
 <div class="bg-slate-900 border border-slate-800">
 <p class="text-[10px] uppercase tracking-wider">
 <p class="font-mono text-emerald-300">
 </div>
 </div>
 </div>
 </div>

```

```

 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-1.5 pr-2 font-mono">Операция
 <th class="py-1.5 pr-2 font-mono">Выражение
 <th class="py-1.5 font-bold">Сложность
 </tr>
 </thead>
 <tbody class="text-slate-300 font-mono">
 <tr class="border-b border-slate-500">
 x · len</td><td class="py-1.5">lazy[v] += c
 <tr class="border-b border-slate-500">
 c · len</td><td class="py-1.5">lazy[v] = c
 <tr class="border-b border-slate-500">
 len - tree[v]</td><td class="py-1.5">lazy[v] =
 <tr><td class="py-1.5 pr-2">min
 -1.5">lazy[v] = min(lazy[v], c)</td></tr>
 </tbody>
 </table>
 <p class="text-slate-500 text-xs mt-3">Операции выполняются в строгом порядке: сначала присваиваются значения, а затем вычисления.
</div>

<div class="bg-slate-900/40 rounded-lg border">
 <p class="text-slate-300 text-sm mb-2">Операции min, gcd и И являются идемпотентными.
 <p class="text-slate-400 text-sm mb-1">Операции min и gcd являются ассоциативными, но не коммутативными.
за O(1), но строятся один раз и умирают при следующем вызове.
ми.</p>
 <p class="text-slate-400 text-sm mb-1">Операции min, gcd и И являются идемпотентными.
идемпотентные запросы (min, gcd, И) по состоянию.
 <p class="text-slate-400 text-sm mb-1">Операции min и gcd являются ассоциативными, но не коммутативными.
 4 (только для bottom-up без lazy) обходится ровно 4 операциями.
 <p class="text-slate-400 text-sm mb-1">Операции min, gcd и И являются идемпотентными.
ассоциативной, а функция массового обновления является ассоциативной.
ний (f ∘ g)(x) = f(g(x)).
 <p class="text-slate-400 text-sm">• Оценки сложности
 <p class="text-slate-400 text-sm">Построить лень во всё дерево – это O(n) и весь смысл в том, чтобы не делать этого.
</div>

```

```

 return sum(2 * v, l, m, L, R) // части
 + sum(2 * v + 1, m + 1, r, L, R);
 }

```

```

void update(int v, int l, int r, int i, int x) { // a[i]
 if (l == r) { tree[v] += x; return; }
 int m = (l + r) / 2;
 if (i <= m) update(2 * v, l, m, i, x);
 else update(2 * v + 1, m + 1, r, i, x);
 tree[v] = tree[2 * v] + tree[2 * v + 1]; // пересч
}

```

</div>  
</details>

<details class="bg-slate-900/40 rounded-lg l  
<summary class="p-4 font-bold text-slate  
-b border-slate-700/50">  
Код: массовое обновление, lazy pro  
</summary>

<div class="p-5 text-sm text-slate-300  
<pre class="bg-slate-950 p-4 rounded  
y[4 \* MAXN]; // «обещание»: прибавит

```

void push(int v, int l, int r) {
 if (lazy[v] == 0) return;
 int m = (l + r) / 2;
 tree[2 * v] += lazy[v] * (m - l + 1); // п
 lazy[2 * v] += lazy[v];
 tree[2 * v + 1] += lazy[v] * (r - m); // и
 lazy[2 * v + 1] += lazy[v];
 lazy[v] = 0; // о
}

```

```

void add(int v, int l, int r, int L, int R, int x) { //
 if (r < L || R < l) return;
 if (L <= l && r <= R) {
 tree[v] += x * (r - l + 1);
 lazy[v] += x; // д
 }
}

```

```
 </details>

 <details class="bg-slate-900/40 rounded-lg border-2 border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Вопросы, которые любят задавать (и не любят)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p>• Почему $4n$, а не $2n$? – при n , например: $n = 6$).</p>
 <p>• Сколько вершин посещает запрос?</p>
 <p>• Почему построение $O(n)$, а не $O(n^2)$?</p>
 <p>• Какие операции подходят? – любые, но количество различных на отрезке» обычное дерево.</p>
 <p>• Зачем push в запросе, если ничего не добавляется?</p>
 <p>• push делает данные детей достоверными.</p>
 </div>
 </details>
 </div>
</div>
</section>
},
];
```

---

ФАЙЛ 27/113: src/data/tickets/ticket14\_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 1237 | РАЗМЕР: 10240 байт

---

```
import { Chapter } from '../types';

export const tickets14to20: Chapter[] = [
 {
 id: "dijkstra",
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра"
```

```

 <p class="font-mono text-emerald-300">
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300"
 </div>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Декстер Дуглас и Фр
 </div>
 <div class="my-3 rounded-lg overflow

 </div>
 <p class="text-slate-300 text-sm mb
елевизор). Ярко-синий электрический вирус <
а переходит от текущего узла к самым близким
ext-emerald-300"> $w \geq 0$ }, зафиксирован
ию расстояния!</p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Ограничения: минус ломает жадно
 </div>
 <p class="text-slate-300 text-sm mb
ления в кучу). Допустим: <span class="font-m
= 3, <span class="font-mono text-em
. Настоящий кратчайший путь до <span class=
class="font-mono">3 - 2 + 1 = 2. Но ,
ono">d[a] = 2 и установит <span clas
т b→a, зафикс
t-mono">t, и алгоритм выдаст неверный

```

```

 <th class="py-1.5 pr-2 font
 <th class="py-1.5 font-bold
 </tr>
</thead>
<tbody class="text-slate-300">
 <tr class="border-b border-slate
ld">Пиццерия A (0)</td><td class="py-2 font
 <tr class="border-b border-slate
ld">Парк C (2)</td><td class="py-2 font-mono
 <tr class="border-b border-slate
ld">Дом 1 B (3)</td><td class="py-2 font-mono
 <tr class="border-b border-slate
ld">Метро G (5)</td><td class="py-2 font-mono
 <tr class="border-b border-slate
ld">Офис D (6)</td><td class="py-2 font-mono
 <tr class="border-b border-slate
ld">Дом 2 E (7)</td><td class="py-2 font-mono
 <tr class="align-top"><td class="
="py-2 font-mono">исходящих рёбер нет</td><
 </tbody>
</table>
 <p class="text-slate-400 text-xs mt-3">
F 8. Из 11 проверок рёбер 10 улучши
/span>: A → C → G → E → F = 2 + 3 + 2 + 1 =
</div>

<div class="bg-slate-800/70 p-5 rounded-lg l
 <p class="text-sm font-bold text-white m
 <table class="w-full text-xs text-left l
 <thead>
 <tr class="text-slate-500 borde
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 font-bold
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate

```

0">u</span> нет. ■</p>

<p class="text-slate-400 text-xs">Г,  
u)</span>». С отрицательным ребром хвост п  
о контрпример из карточки «Ограничения».</p>

</div>

</details>

<details class="bg-slate-900/40 rounded-lg l

<summary class="p-4 font-bold text-slate  
-b border-slate-700/50">> Откуда берётся сл

<div class="p-5 text-sm text-slate-300

<ul class="list-disc pl-5 space-y-2

<li><b>Куча.</b> Каждая вершина  
span class="font-mono">pop</span>), каждое

o">push</span>. Итого <span class="font-mono

жать до E пар, но логарифм от этого не меня  
<li><b>Массив.</b> Поиск миниму

class="font-mono">V</span>: итого <span cla  
ont-mono">E ≈ V<sup>2</sup></span>) это быстрее кучи и

<li><b>Куча Фибоначчи.</b> <span  
константа: в олимпиадах почти не используе

<li><b>Специальные веса.</b> Ве  
>; малые целые веса → корзины Диала. Всё эт

</ul>

</div>

</details>

</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord

<h3 class="text-xl font-bold text-emerald-4

<div class="overflow-x-auto my-4">

<table class="w-full text-xs text-left l

<thead>

<tr class="text-slate-500 borde

<th class="py-2 pr-3 font-b

<th class="py-2 pr-3 font-b

<th class="py-2 font-bold">

```

 if u in done: # устаревшая п
 continue
 done.add(u) # dist[u] окон
 for v, w in graph[u]:
 if du + w < dist[v]: # релаксаци
 dist[v] = du + w
 prev[v] = u
 heapq.heappush(pq, (dist[v], v))
 return dist, prev

def restore(prev, start, target):
 if prev[target] is None and target != start:
 return None # вершина недо
 path = [target]
 while path[-1] != start:
 path.append(prev[path[-1]])
 return path[::-1]

демо-граф страницы (доставка пиццы, рёбра односторонни
g = {
 "A": [("B", 4), ("C", 2)],
 "B": [("G", 4), ("D", 5)],
 "C": [("B", 1), ("G", 3), ("E", 8)],
 "D": [("F", 3)],
 "E": [("F", 1)],
 "F": [],
 "G": [("D", 1), ("E", 2)],
}
dist, prev = dijkstra(g, "A")
print(dist) # A 0, B 3, C 2, D 6, E 7,
print(restore(prev, "A", "F")) # ['A', 'C', 'G', 'E',
<p class="text-slate-400 text-xs">П
 же вершины, потому что читает ваши <span c
nt-mono">done.</p>
</div>
</details>

<details class="bg-slate-900/40 rounded-lg

```



```

 return res;
}

```

уть из  $10^5$  рёбер веса  $10^9$  даёт  $10^{14}$  –

Код:  $O(V^2)$  без куч

```

ти $n \times n$, $w[i][j] = \text{INF}$ если ребра нет
d = [INF] * n
used = [False] * n
d[s] = 0

```

```

for _ in range(n):
 u = -1
 for v in range(n):
 if not used[v] and (u == -1 or d[v] < d[u]):
 u = v
 if u == -1 or d[u] == INF:
 break
 used[u] = True
 for v in range(n):
 if d[u] + w[u][v] < d[v]:
 d[v] = d[u] + w[u][v]

```

. Память – матрица

```
</div>
```

```
<div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border">
```

```
 <h3 class="text-xl font-bold text-rose-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
 <p class="text-lg text-rose-300 italic">
```

```
 <p class="text-xl font-mono text-white">
```

```
 есть отрицательный цикл.</p>
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">Форд
```

```
 ьных весов. Плата — скорость: вместо <span
```

```
 ="font-mono text-emerald-300"> $0(V \cdot E)$
```

```
 font-mono text-emerald-300">(u, v, w)
```

```
 ком, релаксируя каждое ребро.</p>
```

```
 <p class="text-slate-300 text-sm mb-4">Логи
```

```
 ез повторов вершин), значит в нём не больше
```

```
 nt-mono text-emerald-300">i-й итераци
```

```
 00"> $\leq i$ рёбер. Поэтому <span class="
```

```
 nt-mono text-emerald-300"> V -я — диаг
```

```
 ь мы попали в отрицательный цикл.</p>
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
```

```
 <div class="bg-slate-900 border border-
```

```
 <p class="text-[10px] uppercase tra
```

```
 <p class="font-mono text-emerald-30
```

```
 </div>
```

```
 <div class="bg-slate-900 border border-
```

```
 <p class="text-[10px] uppercase tra
```

```
 <p class="font-mono text-emerald-30
```

```
 </div>
```

```
 <div class="bg-slate-900 border border-
```

```
 <p class="text-[10px] uppercase tra
```

```
 <p class="font-mono text-emerald-30
```

```
 </div>
```

>, которая в обычном режиме весит `<span class="font-mono text-emerald-300">-6</span>`. В компиляторе `<span>` внутри `<span class="font-mono text-emerald-300">`.

```
<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-700">
 <td class="py-2 text-emerald-300">да</td><td class="py-2">
 <tr class="border-b border-slate-700">
 <td class="py-2">без изменений</td>
 <tr class="border-b border-slate-700">
 <td class="py-2">гарантия V –
 <tr class="align-top"><td class="py-2">
 <td class="py-2 text-emerald-300">отрицатель
 </tbody>
 </table>
 <p class="text-slate-400 text-xs mt-3">
 бном» порядке: S→A,
 порядок рёбер местами – и понадобится до ше
 </div>
```

```
<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <p class="text-slate-300 text-sm mb-3">
 тава A имеет суммарный вес S→A,
 веет» на 2.</p>
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b">
```



```
<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-
 -b border-slate-700/50">» Код: Форд–Беллман
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te

def bellman_ford(n, edges, s):
 """edges = [(u, v, w), ...] – ориентированные рёбра
 dist = [INF] * n
 prev = [-1] * n
 dist[s] = 0
 changed_vertex = -1

 for it in range(n):
 changed_vertex = -1
 for u, v, w in edges:
 if dist[u] != INF and dist[u] + w < dist[v]:
 dist[v] = dist[u] + w
 prev[v] = u
 changed_vertex = v
 if changed_vertex == -1:
 break

 if changed_vertex == -1:
 return dist, prev, None

 # changed_vertex улучшен на n-й итерации:
 # поднимаемся по prev на n шагов – гарантированно з
 y = changed_vertex
 for _ in range(n):
 y = prev[y]

 cycle = [y]
 v = prev[y]
 while v != y:
 cycle.append(v)
 v = prev[v]
```

```

vector<long long> d;
vector<int> p;

// возвращает вершину, улучшенную на «лишней» итерации,
int bellman_ford(int s) {
 d.assign(n, INF); p.assign(n, -1); d[s] = 0;
 int x = -1;
 for (int it = 0; it < n; ++it) { // n-1
 x = -1;
 for (const auto &e : edges) {
 if (d[e.u] != INF && d[e.u] + e.w < d[e.v]) {
 d[e.v] = d[e.u] + e.w;
 p[e.v] = e.u;
 x = e.v;
 }
 }
 if (x == -1) break; // ранний
 }
 return x;
}

vector<int> negative_cycle(int x) {
 int y = x;
 for (int i = 0; i < n; ++i) y = p[y]; // захватили цикл
 vector<int> cycle;
 for (int v = y;; v = p[v]) {
 cycle.push_back(v);
 if (v == y && cycle.size() > 1) break;
 }
 reverse(cycle.begin(), cycle.end());
 return cycle;
}

int main() {
 int s = 0;
 int x = bellman_ford(s);
 if (x == -1) cout <<< "отрицательных циклов не
 else {

```

- <li><b>Ранний выход.</b> Если з  
рафах экономит кратно.</li>
- <li><b>SPFA</b> (Shortest Path  
, у которых только что улучшился <span clas  
an>, но <b>худший случай остаётся <span cla  
кас» из рёбер) SPFA ловит TL. Критерий цикла
- <li><b>Порядок рёбер.</b> Форма  
итерации, против течения – за <span class="font-mon  
ция даёт финальные <span class="font-mono">
- <li><b>DAG.</b> Если граф ацикл  
<span class="font-mono"> $0(V + E)$ </span>, и

```


 </div>
</details>
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-amber-400

```

- ```

<div class="bg-rose-950/30 p-5 rounded-lg b
  <p class="text-sm font-bold text-rose-300
  <ul class="list-disc pl-5 space-y-2 tex
    <li><b> $V - 1$ , а не  $V$ .</b> Рабочих и
ont-mono text-emerald-300"> $V$ </span>-я – тол
отрицательным циклом <span class="font-mono
    <li><b>Переполнение INF.</b> <span <
INT_MAX</span> превращается в отрицательно
запасом и <span class="font-mono">long lon
    <li><b>Неориентированный граф с мин
lass="font-mono"> $-3$ </span> – это цикл длины
рицательном цикле. Минусы в неориентированн
    <li><b>Цикл есть, но не от старта.</b>
</span>. Нужна проверка всего графа – добав
а, билет 17) и запустите от неё.</li>
    <li><b>«Ответ  $-\infty$ ».</b> Если отрицат
s="font-mono">dist[v] =  $-\infty$ </span>: печатать
ходом в обратном графе.</li>
    <li><b>SPFA как «быстрый Форд–Беллм

```

n> в остальных клетках.</p>

```
<div class="my-5 p-5 rounded-xl bg-slate-900">
  <div class="mb-4 rounded-xl overflow-hidden">
    
  </div>
  <div class="flex items-center gap-3">
    <span class="text-2xl"> </span>
    <div>
      <h4 class="text-purple-300 font-sans">
        «Все против Фсем» (Все против Фсем)
      </h4>
      <p class="text-slate-300 text-sm">
        Флойд–Уоршелл – это «Королева» графов.
        Это кратчайший путь к абсолютно КАЖДОЙ другой вершине.
      </p>
    </div>
  </div>
</div>
```

```
<div class="bg-slate-800 p-6 rounded-lg border border-emerald-500 shadow-md">
  <div class="absolute -top-3 left-4 bg-slate-900 text-emerald-300
  Суть фокуса (по шагам)
  </div>
  <p class="text-slate-300 text-sm mb-4">
    Главный герой – <b>внешний цикл</b>
    <span class="font-mono text-emerald-300">k</span> та
    </span>, если в качестве <b>промежуточных</b>
    , ..., k}</span>?». Каждый шаг добавляет в ра
    </p>
    <ul class="text-sm text-slate-300 space-y-2">
      <li><b>Шаг k = -1 (старт):</b> разрешено перемещаться
      <li><b>Шаг k = 0:</b> разрешаем перемещаться
      300">i → 0 → j</span>, чем текущий <span class="font-mono
      <li><b>Шаг k = 1:</b> разрешены перемещения
      s="font-mono">1 → j</span> уже может стоять
```



```
s="font-mono">k</span> <b>один раз</b> и бо
садку не соберётся. На демо-графе этой стра
ono text-emerald-300">d[2][1] = ∞</span>, х
пользует «поздние» посредники). Проверка пе
</div>
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border
<h3 class="text-xl font-bold text-blue-400 m
```

```
<p class="text-slate-300 text-sm mb-4">Демо
вершины 0..6, <span class="font-mono text-em
merald-300">(i, j)</span>, которая улучшает
ляторе переменные <span class="font-mono te
class="font-mono text-emerald-300">j</span>
напрямую: измените <span class="font-mono"
```

```
<div class="bg-slate-800/70 p-5 rounded-lg l
<p class="text-sm font-bold text-white m
<table class="w-full text-xs text-left l
<thead>
<tr class="text-slate-500 borde
<th class="py-1.5 pr-2 font
<th class="py-1.5 pr-2 font
<th class="py-1.5 font-bold
```

```
</tr>
```

```
</thead>
```

```
<tbody class="text-slate-300">
```

```
<tr class="border-b border-slate
-2 font-mono">0</td><td class="py-2">в верш
```

```
<tr class="border-b border-slate
-2 font-mono">4</td><td class="py-2">открыв
```

```
<tr class="border-b border-slate
-2 font-mono">3</td><td class="py-2">2 — вх
```

```
<tr class="border-b border-slate
-2 font-mono">2</td><td class="py-2">3 — вх
```

```
<tr class="border-b border-slate
```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-slate-900">
  <h3 class="text-xl font-bold text-indigo-400">Динамическое программирование
  <div class="bg-slate-900/40 rounded-lg p-4 border border-slate-700/50">
    <summary class="p-4 font-bold text-slate-300">Формальная динамика
    <div class="p-5 text-sm text-slate-300">
      <p>Обозначим  $i \rightarrow j$ , все промежуточные вершины
      <math>i</math> и  $j$ 
      <p><b>База:</b>  $d[i][i] = 0$ 
      <p><b>Переход:</b> путь из  $i$  в  $j$  либо не использует  $k$ 
      <math>i \rightarrow j</math>, – либо использует  $k$  (тогда  $d[i][j] = \min(d[i][k] + d[k][j], d[i][j])$ ), разбиваясь на два подпути с посредником  $k$ 
      <p><b>Почему хватает одной матрицы.</b>
      <math>d[i][j]</math> и столбец  $d[k][k]$ 
      <math>d[i][j] = \min(d[i][k] + d[k][j], d[i][j])</math>, а  $d[k][k]$ 
      <math>d[i][j]</math> не меньше первого. Значит, читая  $d[i][j]$ 
      <math>d[i][j]</math>-й итерации
      <math>d[i][j]</math> без  $d[k][k]$ 
      <p><b>Следствие:</b> если в графе  $d[k][k] < 0$  и рассуждение
    </div>
  </div>
</div>

<div class="bg-slate-900/40 rounded-lg p-4 border border-slate-700/50">
  <summary class="p-4 font-bold text-slate-300">Эксперимент: перебор
  <div class="p-5 text-sm text-slate-300">
    <pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
M = [[0,4,INF,2,INF,INF,INF],
      [INF,0,3,INF,INF,3,INF],
      [INF,INF,0,INF,2,INF,INF],
      [INF,INF,INF,0,4,2,INF],
      [INF,INF,INF,INF,0,INF,1],
      [INF,INF,INF,INF,INF,0,INF]]
    
```

```

-----
nxt = [[-1] * n for _ in range(n)]
for i in range(n):
    d[i][i] = 0                                # диагональ об

for u, v, w in edges:
    if w < d[u][v]:                            # кратчайше
        d[u][v] = w
        nxt[u][v] = v
    if not directed and w < d[v][u]:
        d[v][u] = w
        nxt[v][u] = u

for k in range(n):                            # k – ВНЕШНИЙ
    for i in range(n):
        if d[i][k] == INF:
            continue
        for j in range(n):
            if d[k][j] == INF:
                continue
            if d[i][k] + d[k][j] < d[i][j]:
                d[i][j] = d[i][k] + d[k][j]
                nxt[i][j] = nxt[i][k]

negative = [i for i in range(n) if d[i][i] < 0]
return d, nxt, negative

def path(nxt, i, j):
    if nxt[i][j] == -1:
        return None                            # j недостижим
    p = [i]
    while p[-1] != j:
        p.append(nxt[p[-1]][j])
    return p

nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [("S","A",4), ("S","B",5), ("B","C",-2), ("A","C",1),
      ("C","D",3), ("C","E",4), ("D","F",2), ("E","F",1)]

```

```

        return false;
    }

    long long diameter() {                                     // сама
        long long res = 0;
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (d[i][j] < INF) res = max(res, d[i][j]);
        return res;
    }

    int center() {                                             // верши
        int best = -1; long long bestEcc = INF;
        for (int i = 0; i < n; ++i) {
            long long ecc = 0;
            for (int j = 0; j < n; ++j) {
                if (d[i][j] == INF) { ecc = INF; break; }
                ecc = max(ecc, d[i][j]);
            }
            if (ecc < bestEcc) { bestEcc = ecc; best = i; }
        }
        return best;
    }
}

```

</div>
 </details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">» Та же динамика дл
 <div class="p-5 text-sm text-slate-300
 <table class="w-full text-xs text-l
 <thead>
 <tr class="text-slate-500 b
 <th class="py-1.5 pr-2
 <th class="py-1.5 pr-2
 <th class="py-1.5 font-l
 </tr>
 </thead>

```

        <li><b>Отрицательные циклы.</b> Если
        >k</span> принадлежит отрицательному циклу.
        , <b>достижимых ИЗ цикла в ПРЯМОМ графе</b>
        <li><b>Недостижимость ≠ 0.</b> В ор
        таких пар из 49). Выводить 999 или 0 как «р
        <li><b>Память.</b> <span class="font
        ачений (≈100 МБ на <span class="font-mono">
        истока берите Дейкстру.</li>
        <li><b>Сам путь.</b> Матрица <span
        pan class="font-mono">nxt[i][j]</span> (как
        </ul>
    </div>

    <p class="text-slate-300 text-sm mt-4"><b>С
    от одного истока и дешевле, если все пары н
    n class="font-mono">V</span> Дейкстр быстрее
    </span>). Алгоритм Джонсона (билет 17) комб
    /span> Дейкстр на перевзвешенном графе. Тра
    ции достижимость проверяется по DAG.</p>
    </div>
</div>
</section>`
    },
    {
        id: "johnson-algo",
        title: "17. Графы. Поиск кратчайшего пути. Ускорени
        type: "html",
        description:
            "Джонсон (перевзвешивание потенциалов), встречный
        category: "Графы. Пути",
        content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 r
        <h2 class="text-2xl sm:text-3xl font-bold text-v
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

<div class="overflow-x-auto my-6">
  <table class="w-full text-xs text-left">
    <thead>
      <tr class="text-slate-500 border-b">
        <th class="py-2 pr-3 font-b">
        <th class="py-2 pr-3 font-b">
        <th class="py-2 pr-3 font-b">
        <th class="py-2 font-bold">
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-slate-200">
        <td class="py-2 pr-3">разрешает Дейкс
        <td class="py-2">нет отрицательных
      </tr>
      <tr class="border-b border-slate-200">
        <td class="py-2 pr-3">не обходим весь граф ра
        <td class="py-2">нужен путь до одной цели</td>
      </tr>
      <tr class="border-b border-slate-200">
        <td class="py-2 pr-3">две маленькие волн
        <td class="py-2">е вдвое меньше раскрытий</td>
      </tr>
      <tr class="border-b border-slate-200">
        <td class="py-2 pr-3">раскрывает вершины в сторону цели
        <td class="py-2">есть допустимая эвристика  $h(v)$ </td>
      </tr>
      <tr class="border-b border-slate-200">
        <td class="py-2 pr-3">убирает кучу целиком</td>
        <td class="py-2">только 0 и 1</td>
      </tr>
      <tr class="border-b border-slate-200">
        <td class="py-2 pr-3">куча → массив корзины
        <td class="py-2">С небольшое</td>
      </tr>
      <tr class="border-b border-slate-200">
        <td class="py-2 pr-3">decrease-key за  $O(1)$ 
        <td class="py-2">теоретически; на практике двоичная/d-ар
      </tr>
      <tr class="border-b border-slate-200">
        <td class="py-2 pr-3">не релаксируем все рё
        <td class="py-2">, на практике быстро</td>
      </tr>
      <tr class="align-top">
        <td class="py-2 pr-3">переносит работу в предобработку</td>
        <td class="py-2">миллионы запросов (карты)</td>
      </tr>
    </tbody>
  </table>

```

h3 class="text-lg font-bold text-white mt-3"
p class="text-slate-300 text-sm mb-3">Если
 азад из t (по перевёрнутым рёбрам). Волны р
 : в «широком» графе (дорожная сеть) это экол
 огда волны «встретились» в одной вершине; н
 огда **span class="font-mono text-emerald-300"**
 более короткий путь.

h3 class="text-lg font-bold text-white mt-3"
p class="text-slate-300 text-sm mb-3">Дейк
 по **span class="font-mono">** $g(v)$ **** (уже
 $g(v) = g(v) + h(v)$ ****, где $h(v)$ – оценка
 а требования: **допустимость** ($h(v) \leq$ и
 омонотонность/согласованность) ($h(u) \leq w(u$
 вырождается ровно в Дейкстру – это хороший

h3 class="text-lg font-bold text-white mt-3"
p class="text-slate-300 text-sm mb-3">Если
 кучу заменяют **деком**: ребро веса 0 к
 новременно живут максимум два уровня), поэто
>. Обобщение – **алгоритм Дила (корзины)******
 $E + V \cdot C$). Оба приёма – стандартный ответ на

h3 class="text-lg font-bold text-white mt-3"
p class="text-slate-300 text-sm mb-3">Форд
 оторых расстояние только что уменьшилось****
 ческого прогона, но асимптотика та же, $O(V \cdot E)$
 пользуют осторожно, с эвристиками SLF/LLL).
) уменьшает высоту и на практике быстрее, к
</p>

h3 class="text-lg font-bold text-white mt-3"
p class="text-slate-300 text-sm mb-3">Нави
 в **предобработку**. Сначала – **сжати**
 ндексами $0 \dots N$, и уже по ним строят граф и се
 тояния от нескольких «ориентиров» до всех в
 етрии. **Contraction hierarchies** (так
 и запрос идёт двунаправленным поиском вверх

```

for _ in range(n):                                # V итераций: V-
    changed = False
    for u, v, w in edges:
        if h[u] + w < h[v]:
            h[v] = h[u] + w
            changed = True
    if not changed:
        break
else:
    return None                                    # отрицательный

# 2) перевзвешивание: все  $w' \geq 0$ 
g = [[] for _ in range(n)]
for u, v, w in edges:
    g[u].append((v, w + h[u] - h[v]))

# 3) Дейкстра от каждой вершины + обратный перевод
d = [[INF] * n for _ in range(n)]
for s in range(n):
    d[s][s] = 0
    heap = [(0, s)]
    while heap:
        dist, v = heapq.heappop(heap)
        if dist > d[s][v]:
            continue
        for to, w in g[v]:
            if d[s][to] > dist + w:
                d[s][to] = dist + w
                heapq.heappush(heap, (d[s][to], to))
    for v in range(n):
        if d[s][v] < INF:
            d[s][v] = d[s][v] - h[s] + h[v]          # от
return d

```



```

while heap:
    f, gv, v = heapq.heappop(heap)
    if v == t:
        return gv # ранний выход:
    if v in closed:
        continue
    closed.add(v)
    for to, w in g[v]:
        nd = gv + w
        if nd < gdist.get(to, INF):
            gdist[to] = nd
            heapq.heappush(heap, (nd + h(to), nd, to))
return INF

# --- 0-1 BFS: веса 0 и 1, дека вместо кучи ---
from collections import deque
def bfs01(g, s, n): # g[v] = [(to, w)]
    INF = float("inf")
    d = [INF] * n; d[s] = 0
    dq = deque([s])
    while dq:
        v = dq.popleft()
        for to, w in g[v]:
            if d[v] + w < d[to]:
                d[to] = d[v] + w
                (dq.appendleft(to) if w == 0 else dq.append(to))
    return d

```

<details class="bg-slate-900/40 rounded-lg p-4 font-bold text-slate-300 border-slate-700/50">

Вопросы, которые любят задавать (и не только)

</summary>

<div class="p-5 text-sm text-slate-300">

<p>• Почему после перевзвешивания в алгоритме Дейкстры есть условие отсутствия улучшающей релаксации?

```

    id: "mst",
    title: "18–20. Графы. Остовное дерево: Краскал, Приоритетная очередь",
    type: "html",
    description:
      "Три способа построить минимальное остовное дерево.",
    category: "Графы. Остовные",
    content: `
<section id="mst" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">18–20. Графы. Остовное дерево: Краскал, Приоритетная очередь
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Остовное дерево
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
      <h3 class="text-xl font-bold text-emerald-400">Остовное дерево
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Остовное дерево — это подграф исходного графа, который является деревом и содержит все вершины графа.
        <p class="text-xl font-mono text-white">Остовное дерево — это подграф исходного графа, который является деревом и содержит все вершины графа.
        он связан. MST — такое остовное дерево, у которого минимальная сумма весов рёбер.
      </div>

      <p class="text-slate-300 text-sm mb-4">Все рёбра, которые не входят в некоторое MST, являются рёбрами, которые не входят в некоторое MST. Алгоритмы отличаются тем, что они находят компоненты текущего леса (разрез — это набор рёбер, которые не входят в некоторое MST). Разрезы для всех компонент одновременно.</p>

      <div class="bg-slate-800 p-5 rounded-lg border border-slate-800">
        <h4 class="text-amber-400 font-bold text-sm">Остовное дерево
        <p class="text-slate-200 text-sm font-sans">Остовное дерево — это подграф исходного графа, который является деревом и содержит все вершины графа.
        <p class="text-slate-300 text-sm mb-3">Остовное дерево — это подграф исходного графа, который является деревом и содержит все вершины графа.
        адает буква Р — остаётся <b>остов</b>.</p>
        <p class="text-slate-300 text-sm">Остовное дерево — это подграф исходного графа, который является деревом и содержит все вершины графа.
        т графа: минимум костей, чтобы всё держалось вместе.
      </div>
    </div>
  `

```

```

        </tbody>
    </table>
</div>

<div class="grid grid-cols-1 xl:grid-cols-3"
    <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
der border-indigo-500 shadow-md">
            Аналогия к билету 18 • Краск
        </div>
        <p class="text-slate-300 text-sm">С
    ». По всей карте растут независимые кусочки
        дерево. Порядок рёбер важен, порядок верши
    </div>

    <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            ✦ Аналогия к билету 19 • Прим:
        </div>
        <p class="text-slate-300 text-sm">П
а захватывает ближайший непосещённый город.
ва» против «снаружи».</p>
    </div>

    <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
r border-rose-500 shadow-md">
            Аналогия к билету 20 • Боровк
        </div>
        <p class="text-slate-300 text-sm">С
айшему чужому племени: к вечеру компонент в
        <b>log2 V</b> дней образуется единая федер
    </div>
</div>

<div class="bg-slate-900/40 rounded-lg bord
    <p class="text-slate-300 text-sm mb-2">

```

```

    p[b] = a;
    if (rnk[a] == rnk[b]) rnk[a]++;
    return true;
}

sort(edges.begin(), edges.end());           // O(E log E)
long long cost = 0; int taken = 0;
for (auto [w, uv] : edges) {
    if (unite(uv.first, uv.second)) {        // не замыкаем
        cost += w;
        if (++taken == n - 1) break;        // V - 1 ребро
    }
}
}

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">

Код 19: Прим с кучей (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">

heapq

```

INF = float("inf")
key = [INF] * n           # минимальный вес ребра, ведущего к v
used = [False] * n
key[0] = 0
heap = [(0, 0)]           # (вес ребра, вершина) ← вот это
total = 0

```

```

while heap:
    w, v = heapq.heappop(heap)
    if used[v]:
        continue          # устаревшая запись в куче
    used[v] = True
    total += w

```

```

        break # граф несвя
    for w, u, v in best.values():
        if find(u) == find(v):
            continue # дубль: два
        parent[find(u)] = find(v)
        comps -= 1
        total += w
print(total)</pre>

```

<p class="text-slate-400 text-xs">3
ну), значит фаз не больше $\log_2 V$.</p>

</div>

</details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">

Вопросы, которые любят задавать (
 </summary>

<div class="p-5 text-sm text-slate-300
 <p>• Почему жадность здесь работает
 меном ребра в предполагаемом MST.</p>

<p>• Чем Прим отличается от Дейкстры
 ного веса против дерева кратчайших путей).</p>

<p>• Какой алгоритм лучше на плотном

<p>• Зачем нужна Борувка, если есть
 независимо, поэтому её любят на GPU и в рас

<p>• Что если граф несвязный? – Прим
 минимального остовного леса (MSF) Прим
 т лес автоматически.</p>

<p>• Как найти второе по величине о
 ребро на образовавшемся цикле (LCA + максим

</div>

</details>

</div>

</div>

</section>`,

},

];

```

    </p>
    <ol class="list-decimal pl-5 space-y-1.5">
      <li>Считаем значение <span class="font-mono text-indigo-300">1</span>
      <li>Сравниваем следующий символ пре
- <span class="font-mono text-indigo-300">п</span>
    </ol>
    <div class="bg-slate-950 p-3 rounded-lg">
      <div class="inline-flex items-center justify-center">
        <span class="text-indigo-300 font-mono text-sm">1</span>
        <span class="text-slate-600">|</span>
        <span class="bg-indigo-600 text-white font-mono text-sm">п</span>
      </div>
      <p class="text-xs text-indigo-300 m-0">
        - Рамка смотрит ВЛЕВО: проверяя
      </p>
    </div>
  </div>

  <details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
    <summary class="p-4 font-bold text-slate-300">
      Пример вычисления (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300">
      <p>Тот же пример: <b>abcabcd</b></p>
      <ul class="list-disc pl-5 space-y-2">
        <li><b>a</b>: Префиксов нет. <span class="font-mono text-indigo-300">0</span>
        <li><b>ab</b>: Из начала ("a") не начинаем
      </ul>
      <ul class="list-disc pl-5 space-y-2">
        <li><b>abc</b>: Опять мимо. <span class="font-mono text-indigo-300">0</span>
        <li><b>abca</b>: 0! Начало ("a") не начинаем
      </ul>
      <ul class="list-disc pl-5 space-y-2">
        <li><b>abcab</b>: Начало ("ab") не начинаем
      </ul>
      <ul class="list-disc pl-5 space-y-2">
        <li><b>abcabc</b>: Начало ("abc") не начинаем
      </ul>
      <ul class="list-disc pl-5 space-y-2">
        <li><b>abcabcd</b>: «d» всё исп

```

```

        льшего собственного префикса, равного суффи
        <p class="text-slate-400 text-sm mb-1">
    ый алгоритм.</p>
        <p class="text-slate-400 text-sm mb-1">
    т онлайн (символы могут приходить потоком),
        <p class="text-slate-400 text-sm">• <b>
    5] = 4, период 6 - 4 = 2 («ab»)).</p>
</div>
<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
    -b border-slate-700/50">
        Код: поиск подстроки КМР (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounde
fix_function(s):
n = len(s)
pi = [0] * n
for i in range(1, n):
    j = pi[i - 1]
    while j > 0 and s[i] != s[j]:
        j = pi[j - 1]          # откат по уже п
    if s[i] == s[j]:
        j += 1
    pi[i] = j
return pi

def kmp_search(text, pattern):
    m = len(pattern)
    pi = prefix_function(pattern)      # O(m) один раз
    j = 0
    found = []
    for i, ch in enumerate(text):      # O(n) по тексту
        while j > 0 and ch != pattern[j]:
            j = pi[j - 1]
        if ch == pattern[j]:
            j += 1
        if j == m:

```

```

<div class="bg-slate-900 border border-emerald-500 p-4" style="border-radius: 10px;">
  <h4 class="text-emerald-300 font-bold text-sm">
    Z-функция Z[i] – смотрим ВПРАВО (пока не встретим совпадение)
  </h4>
  <p class="text-slate-300 text-sm mb-4">
    Буква <b>Z</b> запоминается как <b>Z</b> (пока не встретим совпадения)
    и подглядываем <b>ВПРАВО</b> (ПОЗАДИ НЕ СМОТРИМ).
  </p>
  <div class="bg-slate-950 p-3 rounded-lg" style="border: 1px solid #000; border-radius: 10px;">
    <div class="inline-flex items-center justify-center gap-2">
      <span class="text-slate-500">[0]
      <span class="bg-emerald-600 text-emerald-300 font-mono text-sm">a
      <span class="text-slate-600">|
      <span class="text-emerald-300 font-mono text-sm">a
    </div>
    <p class="text-xs text-emerald-300">
      - Рамка смотрит ВПРАВО: измеряет длину совпадения
    </p>
  </div>
</div>

<details class="bg-slate-900/40 rounded-lg p-4" style="border: 1px solid #000; border-radius: 10px;">
  <summary class="p-4 font-bold text-slate-300">
    -b border-slate-700/50"
    Пример вычисления (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300">
    <p>Тот же пример: <b>abcabcd</b></p>
    <ul class="list-disc pl-5 space-y-2">
      <li><code>Z[0]</code> – не считаем (пока не встретим совпадения)
      <li><code>Z[1]</code> (<b>b</b> против <b>a</b>)
      <li><code>Z[2]</code> (<b>c</b> против <b>a</b>)
      <li><code>Z[3]</code> (<b>abcd</b> против <b>a</b>)
    </ul>
    а дальше s[6]='d' против s[3]='a' – мимо.
  </div>

```



```
<div class="bg-slate-900 p-4 rounded-lg mb-6 text-white">
  <p class="text-sm text-blue-300 italic mb-2">0c
  <p class="text-lg font-mono text-white">Бор (Тр
</div>
```

```
<p class="text-slate-300 text-sm">Позволяет найти
  го один проход.</p>
</div>
```

```
<!-- Аналогии -->
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
      Аналогия 1: Паутина (Бор)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      ет — мы падаем по <b>суффиксной ссылке</b> ("
    </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-2 border-slate-800">
  <div class="absolute -top-3 left-4 bg-rose-900
    -500 shadow-md">
    Аналогия 2: Матёшка (Терминальные)
  </div>
  <p class="text-slate-300 text-sm mb-4">Представ
    нашли и "he", потому что оно спрятано внутри
  </div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border border-slate-800">
  <summary class="p-4 font-bold text-slate-400 outline-slate-700/50">
    Код (Скрыто)
  </summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-4">
  <pre class="bg-slate-950 p-4 rounded overflow-x-auto">
int get link(int v) {
```

```
    <p class="text-xl font-mono text-white":
ат). Сведение  $A \leq_p B$  – преобразовать вход  $A$ 
.</p>
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Клас
машина Тьюринга) с выбранным бюджетом ресур
про <b>худший случай</b>, а не про «в средн
```

```
<div class="grid grid-cols-1 xl:grid-cols-2
    <div class="bg-slate-800 p-6 rounded-lg
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Судoku (Р против P)

```
</div>
```

```
    <p class="text-slate-300 text-sm">К
ий путь, MST). Класс <b>NP</b> – когда реша
проверишь, что там нет нарушений правил. Во
и решить?». Открыт с 1971 года и стоит мил.
```

```
</div>
```

```
    <div class="bg-slate-900 p-6 rounded-lg
        <div class="absolute -top-3 left-4 l
rder border-purple-500 shadow-md">
```

Аналогия 2: Переводчик (сведе

```
</div>
```

```
    <p class="text-slate-300 text-sm">С
ский (сведение за полином) и друг-англичани
Отсюда главный инструмент: если к В сводитс
ь не стоит.</p>
```

```
</div>
```

```
</div>
```

```
<h3 class="text-lg font-bold text-white mt-:
<p class="text-slate-300 text-sm mb-3">Изве
ения в цепочке доказывается теоремой об иер
<div class="bg-slate-950 border border-slate
    <p class="font-mono text-sm sm:text-bas
         $P \subseteq NP \subseteq PSPACE \subseteq EXP \subseteq NEXP$ <br>
```

```
<h3 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Стандартная задача — получить сертификат полиномиального размера и проверить, что
— «все вершины по разу и каждая соседняя пара вершин образует полиномиальное сведение  $A \leq_p B$ . Тогда  $B$  — NP-полна. Тогда
<p class="text-slate-300 text-sm mb-3"><b>Теорема 1.1. Если задача SAT NP-полна. Доказывается «в лоб»: вычислительная сложность
ековой формулой размера  $\text{poly}(n)$ , где переменные  $x_i$  принимают значения 0 или 1. Тогда
ильна принятию входа. Дальше всё катится цепочкой: SAT  $\leq_p$  3-SAT  $\leq_p$  TSP.
гамильтонов цикл  $\rightarrow$  TSP.</p>
```

```
<div class="bg-slate-800/70 p-5 rounded-lg border">
  <p class="text-sm font-bold text-white">
  <ul class="list-disc list-inside text-slate-300">
    <li><b>Точный перебор с отсечениями (meet-in-the-middle).</b></li>
    <li><b>Приближённые алгоритмы</b>:
et cover —  $\ln n$ .</li>
    <li><b>Эвристики и метаэвристики</b>:
ke.</li>
    <li><b>Параметризованная сложность</b>
n).</li>
    <li><b>Специальные классы входов</b>:
полиномиальными (независимое множество на д
    <li><b>Готовые решатели</b>: SAT/ILP,
воего алгоритма.</li>
  </ul>
</div>
```

```
<div class="bg-slate-900/40 rounded-lg border">
  <p class="text-slate-300 text-sm mb-2">
  <p class="text-slate-400 text-sm mb-1">
NP: любая задача из P автоматически в NP.
  <p class="text-slate-400 text-sm mb-1">
становки — неразрешима, зато NP-трудна). NP-полнота — это не то же самое, что
  <p class="text-slate-400 text-sm mb-1">
стной трудной  $A \leq_p B$  ( $A \leq_p B$ ), а не наоборот.
  <p class="text-slate-400 text-sm mb-1">
```

```

        if ci == cj:                                # из од
            continue
        if li.lstrip("-") == lj.lstrip("-") and (li
            continue                                # х и н
        edges.append((i, j))
    return verts, edges, len(clauses)                # клика

```

Проверка задачи из P для контраста: двудольность = 2-р

```

def is_bipartite(n, graph):
    from collections import deque
    col = [-1] * n
    for s in range(n):
        if col[s] != -1:
            continue
        col[s] = 0
        dq = deque([s])
        while dq:
            v = dq.popleft()
            for to in graph[v]:
                if col[to] == -1:
                    col[to] = col[v] ^ 1
                    dq.append(to)
                elif col[to] == col[v]:
                    return False
    return True

```

<p class="text-slate-400 text-xs">0
 ТОВЫЙ ОТВЕТ, но не находить его; для задачи
 </div>
 </details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">

Вопросы, которые любят задавать (C
 </summary>
 <div class="p-5 text-sm text-slate-300
 <p>• Дайте определение NP через вер
 нтны.</p>

```

<div class="space-y-8">
  <!-- Единый формат ввода -->
  <div class="bg-slate-700/50 p-6 rounded-xl border
    <h3 class="text-xl font-bold text-indigo-400">
    <p class="text-slate-300 text-sm mb-4">
      Фиксируем один формат, чтобы граф больше
    </p>
    <div class="bg-slate-950 p-4 rounded-lg border
      g mb-4">
        edges[x] = [(y, w), ...]
    </div>
    <p class="text-slate-300 text-xs sm:text-sm">
      То есть <span class="font-mono text-emerald-400">
    </p>
    <ul class="list-disc list-inside space-y-1">
      <li><span class="font-mono text-emerald-400">
      <li><span class="font-mono text-emerald-400">
      <li><span class="font-mono text-emerald-400">
    </ul>
  </div>

  <!-- Исправленный Декстер -->
  <div class="bg-slate-700/50 p-6 rounded-xl border
    <h3 class="text-xl font-bold text-emerald-400">
    <p class="text-slate-300 text-sm mb-4">
      Убираем сортировку: <span class="font-mono text-emerald-400">
      правильный порядок вершин неизвестен.
    </p>

    <pre class="bg-slate-950 p-4 rounded-lg overflow-x-auto
      leading-relaxed mb-4">def dexter(edges, n,
min_way = [float('inf')] * n
min_way[start] = 0
seen = set()

for _ in range(n):
  x = -1

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-500">
  <h3 class="text-xl font-bold text-amber-400">Пример
  <p class="text-slate-300 text-sm mb-4">
    Здесь <span class="font-mono text-amber-400">остовное ребро</b>, которым вершину <span class="font-mono text-amber-400">
  </p>

  <pre class="bg-slate-950 p-4 rounded-lg overflow-x-auto text-slate-400 font-mono">def prima(edges, n, start):
min_way = [float('inf')] * n
min_way[start] = 0

seen = set()
pred = [-1] * n
ostov = []

for _ in range(n):
    x = -1

    # Выбираем вершину, которую дешевле всего
    # присоединить к остову
    for i in range(n):
        if i not in seen:
            if x == -1 or min_way[i] < min_way[x]:
                x = i

    if x == -1 or min_way[x] == float('inf'):
        break

    seen.add(x)

    if pred[x] != -1:
        ostov.append((pred[x], x, min_way[x]))

    for y in edges[x]:
        if y[0] not in seen and y[1] < min_way[y[0]]:
            min_way[y[0]] = y[1]
            pred[y[0]] = x

```

```
</div>
```

```
<!-- Вся разница в одной строке -->
```

```
<div class="bg-slate-900/90 p-6 rounded-xl border">
```

```
  <h3 class="text-xl font-bold text-white mb-4">
```

```
    <div class="grid grid-cols-1 md:grid-cols-3">
```

```
      <div class="bg-slate-950 p-4 rounded-lg">
```

```
        <p class="text-emerald-400 font-bold">
```

```
          <code class="text-emerald-300 font-mono">
```

```
            <p class="text-slate-400 text-xs mt-2">
```

```
          </div>
```

```
      <div class="bg-slate-950 p-4 rounded-lg">
```

```
        <p class="text-amber-400 font-bold">
```

```
          <code class="text-amber-300 font-mono">
```

```
            <p class="text-slate-400 text-xs mt-2">
```

```
          </div>
```

```
    <div class="bg-slate-950 p-4 rounded-lg">
```

```
      <p class="text-sky-400 font-bold mb-2">
```

```
        <code class="text-sky-300 font-mono">
```

```
          <p class="text-slate-400 text-xs mt-2">
```

```
        </div>
```

```
    </div>
```

```
<div class="mt-6 bg-slate-950/60 p-4 rounded">
```

```
  <p className="font-bold text-indigo-300">
```

```
  <ul className="list-disc list-inside space-y-2">
```

```
    <li><b>Декстер:</b> плесень, которая
```

```
    <li><b>Прима:</b> плесень, которая
```

```
    <li><b>BFS:</b> плесень, для которой
```

```
  </ul>
```

```
</div>
```

```
</div>
```

```
<!-- Мнемокарточка графовых алгоритмов -->
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border">
```



```
export const tickets6to13: Chapter[] = [
  {
    id: "graph-dfs-bfs",
    title: "6. Графы. Представление графов, DFS, BFS",
    type: "html",
    description: "Представление графов и базовые обходы",
    category: "Графы. База",
    content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      { /* База представлений */ }
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-slate-300">
          <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-slate-300 italic">
              <p class="text-xl font-mono text-white">
                + E).</p>
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">
              <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                  Аналогия: Матрица = Таблица
                </div>
                <p class="text-slate-300 text-sm mb-4">
                  глупо, но проверка "знает ли Вася Петю" мгр
                </div>
                <div class="bg-slate-900 p-6 rounded-lg">
                  <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
                    Аналогия: Списки = Контакты
                  </div>
                  <p class="text-slate-300 text-sm mb-4">

```

```

        </p>
        <ul class="text-xs text-emerald-300">
            <li><b>Где используется:</b></li>
            <li>Кратчайший путь в Невзвешенном графе</li>
            <li>Проверка двудольности графа</li>
            <li>Алгоритм Ахо-Корасик (построение автомата)</li>
            <li>Двунаправленный BFS для поиска кратчайшего пути (вместо
                йкстра).</li>
        </ul>
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
    <summary class="p-4 font-bold text-slate-200">
        Код обходов C++ (Основа)
    </summary>
    <div class="grid grid-cols-1 md:grid-cols-2">
        <pre class="bg-slate-950 p-3 rounded">
// DFS (Рекурсия с возвратом)
vector<bool> vis;
vector<vector<int>>> adj;

void dfs(int v) {
    vis[v] = true;
    for (int u : adj[v]) {
        if (!vis[u]) dfs(u);
    }
}</pre>

        <pre class="bg-slate-950 p-3 rounded">
// BFS (Очередь FIFO)
queue<int> q;
q.push(start_node);
vis[start_node] = true;

while (!q.empty()) {
    int v = q.front(); q.pop();
    for (int u : adj[v]) {

```

Доказательство: Каждая грань $span>$. Подставляя в формулу Эйлера: $span>$

Для простого двудольного графа
 $span>$.

Доказательство: В двудольном графе $class="font-mono">2E \geq 4F \Rightarrow F \leq E/2$

Пример 1. $class="font-mono">10 \leq 3(5) - 6 = 9$

Пример 2. $class="font-mono">9 \leq 12$ – формально противоречие $2V - 4$: $class="font-mono">9 \leq 12$

Теорема Куратовского (1930) / Вагнера (1936). Любой граф, являющегося подразделением (subdivision) графа K_5 или $K_{3,3}$ является планарным.

Задача 1. Раскраска графа

Рассмотрим граф G с n вершинами и m ребрами. Пусть k – минимальное количество цветов, необходимое для раскраски графа G так, чтобы никакие два смежных вершины не были окрашены в один и тот же цвет. Тогда k называется **хроматическим числом** графа G и обозначается $\chi(G)$.

Сложность задачи раскраски:

- $k = 2$ (2-раскрашиваемость):

```
<p class="text-xl font-mono text-white">
    рой вершин есть путь. Внешний цикл «для каж
    » даёт и разметку, и количество компонент.<
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Два р
ной запускаем DFS или BFS и присваиваем все
mp[v] = c</span>. Счётчик c увеличивается р
Время <b> $O(V + E)$ </b>, память  $O(V)$ . <b>Чере
каждое ребро делаем union(u, v), в конце чи
ту</b> и после каждого добавления спрашиваю
```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Острова

```
</div>
```

```
<p class="text-slate-300 text-sm">Т
ались серые (неисследованные) зоны? Прыгаеш
и компонент. Краска — это comp[v], прыжок ч
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
border border-emerald-500 shadow-md">
```

Аналогия 2: Группы друзей (DSU)

```
</div>
```

```
<p class="text-slate-300 text-sm">К
омпании в одну (union). Кто представитель к
биты на компании, и вопрос «в одной ли комп
рут, только если концы в разных компонентах
```

```
</div>
```

```
</div>
```

```
<h3 class="text-lg font-bold text-white mt-1">
```

```
<div class="overflow-x-auto">
```

```
<table class="w-full text-xs text-left">
```

```
<thead>
```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-blue-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-1"
      <p class="text-lg text-blue-300 italic"
      <p class="text-xl font-mono text-white"
перед.</p>
</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Учеба в вузе
    </div>
    <p class="text-slate-300 text-sm mb
еская сортировка – это расписание, которое
р>
  </div>
  <div class="bg-slate-900 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
      Ограничения: Циклы
    </div>
    <p class="text-slate-300 text-sm mb
получить опыт, нужна работа". Алгоритм выяв
  </div>
</div>

<div class="bg-rose-950/40 p-4 rounded-lg b
  <p class="text-rose-300 font-bold text-
  <p class="text-slate-300 text-xs leadin
    Обычный массив <span class="font-mono
ли в графе есть ориентированный цикл, код с
  <b>Правильно:</b> используем раскраск
  <br>• <span class="font-mono text-sla
  <br>• <span class="font-mono text-amb
  <br>• <span class="font-mono text-еме
  <br>Если во время обхода из серы верши

```

```

    }
    if (has_cycle) return false; // Граф содержит цикл!
    reverse(ans.begin(), ans.end()); // Разворачиваем р
    return true;
}</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "scc-kosaraju",
        title: "10. Графы. Компоненты сильной связности (SCC)",
        type: "html",
        description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
        category: "Графы. Продвинутые",
        content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
        <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
    </div>

    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
            <h3 class="text-xl font-bold text-emerald-400">10.1. Компоненты сильной связности (SCC)

            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-emerald-300 italic">Вспомогательная теорема:
                <div class="text-left font-mono text-sm">
                    <p><strong class="text-white">Компоненты сильной связности (SCC) ориентированного графа, в котором
                </div>
            </div>

            <p class="text-slate-300 text-sm mb-4">
                <strong>«Это просто цикл?» – Нет!</strong>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-500">
  <h3 class="text-xl font-bold text-rose-400">Мосты
  <div class="bg-slate-900 p-4 rounded-lg mb-4">
    <p class="text-lg text-rose-300 italic">Мосты
    <p class="text-xl font-mono text-white">
    <p class="text-xs text-slate-400 mt-1">
  </div>

  <div class="bg-slate-950 p-4 rounded-lg border-1 border-slate-500">
    <p class="text-amber-300 font-bold text">
    <p class="text-slate-300 text-xs leading">
      Проверка <span class="font-mono text-
тиграфе между u и v может быть 2 параллельны
азуют цикл длиной 2.<br>
      <b>Решение:</b> передавать в DFS не v
rald-300">p_edge_id</span> (индекс ребра, по
    </p>
  </div>

  <details class="bg-slate-900/40 rounded-lg border-1 border-slate-500">
    <summary class="p-4 font-bold text-slate-300">
      -b border-slate-700/50">
        Код поиска мостов C++ (Скрыто)
      </summary>
      <div class="p-5 text-sm text-slate-300">
        <pre class="bg-slate-950 p-4 rounded">
// Для мультиграфа с edge_id:
vector<pair<int, int>> adj[MAXN]; // {to, edge_id}
int tin[MAXN], low[MAXN], timer;

void dfs_bridges(int v, int p_edge_id = -1) {
  tin[v] = low[v] = ++timer;
  for (auto [to, edge_id] : adj[v]) {
    if (edge_id == p_edge_id) continue; // Пропускаем
    if (tin[to]) {
      low[v] = min(low[v], tin[to]); // Обратное
    } else {
      dfs_bridges(to, edge_id);
    }
  }
}

```

```

        <p class="text-xs text-slate-400">
    </div>
</div>
</div>

<p class="text-slate-300 text-sm">
    <b>Особый случай:</b> Корень дерева DFS
    моего ребёнка</b> в дереве DFS.
</p>

<details class="bg-slate-900/40 rounded-lg">
    <summary class="p-4 font-bold text-slate-300
    -b border-slate-700/50">
        Код поиска точек сочленения (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300">
        <pre class="bg-slate-950 p-4 rounded">
void dfs_cutpoints(int v, int p = -1) {
    tin[v] = low[v] = ++timer;
    int children = 0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (tin[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs_cutpoints(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1) {
                is_cutpoint[v] = true;
            }
            ++children;
        }
    }
    if (p == -1 && children > 1) {
        is_cutpoint[v] = true;
    }
}
</pre>
        </div>

```



```

<!-- Ориентированные графы -->
<div class="bg-slate-800 p-5 rounded-lg" style="background-color: #2d3748; color: white; padding: 10px;">
  <p class="text-indigo-300 font-bold">Графы</p>
  <ul class="list-disc list-inside" style="list-style-type: none; padding-left: 20px;">
    <li><b>Эйлеров цикл:</b> слабосвязный граф, в котором для каждой вершины in[v] == out[v].</li>
    <li><b>Эйлеров путь:</b> слабосвязный граф, в котором для каждой вершины in[v] - 1 ≤ out[v] ≤ in[v] + 1 (старт), у одной вершины in[v] - out[v] = 1 (финиш).</li>
  </ul>
</div>

<h3 class="text-lg font-bold text-white mt-5">Эйлеров цикл</h3>
<p class="text-slate-300 text-sm mb-3">Используется алгоритм поиска Эйлера (или рекурсивный переход). При возврате (на выходе) нужно проверить, что все ребра пройдены: in[v] == out[v] для всех вершин v.
<details class="bg-slate-900/40 rounded-lg p-4" style="background-color: #1a202c; color: white; padding: 10px; border: 1px solid #2d3748;">
  <summary class="p-4 font-bold text-slate-300">Код построения Эйлерова цикла C++</summary>
  <div class="p-5 text-sm text-slate-300">
    <pre class="bg-slate-950 p-4 rounded" style="background-color: #0f172a; color: white; padding: 10px;">
vector<unordered_multiset<int>>> adj;
vector<int> path;

void find_euler(int v) {
    while (!adj[v].empty()) {
        int u = *adj[v].begin();
        adj[v].erase(adj[v].find(u));
        adj[u].erase(adj[u].find(v)); // Убираем ребро
        find_euler(u);
    }
    path.push_back(v); // Добавляем в путь на выходе!
}</pre>
    </div>
  </div>

```

```

    { id: 4, x: 550, y: 150, label: "4" },
    { id: 5, x: 700, y: 50, label: "5" },
    { id: 6, x: 700, y: 250, label: "6" },
  ];

const edges: Edge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 },
  { u: 3, v: 4 }, // 3 and 4 are articulation points (3
  { u: 4, v: 5 },
  { u: 5, v: 6 },
  { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
  { length: Object.keys(nodes).length },
  () => [],
);

edges.forEach((e) => {
  adj[e.u].push(e.v);
  adj[e.v].push(e.u);
});

interface StepInfo {
  desc: string;
  visited: Set<number>;
  ap: Set<number>;
  currentNode: number | null;
  tin: number[];
  low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
  const [steps, setSteps] = useState<StepInfo[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt

```

```

        if (visited.has(to)) {
            currentLow[v] = Math.min(currentLow[v], currentLow[to]);
            recordStep(
                `Обратное ребро ${v}-${to}. Обновляем low[${v}],
                v,
            );
        } else {
            children++;
            dfs(to, v);
            currentLow[v] = Math.min(currentLow[v], currentLow[to]);

            recordStep(
                `Возврат в ${v} из ${to}. Обновляем low[${v}],
                v,
            );

            if (currentLow[to] >= currentTin[v] && p !== -1) {
                ap.add(v);
                recordStep(
                    `Найдена точка сочленения: вершина ${v},
                    v,
                );
            }
        }
    }

    if (p === -1 && children > 1) {
        ap.add(v);
        recordStep(
            `Корень дерева DFS ${v} имеет >1 детей. Это
            v,
        );
    } else if (p === -1) {
        recordStep(
            `Корень дерева DFS ${v} имеет <=1 ребенка, он
            v,
        );
    }
}

```

```

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Info className="w-5 h-5 text-rose-400" />
        Моделирование поиска точек сочленения
      </h3>

    <div className="flex items-center gap-2 bg-slate-
      <button
        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-
        text-white"
      >
        {isPlaying ? (
          <Pause className="w-4 h-4 ml-1" />
        ) : (
          <Play className="w-4 h-4 ml-1" />
        )}
        {isPlaying ? "Пауза " : "Старт "}
      </button>

      <button
        onClick={reset}
        aria-label="Сбросить обход"
        title="Сбросить обход"
        className="flex items-center justify-center
        ors"
      >
        <RotateCcw className="w-4 h-4" />
      </button>
    </div>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-3 g
    <div className="lg:col-span-2 relative h-96 lg:l
      <svg className="w-full h-full" viewBox="0 0 8

```

```

        : isAp
        ? "#f43f5e"
        : isVisited
        ? "#34d399"
        : "#334155";
const r = isAp ? 24 : 20;

return (
  <g key={node.id}>
    <circle
      cx={node.x}
      cy={node.y}
      r={r}
      fill={fill}
      stroke={stroke}
      strokeWidth="3"
      className="transition-all duration-1
    />
    <text
      x={node.x}
      y={node.y}
      dy=".3em"
      textAnchor="middle"
      fill="white"
      fontSize="14px"
      fontWeight="bold"
      className="select-none pointer-events
    >
      {node.label}
    </text>

    {isVisited && (
      <>
        <text
          x={node.x}
          y={node.y - r - 15}
          textAnchor="middle"
          fill="#94a3b8"

```

```

<div className="space-y-4 flex-1 overflow-y-auto">
  <div className="mb-4">
    <h5 className="text-xs text-slate-500 font-medium">Точка сочленения</h5>
    <div className="flex items-center gap-2">
      <div className="w-3 h-3 rounded-full bg-slate-500"></div>
      Точка сочленения
    </div>
    <div className="flex items-center gap-2">
      <div className="w-3 h-3 rounded-full bg-slate-500"></div>
      Посещена
    </div>
    <div className="flex items-center gap-2">
      <div className="w-3 h-3 rounded-full bg-slate-500"></div>
      Текущая
    </div>
    <div className="flex items-center gap-2">
      <div className="w-8 h-1 bg-rose-500 border-1px border-slate-500"></div>
      Мост
    </div>
  </div>

  <div>
    <h5 className="text-xs text-slate-500 font-medium">ЛОГ ДЕЙСТВИЙ:</h5>
    <div className="space-y-2">
      {steps
        .slice(0, currentStepIndex + 1)
        .reverse()
        .map((step, idx) => (
          <div
            key={idx}
            className={`text-xs p-2 rounded $
xt-slate-500`} `>
            >
            {step.desc}
          </div>
        ))}
    </div>
  </div>
</div>

```

```

    activeCodeLine: number;
}

/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
    и листать вручную.";

export default function BellmanFordViz() {
  const [hasCycle, setHasCycle] = useState(false);
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const runtime = useVizRuntime();
  const [isPlaying, setIsPlaying] = useState(false);

  const nodes: Node[] = [
    { id: 'S', x: 60, y: 150, name: 'База (S)' },
    { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
    { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
    { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
    { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
    { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
    { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
  ];

  const getEdges = (cycle: boolean): Edge[] => {
    const defaultEdges = [
      { u: 'S', v: 'A', w: 4 },
      { u: 'S', v: 'B', w: 5 },
      { u: 'B', v: 'C', w: -2 },
      { u: 'A', v: 'C', w: 1 },
      { u: 'C', v: 'D', w: 3 },
      { u: 'C', v: 'E', w: 4 },
      { u: 'D', v: 'F', w: 2 },
      { u: 'E', v: 'F', w: 1 },
    ];
    if (cycle) {
      return [...defaultEdges, { u: 'D', v: 'A', w: -6 }
    } else {

```

```

const vCount = nodes.length;
for (let iter = 1; iter < vCount; iter++) {
  let anyUpdate = false;
  for (const edge of edges) {
    const uDist = dist[edge.u];
    const vDist = dist[edge.v];

    let updated = false;
    if (uDist !== 999 && uDist + edge.w < vDist) {
      dist = { ...dist, [edge.v]: uDist + edge.w };
      prev = { ...prev, [edge.v]: edge.u };
      updated = true;
      anyUpdate = true;
    }

    simulationSteps.push({
      iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
      distances: { ...dist }, previous: { ...prev },
      log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}.
        ${updated ? 'Релаксация успешна! dist[${edge.v}] = ' + (uDist + edge.w) : ''}
      `,
      activeCodeLine: updated ? 6 : 5,
    });
  }

  if (!anyUpdate && iter < vCount - 1) {
    simulationSteps.push({
      iteration: iter, checkingEdge: null, distances: { ...dist },
      log: `Ранний выход после итерации ${iter}.`,
      activeCodeLine: 10,
    });
    break;
  }
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
  let cycleFound = false;

```



```

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveW = vizNumber(vars?.w);
const liveDist = vizRecord(vars?.dist);
const compilerLinked = liveI !== null || liveU !== null || liveV !== null || liveW !== null || liveDist !== null;
const currentStep: Step = compilerLinked
  ? {
      ...baseStep,
      iteration: liveI ?? 0,
      checkingEdge: liveU && liveV ? { u: liveU, v: liveV } : null,
      distances: Object.fromEntries(nodes.map((node) => [node, liveDist[node]])),
      log: `Компилятор: i=${liveI ?? "-"}, ребро ${liveU ?? "-"} - ${liveV ?? "-"}, вес ${liveW ?? "-"}, расстояние ${liveDist[liveU ?? "-"] ?? "-"}
    }
  : baseStep;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
    <div className="flex flex-wrap items-center justify-center gap-2">
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-blue-600 text-white rounded-lg">
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white">
            <p className="text-sm text-blue-300">Полный цикл
          </div>
        </div>
      </div>

      <div className="flex items-center gap-3">
        <div className="bg-slate-900 p-1 rounded-lg border border-slate-800">
          <button onClick={() => setHasCycle(false)} className="p-1 text-sm bg-blue-600 text-white" : 'text-slate-400'>
            <button onClick={() => setHasCycle(true)} className="p-1 text-sm bg-rose-600 text-white" : 'bg-slate-600 text-white'>
          </div>
        </div>
      </div>
    </div>
  )

```

```

<defs>
  <marker id="arrow-bf-normal" markerWidth=
<path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
  <marker id="arrow-bf-active" markerWidth=
<path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
  <marker id="arrow-bf-cycle" markerWidth=
path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
</defs>

```

```

{edges.map((edge, idx) => {
  const uNode = nodes.find((n) => n.id === u);
  const vNode = nodes.find((n) => n.id === v);
  const isChecking = currentStep.checkingEdges;
  const isNeg = edge.w < 0;
  const isCyclePart = hasCycle && ((edge.u === u && edge.v === 'D' || edge.v === v && edge.u === 'D'));

```

```

  let strokeColor = '#475569';
  let marker = 'url(#arrow-bf-normal)';
  if (isChecking) { strokeColor = '#f59e0b';
  else if (currentStep.hasNegativeCycle && isCyclePart) { strokeColor = '#e11d48';

```

```

  return (
    <g key={idx}>
      <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={isChecking && isCyclePart ? 4 : 2} strokeWidth={isChecking ? '4,4' : 'none'} />
      <circle cx={({uNode.x + vNode.x} / 2) cy={({uNode.y + vNode.y} / 2)} fill={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#64748b'} stroke={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#64748b'} />
      <text x={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)} text={isChecking ? 'Checking' : isNeg ? 'Neg' : 'Dist'} font-size="11" font-weight="bold" />
    </g>
  );
}}

```

```

{nodes.map((node) => {
  const dist = currentStep.distances[node.id];

```

```

        )
      }}}}
    </div>
  }}}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div>
      <h4 className="text-lg font-bold text-rose-950">
        Таблица расстояний
      </h4>
      <div className="grid grid-cols-4 gap-2 text-rose-950">
        {nodes.map(n => {
          const d = currentStep.distances[n.id];
          const isTgt = currentStep.checkingEdge === n.id;
          return (
            <div key={n.id} className={`p-2 rounded-lg
              ${isTgt ? 'border-slate-700/60 bg-slate-800/40' : ''}`}>
              <div className="text-xs text-slate-400">
                {n.id}
              </div>
              <div className={`font-mono font-bold text-rose-950`}
                >
                {d}
              </div>
            </div>
          );
        })}
      </div>
    </div>
  </div>
</div>

  {/* Код алгоритма */}
  <div className="w-full lg:w-1/2 bg-slate-900/60">
    <div>
      <h4 className="text-lg font-bold text-slate-400">
        Код алгоритма
      </h4>
      <div className="bg-slate-950/80 rounded-lg p-4">
        {codeLines.map(item => (
          <div key={item.line} className={`py-1.5 px-2
            ${item.line === currentStep.line ? 'bg-blue-900/80 text-amber-400' : ''}`}>
            {item.line} {item.code}
          </div>
        ))}
      </div>
    </div>
  </div>

```

```
0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],  
};
```

```
interface Step {  
  activeLine: number;  
  queue: number[];  
  visited: number[];  
  currentNode: number | null;  
  logs: string;  
}
```

```
const generateBfsSteps = (): Step[] => {  
  const steps: Step[] = [];  
  const queue: number[] = [];  
  const visited = new Set<number>();
```

```
  // Init
```

```
  steps.push({  
    activeLine: 1,  
    queue: [],  
    visited: [],  
    currentNode: null,  
    logs: "Инициализация: начинаем с корня (0)."  
  });
```

```
  queue.push(0);
```

```
  visited.add(0);
```

```
  steps.push({  
    activeLine: 2,  
    queue: [...queue],  
    visited: Array.from(visited),  
    currentNode: null,
```

```
    logs: "Добавляем стартовую вершину 0 в очередь и по  
  });
```

```
  while (queue.length > 0) {  
    steps.push({
```

```
      activeLine: 4,
```

```

    }
  } else {
    steps.push({
      activeLine: 6,
      queue: [...queue],
      visited: Array.from(visited),
      currentNode: curr,
      logs: `Соседей нет или все уже посещены.`
    });
  }
}

steps.push({
  activeLine: 12,
  queue: [],
  visited: Array.from(visited),
  currentNode: null,
  logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
  const [stepIdx, setStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const step = STEPS[stepIdx];

  const handleNext = useCallback(() => {
    setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
  }, []);

  // @ts-expect-error зарезервировано под кнопку «назад»
  const handlePrev = useCallback(() => {
    setStepIdx(prev => Math.max(prev - 1, 0));
  }, []);
}

```

```

        disabled={stepIdx >= STEPS.length - 1}
        className="p-2 text-emerald-400 hover:text-
ed:hover:bg-transparent"
        title="Шаг вперед"
      >
        <StepForward className="w-5 h-5" />
      </button>
    </div>
  </div>

<div className="grid grid-cols-1 lg:grid-cols-2 g
  {/* Визуал графа */}
  <div className="bg-slate-800 border border-slate
    50px]">
    <svg className="w-full h-full min-h-[350px]" *
      {/* Рёбра */}
      {EDGES.map((e, idx) => {
        const n1 = NODES.find(n => n.id === e.from)
        const n2 = NODES.find(n => n.id === e.to)
        const isVisited = step.visited.includes(n
        return (
          <line
            key={idx}
            x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
            className={`transition-all duration-5
          `} `}
        ) />
      )};
    }}}

    {/* Вершины */}
    {NODES.map(n => {
      const isCurrent = step.currentNode === n
      const isInQueue = step.queue.includes(n.id)
      const isVisited = step.visited.includes(n

      let fill = '#1e293b'; // slate-800
      let stroke = '#475569'; // slate-600

```

```

    });
  }}}
</svg>
</div>

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
  <div className="bg-slate-900 border border-slate-800 p-4">
    <div className="bg-slate-800 text-slate-400 p-4 font-mono font-medium">
      <span>bfs(graph, start)</span>
      <span className="text-blue-400">Mar {step}</span>
    </div>
    <div className="p-4">
      {[
        { line: 1, code: 'queue = Queue()' },
        { line: 2, code: 'queue.push(start); visited.add(start)' },
        { line: 3, code: '' },
        { line: 4, code: 'while not queue.isEmpty():' },
        { line: 5, code: '    node = queue.pop()' },
        { line: 6, code: '    for neighbor in graph[node]:' },
        { line: 7, code: '        if neighbor not in visited:' },
        { line: 8, code: '            visited.add(neighbor)' },
        { line: 9, code: '            queue.push(neighbor)' },
      ]}.map((cl) => {
        const isActive = step.activeLine === cl.line
        return (
          <div key={cl.line} className={`px-2 py-2 border border-l-2 border-blue-400 -ml-0.5` : 'text-blue-400'}>
            <span className="opacity-40 w-6 inline-block"> </span>
            <span className="whitespace-pre">{cl.code}</span>
          </div>
        )
      })
    </div>
  </div>
</div>

```

```
-----
const imageMap: Record<string, { src: string; alt: string;
  dijkstra: {
    src: dijkstraImg,
    alt: 'Декстер Дуглас и Фриказоид',
    caption: 'Декстер Дуглас и Фриказоид – волна вирусов',
    borderColor: 'border-blue-500/30',
    captionColor: 'text-blue-400',
  },
  'bellman-ford': {
    src: bellmanImg,
    alt: 'Бэтмен на машине Генри Форда',
    caption: 'Бэтмен на машине Форда – n-1 ночей патрулирует',
    borderColor: 'border-rose-500/30',
    captionColor: 'text-rose-400',
  },
  floyd: {
    src: floydImg,
    alt: 'Fortnite Все против Всех (Флойд–Уоршелл)',
    caption: 'Fortnite: Все против Всех – Флойд-Уоршелл',
    borderColor: 'border-emerald-500/30',
    captionColor: 'text-emerald-400',
  },
};

export default function ChapterImage({ vizType }: { vizType: string }) {
  const info = imageMap[vizType];
  if (!info) return null;

  return (
    <div>
      <div className={`rounded-2xl overflow-hidden border-2 border-${info.borderColor}>
        <img src={info.src} alt={info.alt} className="w-full h-full" />
      </div>
      <p className={`text-center text-xs mt-2 font-bold text-${info.captionColor}>
        {info.caption}
      </p>
    </div>
  );
}
```



```

        {index + 1} / {total}
      </span>
      <button
        disabled={!next}
        onClick={() => next && onGo(next.id)}
        title={next ? next.title : "Это последняя тема"}
        className="flex items-center gap-1 px-2.5 py-1.5
          bg-slate-700 enabled:hover:text-white disabled:opacity-50"
      >
        <span className="hidden sm:inline">Вперёд</span>
        <ChevronRight className="w-4 h-4" />
      </button>
    </div>
  );
}

return (
  <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" >
    {prev ? (
      <button
        onClick={() => onGo(prev.id)}
        className="group text-left p-4 rounded-xl bg-slate-700
          transition-all"
      >
        <div className="flex items-center gap-1.5 text-slate-400 mb-1">
          <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая
        </div>
        <div className="text-sm font-bold text-slate-400">
          {prev.title}
        </div>
      </button>
    ) : (
      <div className="hidden sm:block" />
    )}
  </nav>
  <div className="flex justify-between items-center gap-4" >
    {next && (
      <button
        onClick={() => onGo(next.id)}
        className="group text-right p-4 rounded-xl bg-slate-700
          transition-all"
      >
        <div className="flex items-center gap-1.5 text-slate-400 mb-1">
          Следующая <ChevronRight className="w-3.5 h-3.5" />
        </div>
        <div className="text-sm font-bold text-slate-400">
          {next.title}
        </div>
      </button>
    )}
  </div>
);

```

```
export const ChapterQuiz: React.FC<Props> = ({ chapterId }) => {
  const list = quizzes[chapterId];
  const [idx, setIdx] = useState(0);
  const [picked, setPicked] = useState<number | null>(null);
  const [score, setScore] = useState(0);
  const [finished, setFinished] = useState(false);

  // другая тема – другой блиц с нуля
  useEffect(() => {
    setIdx(0);
    setPicked(null);
    setScore(0);
    setFinished(false);
  }, [chapterId]);

  if (!list || list.length === 0) return null;

  const q = list[idx];
  const answered = picked !== null;
  const correct = answered && picked === q.correctIndex;

  const pick = (i: number) => {
    if (answered) return;
    setPicked(i);
    if (i === q.correctIndex) setScore((s) => s + 1);
  };

  const next = () => {
    if (idx + 1 < list.length) {
      setIdx(idx + 1);
      setPicked(null);
    } else {
      setFinished(true);
    }
  };

  const restart = () => {
    setIdx(0);
  };
};
```

```

    </div>
  ) : (
    <>
    <p className="text-sm font-semibold text-wh
    <div className="space-y-2">
      {q.options.map((option, i) => {
        const isRight = i === q.correctIndex;
        const chosen = picked === i;
        const tone = !answered
          ? "border-slate-600 bg-slate-800/70 t
          : isRight
            ? "border-emerald-500 bg-emerald-50
            : chosen
              ? "border-rose-500 bg-rose-500/15
              : "border-slate-700 bg-slate-800/
        return (
          <button
            key={i}
            type="button"
            onClick={() => pick(i)}
            disabled={answered}
            className={`w-full text-left flex i
            lons disabled:cursor-default ${tone}`}
          >
            {answered && isRight && <CheckCirc
            {answered && chosen && !isRight &&
            <span>{option}</span>
          </button>
        );
      })}
    </div>

    {answered && (
      <div role="status" className="mt-3 rounde
        <p className={`text-xs font-bold mb-1 $
          {correct ? "Верно" : "Неверно"}
        </p>
        <p className="text-[13px] leading-relax

```

```
-----
import { VizLiveStatus } from "../VizLiveStatus";
import { ChapterNav } from "../ChapterNav";
import { ChapterQuiz } from "../ChapterQuiz";
import { downloadChapterHtml } from "../utils/exportHtml";
import { useTicketLinks } from "../utils/ticketLinks";
import ChapterImage from "../ChapterImage";
import MnemonicCards from "../MnemonicCards";
```

```
interface Props {
  chapter: Chapter;
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
  onOpenSimulator: (vizId: string) => void;
  /** Перейти во вкладку компилятора – там уже подставлен код */
  onOpenCompiler?: () => void;
}
```

```
const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;
```

```
export const ChapterView: React.FC<Props> = ({ chapter,
  const contentRef = useRef<HTMLDivElement>(null);
  const [anchor, setAnchor] = useState<HintAnchor | null>(null);
  const hideTimer = useRef<number | null>(null);
  const [saving, setSaving] = useState<"idle" | "work">("idle");
```

```
  useTermHints(contentRef, [chapter.id]);
```

```
  /**
```

```
   * Упоминания «билет 15» становятся переходами на стр.
```

```
   * Разметка идёт ПОСЛЕ терминов: подсказки глоссария
```

```
   */
```

```
  useTicketLinks(contentRef, [chapter.id], { selfId: chapter.id });
```

```
  const cancelHide = useCallback(() => {
```

```
    if (hideTimer.current) {
```

```
      window.clearTimeout(hideTimer.current);
```

```

    };
    }, [chapter.id]));

const saveHtml = useCallback(async () => {
  setSaving("work");
  try {
    await downloadChapterHtml(chapter);
    setSaving("done");
    window.setTimeout(() => setSaving("idle"), 2500);
  } catch {
    setSaving("idle");
  }
}, [chapter]);

const open = useCallback(
  (el: HTMLElement) => {
    const termId = el.dataset.termId;
    if (!termId) return;
    cancelHide();
    setAnchor({ termId, rect: el.getBoundingClientRect() });
  },
  [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
  if (isTouch()) return;
  const el = (e.target as HTMLElement).closest<HTMLElement>(".chapter-link");
  if (el) open(el);
};

const handlePointerOut = (e: React.MouseEvent) => {
  if (isTouch()) return;
  const el = (e.target as HTMLElement).closest<HTMLElement>(".chapter-link");
  if (el) scheduleHide();
};

const handleClick = (e: React.MouseEvent) => {
  // ссылка на другой билет: переходим внутри приложения

```

```

    })
    {saving === "done" ? "Файл сохранён" : "Скачать"}
  </button>
  <button
    onClick={() => window.print()}
    title="Распечатать или сохранить эту тему в PDF-формате. Это версия документа, без шапки, содержания и панели комментариев"
    className="flex items-center gap-1.5 text-x-small text-slate-500 hover:text-white hover:border-indigo-500"
  >
    <Printer className="w-3.5 h-3.5" /> PDF этого документа
  </button>
  <a
    href={`?lite=1&topic=${encodeURIComponent(topic)}`}
    target="_blank"
    rel="noreferrer"
    title="Текстовая версия темы без навигации, комментариев и панели поиска"
    className="flex items-center gap-1.5 text-x-small text-slate-500 hover:text-white hover:border-sky-500"
  >
    <FileText className="w-3.5 h-3.5" /> Текстовая версия
  </a>
  <span className="text-[11px] text-slate-500">
    {chapter.id === "intro" && (
      <div className="my-8 print:hidden">

```

```

        <Terminal className="w-3.5 h-3.5" />
      </button>
    })
  </div>
</div>
{viz.hint && <p className="text-xs text-slate-500">
  <VizChapterContext.Provider value={chapter}>
    <VizLiveStatus demoTitle={viz.title} className="text-slate-500">
      <viz.Component />
    </VizChapterContext.Provider>
  </section>
}&#125;

  </* прочитал → посмотрел, как работает → проверить &#125;
  <ChapterQuiz chapterId={chapter.id} />

  <ChapterNav prev={prev} next={next} index={index} />
</article>

<TermPopover
  anchor={anchor}
  onClose={() => setAnchor(null)}
  onOpenSimulator={(id) => {
    setAnchor(null);
    onOpenSimulator(id);
  }}
  onCardEnter={cancelHide}
  onCardLeave={scheduleHide}
/>
</>
);
};

```

```

    ];

    const n = nodes.length;
    const adjList: { [key: number]: [number, number][] } = {};
    for (let i = 0; i < n; i++) adjList[i] = [];
    rawEdges.forEach(({ u, v, w }) => {
        adjList[u].push([v, w]);
        adjList[v].push([u, w]);
    });

    const [steps, setSteps] = useState<Step[]>([]);
    const [stepIdx, setStepIndex] = useState(0);
    const [isPlaying, setIsPlaying] = useState(false);
    const runtime = useVizRuntime();

    useEffect(() => {
        const simSteps: Step[] = [];

        if (activeTab === 'dexter') {
            const minWay = Array(n).fill(999);
            minWay[0] = 0;
            const seen = new Set<number>();

            simSteps.push({
                x: null, checkingEdge: null,
                minWay: [...minWay], seen: Array.from(seen),
                log: ' Декстер: min_way[0] = 0, остальные ∞. Г
            });

            for (let iter = 0; iter < n; iter++) {
                let x = -1;
                for (let i = 0; i < n; i++) {
                    if (!seen.has(i)) {
                        if (x === -1 || minWay[i] < minWay[x]) x = i;
                    }
                }
                if (x === -1 || minWay[x] === 999) break;
            }
        }
    }, [activeTab]);

```



```

simSteps.push({
  x: null, checkingEdge: null,
  minWay: [...minWay], seen: Array.from(seen), pr
  log: `  Прима: min_way[0] = 0 (стоимость отдель
});

for (let iter = 0; iter < n; iter++) {
  let x = -1;
  for (let i = 0; i < n; i++) {
    if (!seen.has(i)) {
      if (x === -1 || minWay[i] < minWay[x]) x = i;
    }
  }
  if (x === -1 || minWay[x] === 999) break;

  seen.add(x);
  if (pred[x] !== -1) {
    ostov.push([pred[x], x, minWay[x]]);
  }

  simSteps.push({
    x, checkingEdge: null,
    minWay: [...minWay], seen: Array.from(seen), pr
    log: `  Вершина x=${x} присоединена к остову
  });

  for (const [neighbor, w] of adjList[x]) {
    if (!seen.has(neighbor)) {
      simSteps.push({
        x, checkingEdge: { u: x, v: neighbor, w },
        minWay: [...minWay], seen: Array.from(seen), pr
        log: `  Проверяем укус ребра (${x} → ${ne
        [neighbor] === 999 ? '∞' : minWay[neighbor]
      });

      if (w < minWay[neighbor]) {
        minWay[neighbor] = w;

```

```

        seen.add(neighbor);
        minWay[neighbor] = minWay[x] + 1;
        queue.push(neighbor);

        simSteps.push({
            x, checkingEdge: { u: x, v: neighbor, w }
            minWay: [...minWay], seen: Array.from(seen),
            log: ` Волна накрывает соседа ${neighbor}
        });
    }
}
simSteps.push({
    x: null, checkingEdge: null,
    minWay: [...minWay], seen: Array.from(seen), qu
    log: ` BFS завершил работу. Рассчитаны расстоя
});
}

setSteps(simSteps);
setStepIndex(0);
setIsPlaying(false);
}, [activeTab]));

useEffect(() => {
    let timer: any = null;
    if (isPlaying && stepIdx < steps.length - 1) {
        timer = setTimeout(() => setStepIndex((prev) => p
    } else if (stepIdx >= steps.length - 1) {
        setIsPlaying(false);
    }
    return () => clearTimeout(timer);
}, [isPlaying, stepIdx, steps.length]);

const baseStep = steps[stepIdx] || null;
const vars = runtime?.variables;

// Compiler live vars

```

```

    </button>
    <button
      onClick={() => setActiveTab('prima')}
      className={`px-3 py-1.5 rounded-lg text-xs
        activeTab === 'prima' ? 'bg-amber-600 text-
      }`}
    >
      Прима (Следующий укус)
    </button>
    <button
      onClick={() => setActiveTab('bfs')}
      className={`px-3 py-1.5 rounded-lg text-xs
        activeTab === 'bfs' ? 'bg-sky-600 text-wh
      }`}
    >
      BFS (Ребро = 1)
    </button>
  </div>

  {/* Управление */}
  <div className="flex items-center gap-2">
    <button
      onClick={() => { setIsPlaying(false); setSt
      disabled={compilerLinked}
      className="bg-slate-700 hover:bg-slate-600
    >
      Сброс
    </button>
    <button
      onClick={() => setIsPlaying(!isPlaying)}
      disabled={compilerLinked}
      className={`px-3 py-1.5 rounded-lg text-xs
        isPlaying ? 'bg-amber-600 hover:bg-amber-
      }`}
    >
      {isPlaying ? 'Пауза' : 'Пуск'}
    </button>
    <button

```

```

        ([p, c]) => (p === e.u && c === e.v) ||
    );

    return (
      <g key={idx}>
        <line
          x1={uNode.x} y1={uNode.y}
          x2={vNode.x} y2={vNode.y}
          stroke={isChecking ? '#f59e0b' : in0str}
          strokeWidth={isChecking ? 4 : in0str}
        />
        <circle
          cx={({uNode.x + vNode.x) / 2} cy={({uNode.y + vNode.y) / 2}
          r="10" fill="#1e293b" stroke={isChecking ? '#f59e0b' : '#cb0000'}
        />
        <text
          x={({uNode.x + vNode.x) / 2} y={({uNode.y + vNode.y) / 2}
          fill={isChecking ? '#f59e0b' : '#cb0000'}
          fontSize="10" fontWeight="bold" text={e.w}
        />
      </g>
    );
  }
}
}

```

```

  { /* Вершины */ }
  { nodes.map((node) => {
    const isX = currentStep.x === node.id;
    const isSeen = currentStep.seen.includes(node.id);
    const val = currentStep.minWay[node.id];

    return (
      <g key={node.id}>
        <circle
          cx={node.x} cy={node.y}
          r={isX ? 20 : 16}
          fill={isX ? '#4f46e5' : isSeen ? '#f59e0b' : '#1e293b'}
        />
      </g>
    );
  }) }

```

```

        >
        <span className="text-amber-300 font-
        <span>
          [{adjList[u].map(([y, w]) => `(${y}
        </span>
      </div>
    );
  }}}
</div>
</div>

<div className="bg-slate-950 p-3 rounded-lg b
  <p><span className="text-indigo-400 font-bo
    '}}]</p>
  <p><span className="text-emerald-400 font-b
    {activeTab === 'prima' && currentStep.ostov
      <p><span className="text-amber-400 font-b
        ('', '')}}</p>
    }}
    {activeTab === 'bfs' && currentStep.queue &
      <p><span className="text-sky-400 font-bol
    }}
  </div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 40/113: src/components/ComplexityViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useContext, useEffect, useMemo, useState } from 'react';
import { ChevronLeft, ChevronRight, Play, Pause, Rotate } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime,

```

```
};
```

```
const NP_HARD_NOTE = "NP-трудные — всё, к чему сводится  
амильтонов цикл. Проблема остановки NP-трудна, но в NP."
```

```
/* ----- Сведение -----
```

```
const REDUCE_STEPS = [
  { title: "Дано: задача A, для которой быстрого алгоритма  
— то есть B NP-трудная.", boxes: ["A"], active: 0 },
  { title: "Берём вход x задачи A", note: "x — конкретный  
"x"]}, active: 1 },
  { title: "Строим  $f(x) = y$  за полиномиальное время", note: "Полиномиальное  
шагами из разных дизъюнктов, если литералы не противоречивы", active: 2 },
  { title: "Решаем B на входе y", note: "Гипотетический алгоритм B  
— но допустим, что он есть.", boxes: ["A", "x", "f"], active: 3 },
  { title: "Переводим ответ обратно:  $A(x) = B(f(x))$ ", note: "Если B решается  
⇔ формула выполнима. Полиномиальное время сохранилось", active: 4 },
  { title: "Вывод: B не проще A", note: "Раз  $A \leq_p B$  и A NP-полна.  
— полна.", boxes: ["A", "x", "f", "y", "B", "ans"], active: 5 },
];
```

```
const BOX_LABEL: Record<string, { t: string; c: string }> = {
  A: { t: "задача A\n(NP-полная)", c: "#f43f5e" },
  x: { t: "вход x\n|x| = n", c: "#64748b" },
  f: { t: "f: сведение\nO(poly(n))", c: "#f59e0b" },
  y: { t: "вход y\n|y| = poly(n)", c: "#64748b" },
  B: { t: "решатель B\n(гипотеза: быстрый)", c: "#6366f1" },
  ans: { t: "ответ\nA(x) = B(f(x))", c: "#10b981" },
};
```

```
/* ----- Верификатор -----
```

```
/** 3-SAT формула: 4 дизъюнкта по 3 литерала, переменные x, y, z
const CLAUSES: [number, number, number][] = [
  [1, 2, -3],
  [1, -2, 3],
  [-1, 2, 3],
  [-1, -2, -3],
];
```

```

    if (chapterId) emitVizDemo(chapterId, tab);
  }, [chapterId, tab]);

const maxStep = tab === "reduce" ? REDUCE_STEPS.length : 1;
useEffect(() => {
  setStep(0);
  setPlaying(false);
}, [tab]);

useEffect(() => {
  if (!playing) return;
  const t = window.setInterval(() => {
    setStep((s) => {
      if (s >= maxStep) {
        setPlaying(false);
        return s;
      }
      return s + 1;
    });
  }, 1400);
  return () => window.clearInterval(t);
}, [playing, maxStep]);

/* живые переменные компилятора */
const liveStep = vars ? (vizNumber(vars.step) ?? vizNumber(0)) : null;
useEffect(() => {
  if (liveStep === null || !Number.isFinite(liveStep))
    setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);

const liveCls = vars ? vizString(vars.cls) : null;
const liveK = vars ? vizNumber(vars.k) : null;
const liveCert = vars ? vizArray(vars.cert) ?? vizArray(0) : null;
const liveOk = vars ? (vars.ok === undefined ? null : vizBoolean(true)) : null;
const liveN = vars ? vizNumber(vars.n) : null;
const liveOps = vars ? (vizNumber(vars.ops) ?? vizNumber(0)) : null;

/** Копим точки (n, ops): код пользователя крутит цикл */

```

```

        nn += (lx[i] - ax) * (ly[i] - ay);
        dd += (lx[i] - ax) ** 2;
    }
    return dd === 0 ? slope : nn / dd;
  })();
  const cls =
    slopeTail > slope + 0.8 || slopeTail > 4.5
      ? { name: "экспонента / суперполином", color: "#f59e00",
        chi }
      : slope <= 1.25
      ? { name: "≈ линейный,  $O(n)$ ", color: "#10b981",
        chi }
      : slope <= 1.8
      ? { name: "≈  $O(n \log n)$ ", color: "#06b6d4", hinc }
      : slope <= 2.4
      ? { name: "≈ квадратичный,  $O(n^2)$ ", color: "#f59e00",
        chi }
      : slope <= 3.4
      ? { name: "≈ кубический,  $O(n^3)$ ", color: "#fb9238",
        chi }
      : { name: "полином высокой степени", color: "#fb9238",
        chi };
  return { slope, slopeTail, ...cls };
}, [samples]);

```

```

const reduceFrame = REDUCE_STEPS[Math.min(step, REDUCE_STEPS.length)];
const verifyIdx = Math.min(step, VERIFY_STEPS.length);
const verifyDone = verifyIdx >= VERIFY_STEPS.length;
const allOk = VERIFY_STEPS.every((s) => s.ok);

```

```

return (
  <div className="bg-slate-800/90 p-5 sm:p-6 rounded-1" >
    <div className="flex flex-wrap items-center justify-center" >
      <div className="flex items-center gap-3" >
        <div className="p-2.5 bg-purple-600 text-white" >
          <Boxes className="w-5 h-5" />
        </div>
        <div>
          <h3 className="text-lg sm:text-xl font-bold" >
            <p className="text-xs text-purple-300">P · I
          </p>
        </div>
      </div>
    </div>
  </div>
)

```



```

        <span key={i} className={"font-mono text-
ld-600/50 text-emerald-300" : "bg-slate-900
        x{i + 1} = {liveCert && typeof liveCe
        </span>
    )))
    {liveCert && <span className="font-mono t
le-950/40">cert из кода</span>}
</div>
<div className="space-y-2">
    {CLAUSES.map((cl, i) => {
        const checked = liveK !== null ? i < li
        const current = liveK !== null ? i ===
        const st = VERIFY_STEPS[i];
        return (
            <div
                key={i}
                className={"flex items-center gap-3
amber-950/20" : checked ? {st.ok ? "border-
slate-800 bg-slate-900/40"}}
            >
                <span className="text-[10px] font-b
                <span className="font-mono text-xs
span>
                <span className="font-mono text-[10
                ")}</span>
                <span className={"text-[11px] font-l
0"))}>
                    {checked ? {st.ok ? "✓ истинно" :
                    </span>
                </div>
            );
        })}
    </div>
    {(verifyDone || liveOk !== null) && (
        <div className={"mt-3 rounded-lg border p
/30 text-emerald-300" : "border-rose-600/40
        {(liveOk ?? allOk) ? "Сертификат принят
вергнут: хотя бы один дизъюнкт ложен."}
    )}

```

```

    }
    return <polyline key={g.id} points={pts
acity={0.55} />;
  }}}
  {samples.map((s) => {
    const x = 44 + (Math.log(Math.max(1, s.n)
    const y = 200 - (Math.log(Math.max(1, s.n)
    return <circle key={`_${s.n}`} cx={Math.r
strokeWidth={2} />;
  }}}
  {GROWTH_LINES.map((g, i) => (
    <g key={g.id + "l"}>
      <rect x={330 + (i % 3) * 58} y={16 + 1
      <text x={343 + (i % 3) * 58} y={24 + 1
    </g>
  ))}
</svg>
{samples.length === 0 && (
  <p className="text-xs text-slate-500 text
    Пусто. Запусти в компиляторе код с пере
t-mono text-purple-300">ops</code> – напри
  </p>
)}
</div>
<div className={"rounded-xl border p-3.5 " +
/60"}}>
  <p className="text-sm font-bold text-white
    {verdict ? `Похоже на ${verdict.name}` :
  </p>
  <p className="text-xs text-slate-400 leading
    {verdict
      ? `Наклон в лог-лог координатах ≈ ${ver
dict.slopeTail.toFixed(2)} – кривая загибае
      : "Как только в коде появятся n и ops,
e с n)."}
  </p>
</div>
</div>

```

```
interface CodeLine {
  line: string;
  highlight: string;
}

const CODE_LINES: CodeLine[] = [
  { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
  { line: "    visited[v] = true;", highlight: "normal" },
  { line: "    tin[v] = low[v] = timer++;", highlight: "normal" },
  { line: "    for (int to : adj[v]) {", highlight: "normal" },
  { line: "        if (to == p) continue;", highlight: "normal" },
  { line: "        if (visited[to]) {", highlight: "normal" },
  { line: "            low[v] = min(low[v], tin[to]);", highlight: "normal" },
  { line: "        } else {", highlight: "normal" },
  { line: "            dfs(to, v);", highlight: "normal" },
  { line: "            low[v] = min(low[v], low[to]);", highlight: "normal" },
  { line: "        }", highlight: "normal" },
  { line: "        if (low[to] > tin[v]) {", highlight: "normal" },
  { line: "            // ЭТО МОСТ!", highlight: "normal" },
  { line: "        }", highlight: "normal" },
  { line: "    }", highlight: "normal" },
  { line: "}", highlight: "normal" },
];
```

```
interface DfsStep {
  currentNode: number | null;
  visitedIndices: number[];
  tin: Record<number, number>;
  low: Record<number, number>;
  stack: number[];
  bridges: string[];
  log: string;
  activeEdge: [number, number] | null;
  backEdge: [number, number] | null;
  activeCodeLine: number[];
}
```

```

{
  currentNode: 6, visitedIndices: [0,1,6],
  tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1],
  activeEdge: [1,6], backEdge: null,
  log: "Из В идём в G. tin[G]=3, low[G]=3.",
  activeCodeLine: [4,5,7,10,11]
},
{
  currentNode: 1, visitedIndices: [0,1,6],
  tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1],
  activeEdge: null, backEdge: null,
  log: "У G нет соседей (кроме В). Возврат. low[G]=3 : ",
  activeCodeLine: [13,14,15]
},
{
  currentNode: 2, visitedIndices: [0,1,6,2],
  tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,2],
  activeEdge: [1,2], backEdge: null,
  log: "Из В идём в С. tin[C]=4, low[C]=4.",
  activeCodeLine: [4,5,7,10,11]
},
{
  currentNode: 2, visitedIndices: [0,1,6,2],
  tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, stack: [0,1,2],
  activeEdge: null, backEdge: [2,0],
  log: "Видим соседа А (уже посещён)! Обратное ребро ",
  activeCodeLine: [7,8,9]
},
{
  currentNode: 3, visitedIndices: [0,1,6,2,3],
  tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3:2},
  activeEdge: [2,3], backEdge: null,
  log: "Идём C→D. tin[D]=5, low[D]=5.",
  activeCodeLine: [4,5,7,10,11]
},
{
  currentNode: 4, visitedIndices: [0,1,6,2,3,4],
  tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2:1,3:2,4:3}
}

```



```

<button onClick={() => { setDfsAuto(false); s
  disabled={dfsIdx === DFS_STEPS.length-1}
  aria-label="Шаг вперёд" title="Шаг вперёд"
  className="p-1.5 hover:bg-slate-700 rounded
  <SkipForward className="h-3.5 w-3.5" />
</button>
<button onClick={() => { setDfsAuto(false); s
  aria-label="Сбросить DFS" title="Сбросить D
  className="p-1.5 hover:bg-slate-700 rounded
  <RefreshCw className="h-3.5 w-3.5" />
</button>
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-5 g
  { /* Graph visualization */}
  <div className="md:col-span-3 bg-slate-950 roun
    <div className="absolute inset-0 bg-[radial-g
      >
    <svg className="w-full h-[280px] z-10 relative
      {GRAPH_EDGES.map(e => {
        const u = GRAPH_NODES.find(n => n.id ===
        const v = GRAPH_NODES.find(n => n.id ===
        const bridge = step.bridges.includes(e.key
        const activeTree = step.activeEdge && (
          (step.activeEdge[0]===e.u && step.active
          (step.activeEdge[0]===e.v && step.active
        ));
        const activeBack = step.backEdge && (
          (step.backEdge[0]===e.u && step.backEdge
          (step.backEdge[0]===e.v && step.backEdge
        ));
        let sc = "#475569", sw = 2, dash = "none"
        if (bridge) { sc = "#eab308"; sw = 4; }
        else if (activeTree) { sc = "#10b981"; sw
        else if (activeBack) { sc = "#f43f5e"; sw
        else if (step.visitedIndices.includes(e.u
        return <line key={e.key} x1={u.x} y1={u.y

```

```

    </div>
  </div>

  {/** Code panel + stack */}
  <div className="md:col-span-2 space-y-3">
    {/** Code panel */}
    <div className="bg-slate-950 rounded-lg border" style={{borderColor: 'black', borderStyle: 'solid', borderWidth: 1px}}>
      <div className="bg-slate-900 px-3 py-1.5 text-slate-400">
        >
        <div className="p-2 overflow-x-auto">
          <pre className="text-[10px] font-mono leading-relaxed" style={{margin: 0, padding: 0}}>
            {CODE_LINES.map(({cl, i}) => {
              const hl = step.activeCodeLine.includes(cl)
              return <div key={i}
                className={`px-1 transition-all duration-200 ${
                  hl ? "bg-indigo-600/20 text-indigo-400" : "text-slate-400"
                }`}
                <span className="text-slate-600 mr-1 font-mono font-size-[0.8em] font-weight-normal">
                  {cl.line || ' '}
                </div>
              </div>
            )}}
          </pre>
        </div>
      </div>

    {/** Stack */}
    <div className="bg-slate-900/50 rounded-xl border" style={{borderColor: 'black', borderStyle: 'solid', borderWidth: 1px}}>
      <div className="text-[10px] font-bold text-slate-400" style={{padding: 5px 10px 5px 10px}}>
        <span>Стек DFS</span>
        <span className="text-indigo-400">{step.stack.length}</span>
      </div>
      <div className="flex flex-col-reverse gap-1" style={{padding: 5px 10px 5px 10px}}>
        {step.stack.length === 0
          ? <div className="text-[10px] text-slate-400">Пустой стек</div>
          : step.stack.map((id, i) => {
              <div key={id} className={`px-2 py-0.5 text-[0.8em] font-mono font-weight-normal`
                + (i === step.stack.length - 1 ? "bg-emerald-600/20 text-emerald-400" : "text-slate-400")}
            >
              {id}
            </div>
            )}
        </div>
      </div>
    </div>
  </div>

```



```

currentNode: string | null;
checkingEdge: { u: string; v: string } | null;
distances: { [key: string]: number };
visited: { [key: string]: boolean };
previous: { [key: string]: string | null };
log: string;
activeCodeLine: number;
}

/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
    и листать вручную.";

export default function DijkstraViz() {
  const nodes: Node[] = [
    { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
    { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
    { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
    { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
    { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
    { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
    { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
  ];

  const edges: Edge[] = [
    { u: 'A', v: 'B', w: 4 },
    { u: 'A', v: 'C', w: 2 },
    { u: 'C', v: 'B', w: 1 },
    { u: 'C', v: 'G', w: 3 },
    { u: 'C', v: 'E', w: 8 },
    { u: 'B', v: 'G', w: 4 },
    { u: 'B', v: 'D', w: 5 },
    { u: 'G', v: 'D', w: 1 },
    { u: 'G', v: 'E', w: 2 },
    { u: 'D', v: 'F', w: 3 },
    { u: 'E', v: 'F', w: 1 },
  ];
};

```

```

for (let iter = 0; iter < nodes.length; iter++) {
  let minD = 999;
  let u = null;
  for (const n of nodes) {
    if (!visited[n.id] && dist[n.id] < minD) {
      minD = dist[n.id];
      u = n.id;
    }
  }

  if (!u) break;

  visited[u] = true;
  simulationSteps.push({
    currentNode: u, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: `  Выбираем вершину ${u} с мин. непосещённой вершиной`,
    activeCodeLine: 5,
  });

  const neighbors = edges.filter(e => e.u === u);
  for (const edge of neighbors) {
    const v = edge.v;
    if (visited[v]) continue;

    simulationSteps.push({
      currentNode: u, checkingEdge: { u, v },
      distances: { ...dist }, visited: { ...visited },
      log: `  Смотрим соседа ${v}. Текущее расстояние: ${dist[u] + edge.w}.`,
      activeCodeLine: 8,
    });

    if (dist[u] + edge.w < dist[v]) {
      dist[v] = dist[u] + edge.w;
      prev[v] = u;
      simulationSteps.push({
        currentNode: u, checkingEdge: { u, v },

```

```

const liveDone = vizArray(vars?.done) ?? vizArray(var
const compilerLinked = liveU !== null || liveV !== nu
const currentStep: Step = compilerLinked
  ? {
    ...baseStep,
    currentNode: liveU,
    checkingEdge: liveU && liveV ? { u: liveU, v: l
    distances: Object.fromEntries(nodes.map((node) =>
    previous: Object.fromEntries(nodes.map((node) =>
    visited: Object.fromEntries(nodes.map((node) =>
    log: `Компилятор: u=${liveU ?? "-"}, v=${liveV
  }
  : baseStep;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl bor
    {/* Шапка симулятора */}
    <div className="flex flex-wrap items-center justifi
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-indigo-600 text-white
          <span className="text-2xl"> </span>
        </div>
      <div>
        <h3 className="text-2xl font-bold text-white
        <p className="text-sm text-indigo-300">Полн
      </div>
    </div>

    <div className="flex flex-wrap items-center jus
      <button
        onClick={() => { setIsPlaying(false); setCu
        disabled={compilerLinked}
        title={compilerLinked ? HINT_TEXT : "Вернуть
        className="flex items-center gap-1 bg-slate
        ver:bg-slate-700 text-white px-3 py-2 round
      >
        Сброс
      </button>

```

```
</div>
```

```
<svg className="w-full h-auto max-h-[500px]"
  <defs>
    <marker id="arrow-dh-normal" markerWidth=
      <path d="M0,0 L6,3 L0,6 z" fill="#47556"
    </marker>
    <marker id="arrow-dh-active" markerWidth=
      <path d="M0,0 L6,3 L0,6 z" fill="#f59e0"
    </marker>
    <marker id="arrow-dh-opt" markerWidth="6"
      <path d="M0,0 L6,3 L0,6 z" fill="#10b98"
    </marker>
  </defs>
  {/* Рёбра */}
  {edges.map((edge, idx) => {
    const uNode = nodes.find((n) => n.id ===
    const vNode = nodes.find((n) => n.id ===
    const isChecking =
      currentStep.checkingEdge &&
      (currentStep.checkingEdge.u === edge.u &
    }

    const isOptimal = currentStep.previous[ed

    let marker = 'url(#arrow-dh-normal)';
    if (isChecking) marker = 'url(#arrow-dh-a
    else if (isOptimal) marker = 'url(#arrow-d

    return (
      <g key={idx}>
        <line
          x1={uNode.x} y1={uNode.y}
          x2={vNode.x} y2={vNode.y}
          stroke={isChecking ? '#f59e0b' : is
          strokeWidth={isChecking ? 5 : isOpt
          markerEnd={marker}
          className="transition-all duration-
          strokeDasharray={isChecking ? '4,4'
```

```

        {dist === 999 ? '∞' : `d=${dist}`}
      </text>
      <text x={node.x} y={node.y + 35} fill={
        node.name.split(' ')[0]}
      </text>
    </g>
  );
  }}}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded-2xl">
  <p className="font-semibold text-amber-400">
    <p className="min-h-[40px] flex items-center">
  </p>
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/50 p-4 rounded-2xl">
  <h4 className="text-lg font-bold text-emerald-400">
    <div className="space-y-2.5 flex-1 overflow-y-hidden">
      {Object.keys(adjList).map((u) => {
        const isCurrentNode = currentStep.currentNode === u;
        return (
          <div key={u} className={`p-3 rounded-lg ${
            isCurrentNode ? 'bg-emerald-900/60 text-slate-300' : 'bg-slate-950/80 text-slate-400'
          }`} >
            <div className="flex items-center gap-2">
              <span className={`px-2 py-0.5 rounded`}>
                {u}
              </span>
              <span className="text-slate-400">→</span>
            </div>
            <div className="flex flex-wrap gap-2">
              {adjList[u].map((edge, idx) => {
                const isCheckingThis = currentStep.currentNode === edge.to;
                return (
                  <span key={idx} className={`px-2 py-0.5 rounded`}>

```

```

        return (
          <tr key={n.id} className={`border
            isCurr ? 'bg-indigo-950/60 fo
          }`}>
            <td className="py-2.5 px-3 flex
              <span className={`w-2 h-2 rou
                {n.name}
              </td>
            <td className="py-2.5 px-3 font
              {dist === 999 ? '∞' : dist}
            </td>
            <td className="py-2.5 px-3 text
              {prev || '-'}
            </td>
          </tr>
        );
      }}}}
    </tbody>
  </table>
</div>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60
  <div>
    <h4 className="text-lg font-bold text-slate
    <div className="bg-slate-950/80 rounded-lg p
      {codeLines.map((item) => {
        const isActive = currentStep.activeCode
        return (
          <div key={item.line} className={`py-1
            isActive ? 'bg-indigo-900/80 text
          }`}>
            <span className="text-slate-600 sel
            <span className="whitespace-pre fle
              {isActive && <span className="text-
            </span>}

```

```

export function DPVisualizer() {
  const [targetN, setTargetN] = useState<number>(5);
  const [steps, setSteps] = useState<DPStep[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const runtime = useVizRuntime();
  const [isPlaying, setIsPlaying] = useState<boolean>(f
  const [speed] = useState<number>(800);
  const [activeTab, setActiveTab] = useState<'table' |

  useEffect(() => {
    const generatedSteps: DPStep[] = [];
    let stepId = 0;
    const memoMap: { [key: number]: number } = {};

    function simulateFib(n: number): number {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'call',
        desc: `Вызов fibМемо(${n}). Проверяем наличие в
        codeLine: 2,
        memoState: { ...memoMap }
      });

      if (n in memoMap) {
        generatedSteps.push({
          id: stepId++,
          n,
          action: 'memo_hit',
          desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано
          codeLine: 2,
          memoState: { ...memoMap }
        });
        return memoMap[n];
      }

      if (n <= 2) {

```

```

        generatedSteps.push({
          id: stepId++,
          n: targetN,
          action: 'done',
          desc: `  Расчет завершен! fibMemo(${targetN}) = $
          codeLine: 6,
          memoState: { ...memoMap }
        });

        setSteps(generatedSteps);
        setCurrentStepIndex(0);
        setIsPlaying(false);
      }, [targetN]));

    useEffect(() => {
      let timer: any = null;
      if (isPlaying && currentStepIndex < steps.length - 1) {
        timer = setTimeout(() => {
          setCurrentStepIndex(prev => prev + 1);
        }, speed);
      } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
      }
      return () => clearTimeout(timer);
    }, [isPlaying, currentStepIndex, steps, speed]);

    const baseStep = steps[currentStepIndex] || null;
    const vars = runtime?.variables;
    const liveK = vizNumber(vars?.k) ?? vizNumber(vars?.n);
    const liveMemo = vizRecord(vars?.memo);
    const compilerLinked = liveK !== null || !!liveMemo;
    const memoState = liveMemo
      ? Object.fromEntries(Object.entries(liveMemo).flatMap(
          ([key, value]) => {
            const number = vizNumber(value);
            return number === null ? [] : [[Number(key), number]];
          }
        ))
      : baseStep?.memoState || {};
    const currentStep: DPStep | null = baseStep && compilerLinked

```



```
</div>
```

```
<div className="flex items-center bg-slate-950" >
  <button
    onClick={() => { setIsPlaying(false); set
    className="p-2 text-slate-400 hover:text-
    title="Сбросить"
  >
    <RotateCcw className="w-4 h-4" />
  </button>
  <button
    onClick={() => { setIsPlaying(false); set
    disabled={currentStepIndex === 0}
    className="p-2 text-slate-400 hover:text-
    title="Шаг назад"
  >
    <ChevronLeft className="w-5 h-5" />
  </button>
  <button
    onClick={() => setIsPlaying(!isPlaying)}
    className={`flex items-center gap-1.5 px-
      isPlaying
        ? 'bg-amber-500 text-slate-950 hover:
        : 'bg-emerald-600 text-white hover:
    }`
  >
    {isPlaying ? <Pause className="w-4 h-4" />
    {isPlaying ? 'Пауза' : 'Пуск'}}
  </button>
  <button
    onClick={() => { setIsPlaying(false); set
    disabled={currentStepIndex === steps.leng
    className="p-2 text-slate-400 hover:text-
    title="Шаг вперед"
  >
    <ChevronRight className="w-5 h-5" />
  </button>
</div>
```

```

        </div>
        <div className="text-xs font-mono bg-slate-500 text-emerald-400 p-2 rounded-2xl">
          Текущий N: <strong className="text-emerald-400">{currentN}</strong>
        </div>
      </div>
    )}
  </div>

  { /* Tabs */ }
  <div className="flex items-center gap-2 border-b border-slate-500 p-2">
    <button
      onClick={() => setActiveTab('table')}
      className={`flex items-center gap-2 px-6 py-3 border rounded-2xl
        ${activeTab === 'table' ? 'border-emerald-500 text-emerald-400 bg-slate-500' : 'border-transparent text-slate-400 hover:text-slate-300'}
      `}
    >
      <Eye className="w-4 h-4" />
      Степ-бай-степ состояние
    </button>
    <button
      onClick={() => setActiveTab('code')}
      className={`flex items-center gap-2 px-6 py-3 border rounded-2xl
        ${activeTab === 'code' ? 'border-emerald-500 text-emerald-400 bg-slate-500' : 'border-transparent text-slate-400 hover:text-slate-300'}
      `}
    >
      <Code className="w-4 h-4" />
      Интерактивный код
    </button>
  </div>

  { /* Tab Content */ }
  <div className="min-h-[320px] flex flex-col justify-start p-2">
    {activeTab === 'table' && (
      <div className="bg-slate-950 p-6 rounded-2xl">

```



```

const [c1Done, setC1Done] = useState(false);

useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, "walk");
}, [chapterId]);

const resetC1 = () => {
  setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли");
};

const clickC1 = (vId: number) => {
  if (c1Done && c1Path.length > 0) return;
  if (c1Path.length === 0) {
    setC1Path([vId]);
    setC1Msg("Старт! Кликай соседние вершины, чтобы п");
    return;
  }
  const last = c1Path[c1Path.length - 1];
  const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
  if (!edge) {
    setC1Msg("Нет ребра между этими вершинами! Выбери");
    return;
  }
  const key = ekey(last, vId);
  if (c1VisitedEdges.includes(key)) {
    setC1Msg("Это ребро уже пройдено! Эйлер не прощает");
    setC1Done(true); return;
  }
  const nextEdges = [...c1VisitedEdges, key];
  const nextPath = [...c1Path, vId];
  setC1VisitedEdges(nextEdges);
  setC1Path(nextPath);

  if (nextEdges.length === C1_EDGES.length) {
    if (nextPath[0] === vId) {
      setC1Msg("    ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф");
    } else {
      setC1Msg("`    ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С");
    }
  }
}

```



```

        fill={isLast ? "#6366f1" : visited ? "#6366f1" : "#fff"}
        stroke={isLast ? "#fff" : visited ? "#6366f1" : "#6366f1"}
        strokeWidth={2.5}
        className="transition-all duration-200"
      <text x={n.x} y={n.y + 4} textAnchor="middle">{n.label}</text>
      <text x={n.x} y={n.y - 16} textAnchor="middle">deg {deg[n.id]}{isOdd ? " · нечёт" : ""}</text>
    </g>
  </svg>
  <div className="absolute top-2 left-2 bg-slate-900 text-white p-2">
    Ребёп: {visitedKeys.length}/{C1_EDGES.length}
  </div>
</div>

<div className={`p-3 rounded-xl border text-sm ${
  done && visitedKeys.length === C1_EDGES.length
    ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
    : c1Done && !live
    ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
    : "bg-slate-900/50 border-slate-700 text-slate-50"
}`}>
  {msg}
</div>

{/* Критерий Эйлера */}
<div className={`p-3 rounded-xl border text-xs font-mono ${
  verdict ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50" : "bg-rose-950/40 border-rose-500/30 text-rose-50"
}`}>
  Критерий Эйлера (для связного графа): {verdict}
  <span className="block font-normal opacity-80 mt-1">
    Степени: {C1_NODES.map((n) => `${n.label}=${deg[n.id]}`).join(" ")}
    две нечётны. Алгоритм Иера строит маршрут за
  </span>
</div>

<div className="flex flex-wrap items-center gap-2">
  <span className="text-[10px] uppercase tracking-wider">
    /> из компилятора:</span>

```

```
-----  
/** Подсказка для кнопок, когда шаги ведёт компилятор.  
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →  
    ва листать вручную.";
```

```
export default function FloydViz() {  
  const [steps, setSteps] = useState<Step[]>([]);  
  const [currentStepIndex, setCurrentStepIndex] = useSt  
  const runtime = useVizRuntime();  
  // k/i/j/d приходят напрямую; номер строки произвольн  
  const [isPlaying, setIsPlaying] = useState(false);
```

```
  const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',  
  const nodesSvg = [  
    { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },  
    { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },  
    { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },  
    { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },  
    { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },  
    { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },  
    { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },  
  ];
```

```
  const initialMatrix = [  
    [0, 4, 999, 2, 999, 999, 999],  
    [999, 0, 3, 999, 999, 3, 999],  
    [999, 999, 0, 999, 2, 999, 999],  
    [999, 999, 999, 0, 4, 2, 999],  
    [999, 999, 999, 999, 0, 999, 1],  
    [999, 999, 999, 999, 999, 0, 5],  
    [999, 1, 999, 999, 999, 999, 0]  
  ];
```

```
  const adjList: { [key: number]: { v: number; w: numbe  
    0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],  
    1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],  
    2: [{ v: 4, w: 2 }],  
    3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],  
    4: [{ v: 6, w: 1 }],
```

```

        const viaVal = dist[i][k] + dist[k][j];
        if (dist[i][k] !== 999 && dist[k][j] !== 999) {
            dist[i][j] = viaVal;
            simulationSteps.push({
                k: i, j: k, matrix: dist.map(r => [...r]),
                log: `Улучшение пути [${i}]->[${j}] через [${k}]`,
                activeCodeLine: 7,
            });
        }
    }
}

simulationSteps.push({
    k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]),
    log: 'Все пары отработаны. Алгоритм Флойда завершен',
    activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []);

useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length - 1) {
        timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
    return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;

```



```

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border"
    <div className="flex flex-wrap items-center justify"
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-emerald-600 text-white"
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white"
            <p className="text-sm text-emerald-300">Полн
          </div>
        </div>

        <div className="flex items-center gap-2">
          <button onClick={() => { setIsPlaying(false);
            TEXT : "Вернуться к первому шагу"} className="
            d:cursor-not-allowed">Сброс</button>
          <button onClick={() => setIsPlaying(!isPlaying
            нную демонстрацию"} className={`px-4 py-2 ro
            wed ${isPlaying ? 'bg-amber-600' : 'bg-emera
          <button onClick={() => { setIsPlaying(false);
            ; }} disabled={compilerLinked} title={compil
            ld-600 text-white rounded-lg text-sm disabl
          </div>
        </div>

    <div className="flex flex-col lg:flex-row gap-6 m
      {/* Граф */}
    <div className="w-full lg:w-2/3 bg-indigo-950/20
      stify-center min-h-[400px]">
      <div className="absolute top-3 left-3 bg-indi
        ont-bold shadow-sm">
        {currentStep.k >= 0 && currentStep.k < 7 ?
      </div>

      <svg className="w-full h-auto max-h-[500px]"
        <defs>

```

```

        <g key={node.id} className="transition-
          <circle cx={node.x} cy={node.y} r={is
s] ? 4 : 2} className="transition-colors du
          <text x={node.x} y={node.y + 5} fill=
          <text x={node.x} y={node.y + 32} fill=
        </text>
      </g>
    );
  }}}
</svg>

```

```

<div className="w-full bg-slate-950/80 p-4 ro
  <p className="font-semibold text-amber-400 r
  <p className="min-h-[40px] flex items-cente
</div>
</div>

```

```

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
  <h4 className="text-lg font-bold text-emerald
  <div className="space-y-2 text-sm font-mono t
    {Object.keys(adjList).map((uStr) => {
      const u = parseInt(uStr);
      const isI = currentStep.i === u;
      const isK = currentStep.k === u;
      return {
        <div key={u} className={`flex flex-wrap
          isI ? 'bg-amber-900/40 border-ambe
          isK ? 'bg-indigo-900/40 border-ind
        }`}>
        <span className="font-bold text-whi
e-400">→</span>
        {adjList[u].map((edge, idx) => {
          const isUpd = currentStep.i === u
          const isVia = currentStep.k !== -
rentStep.j === edge.v);
          return {
            <span key={idx} className={`px-1

```

```

        </td>
        {row.map((val, j) => {
            const isUpd = currentStep.i === i
            const isVia = currentStep.k !== -1 &&
            tep.j === j));
            let bg = 'bg-slate-900/40 text-slate-600'
            if (isUpd) bg = 'bg-emerald-900/80 text-emerald-50'
            else if (isVia) bg = 'bg-indigo-950/60 text-indigo-50'
            else if (currentStep.i === -1 && currentStep.k === -1)
                bg = 'bg-indigo-950/60 text-indigo-50'
            else if (i === j) bg = 'bg-slate-900/40 text-slate-600'
            const shown = val === null ? '.' : val
            return (<td key={j} className={bg} >{shown})
        })}
    </tr>
  </tbody>
</table>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 rounded-lg p-4">
  <div>
    <h4 className="text-lg font-bold text-slate-600">Алгоритм
    <div className="bg-slate-950/80 rounded-lg p-4">
      {codeLines.map(item => (
        <div key={item.line} className={`${py-1.5} ${
          item.line === currentStep.i ? 'bg-blue-900/80 text-amber-50' :
          'bg-slate-950/60 text-slate-600'}`}>
          <span className="text-slate-600 w-5 text-right">{item.line}</span>
          <span className="whitespace-pre flex-grow-1">{item.value}</span>
        </div>
      ))}
    </div>
  </div>
</div>
</div>
</div>
</div>

```

```

}

/** Граф из 10 вершин и трёх компонент – ровно как в те
const NODES: Node[] = [
  { id: 0, label: "0", x: 90, y: 80 },
  { id: 1, label: "1", x: 175, y: 45 },
  { id: 2, label: "2", x: 205, y: 140 },
  { id: 3, label: "3", x: 105, y: 175 },
  { id: 4, label: "4", x: 330, y: 70 },
  { id: 5, label: "5", x: 420, y: 45 },
  { id: 6, label: "6", x: 380, y: 145 },
  { id: 7, label: "7", x: 130, y: 290 },
  { id: 8, label: "8", x: 230, y: 320 },
  { id: 9, label: "9", x: 330, y: 275 },
];

const EDGES: [number, number][] = [
  [0, 1], [1, 2], [2, 3], [0, 3], [1, 3], // компонента
  [4, 5], [5, 6], [4, 6],                // компонента
  [7, 8], [8, 9],                        // компонента
];

const ADJ: number[][] = NODES.map(() => []);
EDGES.forEach(([u, v]) => {
  ADJ[u].push(v);
  ADJ[v].push(u);
});
ADJ.forEach((a) => a.sort((x, y) => x - y));

/** Палитра компонент. */
const COLORS = ["#6366f1", "#10b981", "#f43f5e", "#f59e0b"];

interface DfsFrame {
  title: string;
  note: string;
  comp: number[];           // 0 = не покрашена, иначе н
  cur: number | null;       // текущая вершина
  frontier: number[];       // стек/очередь

```

```

    }
    push(null, [], с, "Готово: компонент = " + с, `Внешни
      вязности.`);
    return frames;
  }

```

```

interface DsuFrame {
  title: string;
  note: string;
  parent: number[];
  comps: number;
  edge: [number, number] | null;
  merged: boolean;
}

```

```

function buildDsuFrames(): DsuFrame[] {
  const frames: DsuFrame[] = [];
  const parent = NODES.map((n) => n.id);
  const find = (v: number): number => (parent[v] === v
    ? v : find(parent[v]));
  let comps = NODES.length;
  const push = (title: string, note: string, edge: [numbe
    frames.push({ title, note, parent: [...parent], comp
  push("Старт: каждая вершина сама по себе", `parent[v]
    .`, null, false);
  EDGES.forEach(([u, v], i) => {
    const ru = find(u);
    const rv = find(v);
    if (ru === rv) {
      push(`Ребро ${i}: (${u}, ${v}) — корни совпали`,
        `к, union ничего не меняет, comps остается ${com
      return;
    }
    parent[rv] = ru;
    comps -= 1;
    push(`Ребро ${i}: union(${u}, ${v})`, `find(${u}) =
      ент стало ${comps}`.`, [u, v], true);
  });
  push(`Готово: comps = ${comps}`, `Все ${EDGES.length}

```

```

    timer.current = window.setInterval(() => {
      setStep((s) => {
        if (s >= maxStep) {
          setPlaying(false);
          return s;
        }
        return s + 1;
      });
    }, 900);
    return () => {
      if (timer.current) window.clearInterval(timer.current);
    };
  }, [playing, maxStep]);

const goTo = useCallback((n: number) => setStep(Math.floor(n / 2)))

/* — переменные компилятора, которые рисуются НАПРЯМУЮ
const vars = runtime?.variables;
/** comp / color / used из кода пользователя → своя р-ция
const liveComp = useMemo(() => {
  const arr = vars ? (vizArray(vars.comp) ?? vizArray(0)) : []
  if (!arr) return null;
  return arr.slice(0, NODES.length).map((v) => (typeof v === 'number' ? v : 0))
}, [vars]);
/** parent / p из кода → DSU-множества по корням. */
const liveParent = useMemo(() => {
  const arr = vars ? (vizArray(vars.parent) ?? vizArray(0)) : []
  if (!arr || arr.length < NODES.length) return null;
  return arr.slice(0, NODES.length).map((v) => vizNumber(v))
}, [vars]);
const liveCur = vars ? (vizNumber(vars.v) ?? vizNumber(0)) : 0;
const liveComps = vars ? (vizNumber(vars.comps) ?? vizNumber(0)) : 0;
const liveFrontier = useMemo(() => {
  if (!vars) return null;
  const arr = vizArray(vars.stack) ?? vizArray(0);
  if (!arr) return null;
  return arr.map((v) => vizNumber(v)).filter((v): v is number => v !== 0)
}, [vars]);

```

```

return (
  <div className="bg-slate-800/90 p-5 sm:p-6 rounded-3xl">
    <div className="flex flex-wrap items-center justify-between">
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-emerald-600 text-white rounded-lg">
          <Network className="w-5 h-5" />
        </div>
        <div>
          <h3 className="text-lg sm:text-xl font-bold">
            <p className="text-xs text-emerald-300">10
          </p>
        </div>
      </div>
      <div className="flex items-center bg-slate-950 p-2">
        <button
          onClick={() => switchMode("dfs")}
          className={"flex items-center gap-1.5 px-3 py-2 bg-emerald-600 text-white shadow" : "text-slate-400 hover:bg-emerald-700"}
        >
          <Layers className="w-3.5 h-3.5" /> 06ход (DFS)
        </button>
        <button
          onClick={() => switchMode("dsu")}
          className={"flex items-center gap-1.5 px-3 py-2 bg-emerald-600 text-white shadow" : "text-slate-400 hover:bg-emerald-700"}
        >
          <Network className="w-3.5 h-3.5" /> DSU (path)
        </button>
      </div>
    </div>

    {/* панель управления */}
    <div className="flex flex-wrap items-center gap-2">
      <button onClick={() => { setStep(0); setPlaying(true); }}
        className="p-2 bg-slate-700 text-slate-300 hover:text-white transition"
      >
        <RotateCcw className="w-4 h-4" />
      </button>
      <button onClick={() => { setPlaying(false); goToStep(0); }}
        className="p-2 bg-slate-900 border border-slate-700 text-slate-300 hover:bg-slate-800"
      >
        <RotateCw className="w-4 h-4" />
      </button>
    </div>
  </div>
)

```

```

        />
    );
  }}}
  {NODES.map((n) => {
    const c = compOf(n.id);
    const color = c > 0 ? COLORS[(c - 1) % COLORS.length];
    const isCur = cur === n.id;
    const inFrontier = frontier.includes(n.id);
    return (
      <g key={n.id} onClick={() => goTo(step)} >
        {isCur && <circle cx={n.x} cy={n.y} r={13} fill={color} />}
        {inFrontier && !isCur && <circle cx={n.x} cy={n.y} r={13} fill="none" />}
      <circle cx={n.x} cy={n.y} r={13} fill={color} />
      <text x={n.x} y={n.y + 4} textAnchor="middle">{n.label}</text>
      {c > 0 && <text x={n.x} y={n.y - 19} textAnchor="middle">{c}</text>}
    </g>
  );
  }}}
  /* стек/очередь и DSU-таблица справа */
  <g>
    <text x={470} y={24} textAnchor="end" fill={color} font-weight="bold">
      {mode === "dfs" ? "стек DFS" : "parent[]"}
    </text>
    {mode === "dfs"
      ? frontier.slice(-8).map((v, i) => (
        <g key={i}>
          <rect x={440} y={32 + i * 20} width={20} height={10} fill={color} />
          <text x={453} y={44 + i * 20} textAnchor="left">{v}</text>
        </g>
      ))
      : (parentNow ?? []).slice(0, 10).map((pv, i) => (
        const color = COLORS[(setOf(parentNow).get(pv) - 1) % COLORS.length];
        return (
          <g key={i}>

```



```
    </div>
  );
};

export default GraphComponentsViz;
```

ФАЙЛ 47/113: src/components/GraphTraversalViz.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from '...';
import { Play, Pause, SkipForward, Undo, RefreshCw } from '...';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
  { id: 'A', label: 'A', x: 200, y: 40 },
  { id: 'B', label: 'B', x: 100, y: 120 },
  { id: 'C', label: 'C', x: 300, y: 120 },
  { id: 'D', label: 'D', x: 50, y: 200 },
  { id: 'E', label: 'E', x: 150, y: 200 },
  { id: 'F', label: 'F', x: 250, y: 200 },
  { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
  { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
  { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
  { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
  'A': ['B', 'C'],
  'B': ['A', 'D', 'E'],
```

```

    dfs(start);
    return steps;
}

function generateBFSSteps(start: string) {
    const steps: Action[] = [];
    const vis = new Set<string>();
    const q: string[] = [];

    q.push(start);
    vis.add(start);
    steps.push({ type: 'visit', node: start, desc: `Начинаем с узла ${start}.`, visitedNodes: [...Array.from(vis)] });

    while (q.length > 0) {
        const v = q[0];
        steps.push({ type: 'process', node: v, desc: `Обработка узла ${v}.`, visitedNodes: [...Array.from(vis)] });
        q.shift();

        for (const u of adj[v]) {
            if (!vis.has(u)) {
                vis.add(u);
                q.push(u);
                steps.push({ type: 'visit', node: u, desc: `Визит узла ${u}.`, visitedNodes: [...Array.from(vis)] });
            }
        }
    }

    steps.push({ type: 'backtrack', node: '', desc: `Завершение поиска.`, visitedNodes: [...Array.from(vis)] });

    return steps;
}

```

```

    }
  }, [autoPlay, steps.length]);

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
  <div className="space-y-4">
    <div className="flex bg-slate-900 border border-slate-800 p-4">
      <button onClick={() => setMode("dfs")}
        className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
        DFS (В глубину)
      </button>
      <button onClick={() => setMode("bfs")}
        className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
        BFS (В ширину / Волна)
      </button>
    </div>

    <div className="flex flex-col md:flex-row gap-4">
      <div /* SVG Graph View */
        className="bg-slate-900 border border-slate-800 p-4 flex flex-grow-1 min-h-[300px]">
        <svg className="w-full h-full max-w-[600px] max-h-[300px]">
          <!-- Edges -->
          {edges.map((e, i) => {
            const u = nodes.find(n => n.id === e.u);
            const v = nodes.find(n => n.id === e.v);
            // Check if edge is part of current step
            const edgeTraversed = isVisited(u.id) || isVisited(v.id);

            return (
              <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
                className={`stroke-slate-400 stroke-2 ${edgeTraversed ? 'stroke-slate-300'} `} />
            );
          })}
        </svg>
      </div>
    </div>
  </div>
);

```

```

                                className="stroke"
                                ))
                                </g>
                            )
                        }}}}
                    </svg>
                </div>

                {/* Structure View (Stack / Queue) */}
                <div className="bg-slate-900 border border-
r">
                    <div className={`absolute -top-3 left-
                                ${mode === 'dfs' ? 'bg-
ld-300 border border-emerald-500'}`}>
                        {mode === 'dfs' ? 'Стек вызовов' : ''}
                    </div>

                    <div className="flex flex-col gap-2 w-
                        {step.queueOrStack && step.queueOrStack.length > 0}
                        <div key={idx} className={`w-
                            ${mode === 'dfs' ? 'bg-slate-900' : 'bg-emerald-200'}
                            text-emerald-200`}
                            ${idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
                                ? `border border-emerald-500` : ''}
                            `}>
                                Узел: {item}
                                {idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
                                    ? <span className="ml-2">{step.queueOrStack[idx]}</span>
                                    : ''}
                                })
                            </div>
                        }}}}
                    {(!step.queueOrStack || step.queueOrStack.length === 0)
                        ? <div className="text-slate-500">Пусто</div>
                        : ''}
                    </div>
                </div>
            </div>

```

ФАЙЛ 48/113: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
import { useVizRuntime, vizArray, vizNumber } from '../utils';
```

```
interface Patient {
  id: string;
  name: string;
  emoji: string;
  symptom: string;
  priority: number; // 1 to 99 (Severity)
  animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
```

```
function siftUp(arr: Patient[]): Patient[] {
  const a = arr.map(x => ({ ...x }));
  let idx = a.length - 1;
  while (idx > 0) {
    const parent = Math.floor((idx - 1) / 2);
    if (a[parent].priority < a[idx].priority) {
      [a[parent], a[idx]] = [a[idx], a[parent]];
      idx = parent;
    } else break;
  }
  return a;
}
```

```
function siftDown(arr: Patient[]): Patient[] {
  const a = arr.map(x => ({ ...x }));
  const n = a.length;
  let idx = 0;
  while (true) {
```

```

let arr: Patient[] = [];
const start = [
  { name: 'Иван (Инфаркт)', symptom: ' Боли в сердце', emoji: '👤', priority: 100, animState: 'idle' },
  { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤', priority: 100, animState: 'idle' },
  { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤', priority: 100, animState: 'idle' },
  { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈', priority: 100, animState: 'idle' },
  { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤', priority: 100, animState: 'idle' },
  { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤', priority: 100, animState: 'idle' },
];
start.forEach((p, i) => {
  arr = heapInsert(arr, { id: `init${i}`, ...p, animState: 'idle' });
});
return arr;
})();

```

```

let uid = 300;

```

```

export const HeapViz: React.FC = () => {
  const [heap, setHeap] = useState<Patient[]>();
  const runtime = useVizRuntime();
  const liveHeap = vizArray(runtime?.variables?.h) ?? vizArray([]);
  const liveI = vizNumber(runtime?.variables?.i);
  const visualHeap: Patient[] = liveHeap
    ? liveHeap.map((value, index) => {
        const tuple = vizArray(value);
        const priority = vizNumber(tuple?.[0]) ?? vizNumber(0);
        return {
          id: `python-${index}`,
          name: tuple ? String(tuple[1] ?? value) : String(index),
          emoji: "👤",
          symptom: `индекс ${index}`,
          priority,
          animState: "idle",
        };
      })
    : heap;
  const [customName, setCustomName] = useState('');
  const [customPriority, setCustomPriority] = useState(0);

```

```

    setTimeout(() => {
      setHeap(nextHeap);
      setBusy(false);
    }, 500);
  }, [busy, heap]);

const handleRandomInsert = () => {
  const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
  const randomShift = Math.floor(Math.random() * 10);
  const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
  insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
  e.preventDefault();
  if (!customName.trim()) return;
  insertPatient(customName, customPriority);
  setCustomName('');
};

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
  if (busy || heap.length === 0) {
    setShake(true);
    addLog('⚠ Нет пациентов для экстренной госпитализации');
    setTimeout(() => setShake(false), 400);
    return;
  }
  setBusy(true);
  const topPatient = heap[0];
  setHealing(topPatient);

  addLog(`СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

  // Step 1: Root lights up as winner
  setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, status: 'winning' } : x));
}, [busy, heap]);

```

```

    const p = nodePos(idx);
    const l = 2 * idx + 1;
    const r = 2 * idx + 2;
    if (l < visualHeap.length) {
      const lp = nodePos(l);
      res.push(
        <line key={`el${idx}`}
          x1={`${p.x}%`} y1={`${p.y + 4}%`}
          x2={`${lp.x}%`} y2={`${lp.y - 4}%`}
          stroke="#475569" strokeWidth="2"
        />
      );
    }
    if (r < visualHeap.length) {
      const rp = nodePos(r);
      res.push(
        <line key={`er${idx}`}
          x1={`${p.x}%`} y1={`${p.y + 4}%`}
          x2={`${rp.x}%`} y2={`${rp.y - 4}%`}
          stroke="#475569" strokeWidth="2"
        />
      );
    }
    return res;
  });

  return (
    <div className="flex flex-col gap-6">
      {/* МНЕМОНИКА / ПРАВИЛО */}
      <div className="bg-emerald-950/40 border border-emerald-500">
        <div className="text-3xl"> </div>
        <div>
          <h3 className="font-black text-emerald-400 text-center">
            Типы Queue</h3>
          <p className="text-xs text-slate-300 mt-1 leading-relaxed">
            В реанимации не важно, кто пришел раньше, а кто
            остояние</b> (максимальный приоритет).
            Куча (Heap) автоматически перестраивает пац

```



```

        className="w-20 accent-emerald-500 cursor-pointer"
      />
    </div>
    <button
      type="submit"
      disabled={busy || !customName.trim() || heap.length === 0}
      className="bg-teal-600 hover:bg-teal-500 disabled:opacity-50 w-20 h-10 px-4 rounded-xl shadow-lg active:scale-95 transition-all"
    >
      Везти
    </button>
  </form>

  { /* Забрать самого тяжелого */ }
  <button
    onClick={extractMax}
    disabled={busy || heap.length === 0}
    className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-medium rounded-xl shadow-lg transition-all cursor-pointer flex items-center justify-center"
  >
    <span> В реанимацию (EXTRACT MAX)</span>
  </button>

  <button
    onClick={reset}
    className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-medium cursor-pointer border border-slate-700"
  >
    Сброс
  </button>
</div>

{ /* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */ }
<div className="grid grid-cols-1 lg:grid-cols-12 gap-10" >

  { /* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */ }
  <div className={`lg:col-span-8 bg-slate-900 rounded-2xl shadow-2xl`} >

```

```

        { /* Tooltip */ }
        <div className="absolute bottom-full right-0 shadow-xl">
          <div className="font-bold text-white">
            <div className="text-emerald-400">
              </div>
            </div>
          </div>
        </div>
      );
    }
  }
</div>

<div className="w-full h-3 bg-gradient-to-r from-emerald-400 to-teal-500">
</div>

{ /* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */ }
<div className="lg:col-span-4 bg-slate-900 rounded-2xl p-4">
  <div className="absolute top-4 left-4 text-white">
    Операционная койка
  </div>

  <div className="flex-1 flex flex-col justify-between p-4">
    {healingPatient ? (
      <div className="flex flex-col items-center justify-between h-100">
        <div className="relative w-full h-100">
          <span className="text-6xl block animate-pulse text-white">
            <span className="absolute -top-4 -right-4 text-emerald-400">
              <span className="absolute -bottom-2 -left-2 text-emerald-400">
                </div>
              </div>
            </div>
          <div>
            <h4 className="text-white font-black">
              <span>Пациент: {healingPatient.name}</span>
            </h4>
            <p className="text-emerald-400 text-xl">
              Состояние: Стабилизация ({healingPatient.status})
            </p>
            <div className="flex justify-center gap-4">

```

```
import React, { createContext, useContext, useEffect, use } from 'react';

/** Общие примитивы для мини-визуализаций подсказок. */
export const W = 260;
export const H = 130;

/**
 * Общий множитель скорости. Кадры были слишком короткими,
 * заметить, что именно переехало, и анимация читалась
 */
export const SPEED = 2.1;

/** Длительность переезда узлов. Должна быть заметно меньше 1000 мс. */
export const MOVE_MS = 800;
export const EASE = "cubic-bezier(.34,.01,.2,1)";
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;

/** Наведение на карточку ставит анимацию на паузу – можно увидеть, что именно переехало. */
export const DemoPauseContext = createContext(false);

export function useLoop(steps: number, ms = 900) {
  const [step, setStep] = useState(0);
  const paused = useContext(DemoPauseContext);
  const period = Math.round(ms * SPEED);

  useEffect(() => {
    if (steps <= 1 || paused) return;
    const t = setInterval(() => setStep((s) => (s + 1) % steps), period);
    return () => clearInterval(t);
  }, [steps, period, paused]);

  return step;
}
```

```

} > = ({ x, y, r = 13, fill, label, stroke }) => (
  <g style={{ transition: MOVE }}>
    <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}
      {label !== undefined && {
        <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
          {label}
        </text>
      }}
    </g>
  );

export const Edge: React.FC<{
  a: [number, number];
  b: [number, number];
  color?: string;
  width?: number;
  dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
  <line
    x1={a[0]}
    y1={a[1]}
    x2={b[0]}
    y2={b[1]}
    stroke={color}
    strokeWidth={width}
    strokeDasharray={dashed ? "4 3" : undefined}
    style={{ transition: MOVE }}
  />
);

```

ФАЙЛ 50/113: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../de

```

/** Компоненты связности: несколько «островов», которые

```
export const ComponentsDemo: React.FC = () => {
  const comps: [string, string][][] = [
    ["A", "B"], ["B", "C"],
    ["D", "E"],
    ["F", "G"], ["G", "H"], ["F", "H"],
  ];
  const pos: Record<string, [number, number]> = {
    A: [26, 40], B: [62, 92], C: [26, 92],
    D: [120, 40], E: [120, 92],
    F: [190, 34], G: [232, 78], H: [178, 96],
  };
  const colors = [C.active, C.warn, C.done];
  const step = useLoop(comps.length + 1, 900);
  const compOf = (v: string) => comps.findIndex((e) =>

  return (
    <Svg caption={step === 0 ? "Граф распался на острова"
      {comps.flatMap((edges, ci) =>
        edges.map(([u, v], i) => {
          <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]}
        })
      })}
    {Object.entries(pos).map(([k, [x, y]]) => {
      const ci = compOf(k);
      return <Node key={k} x={x} y={y} r={12} label={
    }}}
    </Svg>
  );
};
```

/** Что такое граф: вершины, рёбра, направление и вес.

```
export const GraphBasicsDemo: React.FC = () => {
  const step = useLoop(3, 1300);
  const pos: Record<string, [number, number]> = { A: [50, 40],
  const edges: [string, string, number][] = [["A", "B",
  return (
```

```
export const CoordCompressDemo: React.FC = () => {
  const raw = [1000, 5, 74000, 5, 300];
  const sorted = [5, 300, 1000, 74000];
  const step = useLoop(3, 1200);
  const boxW = 46;
  const startX = (260 - raw.length * (boxW + 4)) / 2;

  return (
    <Svg
      caption={
        step === 0
          ? "Исходные координаты – огромные и с дырами"
          : step === 1
            ? "Сортируем уникальные значения"
            : "Каждое число заменяем его номером → плотн
      }
    >
    {raw.map((v, i) => {
      <g key={i}>
        <rect x={startX + i * (boxW + 4)} y={20} width
        <text x={startX + i * (boxW + 4) + boxW / 2}
          {v}
        </text>
      </g>
    })}

    {step >= 1 &&
      sorted.map((v, i) => {
        <g key={v}>
          <rect x={30 + i * 52} y={62} width={46} hei
          <text x={53 + i * 52} y={77} textAnchor="mi
            {v}
          </text>
        </g>
      })}

    {step >= 2 &&
      raw.map((v, i) => {
```



```

        fill={wave.includes(k) ? C.active : visited.h
        stroke={wave.includes(k) ? "#a5b4fc" : undefi
    }}}
  </Svg>
);
};

export const DfsDemo: React.FC = () => {
  const path = ["A", "B", "D", "E", "F", "C"];
  const step = useLoop(path.length + 1, 750);
  const visited = path.slice(0, step);
  const head = visited[visited.length - 1];
  return (
    <Svg caption={head ? `Идём вглубь: ${visited.join("
      {GRAPH_EDGES.map(([u, v], i) => {
        const iu = visited.indexOf(u);
        const iv = visited.indexOf(v);
        const onPath = iu >= 0 && iv >= 0 && Math.abs(i
        return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_P
      }}}
      {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
        <Node key={k} x={x} y={y} label={k} fill={k ===
      )}}
    </Svg>
  );
};

```

```

export const ToposortDemo: React.FC = () => {
  const nodes = [
    { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
    { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
  ];
  const edges: [string, string][] = [
    ["труссы", "штаны"],
    ["труссы", "носки"],
    ["штаны", "боты"],
    ["носки", "боты"]
  ];
  const order = ["труссы", "носки", "штаны", "боты"];
  const step = useLoop(order.length + 1, 800);
  const placed = order.slice(0, step);
  const pos = Object.fromEntries(nodes.map((n) => [n.id, n.x, n.y]));
  return (

```



```

    </Svg>
  );
};

export const BridgeDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
  ];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""}>
      {edges.map(([u, v], i) => {
        const isBridge = u === "C" && v === "D";
        if (isBridge && cut) return null;
        return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />
      })}
      {Object.entries(pos).map(([k, [x, y]]) => (
        <Node key={k} x={x} y={y} label={k} fill={cut ? "white" : "black"} />
      ))}
    </Svg>
  );
};

```

```

export const ArticulationDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["A", "D"], ["B", "C"], ["B", "D"], ["C", "D"]
  ];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : ""}>
      {edges.map(([u, v], i) => (cut && (u === "C" || v === "C")) ? null : <Edge key={i} a={pos[u]} b={pos[v]} color="black" />)}
      {Object.entries(pos).map(([k, [x, y]]) => (
        <Node key={k} x={x} y={y} label={k} fill={cut ? "white" : "black"} />
      ))}
    </Svg>
  );
};

```



```

    </marker>
  </defs>
  <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХ
  <text x={206} y={44} fontSize={9} fill={C.dim}>ВХО
  {items.map((v, i) => (
    <g key={v} style={{ transition: "all .3s" }}>
      <rect x={30 + i * 54} y={56} width={46} height={20} fill={C.dim} />
      <text x={53 + i * 54} y={71} textAnchor="middle" />
    </g>
  ))}
  <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
</Svg>
);
};

export const DsuDemo: React.FC = () => {
  const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
  const step = useLoop(parents.length, 950);
  const p = parents[step];
  const pos: [number, number][] = [[50, 95], [105, 95], [160, 95], [215, 95]];
  const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#ff7f0e"];
  const root = (i: number): number => (p[i] === i ? i : root(p[i]));
  return (
    <Svg caption="find = «кто твой староста», union = 0" />
    {p.map((par, i) =>
      par !== i ? (
        <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}`}
          fill="none" stroke={rootColor[root(i)]} stroke-width={2} />
      ) : null
    )}
    {pos.map(([x, y], i) => <Node key={i} x={x} y={y} />)}
  </Svg>
);
};

const TRIE_NODES = [
  { id: 0, x: 130, y: 22, ch: "*", parent: null as number },

```



```

        </g>
      )})
    </Svg>
  );
};

export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const len = [1, 2, 4][step];
  const count = 8 - len + 1;
  return (
    <Svg caption={`Уровень j=${step}: готовые ответы для`
      {Array.from({ length: 8 }).map((_, i) => {
        <g key={i}>
          <rect x={12 + i * 30} y={18} width={26} height={10} />
          <text x={25 + i * 30} y={29} textAnchor="middle" />
        </g>
      })}
      {Array.from({ length: count }).map((_, i) => {
        <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 10} width={26} height={10} />
        fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.done} />
      })}
    </Svg>
  );
};

```

```

export const AmortizedDemo: React.FC = () => {
  const costs = [1, 1, 1, 8, 1, 1, 1, 1];
  const step = useLoop(costs.length + 1, 550);
  const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
  const avg = step ? (total / step).toFixed(2) : "-";
  return (
    <Svg caption={`Сделано ${step} операций, суммарно $`
      {costs.map((c, i) => {
        <rect key={i} x={14 + i * 30} y={104 - c * 10} width={26} height={10} />
        fill={i < step ? (c > 2 ? C.bad : C.done) : C.done} />
      })}
      <line x1={8} y1={106} x2={252} y2={106} stroke={C.done} />
    </Svg>
  );
};

```

```

        </g>
      )))
      <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
      <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
      <text x={130} y={106} textAnchor="middle" fontSize={10}
        π[i] – длина самого длинного «префикс = суффикс»
      </text>
    </Svg>
  );
};

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);
  return (
    <Svg caption={`Заполняем таблицу подзадач: готово $
      {Array.from({ length: rows }).map((_, r) =>
        Array.from({ length: cols }).map((_, c) => {
          const idx = r * cols + c;
          const filled = idx < step;
          return (
            <g key={idx}>
              <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
                fill={idx === step - 1 ? C.active : C.dim}
                {filled && (
                  <text x={57 + c * 38} y={36 + r * 32} textAnchor="middle"
                    fontSize={10} fontWeight="700" fill={C.dim}>
                    {idx}</text>
                )}
              </g>
            );
          )}
        )}
      </Svg>
    );
  );
}
<text x={130} y={122} textAnchor="middle" fontSize={10}
  каждая клетка считается один раз и переиспользуется
</text>
</Svg>
);
};

```

```

    "prefix-sum": PrefixSumDemo,
    "sparse-table": SparseTableDemo,
    "segment-tree": SegmentTreeDemo,
    bridge: BridgeDemo,
    articulation: ArticulationDemo,
    mst: MstDemo,
    amortized: AmortizedDemo,
    np: NpDemo,
    scc: SccDemo,
    "prefix-function": PrefixFunctionDemo,
    dp: DpDemo,
    floyd: FloydDemo,
    components: ComponentsDemo,
    graph: GraphBasicsDemo,
    "coord-compress": CoordCompressDemo,
    zig: ZigDemo,
    "zig-zig": ZigZigDemo,
    "zig-zag": ZigZagDemo,
  };

  /** Список работающих мини-демо — его проверяет `npm run`
  export const MINI_DEMO_KINDS = Object.keys(REGISTRY);

  export const MiniDemo: React.FC<{ kind: DemoKind }> = ({
    const [paused, setPaused] = useState(false);
    const Cmp = REGISTRY[kind];
    if (!Cmp) return null;
    return (
      <DemoPauseContext.Provider value={paused}>
        <div
          className="bg-slate-950/70 rounded-lg border bo
          onMouseEnter={() => setPaused(true)}
          onMouseLeave={() => setPaused(false)}
        >
          <Cmp />
        </div>
      </DemoPauseContext.Provider>
    );
  });

```

```

return createPortal(
  <div className="fixed inset-0 z-[110] flex items-end"
    <div className="absolute inset-0 bg-slate-950/80"
      <div className="relative w-full sm:max-w-5xl max-h-
        ded-2xl shadow-2xl flex flex-col">
        <div className="flex items-center gap-3 px-4 py-4"
          <div className="min-w-0 flex-1">
            <h3 className="font-bold text-white text-sm">
              {viz.hint && <p className="text-[11px] text-white">
                <VizLiveStatus demoTitle={viz.title} className="font-normal">
              </div>
            <button
              onClick={onClose}
              className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
              aria-label="Заккрыть симулятор"
            >
              <X className="w-5 h-5" />
            </button>
          </div>
        </div>
        <div className="overflow-y-auto overscroll-contain">
          {/* Тот же id главы: развёрнутая демонстрация */}
          <VizChapterContext.Provider value={vizId}>
            <Component />
          </VizChapterContext.Provider>
        </div>
      </div>
    </div>,
    document.body
  );
};

```



```

    left = Math.max(8, Math.min(left, vw - width - 8));

    const below = anchor.rect.top < height + GAP + 8;
    const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - GAP;

    setPos({ top: Math.max(8, Math.min(top, vh - height - 8)), left: left }, [anchor]);

useEffect(() => {
  if (!anchor) return;
  const onKey = (e: KeyboardEvent) => {
    if (e.key === "Escape") onClose();
  };
  window.addEventListener("keydown", onKey);
  window.addEventListener("scroll", onClose, true);
  return () => {
    window.removeEventListener("keydown", onKey);
    window.removeEventListener("scroll", onClose, true);
  };
}, [anchor, onClose]);

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
  <div
    ref={cardRef}
    onMouseEnter={onCardEnter}
    onMouseLeave={onCardLeave}
    className="fixed z-[100] term-popover"
    style={{
      top: pos ? pos.top : -9999,
      left: pos ? pos.left : -9999,
      width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
      visibility: pos ? "visible" : "hidden",
    }}
  >
    {viz}
  </div>
, document.body);

```

```
        <Play className="w-3.5 h-3.5" /> Показать
      </button>
    })
  </div>
</div>
</div>,
document.body
);
};
```

ФАЙЛ 56/113: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizSteps';

export function JohnsonViz() {
  const [step, setStep] = useState(0);
  const MAX_STEP = 7;
  useVizStepSync(step, setStep, MAX_STEP, (vars) => {
    const edge = `${vizString(vars.u) ?? ""}${vizString(vars.v) ?? ""}`;
    return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
  });

  const nodes = [
    { id: 'S', x: 200, y: 150, label: 'S' },
    { id: 'A', x: 200, y: 50, label: 'A' },
    { id: 'B', x: 320, y: 220, label: 'B' },
    { id: 'C', x: 80, y: 220, label: 'C' },
  ];

  const edges = [
    { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'AB', u: 'A', v: 'B', weight: 4, labelDx: 20, labelDy: 0 },
    { id: 'BC', u: 'B', v: 'C', weight: 5, labelDx: 0, labelDy: 0 },
    { id: 'AC', u: 'A', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
  ];
```

```

        if (step === 6) return "stroke-amber-400 stroke-emerald-500";
        if (step > 6) return "stroke-emerald-500";
    }

    if (step >= 4) return "stroke-slate-600"; // otherwise
    return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
    if (e.u === 'S') {
        if (step === 0 || step === 7) return "";
        if (step === 1) return "url(#arrow-amber)";
        if (step === 2) return "url(#arrow-pink)";
        return "url(#arrow-slate-dim)";
    }
    if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
    if (e.id === 'AB' && step > 4) return "url(#arrow-slate-dim)";
    if (e.id === 'BC' && step === 5) return "url(#arrow-pink)";
    if (e.id === 'BC' && step > 5) return "url(#arrow-slate-dim)";
    if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
    if (e.id === 'CA' && step > 6) return "url(#arrow-slate-dim)";

    if (step >= 4) return "url(#arrow-slate-dim)";
    return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
    if (e.u === 'S') return e.weight;

    if (step < 4) return e.weight;

    if (e.id === 'AB') {
        if (step === 4) return `${e.weight} + (${0})`;
        return e.newWeight;
    }
    if (e.id === 'BC') {
        if (step < 5) return e.weight;
        if (step === 5) return `${e.weight} + (${-2})`;
    }
};

```

```

        </div>
    );
    case 2: return (
        <div>
            <b>Шаг 2: Ищем потенциалы  $h(v)$ .</b>
            Запускаем медленный, но мощный алгоритм
            ных вершин:
            <span className="text-pink-300 ml-1">
        </div>
    );
    case 3: return (
        <div>
            <b>Шаг 3: Магическая формула.</b><b>Шаг 3.1: Пересчет ребра  $A \rightarrow B$ .</b>
            Всем ребрам графа назначаем новый вес.
            <div className="text-center font-monospace">
            Это повышает или понижает вес ребра
        </div>
    );
    case 4: return (
        <div>
            <b>Шаг 3.1: Пересчет ребра  $A \rightarrow B$ .</b>
            Старый вес: <span className="text-pink-300">
            Вычисляем:  $-2 + 0 - (-2) =$  <span className="text-pink-300">
        </div>
    );
    case 5: return (
        <div>
            <b>Шаг 3.2: Пересчет ребра  $B \rightarrow C$ .</b>
            Старый вес: 1. <br/>
            Вычисляем:  $1 + (-2) - (-1) =$  <span className="text-pink-300">
        </div>
    );
    case 6: return (
        <div>
            <b>Шаг 3.3: Пересчет ребра  $C \rightarrow A$ .</b>
            Старый вес: 2. <br/>
            Вычисляем:  $2 + (-1) - 0 =$  <span className="text-pink-300">
        </div>
    );

```

```

    tart-reverse">
        <path d="M 0 0 L 10 5 L 10 10" />
    </marker>
    <marker id="arrow-emerald" />
    o-start-reverse">
        <path d="M 0 0 L 10 5 L 10 10" />
    </marker>
</defs>

{/* Edges */}
{edges.map((e, i) => {
    const u = nodes.find(n => n.id === e.u);
    const v = nodes.find(n => n.id === e.v);

    const isS = e.u === 'S';
    if (isS && (step === 0 || step === 1)) return;

    const midX = (u.x + v.x) / 2;
    const midY = (u.y + v.y) / 2;

    const edgeColorClass = getEdgeColorClass(e);
    const textHigh = isEdgeTextHigh(e);
    const textEmer = isEdgeTextEmer(e);

    return (
        <g key={i}>
            <line
                x1={u.x} y1={u.y}
                x2={v.x} y2={v.y}
                className={`stroke-${edgeColorClass}`}
                markerEnd={getMarkerId(e)}
            />

            <text
                x={midX} y={midY}
                textAnchor="middle"
                dominantBaseline="bottom"
                className={`text-${textHigh} ${textEmer}`}
                style={{textHigh}}
            />
        </g>
    );
})}

```

```

    ))
  </g>
)
}}
</svg>
</div>

<div className="w-full md:w-[320px] flex
  {/* Log Box */}
  <div className="bg-[#0f172a] p-5 rounded
l justify-center leading-relaxed text-sm text-white
    {getDesc()}
  </div>

  {/* Controls */}
  <div className="flex bg-slate-900 border
    <button onClick={() => setStep(1)}
      className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Сбросить"
      <RotateCcw strokeWidth={3}
    </button>
    <button onClick={() => setStep(1)}
      className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Вар на
      <Undo strokeWidth={3} size={24}
    </button>
    <button onClick={() => setStep(
      className={`flex-1 font-bold text-white
        ${step === MAX_STEP
          ? 'bg-emerald-600/50'
          : 'bg-indigo-600 hover:bg-indigo-500'}
        {step === MAX_STEP ? "Готово" : "В процессе"}
        {step !== MAX_STEP && <Skip
    </button>
  </div>

  {/* Progress Dots */}
  <div className="flex justify-center

```

```

-----
const initialNodes: GNode[] = [
  { id: 0, x: 150, y: 100, label: "0" },
  { id: 1, x: 300, y: 100, label: "1" },
  { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
  { id: 3, x: 450, y: 160, label: "3" },
  { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 }, // Connection edge
  { u: 3, v: 4 },
  { u: 4, v: 3 },
];

interface AlgoStep {
  phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
  desc: string;
  visited: Set<number>;
  currentNode: number | null;
  order: number[];
  transposed: boolean;
  sccMap: Record<number, number>; // node -> sccId
  currentSccId: number;
  activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
  const [steps, setSteps] = useState<AlgoStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
  const v = vizNumber(vars.v);
  const orderLength = vizArray(vars.order)?.length;
  if (v === null && orderLength === undefined) return null;
  const candidates = steps
    .map((step, index) => ({ step, index }));

```

```

        false,
        -1,
    );

    // Adj list
    const adj: number[][] = Array.from({ length: 5 }, () => []);
    initialEdges.forEach((e) => adj[e.u].push(e.v));

    // DFS 1: post-order list
    const dfs1 = (u: number) => {
        visited.add(u);
        recordStep(
            "dfs1",
            `DFS 1: зашли в вершину ${u}, помечаем посещение`,
            u,
            false,
            -1,
        );

        for (const to of adj[u]) {
            if (!visited.has(to)) {
                dfs1(to);
            }
        }

        order.push(u);
        recordStep(
            "dfs1",
            `DFS 1: вершина ${u} полностью обработана. Записываем в порядок`,
            u,
            false,
            -1,
        );
    };

    // Run DFS1 on all unvisited
    for (let i = 0; i < 5; i++) {
        if (!visited.has(i)) {
            dfs1(i);
        }
    }

```



```

    };

    // Run DFS2 using the order (reversed from back to front)
    for (let i = order.length - 1; i >= 0; i--) {
        const u = order[i];
        if (!visited.has(u)) {
            sccId++;
            recordStep(
                "dfs2",
                `DFS 2: берем следующую непосещенную из стека`,
                u,
                true,
                sccId,
            );
            dfs2(u, sccId);
        } else {
            recordStep(
                "dfs2",
                `DFS 2: вершина ${u} из стека уже покрашена в`,
                u,
                true,
                sccId,
            );
        }
    }

    recordStep(
        "finished",
        `Алгоритм успешно завершен! Найдено ${sccId} компонент`,
        null,
        true,
        sccId,
    );

    setSteps(generatedSteps);
    setCurrentStepIdx(0);
}, []);

```

```

<div className="bg-slate-800 p-4 border-b border-
  <h3 className="font-bold text-white flex items-
    <Layers className="w-5 h-5 text-emerald-400"
      Визуализатор Алгоритма Косарайю (Поиск SCC)
    </h3>

  <div className="flex items-center gap-2 bg-slate-
    <button
      onClick={togglePlay}
      className="flex items-center gap-2 px-3 py-
        d-500 text-white"
    >
      {isPlaying ? {
        <Pause className="w-4 h-4 ml-1" />
      } : {
        <Play className="w-4 h-4 ml-1" />
      }}
      {isPlaying ? "Пауза " : "Старт "}
    </button>

    <button
      onClick={reset}
      aria-label="Сбросить обход"
      title="Сбросить обход"
      className="flex items-center justify-center
        ors"
    >
      <RotateCcw className="w-4 h-4" />
    </button>
  </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12
  {/* Playfield SVG */}
  <div className="lg:col-span-8 bg-[#0B1120] rela
    <svg className="w-full h-full max-w-[650px]"
      {/* Markers for directed arrows */}
    <defs>

```

```

const v = initialNodes.find((n) => n.id ==

// If transposed, draw reversed coordinates
const x1 = step.transposed ? v.x : u.x;
const y1 = step.transposed ? v.y : u.y;
const x2 = step.transposed ? u.x : v.x;
const y2 = step.transposed ? u.y : v.y;

const isSccEdge =
  step.sccMap[edge.u] &&
  step.sccMap[edge.v] &&
  step.sccMap[edge.u] === step.sccMap[edge.v];
let stroke = "#475569";
let markerId = "arrow";

if (step.transposed) {
  stroke = isSccEdge ? "#818cf8" : "#4f4682";
  markerId = "arrow-transposed";
} else {
  if (
    step.currentNode === edge.u ||
    step.currentNode === edge.v
  ) {
    stroke = "#10b981";
    markerId = "arrow-active";
  }
}

// Adjust slightly for bidirectional link
const isBidi =
  (edge.u === 3 && edge.v === 4) ||
  (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
  <line
    key={`edge-${idx}`}
    x1={x1}
    x2={x2}
    y1={y1}
    y2={y2}
    stroke={stroke}
    markerId={markerId}
  />
)

```

```

        x={node.x}
        y={node.y}
        dy=".3em"
        textAnchor="middle"
        fill="white"
        fontSize="13px"
        fontWeight="bold"
        className="select-none"
    >
        {node.label}
    </text>

    {sccId && (
        <text
            x={node.x}
            y={node.y - 28}
            textAnchor="middle"
            fill="#10b981"
            fontSize="10px"
            fontWeight="bold"
        >
            SCC #{sccId}
        </text>
    )}
    </g>
    );
    }}}
</svg>

{step.transposed && (
    <div className="absolute top-4 left-4 bg-in
    1 rounded">
        Граф транспонирован (Ребра обратные)
    </div>
    )}
</div>

{/* Logs & Stacks */}

```

ЛОГ ОПЕРАЦИЙ:

```

</span>
<div className="space-y-1.5 max-h-[150px]"
  {steps
    .slice(0, currentStepIdx + 1)
    .reverse()
    .slice(0, 5)
    .map(({s, idx}) => {
      <div
        key={idx}
        className={`text-[11px] p-1.5 rounded`
      >
        {s.desc}
      </div>
    })}
  </div>
</div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200"
  <span>
    Шаг {currentStepIdx + 1} из {steps.length}
  </span>
  <div className="flex gap-1.5">
    <button
      disabled={currentStepIdx === 0}
      onClick={() => setCurrentStepIdx((prev) => prev - 1)}
      className="bg-slate-900 border border-slate-700 text-white px-2 py-1"
    >
      Назад
    </button>
    <button
      disabled={currentStepIdx >= steps.length - 1}
      onClick={() => setCurrentStepIdx((prev) => prev + 1)}
      className="bg-slate-900 border border-slate-700 text-white px-2 py-1"
    >
      Вперед
    </button>
  </div>
</div>

```

```

    title: string;
    description: string;
    type: 'info' | 'success' | 'warning' | 'error';
}

// 9 Vertices layout
const initialVertices: Vertex[] = [
  { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
  { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
  { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
  { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
  { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
  { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
  { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
  { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
  { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
  { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
  { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
  { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
  { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
  { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
  { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
  { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
  { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
  { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
  { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
  { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
  { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
  const [activeAlgo, setActiveAlgo] = useState<'kruskal'>
  const [vertices, setVertices] = useState<Vertex[]>(in
  const [edges, setEdges] = useState<Edge[]>(initialEdge

```

```
title: 'Шаг 2: Берем второе ребро D-E (вес 3)',  
description: 'Оно соединяет две другие бесцветные вершины',  
type: 'info'  
}, ...prev]));  
  
// Step 4: Include D-E  
() => {  
  setVertices(prev => prev.map(v => v.id === 3 || v.id === 4 ? {color: 'blue'} : v), ...prev);  
  setEdges(prev => prev.map(e => e.id === '3-4' ? {weight: 3} : e), ...prev);  
  setLogs(prev => [{  
    title: 'Успех: Ребро D-E в остове',  
    description: 'Вершины D и E окрашены в синий цвет',  
    type: 'success'  
}], ...prev));  
  
// Step 5: Inspect B-C (4)  
() => {  
  setEdges(prev => prev.map(e => e.id === '1-2' ? {weight: 4} : e), ...prev);  
  setLogs(prev => [{  
    title: 'Шаг 3: Берем ребро B-C (вес 4)',  
    description: 'Оно соединяет красную вершину В и синюю вершину С',  
    type: 'info'  
}], ...prev));  
  
// Step 6: Include B-C  
() => {  
  setVertices(prev => prev.map(v => v.id === 2 ? {color: 'red'} : v), ...prev);  
  setEdges(prev => prev.map(e => e.id === '1-2' ? {weight: 4} : e), ...prev);  
  setLogs(prev => [{  
    title: 'Успех: Ребро B-C в остове',  
    description: 'Вершина C перекрашена в красный цвет',  
    type: 'success'  
}], ...prev));  
  
// Step 7: Inspect A-D (5)  
() => {  
  setEdges(prev => prev.map(e => e.id === '0-3' ? {weight: 5} : e), ...prev);  
  setLogs(prev => [{
```

```

        title: 'Шаг 6: Берем ребро E-F (вес 7)',
        description: 'Оно соединяет фиолетовую вершину I
        type: 'info'
    }, ...prev]));
},
// Step 12: Include E-F
() => {
    setVertices(prev => prev.map(v => v.id === 5 ? {
    setEdges(prev => prev.map(e => e.id === '4-5' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро E-F в осто́ве',
        description: 'Вершина F окрашена в фиолетовый ц
        type: 'success'
    }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Шаг 7: Берем ребро D-G (вес 8)',
        description: 'Оно соединяет фиолетовую вершину I
        type: 'info'
    }, ...prev]));
},
// Step 14: Include D-G
() => {
    setVertices(prev => prev.map(v => v.id === 6 ? {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро D-G в осто́ве',
        description: 'Вершина G окрашена в фиолетовый.'
        type: 'success'
    }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{

```



```

        description: 'Оба конца E и H уже фиолетовые. С
        type: 'warning'
    }, ...prev]);
},
// Step 20: Reject E-H
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setLogs(prev => [{
        title: 'Отказ: Цикл обнаружен!',
        description: 'Выбрасываем ребро E-H.',
        type: 'error'
    }, ...prev]));
},
// Step 21: Inspect H-I (12)
() => {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: 'Шаг 11: Берем ребро H-I (вес 12)',
        description: 'Соединяет фиолетовую H и последнюю
        type: 'info'
    }, ...prev]));
},
// Step 22: Include H-I
() => {
    setVertices(prev => prev.map(v => v.id === 8 ? {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: 'Успех: MST построено Краскалом!',
        description: 'Все 9 вершин окрашены в единый фи
            + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
        type: 'success'
    }, ...prev]));
},
];

// --- PRIM STEPS ---
const primSteps = [
    // Step 1: Start at E

```

```

        type: 'info'
    }, ...prev]);
},
// Step 5: Include A-D, add A's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 0 ? {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
        eap' } : e));
    setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
    setLogs(prev => [{
        title: 'Успех: Плесень поглотила вершину A',
        description: 'Новое ребро A-B(2) добавлено в кучу',
        type: 'success'
    }, ...prev]);
},
// Step 6: Pop A-B (2)
() => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
        title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
        description: 'Плесень стремительно бежит к вершине A',
        type: 'info'
    }, ...prev]);
},
// Step 7: Include A-B, add B's edges
() => {
    setVertices(prev => prev.map(v => v.id === 1 ? {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
        eap' } : e));
    setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)
    setLogs(prev => [{
        title: 'Успех: Вершина B захвачена плесенью',
        description: 'В кучу добавлено B-C(4). Ребро B-E(6)
        type: 'success'
    }, ...prev]);
},
// Step 8: Pop B-C (4)
() => {

```

```
// Step 12: Pop E-F (7)
() => {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
  setLogs(prev => [{
    title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
    description: 'Плесень из центра Токио (E) тянется к F.',
    type: 'info'
  }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
  setVertices(prev => prev.map(v => v.id === 5 ? {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
    eap' } : e));
  setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (10)']);
  setLogs(prev => [{
    title: 'Успех: Вершина F захвачена',
    description: 'Ребро F-I(13) добавлено в кучу. Ресурс перемещен к F.',
    type: 'success'
  }, ...prev]));
},
// Step 14: Pop D-G (8)
() => {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
  setLogs(prev => [{
    title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
    description: 'Плесень расширяется вниз к G.',
    type: 'info'
  }, ...prev]));
},
// Step 15: Include D-G, add G's edges
() => {
  setVertices(prev => prev.map(v => v.id === 6 ? {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
    eap' } : e));
  setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H (10)']);
  setLogs(prev => [{
    title: 'Успех: Вершина G захвачена',
```

```

        setLogs(prev => [{
            title: 'Успех: Вершина H захвачена',
            description: 'В кучу добавлено H-I(12).',
            type: 'success'
        }, ...prev]));
    },
    // Step 20: Pop E-H (11) - Cycle!
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setLogs(prev => [{
            title: 'Шаг 11: Куча выдает E-H (вес 11)',
            description: 'Обе вершины E и H уже в плесени.',
            type: 'warning'
        }, ...prev]));
    },
    // Step 21: Reject E-H
    () => {
        setEdges(prev => prev.map(e => e.id === '4-7' ? {
        setHeapQueue(['H-I (12)', 'F-I (13)']));
        setLogs(prev => [{
            title: 'Отказ: Игнорируем E-H',
            description: 'Выбрасываем из кучи.',
            type: 'error'
        }, ...prev]));
    },
    // Step 22: Pop H-I (12)
    () => {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setLogs(prev => [{
            title: 'Шаг 12: Куча выдает H-I (вес 12)',
            description: 'Плесень тянется к последней чистой',
            type: 'info'
        }, ...prev]));
    },
    // Step 23: Include H-I
    () => {
        setVertices(prev => prev.map(v => v.id === 8 ? {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {

```

```

// Component 1: {A, B, C} (colored red)
// Component 2: {D, E, F, G, H, I} (colored gold)
setVertices(prev => prev.map(v =>
  [0, 1, 2].includes(v.id) ? { ...v, color: 'red' }
));
setEdges(prev => prev.map(e => {
  if (['0-1', '1-2'].includes(e.id)) {
    return { ...e, status: 'included', color: 'red' }
  }
  if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
    return { ...e, status: 'included', color: 'blue' }
  }
  return e;
}));
setLogs(prev => [{
  title: 'Слияние: Деревни объединились в колхозы',
  description: 'Все выбранные дороги построены радом с {D, E, F, G, H, I}.',
  type: 'success'
}, ...prev]);
},
// Step 3: Five-Year Plan 2 (Inspecting bridge between two kolkhozes)
() => {
  // Now both kolkhozes look at all outgoing roads
  // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}
  // A-D (5), B-E (6), C-F (9).
  // Cheapest is A-D (5).
  setEdges(prev => prev.map(e => e.id === '0-3' ? {
    status: 'included', color: 'red'
  } : e));
  setLogs(prev => [{
    title: 'Вторая пятилетка: Колхозы выбирают мост',
    description: 'Теперь Красный и Синий колхозы объединились в один колхоз. Самый дешевый мост A-D с весом 5. Остальные мосты B-E(6) и C-F(9) остаются в наметке.',
    type: 'warning'
  }, ...prev]);
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
  setVertices(prev => prev.map(v => ({ ...v, color: 'blue' })));
  setEdges(prev => prev.map(e => ({ ...e, status: 'included', color: 'blue' })));
  setLogs(prev => [{
    title: 'Завершение: Все деревни объединились в один колхоз',
    description: 'Все дороги построены. Колхозы объединились в один колхоз.',
    type: 'success'
  }, ...prev]);
}
}

```

```

        type: 'info',
    }]);
}
if (algo === 'prim') {
    return [{
        title: 'Введение в Плесень (Прима)',
        description: 'Логика «Плесени»: выбираем стартовый (Heap) и захватываем соседей!',
        type: 'info',
    }]);
}
return [{
    title: 'Введение в Коллективизацию (Борувка)',
    description: 'Логика «Борувки»: Каждая из 9 деревень объединиться в колхоз.',
    type: 'info',
}]);
};

const handleReset = (algo: 'kruskal' | 'prim' | 'boruvka') => {
    setActiveAlgo(algo);
    setVertices(initialVertices);
    setEdges(initialEdges);
    setStepIndex(0);
    setHeapQueue([]);
    setLogs(introLog(algo));
};

/* ————— связь с Python-компилятором ————— */
const chapterId = useContext(VizChapterContext);
const runtime = useVizRuntime();
const vars = runtime?.variables;

// активная вкладка = подрежим страницы: код в панели
useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, activeAlgo);
}, [chapterId, activeAlgo]);

```

```

    const letters = item.match(/[A-Ia-i]/g);
    let u: number | null = null;
    let v: number | null = null;
    if (letters && letters.length >= 2) { u = idOf(letters[0]); v = idOf(letters[1]); }
    else if (nums && nums.length >= 2) { u = idOf(nums[0]); v = idOf(nums[1]); }
    if (u === null || v === null) return null;
    return { u, v, w: nums && nums.length ? Number(nums[0]) : null };
}
if (Array.isArray(item)) {
    const letters = item.filter((x) => typeof x === 'string');
    const nums = item.filter((x) => typeof x === 'number');
    if (letters.length >= 2) {
        const u = idOf(letters[0]); const v = idOf(letters[1]);
        if (u === null || v === null) return null;
        return { u, v, w: nums.length ? nums[0] : null };
    }
    if (nums.length >= 2) {
        const u = idOf(nums[nums.length - 2]); const v = idOf(nums[nums.length - 1]);
        if (u === null || v === null) return null;
        return { u, v, w: nums.length > 2 ? nums[0] : null };
    }
}
return null;
};

```

```

const PALETTE = ['red', 'blue', 'purple', 'mold'];
/** Цвет по корне компоненты: одинаковый корень → один цвет */
const colorOfComp = (root: string | undefined): string {
    if (!root) return 'none';
    let h = 0;
    for (let i = 0; i < root.length; i++) h = (h * 31 + root.charCodeAt(i)) % PALETTE.length;
    return PALETTE[h];
};

```

```

/** Всё, что код пользователя рассказал про граф. */
const live = useMemo(() => {
    if (!vars) return null;
    const parentRec = vizRecord(vars.parent) ?? vizRecord(vars.parent);
    const childRec = vizRecord(vars.child) ?? vizRecord(vars.child);
    const edgeRec = vizRecord(vars.edge) ?? vizRecord(vars.edge);
    const nodeRec = vizRecord(vars.node) ?? vizRecord(vars.node);
    const rootRec = vizRecord(vars.root) ?? vizRecord(vars.root);
    const graphRec = vizRecord(vars.graph) ?? vizRecord(vars.graph);
    return {
        parentRec,
        childRec,
        edgeRec,
        nodeRec,
        rootRec,
        graphRec,
    };
});

```

```

        const nxt = parentIdx.get(cur);
        if (nxt === undefined || nxt === cur || seen.has(nxt)) {
            seen.add(nxt);
            cur = nxt;
        }
        compOfVertex.set(id, String.fromCharCode(65 + cur));
    });
    const takenRaw = vizArray(vars.mst) ?? vizArray(vars.mst);
    const takenEdges = (takenRaw ?? []).map(parseEdge);
    const takenVertices = (vizArray(vars.taken) ?? vizArray(vars.taken))
        .map((x) => idOf(typeof x === 'string' ? x : vizNumber(x)))
        .filter((x): x is number => x !== null);
    const curU = idOf(vizString(vars.u) ?? vizNumber(vars.u));
    const curV = idOf(vizString(vars.v) ?? vizNumber(vars.v));
    const curW = vizNumber(vars.w);
    const heapRaw = vizArray(vars.heap) ?? vizArray(vars.heap);
    const heapEdges = (heapRaw ?? []).map(parseEdge);
    const cheapestRaw = vizRecord(vars.cheapest) ?? vizRecord(vars.cheapest);
    const cheapestEdges = cheapestRaw ? Object.values(cheapestRaw)
        .filter((e): e is number => e !== null) : [];
    const total = vizNumber(vars.total) ?? vizNumber(vars.total);
    const any = compOfVertex.size > 0 || takenEdges.length > 0 ||
        (curU !== null && curV !== null) || heapEdges.length > 0;
    return any ? { compOfVertex, takenEdges, takenVertices, total,
        // eslint-disable-next-line react-hooks/exhaustive-deps
    }, [vars] );
}

```

```

/** Рёбра, которые рисуем: код пользователя важнее ка...
const effEdges: Edge[] = useMemo(() => {
    if (!live) return edges;
    const key = (a: number, b: number) => `${Math.min(a, b)}-${Math.max(a, b)}`;
    const takenKey = new Set(live.takenEdges.map((e) => key(e.u, e.v)));
    const heapKey = new Set(live.heapEdges.map((e) => key(e.u, e.v)));
    const cheapKey = new Set(live.cheapestEdges.map((e) => key(e.u, e.v)));
    return edges.map((e) => {
        const k = key(e.u, e.v);
        const isCur = (live.curU === e.u && live.curV === e.v);
        if (isCur) return { ...e, status: 'inspecting' as 'inspecting' };
    });
}

```



```

    if (status === 'in_heap') return '#0284c7';
    if (status === 'included') {
      if (color === 'red') return '#ef4444';
      if (color === 'blue') return '#3b82f6';
      if (color === 'purple') return '#a855f7';
      if (color === 'mold') return '#10b981';
    }
    return '#475569';
  };

  return (
    <div className="space-y-12">

      {/* 3-in-1 MST Simulator Box */}
      <div className="bg-slate-900/90 border border-slate-800" >
        <div className="flex flex-col lg:flex-row justify-between" >
          <div>
            <div className="flex items-center gap-2 mb-2" >
              <span className="bg-gradient-to-r from-indigo-500 to-purple-500 text-white font-bold uppercase tracking-wider" >
                Сунер-Интерактив 3 в 1
              </span>
              <h3 className="text-2xl font-bold text-white" >
                Алгоритмы
              </h3>
            </div>
            <p className="text-slate-400 text-sm" >
              Наблюдай за работой Краскала (Раскраска),
            </p>
          </div>

          {/* Algorithm Tabs */}
          <div className="flex flex-wrap items-center gap-2" >
            <div className="flex items-center bg-slate-800" >
              <button
                onClick={() => handleReset('kruskal')}
                className={
                  "flex items-center gap-1.5 px-3 py-2"
                  +
                  (activeAlgo === 'kruskal'

```

```

-200 font-semibold text-xs rounded-xl border
>
  <RotateCcw className="w-3.5 h-3.5" />
  <span>Сбросить</span>
</button>

<button
  onClick={handleNextStep}
  disabled={stepIndex >= currentSteps.length}
  className={
    "flex items-center justify-center gap-2
initial cursor-pointer " +
    (stepIndex >= currentSteps.length
      ? "bg-slate-800 text-slate-500 border
      : activeAlgo === 'kruskal'
      ? "bg-indigo-600 hover:bg-indigo-500
      : activeAlgo === 'prim'
      ? "bg-emerald-600 hover:bg-emerald-500
      : "bg-rose-600 hover:bg-rose-500 text
    )
  >
  <SkipForward className="w-4 h-4" />
  <span>{stepIndex === 0 ? 'Начать' : stepIndex}
</button>
</div>
</div>

/* Что компилятор сообщил графу прямо сейчас */
<div className="mb-6 flex flex-wrap items-center" >
  <span className="flex items-center gap-1.5 text-sm" >
    <Terminal className="w-3.5 h-3.5" /> из компилятора
  </span>
  {live ? (
    <div>
      {live.curU !== null && live.curV !== null
        ? <span className="text-[11px] font-mono" >
            {live.curU} - {live.curV}
          </span>
        : ''
      }
    </div>
  ) : ''}

```

```

        e className="font-mono text-purple-300">u</code>
        mono text-purple-300">w</code> перекрасят э
    </span>
  })
</div>

<div className="grid grid-cols-1 lg:grid-cols-3"
  {/* Graph Canvas */}
  <div className="lg:col-span-2 bg-slate-950 rounded-3xl p-4
    active min-h-[480px] overflow-hidden">

    {/* Legend */}
    <div className="absolute top-4 left-4 bg-slate-900 p-4
      -center gap-3 text-xs">
      <div className="flex items-center gap-1.5">
        <div className="w-3 h-3 rounded-full bg-slate-400">
      </div>
      {activeAlgo === 'kruskal' ? (
        <>
          <div className="flex items-center gap-1.5">
            <div className="w-3 h-3 rounded-full bg-slate-400">
          </div>
          <div className="flex items-center gap-1.5">
            <div className="w-3 h-3 rounded-full bg-slate-400">
          </div>
          <div className="flex items-center gap-1.5">
            <div className="w-3 h-3 rounded-full bg-slate-400">
          </div>
        </>
      ) : activeAlgo === 'prim' ? (
        <>
          <div className="flex items-center gap-1.5">
            <div className="w-3 h-1 bg-sky-600">
          </div>
          <div className="flex items-center gap-1.5">
            <div className="w-3 h-3 rounded-full bg-slate-400">
          </div>
        </>
      ) : null
    }
  </div>
  </div>

```

```

        stroke={strokeColor}
        strokeWidth={isInspecting ? 4.5 : 1}
        strokeDasharray={isRejected ? '4,4' : ''}
        className={"transition-all duration-300"}
      />
      <text
        x={({u.x + v.x} / 2 + "%")}
        y={({u.y + v.y} / 2 + "%")}
        fill={isInspecting ? '#f59e0b' : '#94a3b8'}
        fontSize="4.5"
        fontFamily="monospace"
        fontWeight="bold"
        textAnchor="middle"
        dominantBaseline="middle"
        className="select-none filter drop-shadow"
        dy="-1.5"
      >
        {edge.weight}
      </text>
    </g>
  );
  }}}
</svg>

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full" >
  {effVertices.map(vertex => {
    <div
      key={vertex.id}
      style={{ top: vertex.y + "%", left: vertex.x + "%" }}
      className={"absolute -translate-x-1/2 -translate-y-1/2"}
      font-weight="bold" text-align="center" border="2px solid black" transition="all 0.3s ease-in-out"
      id={vertex.id}
    >
      <span>{vertex.label}</span>
      {vertex.id === 4 && activeAlgo === 'path' ? (
        <span className="text-[8px] block text-center">

```

```

    0 shadow-md animate-pulse" : "bg-slate-800
      {item}
    </span>
  ))
  })
</div>
</div>
})

<div className="flex-1 p-4 overflow-y-auto
  {logs.map((log, idx) => {
    <div
      key={idx}
      className={
        "p-4 rounded-xl border transition-a
        {log.type === 'success'
          ? 'bg-emerald-950/40 border-emera
          : log.type === 'warning'
          ? 'bg-amber-950/40 border-amber-5
          : log.type === 'error'
          ? 'bg-rose-950/40 border-rose-500
          : 'bg-slate-900/60 border-slate-7
        }
      }
    >
      <div className="flex items-center gap
        {log.type === 'success' && <CheckCi
        {log.type === 'warning' && <HelpCir
        {log.type === 'error' && <XCircle c
        {log.type === 'info' && <Play class
        <h5 className="font-bold text-sm le
      </div>
      <p className="text-xs text-slate-300
    </div>
  )})
</div>
</div>
</div>
</div>

```

```

        <p className="text-slate-400 text-xs font-medium">
            <Clock className="w-3.5 h-3.5 text-indigo-500" />
        </p>
        <p className="text-2xl font-black text-slate-900">
            <p className="text-slate-400 text-xs font-medium">
                <Award className="w-3.5 h-3.5 text-indigo-500" />
            </p>
            <p className="text-2xl font-black text-slate-900">
                <p className="text-slate-400 text-xs font-medium">
                    <Award className="w-3.5 h-3.5 text-indigo-500" />
                </p>
            </p>
        </div>
        <div className="bg-slate-950 p-4 rounded-2xl">
            <p className="text-slate-400 text-xs font-medium">
                <Award className="w-3.5 h-3.5 text-indigo-500" />
            </p>
            <p className="text-2xl font-black text-slate-900">
                <p className="text-slate-400 text-xs font-medium">
                    <Award className="w-3.5 h-3.5 text-indigo-500" />
                </p>
            </p>
        </div>
        <div className="bg-slate-950 p-4 rounded-2xl">
            <p className="text-slate-400 text-xs font-medium">
                <Award className="w-3.5 h-3.5 text-indigo-500" />
            </p>
            <p className="text-2xl font-black text-slate-900">
                <p className="text-slate-400 text-xs font-medium">
                    <Award className="w-3.5 h-3.5 text-indigo-500" />
                </p>
            </p>
        </div>
        <div className="lg:col-span-2">
            <div className="bg-slate-950 p-4 rounded-2xl">
                <p className="text-slate-400 text-xs font-medium">
                    <Code className="w-3.5 h-3.5 text-indigo-500" />
                </p>
                <pre className="text-xs text-emerald-400 font-mono">
                    80 flex-1">
                </pre>
            </div>
            {`# 1. Инициализируем DSU
def find(i):
    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]

def union(i, j):
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
    return True`}
```

```

<div className="lg:col-span-2">
  <div className="bg-slate-950 p-4 rounded-1">
    <p className="text-slate-400 text-xs font-medium">
      <Code className="w-3.5 h-3.5 text-emerald-400" />
    </p>
    <pre className="text-xs text-emerald-400 font-family-mono">
80 flex-1">
{`import heapq

def prim(graph, start):
    visited = [False] * len(graph)
    heap = [(0, start, -1)] # (weight, node, parent)
    mst = []

    while heap:
        w, u, prev = heapq.heappop(heap)
        if visited[u]: continue
        visited[u] = True
        if prev != -1: mst.append((prev, u, w))

        for next_w, v in graph[u]:
            if not visited[v]:
                heapq.heappush(heap, (next_w, v, u))
    return mst`}
    </pre>
  </div>
</div>
</div>
})

{activeCheatTab === 'boruvka' && (
  <div className="grid grid-cols-1 lg:grid-cols-2">
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-1">
        <p className="text-slate-400 text-xs font-medium">
          <Clock className="w-3.5 h-3.5 text-rose-500" />
        </p>
        <p className="text-2xl font-black text-white">

```

```
        mst.append(bridge)` }
      </pre>
    </div>
  </div>
</div>
  })
</div>
</div>
  );
};
```

ФАЙЛ 59/113: src/components/LiteGuide.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useMemo } from "react";
import { chapters, chapterTopics } from "../data/content";
import { getViz } from "../vizRegistry";
import { PAGE_SYNC } from "../data/vizSync";
import { linkifyTicketsHtml } from "../utils/ticketLink";
import { quizzes } from "../data/quizzes";
```

```
/**
```

```
 * «Лёгкая» текстовая версия пособия.
```

```
 *
```

```
 * Открывается по адресу `?lite=1` (весь список тем) или
 * (одна тема). Здесь НЕТ навигации, сайдбара, интерактивных
 * панели компилятора – только сам текст тем, чтобы:
```

```
 * 1) любую страницу можно было распечатать/сохранить
```

```
 * 2) внешний парсер (ридер, «версия для слабовидящих»
```

```
 * LLM-индексатор) получал чистый текст без служебных
```

```
 *
```

```
 * Все спойлеры <details> раскрыты, визуализация описания
```

```
 * показывает, какие у неё вкладки и какие переменные к
```

```
 */
```



```

    {syncKeys.map((k) => {
      const sync = PAGE_SYNC[k];
      const demo = k.includes("#") ? k.split("#")[1] : k;
      return {
        <div key={k}>
          <p>
            <b>{demo ? `Вкладка «${demo}»` : `Синхронизация`}
            {sync.vizTitle ? ` – ${sync.vizTitle}` : ''}
            {sync.stepNote ? ` <> Шаг демонстрации` : ''}
          </p>
          {sync.variables.length > 0 && (
            <ul>
              {sync.variables.map((v) => {
                <li key={v.name}>
                  <code>{v.name}</code> – {v.value}
                  {v.range ? ` (${v.range})` : ''}
                </li>
              })}
            </ul>
          )}
        </div>
      );
    })}
  <p className="lite-meta" style={{ margin: 0; padding: 0; font-size: 0.8em; color: #666; font-family: sans-serif; font-weight: normal; line-height: 1.2; }}>
    В интерактивной версии эта демонстрация переменных перерисовывают её напрямую
  </p>
</div>
)}

{quizzes[chapter.id] ?? []}.length > 0 &&
  <div className="lite-quiz">
    <h3>Блиц: вопросы по теме</h3>
    <ol>
      {quizzes[chapter.id] ?? []}.map((q, i) => {
        <li key={i}>
          <p>
            <b>{q.question}</b>

```

```

-----
  /** Идентификаторы вершин, нарисованных на схеме демо
  demoIds: string[];
  /** Подпись таблицы (по умолчанию «Ваши расстояния»).
  title?: string;
}

const fmt = (value: DebugValue | undefined): string =>
  if (value === undefined || value === null) return "—";
  if (typeof value === "number") {
    if (value === Number.POSITIVE_INFINITY) return "∞";
    if (value === Number.NEGATIVE_INFINITY) return "−∞";
    return String(value);
  }
  if (typeof value === "string") {
    const t = value.trim().toLowerCase();
    if (t === "inf" || t === "infinity") return "∞";
    if (t === "-inf" || t === "-infinity") return "−∞";
    return value;
  }
  return String(value);
};

/**
 * Таблица «ваших» расстояний, когда имена вершин в коде
 *
 * Схема демонстрации нарисована на фиксированном наборе
 * S...F у Форда–Беллмана). Если в коде пользователя верши
 * (0...6, v1...v9), картинка физически не может их показат
 * просто пропадали, и казалось, что визуализация живёт
 * выводятся рядом, а подсказка объясняет, как связать
 */
export function LiveDistTable({ dist, prev, done, demoIds
  if (!dist) return null;

  const keys = Object.keys(dist);
  if (keys.length === 0) return null;

  const demoSet = new Set(demoIds);

```

```
        {fixed ? <span className="text-emerald-400" style={{fontFamily: 'monospace'}}>
          </span>
        );
      }}}
    {keys.length > shown.length ? (
      <span className="inline-flex items-center rounded-md px-2 py-1 text-emerald-400">
        +{keys.length - shown.length} ещё
      </span>
    ) : null}
  </div>
</div>
);
}
```

ФАЙЛ 61/113: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import dijkstraImg from '../assets/dijkstra-freakazoid.jpg';
import bellmanImg from '../assets/bellman-ford-batman.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const cards = [
  {
    img: dijkstraImg,
    alt: 'Декстер Дуглас и Фриказоид (Дейкстра)',
    title: '⚡ Фриказоид и Декстер Дуглас',
    desc: 'Декстер смотрит в CRT-монитор; синий электрический ток',
    highlight: 'неотрицательными',
    highlightColor: 'text-emerald-400',
    rest: 'весами ( $w \geq 0$ ). Подсунь отрицательное ребро -',
    complexity: ' $O((V+E) \log V)$ ',
    border: 'border-blue-500/30 hover:border-blue-400/60',
    titleColor: 'text-blue-400',
    badge: 'text-blue-400/70 bg-blue-950/40',
  },
];
```

```
        <div className="p-4">
          <h3 className={`text-lg font-bold mb-1 ${c.className}`}>
            <p className="text-slate-400 text-sm">
              {c.desc} <span className={`font-bold ${c.className}`}>
            </p>
          <div className={`mt-3 text-xs font-mono px-1`}
        </div>
      </div>
    )})
  </div>
);
}
```

ФАЙЛ 62/113: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Заголовок текущей темы – показываем в кнопке, что */
  currentTitle: string;
  index: number;
  total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedChapterId,
  const [open, setOpen] = useState(false);
```



```
</div>
```

```
<div className="flex items-center flex-wrap sm:items-center sm:justify-between sm:w-700">
  <span className="hidden sm:flex items-center justify-center gap-2">
    <div style="background-color: #4b4b9b; color: white; padding: 5px 10px; border-radius: 5px; text-align: center; font-weight: bold; font-size: 0.9em;">Учебное пособие</div>
    <BookOpen className="w-4 h-4" /> Учебное пособие
  </span>
```

```
  <button
    id="tab-compiler-btn"
    onClick={onToggleCompiler}
    aria-pressed={compilerOpen}
    title="Боковая панель Python-компилятора (настройка)"
    aria-label={compilerOpen ? "Скрыть панель Python-компилятора" : "Показать панель Python-компилятора"}
    className={`shrink-0 flex items-center justify-center gap-2 duration-200 whitespace-nowrap ${
      compilerOpen
        ? 'bg-emerald-600 text-white shadow-lg'
        : 'text-slate-400 hover:text-white bg-slate-700'
    }`}
  >
    <Terminal className="w-4 h-4" /> Python
  </button>
```

```
<div id="download-menu" className="flex items-center justify-between gap-2">
  <a
    id="download-guide-pdf-btn"
    href={` ${import.meta.env.BASE_URL}export/guide/download/guide_lite.pdf`}
    download="guide_lite.pdf"
    target="_blank"
    rel="noreferrer"
    className="flex-1 sm:flex-none flex items-center justify-center gap-2"
    style="background-color: #4b4b9b; color: white; padding: 5px 10px; border-radius: 5px; text-align: center; font-weight: bold; font-size: 0.9em;">
    Скачать PDF пособия: компактный код
  </a>
  <FileDownload className="w-4 h-4 text-indigo-400" />
</div>
```



```

let conflict: [number, number] | null = null;
frames.push({
  title: "Старт: пробуем покрасить в 2 цвета",
  note: "BFS красит вершину в цвет, противоположный р
    двудольный ( $\chi > 2$ ).",
  color: [...color], v: null, u: null, e: null,
});
for (let start = 0; start < g.n && !conflict; start++)
  if (color[start] !== 0) continue;
  color[start] = 1;
  queue.push(start);
  frames.push({
    title: `Запускаем BFS из ${g.names[start]}: цвет
    note: `Вершина ${g.names[start]} ещё не покрашена
      и 2.`,
    color: [...color], v: start, u: null, e: null,
  });
while (queue.length && !conflict) {
  const v = queue.shift() as number;
  for (const u of g.adj[v]) {
    if (color[u] === 0) {
      color[u] = 3 - color[v];
      queue.push(u);
      frames.push({
        title: `${g.names[v]} → ${g.names[u]}: цвет
        note: `Сосед не покрашен: color[${g.names[u]
          x}).join(", ") || "пуста"}].`,
        color: [...color], v, u, e: null,
      });
    } else if (color[u] === color[v]) {
      conflict = [v, u];
      frames.push({
        title: `Конфликт: ${g.names[v]} и ${g.names
        note: `Ребро (${g.names[v]}, ${g.names[u]})
          покрасить:  $\chi \geq 3$ .`,
        color: [...color], v, u, e: null, bad: conf
      });
    }
  }
}

```

```

        note: `В вершине ${g.names[a]} заняты цвета ${us
            y) => x - y).join(", ") || "-"}}. Наименьший св
            да.`
        color: new Array(g.n).fill(0), ecolord: [...ecolor
    });
});
const chiE = Math.max(...ecolor);
frames.push({
    title: `Готово:  $\chi' = \chiE$  при  $\Delta = \Delta$ `,
    note:
        g.key === "bip"
        ? `Двудольный граф:  $\chi' = \Delta = \Delta$  — это теор
            сс 1 по Визингу.`
        : ` $\chi' = \chiE$ . Теорема Визинга:  $\Delta \leq \chi' \leq \Delta +$ 
             $\chi' = \Delta + 1$  — классом 2 (например, K3 и любой
        color: new Array(g.n).fill(0), ecolord: [...ecolor],
    });
return frames;
}

```

```

const TABS: { id: Tab; label: string; icon: React.React
    { id: "planar", label: "1 · Планарность: K5 и K3,3",
    { id: "greedy", label: "2 · Покраска вершин ( $\chi \leq \Delta + 1$ )",
    { id: "bipartite", label: "3 · Двудольность (2 краски)",
    { id: "edges", label: "4 · Покраска рёбер (Визинг, Кё
];

```

```

export const PlanarColoringViz: React.FC = () => {
    const chapterId = useContext(VizChapterContext);
    const runtime = useVizRuntime();
    const vars = runtime?.variables;
    const [tab, setTab] = useState<Tab>("greedy");
    const [graphKey, setGraphKey] = useState<GraphKey>("m
    const [step, setStep] = useState(0);
    const [playing, setPlaying] = useState(false);

    useEffect(() => {
        if (chapterId) emitVizDemo(chapterId, tab);
    });
}

```

```

const liveEcolor = vars ? vizArray(vars.ecolor) ?? vizArray(0)
const liveV = vars ? vizNumber(vars.v) : null;
const liveU = vars ? vizNumber(vars.u) : null;
const liveE = vars ? vizNumber(vars.e) : null;
const liveChi = vars ? vizNumber(vars.chi) ?? vizNumber(0) : null;
const linked = Boolean(liveColor || liveEcolor || (liveV || liveU || liveE || liveChi));

const frame = frames[Math.min(step, maxStep)] ?? ({ t: 0, color: {}, ecolor: {} });
const nodeColor = (id: number): number => {
  if (liveColor && typeof liveColor[id] === "number")
    return frame.color?.[id] ?? 0;
};
const edgeColor = (i: number): number => {
  if (liveEcolor && typeof liveEcolor[i] === "number")
    return frame.ecolor?.[i] ?? 0;
};
const curV = liveV !== null && Number.isFinite(liveV);
const curU = liveU !== null && Number.isFinite(liveU);
const curE = liveE !== null && Number.isFinite(liveE);
const delta = Math.max(...graph.adj.map((a) => a.length));
const usedColors = Array.from(new Set(graph.pos.map((p) => p.color)));
const usedEcolors = Array.from(new Set(graph.edges.map((e) => e.ecolor)));

const go = useCallback((d: number) => {
  setPlaying(false);
  setStep((s) => Math.max(0, Math.min(maxStep, s + d)));
}, [maxStep]);

if (tab === "planar") {
  return (
    <div className="bg-slate-800/90 p-5 sm:p-6 rounded">
      <Header />
      <Tabs tab={tab} setTab={setTab} />
      <PlanarityDemo />
      <p className="text-[11px] text-slate-500 mt-2">
        Формула Эйлера  $V - E + F = 2$  и оценка  $E \leq 3V$ 
      </p>
    </div>
  );
}

```

```

? <span className="bg-slate-950 border border-
rs}</span>
: <span className="bg-slate-950 border border
}</span>}
</span>
</div>

<div className="grid grid-cols-1 lg:grid-cols-5 g
  <div className="lg:col-span-3 bg-slate-950 round
    <svg viewBox="0 0 480 300" className="w-full l
      {graph.edges.map(([a, b], i) => {
        const ec = edgeColor(i);
        const isCur = i === curE;
        const isBad = Boolean(frame.bad && ({frame
      }));
      return (
        <g key={i}>
          <line
            x1={graph.pos[a].x} y1={graph.pos[a
            stroke={isBad ? "#f43f5e" : ec > 0
            strokeWidth={isCur ? 6 : isBad ? 5
            strokeDasharray={ec === 0 && !isBad
            className="transition-all duration-
            opacity={isCur || isBad ? 1 : 0.85}
          />
          {tab === "edges" && ec > 0 && (
            <text x={{graph.pos[a].x + graph.po
            ={{colorOf(ec)}} fontSize="10" fontWeight="bo
          }}
        </g>
      );
    }}}}
    {graph.pos.map((p, id) => {
      const c = nodeColor(id);
      const isCur = id === curV;
      const isNbr = id === curU;
      return (
        <g key={id}>

```

```

        Δ (Кёниг). Задача « $\chi' = \Delta?$ » NP-полна даже
      </div>
    </div>
  </div>

  <div className="mt-3 flex flex-wrap items-center">
    <span className="text-[10px] uppercase tracking-widest">
      /> из компилятора:</span>
    {vars && linked ? (
      ["v", "u", "e", "color", "ecolor", "chi"].map(
        (name) => {
          <span key={name} className="text-[11px] font-mono">
            {vars[name]}
          </span>
        }
      )
    ) : (
      <span className="text-[11px] text-slate-500">
        открой панель Python: массив <code className="text-fuchsia-300">ecolor</code> для рёбер) и вершина
      </span>
    )}
  </div>
</div>
);
};

```

```

const Header: React.FC = () => {
  <div className="flex flex-wrap items-center justify-between">
    <div className="flex items-center gap-3">
      <div className="p-2.5 bg-fuchsia-600 text-white rounded">
        <Palette className="w-5 h-5" />
      </div>
      <div>
        <h3 className="text-lg sm:text-xl font-bold text-white">
          <p className="text-xs text-fuchsia-300">K5/K3,3
        </p>
      </div>
    </div>
  </div>

```

```

<svg className="w-full h-full absolute inset-0"
  {(() => {
    const pts = [
      {id:0,x:160,y:40},
      {id:1,x:270,y:100},
      {id:2,x:230,y:230},
      {id:3,x:90,y:230},
      {id:4,x:50,y:100}
    ];
    const edges = [
      [0,1],[0,2],[0,3],[0,4],
      [1,2],[1,3],[1,4],
      [2,3],[2,4],
      [3,4]
    ];
    function findPt(id:number) { return pts.f
    function intersect(a:number,b:number,c:nu
      if (a===c||a===d||b===c||b===d) return
      const p1=findPt(a),p2=findPt(b),p3=find
      function ccw(ax:number,ay:number,bx:num
        return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
      }
      return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
        && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
    }
    const crosses: number[] = [];
    for(let i=0;i<edges.length;i++) {
      for(let j=i+1;j<edges.length;j++) {
        if (intersect(edges[i][0],edges[i][1]
          if (!crosses.includes(i)) crosses.p
          if (!crosses.includes(j)) crosses.p
        }
      }
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
      const p1=findPt(edges[i][0]),p2=findPt(
      const xing = crosses.includes(i);

```

```

        if (!p1||!p2||!p3||!p4) return false;
        function ccw(ax:number,ay:number,bx:number,cx:number,cy:number) {
            return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
        }
        return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
            && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y);
    }
    const crosses:number[]=[];
    for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++) {
        if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
            if (!crosses.includes(i)) crosses.push(i);
            if (!crosses.includes(j)) crosses.push(j);
        }
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
        const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
        const xing=crosses.includes(i);
        lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x} y1={p1.y} y2={p2.y}
            stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
    }
    const labels = ["\u04141","\u04142","\u04143","\u04144"];
    for(const p of pts) {
        circles.push(<g key={`c${p.id}`}>
            <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#4f46e5" stroke-width={2}/>
            <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
        </g>);
    }
    return <>{lines}{circles}</>;
    })()
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20 z-20">
    V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
</div>
</div>
</div>

```



```

    Unlink,
    Variable,
    X,
} from "lucide-react";
import { buildInitTemplate, buildSyncTemplate, getPageS
import { clearVizState, emitVizState, onVizDemo } from
import {
    baseVarName,
    buildDebugRunner,
    changedVars,
    DEBUG_STEP_CAP,
    orderVars,
    type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "../Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
const PLAYBACK_SPEED_KEY = "pythonCompilerPlaybackSpeed
const PLAYBACK_SPEEDS = [0.25, 0.5, 1, 1.5, 2, 4] as co

interface PythonCompilerProps {
    /** Страница, с которой синхронизируется редактор. */
    chapterId: string;
    chapterTitle: string;
    /** Размер панели (px) управляется родителем и сохран
    width?: number;
    height?: number | null;
    onWidthChange?: (w: number) => void;
    onHeightChange?: (h: number) => void;
    /** Перейти к странице пособия (посмотреть визуализац
    /** Кнопка «открыть страницу в пособии» – не нужна, е
    onOpenGuide?: () => void;
    /** Скрыть панель. */
    onClose: () => void;
}

```

```

    if (error instanceof Error) return error.message;
    return String(error);
}

/**
 * Предел вывода в панель «Результат».
 *
 * Трасса ограничена по ЧИСЛУ шагов (DEBUG_STEP_CAP), на
 * напечатанной строки: print("x" * 10_000_000) или print(
 * – это один шаг, после которого весь текст уезжал в <
 * Обрезаем по границе строки и честно пишем, сколько з
 */
const OUTPUT_CHAR_CAP = 20_000;

function clipOutput(text: string): string {
    if (text.length <= OUTPUT_CHAR_CAP) return text;
    const cut = text.slice(0, OUTPUT_CHAR_CAP);
    const lastBreak = cut.lastIndexOf("\n");
    const head = lastBreak > OUTPUT_CHAR_CAP / 2 ? cut.sl
    return `${head}\n... вывод обрезан: показаны первые ${h
}

/* -----
/*  Шаблон синхронизации: короткая шапка + минимальный
/*  скелет, шаг-в-шаг с подсветкой визуализации (см. da
/*  -----

/* -----
/*  Шпаргалка Python: команды сортировки, циклы, структ
/*  «Вставить» кладёт сниппет в редактор под курсор.
/*  -----

interface Snippet {
    label: string;
    tip: string;
    code: string;
}

```



```

const [code, setCode] = useState(() => {
  const saved = savedEditors.get(chapterId);
  if (!saved) return initTemplate;
  if (saved.code === saved.base || saved.code.trim() === saved.base.trim())
    return saved.code;
});
/** «Глаз» – переключатель: эталонный код демо ↔ инициализация
const [refOn, setRefOn] = useState(() => {
  const saved = savedEditors.get(chapterId);
  return !!saved && saved.code.trim() !== "" && saved.base.trim() !== "";
});
const [stdin, setStdin] = useState("");
const [output, setOutput] = useState("Код выполнится");
const [running, setRunning] = useState(false);
const [runtimeReady, setRuntimeReady] = useState(false);
const [runtimeMessage, setRuntimeMessage] = useState("");
/** Принудительный повтор (глаз / Ctrl+Enter), даже если код не готов
const [traceRequest, setTraceRequest] = useState(0);
const [copied, setCopied] = useState(false);
const [stripTab, setStripTab] = useState<"vars" | "highlight">("vars");
const [outputOpen, setOutputOpen] = useState(true);
const editorRef = useRef<HTMLTextAreaElement>(null);
**
* Панель жива? Заккрытие (крестик, «глаз» в шапке, см. код выше)
* компонент, но `runPythonAsync` к тому моменту может быть выполнен
* этого флага доехавший до конца прогон опубликовал результат
* после закрытия: панель исчезла, а визуализация прогона осталась
* кода, которого на экране больше нет.
*/
const aliveRef = useRef(true);
useEffect(() => {
  aliveRef.current = true;
  return () => {
    aliveRef.current = false;
  };
}, []);

// Код хранится по страницам, поэтому при открытии па

```

```

    setDebug(null); // трасса ссылается на строки старого
    setRefOn(false);
    if (code === prev.base || code === prev.full || code === prev.partial) {
      setCode(initTemplate);
      setDesyncedFrom(null);
    } else {
      setDesyncedFrom(chapterId);
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [initTemplate, template]);

const resync = useCallback(() => {
  prevTplRef.current = { base: initTemplate, full: template };
  setCode(initTemplate);
  setRefOn(false);
  setDesyncedFrom(null);
  setStripTab("vars");
  setPlaying(false);
  setEditHold(null);
  setDebug(null);
  setTraceRequest((n) => n + 1);
}, [initTemplate, template]);

const copyCode = useCallback(async () => {
  try {
    await navigator.clipboard.writeText(code);
    setCopied(true);
    setTimeout(() => setCopied(false), 1600);
  } catch {
    setRuntimeMessage("Не получилось скопировать – буфер переполнен");
  }
}, [code]);

/* — Пошаговый отладчик —————

/** Связь больше не зависит от эталонных строк: совпадают строки
const hasViz = sync.variables.length > 0;

```

```

const publishDebug = useCallback(
  (result: DebugResult, idx: number, source: string) => {
    if (!aliveRef.current) return; // панель закрыта
    if (!hasViz || result.steps.length === 0) return;
    const snapshot = snapshotAt(result, idx, source);
    emitVizState(chapterId, {
      line: snapshot.line,
      func: snapshot.func,
      variables: snapshot.variables,
      changed: snapshot.changed,
    });
  },
  [chapterId, hasViz, snapshotAt]
);

// Единственная точка публикации: любое перемещение т
// визуализации согласованный снимок, без side-effect
useEffect(() => {
  if (debug) publishDebug(debug.result, debug.idx, de
}, [debug, publishDebug]);

const goDebug = useCallback(
  (nextIdx: number) => {
    setDebug((d) => {
      if (!d || d.result.steps.length === 0) return d
      const idx = Math.max(0, Math.min(d.result.steps
      return { ...d, idx };
    });
  },
  []
);

const stepBy = useCallback(
  (delta: number) => {
    setDebug((d) => {
      if (!d || d.result.steps.length === 0) return d
      const idx = Math.max(0, Math.min(d.result.steps

```

```
// Пустой редактор: Pyodide не гоняем, панель в режим
// связь с визуализацией снимаем — демо показывает
if (source.trim() === "") {
  setDebug({ result: { steps: [], error: "", truncated: true } });
  setOutput("В редакторе пусто — выполнять нечего.\n\n" +
    «глаз».");
  setRuntimeMessage("Пустой код: связь с визуализацией снята");
  setOutputOpen(true);
  setRunning(false);
  return;
}
```

```
setRuntimeMessage(runtimeReady ? "Обновляю трассу..." : "Загружаю код");
```

```
const stdout: string[] = [];
const stderr: string[] = [];
const inputLines = input.split(/\r?\n/);
```

```
try {
  const runtime = await getPythonRuntime();
  setRuntimeReady(true);
  runtime.setStdout({ batched: (message) => stdout.push(message) });
  runtime.setStderr({ batched: (message) => stderr.push(message) });
  runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
```

```
const raw = await runtime.runPythonAsync(buildDebugCode);
// Панель могли закрыть, пока Python считал: выходим
if (!aliveRef.current) return;
const result = JSON.parse(String(raw)) as DebugResult;
const printed = clipOutput([...stdout, ...stderr]);
```

```
if (result.steps.length === 0) {
  // Шагов нет (только комментарии, синтаксическая ошибка)
  // заменяется пустым листингом, а визуализация — пустой
  // показывает собственный встроенный сценарий (демонстрация)
  setDebug({ result, idx: 0, src: source, input, output: printed });
  clearVizState(chapterId);
}
```

```

    }
  }, [chapterId, code, stdin, traceRequest, running, run

// Код выполняется сам: debounce даёт спокойно допечать
// изменился во время загрузки Pyodide, после завершения
useEffect(() => {
  if (running || editHold === code) return;
  if (debug?.src === code && debug.input === stdin &&
    const timer = window.setTimeout(() => void debugRun
    return () => window.clearTimeout(timer);
  }, [code, stdin, traceRequest, running, runtimeReady,

// Стрелки ← → в режиме отладки листают ШАГИ (а не ст
useEffect(() => {
  if (!debug || debug.result.steps.length === 0) retu
  const onKey = (e: KeyboardEvent) => {
    const target = e.target as HTMLElement | null;
    if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
    if (e.metaKey || e.ctrlKey || e.altKey) return;
    if (e.key === "ArrowRight") {
      e.stopPropagation();
      e.preventDefault();
      setPlaying(false);
      stepBy(1);
    }
    if (e.key === "ArrowLeft") {
      e.stopPropagation();
      e.preventDefault();
      setPlaying(false);
      stepBy(-1);
    }
    if (e.key === " ") {
      e.stopPropagation();
      e.preventDefault();
      setPlaying((v) => !v);
    }
    if (e.key === "Escape") stopDebug();
  };

```



```

* проигрывает её, в отладчике переключает play/pause
const runOrTogglePlayback = useCallback(() => {
  if (running) return;
  if (!debug || debug.result.steps.length === 0) {
    void debugRun(true);
    return;
  }
  if (playing) {
    setPlaying(false);
    return;
  }
  if (debug.idx >= debug.result.steps.length - 1) stepTo(debug.idx);
  setPlaying(true);
}, [debug, debugRun, playing, running, stepTo]);

/** Вставка сниппета из шпаргалки в позицию курсора.
const insertSnippet = useCallback(({snippet: string}) => {
  const ta = editorRef.current;
  setCode((current) => {
    if (!ta) return current + (current.endsWith("\n") ? "" : "\n");
    const start = ta.selectionStart ?? current.length;
    const end = ta.selectionEnd ?? start;
    const padded = (start > 0 && !current.slice(0, start).endsWith("\n")) ? "\n" : "";
    const next = current.slice(0, start) + padded + snippet + current.slice(end, current.length);
    requestAnimationFrame(() => {
      ta.focus();
      ta.selectionStart = ta.selectionEnd = start + padded.length;
    });
    return next;
  });
}, []);

return (
  <aside
    aria-label="Python-компилятор"
    className="fixed z-40 inset-x-0 bottom-0 h-[58dvh] w-[520px] 2xl:w-[600px] flex flex-col bg-slate-900"
    style={typeof window !== "undefined" ? {

```

```

    { /* Левый край меняет ширину на десктопе. */ }
    { onWidthChange && (
      <div
        role="separator"
        aria-orientation="vertical"
        aria-label="Потяните, чтобы изменить ширину п
        title="Потяните, чтобы изменить ширину панели
        className="hidden lg:block absolute top-0 bot
        onPointerDown={(e) => {
          const el = e.currentTarget;
          el.setPointerCapture(e.pointerId);
          const startX = e.clientX;
          const startW = width ?? 560;
          const onMove = (ev: PointerEvent) => {
            const next = Math.round(startW + (startX
              onWidthChange(Math.max(380, Math.min(next
            });
          const onUp = () => {
            el.removeEventListener("pointermove", onM
            el.removeEventListener("pointerup", onUp
          };
          el.addEventListener("pointermove", onMove a
          el.addEventListener("pointerup", onUp as Ev
        })
      >
        <div className="absolute left-1/2 top-1/2 -tr
          digo-500 transition-colors" />
        </div>
      )}
    { /* — Заголовок панели —————
    <div className="shrink-0 flex flex-wrap items-cen
      <span className={`w-2 h-2 rounded-full shrink-0
      <Terminal className="w-4 h-4 text-emerald-400 s
      <span className="text-[13px] font-bold text-whi
      <span className="hidden sm:inline text-[10px] t

      <div className="ml-auto flex flex-wrap items-cen
        <Tooltip

```

```

        <RotateCcw className="w-4 h-4" />
      </button>
    </Tooltip>
    <Tooltip side="bottom" content="Скрыть панель"
      <button
        type="button"
        onClick={onClose}
        className="p-1.5 rounded-lg text-slate-400"
        aria-label="Заккрыть компилятор"
      >
        <X className="w-4 h-4" />
      </button>
    </Tooltip>
    <Tooltip
      side="bottom"
      content={
        refOn
          ? "Убрать эталонный код: редактор станет
            собственному встроенному демо. Кнопка ↺ вернёт
            : "Вставить полный эталонный код этого
            ,
            удет заменён."
        }
    >
      <button
        type="button"
        onClick={() => {
          const next = !refOn;
          setRefOn(next);
          // «Убрать эталон» — значит убрать код
          // выполняется, поэтому демонстрация бо
          // значений и честно показывает встроенн
          // странице скажет об этом явно).
          setCode(next ? template : "");
          setDesyncedFrom(null);
          setPlaying(false);
          setEditHold(null);
          setDebug(null);
          clearVizState(chapterId);

```

```

<div className="flex items-center gap-1 px-2 pt-2" style={{width: 100px}}>
  <button
    type="button"
    onClick={() => setStripTab("vars")}
    className={`inline-flex items-center gap-1.5 px-2 py-2 rounded-md
      stripTab === "vars" ? "bg-slate-800 text-white" : "bg-white text-slate-800"
    }`>
    <Variable className="w-3.5 h-3.5" /> Переменные
  </button>
  <button
    type="button"
    onClick={() => setStripTab("hints")}
    className={`inline-flex items-center gap-1.5 px-2 py-2 rounded-md
      stripTab === "hints" ? "bg-slate-800 text-white" : "bg-white text-slate-800"
    }`>
    <HelpCircle className="w-3.5 h-3.5" /> Подсказки
  </button>
</div>
<div className="ml-auto pr-1">
  <Tooltip
    side="bottom"
    content={
      <div>
        <div>hasViz</div>
        <div>
          ? "Связь идёт по именам ниже: реальные переменные"
          : "На выбранной странице нет интерактивных элементов"
        </div>
      </div>
    }
  >
    <span
      tabIndex={0}
      className={`cursor-help inline-flex items-center gap-1.5 px-2 py-2 rounded-md
        hasViz ? "text-emerald-400" : "text-slate-400"
      }`>
      <Link2 className="w-3 h-3" />
      {hasViz ? "синхронизировано" : "нет визуализации"}
    </span>
  </div>

```

Страница сменилась – подставить её пер

```

    </button>
  </Tooltip>
}}
{hasViz && {
  <div className="flex flex-wrap items-cent
    {sync.variables.map((v) => {
      <Tooltip
        key={v.name}
        side="bottom"
        content={
          <span>
            <span className="block font-bol
              <code className="font-mono te
            </span>
            <span className="block">{v.role
              {v.range && <span className="bl
            </span>
          }
        }
      >
        <code
          tabIndex={0}
          className="cursor-help rounded-md
font-bold text-emerald-300 hover:bg-emeral
        >
          {v.name}
        </code>
      </Tooltip>
    }}}
  {sync.stepNote && {
    <Tooltip side="bottom" content={`War
      <span
        tabIndex={0}
        className="cursor-help inline-fle
text-[10px] text-slate-400 hover:text-white
      >
        <ArrowRightLeft className="w-3 h-3
        <span className="truncate max-w-5

```

```

<Pencil className="w-3.5 h-3.5 text-emerald-400"
  <span className="text-[10px] font-bold uppercase"
    Редактор • выполняется этот текст
  </span>
  {state && (
    <span
      className="inline-flex items-center gap-1"
      font-bold text-amber-300 shrink-0"
      title="Трасса ниже построена по предыдущему"
    >
      <Loader2 className="w-2.5 h-2.5 animate-spin" />
    </span>
  )}
  <span className="ml-auto text-[10px] text-slate-500"
    {code.trim() === "" ? "пусто – ничего не выведено" : ""}
  </span>
</div>

<textarea
  ref={editorRef}
  value={code}
  onChange={(event) => {
    setEditHold(null);
    setRefOn(false);
    setCode(event.target.value);
  }}
  onKeyDown={(event) => {
    if ((event.ctrlKey || event.metaKey) && event.key === 'Enter') {
      event.preventDefault();
      void debugRun();
    }
    event.stopPropagation();
  }}
  spellCheck={false}
  aria-label="Код Python"
  className="flex-[2] min-h-[110px] w-full resize-none"
  line-none focus:ring-2 focus:ring-inset focus:ring-amber-300
  placeholder="Напишите Python-код... (пустой редактор)"
  />

```

```

        <ChevronRight className="w-4 h-4" />
      </button>
    </Tooltip>
    <Tooltip content="В конец трассы">
      <button type="button" aria-label="В конец трассы" disabled={!currentStep}
        className="text-slate-400 hover:text-white hover:bg-emerald-500 rounded-full px-1.5 py-0.5">
        <SkipForward className="w-4 h-4" />
      </button>
    </Tooltip>
    <span className="ml-1.5 text-[10px] text-slate-400">
      {curDebug?.step ? (
        <div>
          шаг {debug.idx + 1}/{debug.result.steps.length}
          {curDebug.step.func !== "<module>" ? (
            <div>
              {curDebug.step.func}
            </div>
          ) : (
            "шагов нет — исправьте ошибку и начните трассировку"
          )}
        </div>
      ) : (
        {debug.result.truncated && (
          <Tooltip content={`Выполнение остановлено`}
            <span tabIndex={0} className="cursor-help rounded-full px-1.5 py-0.5">
              ЛИМИТ
            </span>
          </Tooltip>
        )}
      )}
    </span>
    <Tooltip content={hasViz ? "Визуализация трассы" : "Эталонный текст не обязателен — пишите что хотите, но без имён, связанных с демонстрацией; Python не поддерживает"}>
      <span
        tabIndex={0}
        className={`cursor-help inline-flex items-center gap-1 border-emerald-500/40 bg-emerald-500/10 padding-2`}
      >
        {hasViz ? <Link2 className="w-3 h-3" href="#viz" /> : <div>
          {hasViz ? "имена → демо" : "свободный текст"}
        </div>
      </span>
    </Tooltip>
  </div>
</div>

```

```

        const active = ln === curDebug?.step?.line;
        return (
          <div
            key={i}
            data-line={ln}
            className={`flex whitespace-pre px-1
              active ? "bg-emerald-500/15 text-emerald-300" : ""
            }`
          >
            <span
              className={`w-7 shrink-0 select-none
                active ? "text-emerald-300" : "text-emerald-500"
              }`
            >
              {active ? "▶" : ln}
            </span>
            <span>{line || " "}</span>
          </div>
        );
      })}
    </div>

    /* панель переменных – «Variables», как в VS Code
    <div className="shrink-0 max-h-[34%] overflow-hidden"
      <div className="flex items-center gap-1.5"
        <Variable className="w-3 h-3 text-indigo-300" />
        Переменные на этом шаге
        <span className="normal-case tracking-normal" />
      </div>
      {debug.result.error && (
        <pre className="mt-1 whitespace-pre-wrap" />
      )}
      {debugOrdered.length === 0 && !debug.result.error && (
        <div className="mt-1 text-[11px] text-emerald-300" />
      )}
    </div>

    <div className="mt-1 flex flex-wrap gap-1"
      {debugOrdered.map(([name, value]) => {

```



```

        id="python-stdin"
        value={stdin}
        onChange={(event) => {
            setEditHold(null);
            setStdin(event.target.value);
        }}
        spellCheck={false}
        placeholder="строки ввода через ↵"
        className="w-full bg-transparent font-mono"
    />
</div>
</div>

{/* — Вывод ————— */}
<div className="shrink-0 border-t border-slate-800" >
    <button
        type="button"
        onClick={() => setOutputOpen((o) => !o)}
        className="flex items-center justify-between gap-2"
    >
        <span className="inline-flex items-center gap-2" >
            <Terminal className="w-3.5 h-3.5 text-emerald-500" />
        </span>
        <span className="inline-flex items-center gap-2" >
            {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />}
            <span className="text-slate-600">{outputOpen ? "Закрыть" : "Открыть"}</span>
        </span>
    </button>
    {outputOpen && (
        <pre
            data-testid="compiler-output"
            className="flex-1 min-h-[70px] overflow-auto text-slate-300"
        >
            {output}
        </pre>
    )}
</div>

```

```
];
```

```
let counter = 3;
```

```
export const QueueViz: React.FC = () => {
  const [queue, setQueue] = useState<Customer[]>([
    { id: 'q1', name: 'Студент Вася', emoji: '🍌', color: 'red',
      голода после матана!', animState: 'idle' },
    { id: 'q2', name: 'Кодер Семён', emoji: '🍌', color: 'blue',
      ишу ИИ за порцию борща!', animState: 'idle' },
    { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'green',
      ез сметаны, плиз.', animState: 'idle' },
  ]);
```

```
  const runtime = useVizRuntime();
  const liveQ = vizArray(runtime?.variables?.q);
  const liveV = vizString(runtime?.variables?.v);
  const displayQueue: Customer[] = liveQ
    ? liveQ.map((value, index) => ({
        id: `python-${index}`,
        name: String(value),
        emoji: "🐍",
        color: "from-emerald-500/20 to-emerald-600/20 border",
        bubbleText: `q[${index}] из Python`,
        animState: "idle",
      }))
    : queue;
```

```
  const [servedHistory, setServedHistory] = useState<string[]>([]);
  const [isServing, setIsServing] = useState(false);
  const [shakeEmpty, setShake] = useState(false);
  const [log, setLog] = useState<string[]>([]);
```

```
  const addLog = (msg: string) => setLog(prev => [...prev, msg]);
```

```
  // ENQUEUE: Встать в конец очереди (хвост / Tail)
```

```
  const enqueue = useCallback(() => {
    if (isServing) return;
```



```

        r-pointer border border-slate-700"
    >
    Сброс
  </button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12" >

  {/* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */}
  <div className={`md:col-span-9 bg-slate-900 rounded-
    -[300px] overflow-hidden ${shakeEmpty ? 'anim
  <div className="absolute top-4 left-4 text-[1.2em]
    Очередь голодных гостей (Буфер FIFO / BFS)
    {liveQ && <span className="ml-2 text-emerald-500">
  </div>

  <div className="flex-1 flex flex-col justify-between" >
    <div className="flex items-end justify-between" >

      {/* ПОВАРИХА И КОТЕЛ С СУПОМ */}
      <div className="flex flex-col items-center" >
        <div className="relative" >
          <div className="absolute -top-10 left-1/2" >
            <div className="w-1.5 h-6 bg-slate-900" >
              <div className="w-2 h-4 bg-slate-400" >
            </div>
            <span className="text-5xl block animate-pulse" >
          </div>
          <div className="text-center" >
            <span className="text-3xl block" >
              <span className="text-[10px] font-bold font-mono tracking-wider" >
                ПОВАР
              </span>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

        </div>

        {/* Персонаж */}
        <div className={`
            w-14 h-14 rounded-2xl border-2
            relative
            ${customer.color}
            ${isHead ? 'ring-4 ring-amber-500' : ''}
        `}>
            <span className="text-3xl">{customer.name}</span>

            {/* Индексы HEAD/TAIL */}
            {(isHead || isTail) && (
                <span className={`
                    absolute -bottom-2 text-[0.8em]
                    ${isHead ? 'bg-amber-500' : 'bg-slate-900'}
                `}>
                    {isHead && isTail ? 'h+t' : isHead ? 'h' : 't'}
                </span>
            )}
        </div>
    </div>
);
}}
})
</div>

</div>
</div>

{/* Пол кухни */}
<div className="w-full h-3 bg-gradient-to-r from-slate-900 to-slate-500">
</div>

{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
<div className="md:col-span-3 bg-slate-900 rounded-2xl p-4">
    <div className="absolute top-4 left-4 text-[1.2em] font-medium">
        Сытые гости (Архив FIFO)
    </div>

```

```
      {log.map((entry, idx) => (  
        <div  
          key={idx}  
          className={`text-xs font-mono flex items-center justify-between gap-2`}  
        >  
          {idx === 0 ? <span className="text-indigo-500 font-weight-bold">1</span> :  
            <span>{entry}</span>  
        </div>  
      )})  
    </div>  
  </div>  
);  
};
```

ФАЙЛ 68/113: src/components/SalmonAutomatonWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 101

```
import React, { useState, useEffect, useContext, useMemo } from 'react';  
import { Terminal } from 'lucide-react';  
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from 'viz';  
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';
```

```
interface TrieNode {  
  id: number;  
  char: string;  
  parent: number;  
  children: { [key: string]: number };  
  fail: number;  
  term_link: number;  
  is_terminal: boolean;  
  words: string[];  
  depth: number;  
  x?: number;  
  y?: number;  
}
```

```

    .map((w) => (typeof w === "string" ? w : typeof w !== "string" ? "" : w))
    .filter((w) => Boolean(w))
    .map((w) => w.toUpperCase().replace(/[^A-ZА-Я]/g, ""))
    .filter((w) => w.length > 0)
    .slice(0, 8);
  if (!parsed.length) return;
  if (parsed.join("|") === words.join("|")) return;
  setWords(parsed);
  buildInitialTrie(parsed);
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [liveWordsSig]);

// состояние автомата и шаг BFS – подсвечиваем вершины
useEffect(() => {
  if (liveState !== null && liveState >= 0 && nodes[liveState]) {
    if (liveStep !== null && liveStep >= 0) {
      // simulationStep – грубый статус: 0 = бор готов,
      const total = Object.keys(nodes).length;
      const failReady = liveFail ? Object.keys(liveFail).length : 0;
      setSimulationStep(liveStep <= 0 ? 0 : failReady ? failReady : total);
    }
    if (liveQueue) {
      const q = liveQueue.map((x) => (typeof x === "number" ? x : undefined));
      if (q.length) setBfsQueue(q);
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [vars]);

  const compilerLinked = Boolean(liveWordsSig || liveState);

  // Регулярное моргание рыбы
  useEffect(() => {
    const blinkInterval = setInterval(() => {
      setIsBlinking(true);
      setTimeout(() => setIsBlinking(false), 200);
    }, 4000);
    return () => clearInterval(blinkInterval);
  }, []);

```

```

        newNodes[curr].words.push(word);
        newNodes[curr].is_terminal = true;
    }
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
    const childKeys = Object.values(newNodes[u].children);
    newNodes[u].y = 40 + depth * paddingY;

    if (childKeys.length === 0) {
        newNodes[u].x = 40 + currentX * paddingX;
        currentX++;
    } else {
        let leftX = -1;
        let rightX = -1;
        childKeys.forEach(v => {
            layoutDFS(v, depth + 1);
            if (leftX === -1) leftX = newNodes[v].x!;
            rightX = newNodes[v].x!;
        });
        newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : leftX + paddingX / 2;
        if (leftX === -1) currentX++;
    }
};

layoutDFS(0, 0);

setNodes(newNodes);
setSimulationStep(0);
setBfsQueue([]);
setCurrentNode(null);
setSpeechText(
    `Отлично! Я построил базовое префиксное дерево (БПД) для слова "S", чтобы расставить fail-ссылки!`
);

```



```

    });

    setNodes(updatedNodes);
    setBfsQueue(queue);
    setSimulationStep(1);
    setCurrentNode(null);
    setSpeechText(
        'Начинаем обход в ширину (BFS)! Все вершины на графе  

        уже суффикса просто нет. Жми «Следующий шаг»!');
    });
};

// Один шаг BFS
const stepBFS = () => {
    if (bfsQueue.length === 0) {
        setSimulationStep(2);
        setCurrentNode(null);
        setSpeechText(
            'Ура! Очередь пуста! Мы успешно построили полный  

            граф. Теперь можно полюбоваться таблицей переходов!');
    };
    return;
}

const r = bfsQueue[0];
const newQueue = bfsQueue.slice(1);
const updatedNodes = { ...nodes };
const rNode = updatedNodes[r];
const childrenIds = Object.values(rNode.children);

let logs = `Рассматриваем вершину #${r} (${rNode.children})`;

for (const [char, u] of Object.entries(rNode.children)) {
    newQueue.push(u);
    let v = rNode.fail;
    while (v !== 0 && updatedNodes[v].children[char] !== undefined) {
        v = updatedNodes[v].fail;
    }
}

```

```

<div className="flex items-center gap-2">
  <button
    onClick={() => buildInitialTrie(words)}
    className="flex items-center gap-1 bg-slate
      transition-colors"
    title="Сбросить автомат"
  >
    <RotateCcw className="w-4 h-4" /> Сбросить
  </button>
</div>
</div>

```

```

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 g
  {/* Аватар Лосося (Сеня) с анимацией моргания и
  <div className="flex flex-col items-center just
    <div className="w-40 h-40 relative flex items
      full border border-blue-500/30 shadow-inner
      {/* SVG Лосося */}
    <svg
      viewBox="0 0 200 200"
      className={`w-36 h-36 transition-transform
        isTalking ? 'scale-105 drop-shadow-[0_0
        ]`}
    >
      {/* Тело лосося */}
      <ellipse cx="100" cy="100" rx="70" ry="45"
      <path d="M 40 85 Q 100 55 160 85 Q 100 115
        40 85"
        fill="#be123c"
        className="origin-[35px_100px] animate-
      />

      {/* Плавники */}
      <path d="M 90 55 L 70 30 L 110 55 Z" fill=

```

```

    {/* Пузырь с текстом (Баббл) */}
    <div className="md:col-span-2 bg-slate-800 border-2 border-transparent
      tween min-h-[140px]">
      {/* Треугольник баббла */}
      <div className="absolute -left-3 top-1/2 -transform rotate(-45deg)
        slate-800 border-b-[12px] border-b-transparent border-r-[12px]
          border-r-transparent border-t-[12px] border-t-transparent" />

      <div className="flex items-center space-x-2 text-white">
        <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}`} />
        <span>Прямое вещание из аквариума:</span>
      </div>

      <p className="text-slate-200 text-base leading-relaxed">
        {speechText}
      </p>

      <div className="mt-4 pt-3 border-t border-slate-700">
        <div className="text-xs text-slate-400 flex justify-between">
          <Info className="w-3.5 h-3.5 text-blue-400" />
          Статус: {simulationStep === 0 ? 'Бор готов к запуску!' : 'Бор работает!'}
        </div>

        <div className="flex gap-2">
          {simulationStep === 0 && (
            <button
              onClick={startBFS}
              className="bg-emerald-600 hover:bg-emerald-700 text-white px-3 py-1 shadow-lg shadow-emerald-900/40 transition-colors" />
            <Play className="w-4 h-4" /> Запустить
          )}
          {simulationStep === 1 && (
            <button
              onClick={stepBFS}
              className="bg-blue-600 hover:bg-blue-700 text-white px-3 py-1 shadow-lg shadow-blue-900/40 transition-colors" />
            Шаг
          )}
        </div>
      </div>
    </div>
  )}

```

```

<defs>
  <marker id="arrowhead" markerWidth="6"
    <polygon points="0 0, 6 3, 0 6" fill=
  </marker>
  <marker id="fail-arrow" markerWidth="6"
    <polygon points="0 0, 6 3, 0 6" fill=
  </marker>
</defs>

{/* Draw Edges */}
{Object.values(nodes).map(node => {
  if (node.id === 0) return null;
  const parent = nodes[node.parent];
  if (!parent.x || !parent.y || !node.x || !node.y) return null;

  const isCurrent = currentNode === node.id;

  return (
    <g key={`edge-${node.id}`}>
      <line
        x1={parent.x}
        y1={parent.y + 16}
        x2={node.x}
        y2={node.y - 16}
        stroke={isCurrent ? '#60a5fa' : '#ccc'}
        strokeWidth="2"
        markerEnd="url(#arrowhead)"
      />
      <text
        x={({parent.x + node.x} / 2 - 10)}
        y={({parent.y + node.y} / 2)}
        fill="#94a3b8"
        fontSize="12"
        fontWeight="bold"
      >
        {node.char}
      </text>
    </g>
  )
})}

```

```

    let stroke = '#475569';

    if (isCurrent) {
      fill = '#2563eb';
      stroke = '#60a5fa';
    } else if (isInQueue) {
      fill = '#b45309';
      stroke = '#fbbf24';
    } else if (node.is_terminal) {
      fill = '#059669';
      stroke = '#34d399';
    }

    return (
      <g key={`node-${node.id}`}>
        <circle
          cx={node.x}
          cy={node.y}
          r="16"
          fill={fill}
          stroke={stroke}
          strokeWidth="2"
        />
        <text
          x={node.x}
          y={node.y + 4}
          fill="white"
          fontSize={isRoot ? "10" : "12"}
          fontWeight="bold"
          textAnchor="middle"
        >
          {isRoot ? 'R' : node.char}
        </text>
        {!isRoot && (
          <text
            x={node.x + 12}
            y={node.y - 12}
            fill="#94a3b8"

```

```
</div>
```

```
<form onSubmit={handleAddWord} className="flex flex-col gap-2" >
  <input
    type="text"
    value={newWord}
    onChange={(e) => setNewWord(e.target.value)}
    placeholder="НОВОЕ СЛОВО..."
    maxLength={8}
    className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
  />
  <button
    type="submit"
    className="bg-slate-700 hover:bg-slate-600 transition-colors"
  >
    <Plus className="w-3.5 h-3.5" /> Добавить
  </button>
</form>
</div>
```

```
{/* Таблица Вершин и суффиксных ссылок */}
<div className="lg:col-span-2 bg-slate-900/50 p-4" >
  <h5 className="text-white font-bold mb-3 flex items-center gap-1.5" >
    <span> </span> Таблица переходов и fail-ссылок
    <span className="text-xs text-slate-400 font-normal" >
      Всего вершин: {Object.keys(nodes).length}
    </span>
  </h5>
```

```
<div className="max-h-[220px] overflow-y-auto" >
  <table className="w-full text-left border-collapse" >
    <thead>
      <tr className="border-b border-slate-700" >
        <th className="py-2 px-3" >ID Вершины<
```

```

</td>
<td className="py-2.5 px-3">
  <span className="bg-slate-800 b
    {node.char}
  </span>
</td>
<td className="py-2.5 px-3 text-s
  {node.id === 0 ? '-' : `#${node
</td>
<td className="py-2.5 px-3">
  {node.id === 0 ? {
    <span className="text-slate-5
  } : {
    <span className="bg-indigo-95
      #{node.fail}
    </span>
  }}
</td>
<td className="py-2.5 px-3">
  {node.id === 0 || node.term_lin
    <span className="text-slate-5
  } : {
    <span className="bg-rose-950 l
0 transition-colors cursor-help" title={`Ск
      #{node.term_link}
    </span>
  }}
</td>
<td className="py-2.5 px-3 text-s
  {Object.keys(node.children).len
    ? '0'
    : Object.entries(node.children
      .map([c, id]) => `${c}→#
      .join(', '))}
</td>
<td className="py-2.5 px-3">
  {node.words.length === 0 ? {
    <span className="text-slate-6

```

```

        }}
      </div>
    </div>
  );
};

```

ФАЙЛ 69/113: src/components/SegmentTreeVisualizer.tsx
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
import { useVizRuntime, vizNumber, vizArray } from '../vizRuntime';

```

```

// ===== TYPES =====

```

```

interface Step {
  id: number;
  desc: string;
  codeLine: number;
  treeState: Record<number, number>; // node index to value
  highlightNodes: number[]; // current highlighted nodes
  mergeChildren?: [number, number]; // pair being merged
  arrayHighlight?: [number, number]; // [L, R] range in array
  result?: number; // partial result
}

```

```

// ===== PURE LOGIC: build tree =====

```

```

function buildTreeFull(arr: number[]): Record<number, number> {
  const n = arr.length;
  const tree: Record<number, number> = {};
  function go(v: number, l: number, r: number) {
    if (l === r) { tree[v] = arr[l]; return; }
    const m = (l + r) >> 1;
    go(2 * v, l, m);
    go(2 * v + 1, m + 1, r);
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
  }
  go(1, 0, n - 1);
  return tree;
}

```



```

    { ...tree }, highlightNodes: [v], arrayHighlight:
go(2 * v, l, m);
steps.push({ id: id++, desc: `Вернулись в build(${v}
    lightNodes: [v], arrayHighlight: [l, r] });
go(2 * v + 1, m + 1, r);
tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
steps.push({ id: id++, desc: `Объединяем: tree[${v}]
    state: { ...tree }, highlightNodes: [v], mergeChild
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: `    Дерево построено! Корень
    Highlight: [0, n - 1] });
return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
    const n = arr.length;
    const tree = buildTreeFull(arr);
    const steps: Step[] = [];
    let id = 0;

    function go(v: number, l: number, r: number, ql: number, qr: number) {
        if (ql > r || qr < l) {
            steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]
                codeLine: 2, treeState: tree, highlightNodes: [v]
            return 0;
        }
        if (ql <= l && r <= qr) {
            steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]
                ] = ${tree[v]}`, codeLine: 4, treeState: tree, highlightNodes: [v]
            return tree[v];
        }
        const m = (l + r) >> 1;
        steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]
            codeLine: 6, treeState: tree, highlightNodes: [v]
        const left = go(2 * v, l, m, ql, qr);
        const right = go(2 * v + 1, m + 1, r, ql, qr);
    }
}

```

```

        r--;
        res += tree[r] ?? 0;
        steps.push({ id: id++, desc: `r=${r + 1} нечётный`,
            , treeState: tree, highlightNodes: [r], arrayHighlight: [qL, qR], result: res });
        highlights.push(r);
    }
    l >>= 1;
    r >>= 1;
    if (l < r) {
        steps.push({ id: id++, desc: `Поднимаемся: l = ${l}`,
            , arrayHighlight: [qL, qR], result: res });
    }
}
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}])`,
    , highlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val: number; left: VNode; right: VNode; }
```

```

function buildVisualTree(treeState: Record<number, number>, l: number, r: number): VNode {
    if (l === r) return { v, l, r, val: treeState[v] ?? null, left: null, right: null };
    const m = (l + r) >> 1;
    return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(treeState, l, m),
        right: buildVisualTree(treeState, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```

const BUILD_CODE = [
    "build(v, l, r) {",
    "    if (l === r) { tree[v] = A[l]; return; }",
    "    // -----",
    "    // -----",
    "    mid = (l + r) >> 1;",
    "    build(2*v, l, mid);",
    "    build(2*v+1, mid+1, r);",
    "    tree[v] = tree[2*v] + tree[2*v+1];",
    "}"
]

```

```

    }, [playing, idx, steps, speed]));

    const step = steps[idx] || steps[0];
    if (!step) return null;

    const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-
      border-indigo-500 bg-indigo-600', childRing: 'ring-
      -900' }, emerald: { btn: 'bg-emerald-600 hover:bg-e
      30 border-emerald-500 bg-emerald-600', childRing: '
      -slate-900' }, amber: { btn: 'bg-amber-600 hover:bg
      der-amber-500 bg-amber-600', childRing: 'ring-rose-
      [color];

    return { step, idx, setIdx, playing, setPlaying, speed
  }

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr
  | 'emerald' | 'amber'; arr: number[] }) {
  const p = Player({ steps, color });
  const runtime = useVizRuntime();
  const liveV = vizNumber(runtime?.variables?.v);
  const liveTree = vizArray(runtime?.variables?.tree);
  const liveI = vizNumber(runtime?.variables?.i);

  if (!p) return null;
  const { step, idx, setIdx, playing, setPlaying, speed
  const n = arr.length;

  const fullTree = useMemo(() => buildTreeFull(arr), [a
  const vTree = useMemo(() => buildVisualTree(step.tree

  const renderNode = useCallback((nd: VNode): React.Re
    const isCompilerActive = liveV === nd.v;
    const isHigh = step.highlightNodes.includes(nd.v) |
    const isChild = step.mergeChildren?.includes(nd.v);

    // Live compiler value or step value or pre-calcula

```

```

    ? 'border-emerald-500/30' : 'border-amber-500/30'
  } /* Header */
  <div className={`px-4 py-2.5 flex items-center justify-center
    -emerald-950/50' : 'bg-amber-950/50'} border-b-2 border-amber-400`>
    <h4 className="font-bold text-sm text-white flex items-center">
      <span className="text-[10px] font-mono text-slate-400">Исходный код
    </div>

  { /* Array strip */
    <div className="px-3 py-2 bg-slate-950 border-b-2 border-amber-400">
      <div className="text-[11px] font-bold text-slate-400">
        <span className="text-amber-400">Исходный массив
        <span className="text-slate-400 font-mono text-[10px]">
      </div>
      <div className="flex flex-wrap gap-1 justify-center">
        {arr.map((v, i) => {
          const inRange = step.arrayHighlight && i >= 0;
          const isLiveI = liveI === i;
          return (
            <div key={i} className={`flex flex-col items-center justify-center
              -amber-400 ring-2 ring-amber-400/40' : inRange ? 'bg-emerald-950 border-emerald-500' : 'bg-amber-950 border-amber-400'`>
              <span className="text-[8px] text-amber-400">{v}
              <span className="text-[12px] font-extrabold text-slate-400">{i}
            </div>
          );
        })}
      </div>
    </div>

  { /* Tree */
    <div className="flex-1 p-2 overflow-x-auto bg-slate-950">
      <div className="scale-[0.85] origin-top min-w-max">
    </div>

  { /* Code */
    <pre className="px-3 py-2 text-[9px] md:text-[10px]">

```

```

        </button>
        <button onClick={() => { setPlaying(false); s
          г вперёд" title="Шаг вперёд" className="p-1
          <ChevronRight className="w-3.5 h-3.5" /></b
      </div>
      <select value={speed} onChange={e => setSpeed(N
        text-[9px] font-bold rounded px-1.5 py-1 outl
        <option value={1200}>Медл.</option>
        <option value={700}>Норма</option>
        <option value={350}>Быстро</option>
        <option value={120}>Турбо</option>
      </select>
    </div>

    {/* Progress bar */}
    <div className="h-1 bg-slate-950"><div className=
      + 1) / total) * 100 : 0}%` }} /></div>
  </div>
  );
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
  const runtime = useVizRuntime();
  const liveI = vizNumber(runtime?.variables?.i);
  const liveA = vizArray(runtime?.variables?.A) || vizA
  const [arr, setArr] = useState<number[]>([5, 8, 3, 12
  const [customInput, setCustomInput] = useState('5, 8,
  const [qL, setQL] = useState(1);
  const [qR, setQR] = useState(4);
  const [view, setView] = useState<'build' | 'queries'

  // Update array from live compiler variables A / a if
  useEffect(() => {
    if (liveA && Array.isArray(liveA)) {
      const nums = liveA.filter((x): x is number => typ

```

```

<div className="flex flex-col lg:flex-row gap-4"
  {/* Array input */}
  <form onSubmit={handleApply} className="flex-
    <div className="flex items-center justify-b
      <label className="text-[10px] font-bold up
    label>
      <button type="button" onClick={randomize}
    "><RefreshCw className="w-3.5 h-3.5" /></bu
  </div>
  <div className="flex gap-2">
    <input
      value={customInput}
      onChange={e => setCustomInput(e.target.v
      className="flex-1 bg-slate-900 border b
    one focus:border-indigo-500"
      placeholder="5, 8, 3, 12, 7, 2"
    />
    <button type="submit" className="bg-indigo
    ition-colors">Применить</button>
  </div>
  <div className="flex flex-wrap gap-1.5 pt-1"
    {arr.map((v, i) => {
      <span key={i} className={`border px-2 p
    ext-amber-200 ring-2 ring-amber-400/30" : "l
      <span className={liveI === i ? "text-l
    </span>
    </span>
    }}}
    <span className="text-xs text-slate-500 f
  </div>
</form>

{/* Query range */}
<div className="bg-slate-950 p-4 rounded-xl b
  <label className="text-[10px] font-bold upp
  <div className="flex items-center gap-2">
    <div className="flex items-center gap-1">
      <span className="text-xs text-slate-400
    <input

```

```

        er:text-slate-200'}``}>
        <Search className="w-3.5 h-3.5" /> Запрос: св
    </button>
    <button onClick={() => setView('theory')} class=
        n-colors ${view === 'theory' ? 'border-blue-5
        te-200'}``}>
        <Info className="w-3.5 h-3.5" /> Почему / Спра
    </button>
</div>

{/* ---- TAB CONTENT ---- */}
{view === 'build' && (
    <SimPanel
        key={`build-${arr.join('-')}`}
        title="Построение дерева (рекурсия)"
        icon=" "
        steps={buildSteps}
        code={BUILD_CODE}
        color="indigo"
        arr={arr}
    />
)}

{view === 'queries' && (
    <div className="grid grid-cols-1 xl:grid-cols-2"
        <SimPanel
            key={`qtd-${arr.join('-')}-${qL}-${qR}`}
            title={`Запрос sum([${qL}..${qR}]) – Сверху`
            icon="↑ "
            steps={queryTDSteps}
            code={QUERY_TD_CODE}
            color="emerald"
            arr={arr}
        />
        <SimPanel
            key={`qbu-${arr.join('-')}-${qL}-${qR}`}
            title={`Запрос sum([${qL}..${qR}]) – Снизу-н
            icon="↑ "

```

```

        <p className="text-xs text-slate-300 mb-3">
            Начинаем с корня. Если отрезок узла полн
        наче спускаемся в обоих детей.
        </p>
        <div className="space-y-1.5 text-xs">
            <div className="flex justify-between bo
            <span className="text-rose-400 font-mono">0(
            <div className="flex justify-between bo
            ><span className="text-rose-400 font-mono">
            <div className="flex justify-between"><
            d-400 font-bold">✓ легко</span></div>
            </div>
        </div>
        <div className="bg-slate-950 p-5 rounded-xl"
            <div className="absolute -top-3 left-4 bg
            rder border-amber-500 shadow-md">
                † Запрос Снизу-вверх (Итерация)
            </div>
            <p className="text-xs text-slate-300 mb-3">
                Стартуем с двух указателей (l и r) на л
                ём его левого соседа. Поднимаемся пока l {"
            </p>
            <div className="space-y-1.5 text-xs">
                <div className="flex justify-between bo
                <span className="text-emerald-400 font-mono":
                <div className="flex justify-between bo
                ><span className="text-emerald-400 font-mono
                <div className="flex justify-between"><
                00 font-bold">x сложно</span></div>
                </div>
            </div>
        </div>
    </div>

    {/* Verdict */}
    <div className="bg-slate-950 p-5 rounded-xl b
        <h4 className="font-bold text-blue-400 mb-2"
        <p className="text-sm text-slate-300 leading
            <strong className="text-white">Итеративны

```



```

const groups = useMemo(() => {
  const q = query.trim().toLowerCase();
  const filtered = q ? chapters.filter((c) => c.title
  return SECTIONS.map((section) => ({
    section,
    items: filtered.filter((c) => sectionOf(c.id).id :
  })).filter((g) => g.items.length > 0);
}, [query]);

return (
  <aside className="w-full bg-slate-900/80 lg:bg-tran
    <div className={compact ? "p-2 pb-1.5 shrink-0" :
      {compact && (
        <h3 className="text-[10px] font-bold text-sla
          <Compass className="w-3.5 h-3.5 text-indigo
        </h3>
      )}
    <h3 className={`text-xs font-bold text-slate-400
      }>
      <Compass className="w-4 h-4 text-indigo-400"
    </h3>
    {!compact && (
      <div className="relative">
        <Search className="w-4 h-4 text-slate-500 abs
        <input
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          placeholder="Поиск по темам.."
          className="w-full bg-slate-800/70 border bo
          e-500 focus:outline-none focus:border-indig
        />
        {query && (
          <button
            onClick={() => setQuery("")}
            className="absolute right-2 top-1/2 -tran
            aria-label="Очистить поиск"
          >
            <X className="w-3.5 h-3.5" />

```

```

        aria-current={isSelected ? "page" :
        className={`w-full text-left rounded
" : "px-2 py-1.5"} ${
        isSelected
        ? "bg-indigo-600/15 border-indi
        : "bg-slate-800/40 border-transp
    }`}
    >
    <span className="shrink-0 mt-px rounded
nt-bold text-indigo-300">
        {num ? num[1] : "*"}
    </span>
    <span className={`font-semibold lea
        {label}
        {hasViz && (
            <span className="ml-1.5 inline-l
ая визуализация" />
        )}
    </span>
    {!compact && (
        <ChevronRight
            className={`w-4 h-4 shrink-0 mt
            isSelected ? "text-indigo-400
        }`}
    />
    )}
    </button>
    );
    })}
    </div>
    </div>
    )})
    </div>
    </aside>
    );
};

```

```

const [queue, setQueue] = useState<Person[]>([
  { id: '1', name: 'Студент Вася', icon: ' ' },
  { id: '2', name: 'Голодный кодер', icon: ' ' },
  { id: '3', name: 'Кот Борис', icon: ' ' },
]);
const [dequeueMessage, setDequeueMessage] = useState<

// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
  { id: '1', text: 'Кандидат: "Привет, я искусственный"',
  { id: '2', text: 'Кандидат: "Привет, я испек вкусный"',
  { id: '3', text: 'Кандидат: "Привет, я испанский ин
]);
const [newCustomText, setNewCustomText] = useState('')
const [newCustomProb, setNewCustomProb] = useState(0.)
const [heapMessage, setHeapMessage] = useState<string>('')

// Stack handlers
const addPlate = () => {
  const presets = [
    { name: 'Сковорода от омлета', icon: ' ' },
    { name: 'Кастрюля от супа', icon: ' ' },
    { name: 'Чашка от кофе', icon: '☑' },
    { name: 'Миска с остатками салата', icon: ' ' },
    { name: 'Тарелка от тако', icon: ' ' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPlate = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon
  };
  setStack(prev => [...prev, newPlate]);
  setPopMessage(`Положили поверх стопки: ${newPlate.name}`);
};

const popPlate = () => {
  if (stack.length === 0) {

```

```

        return;
    }
    const firstPerson = queue[0];
    setQueue(prev => prev.slice(1));
    setDequeueMessage(`Выдан суп первому в очереди (F
});

const resetQueue = () => {
    setQueue([
        { id: '1', name: 'Студент Вася', icon: ' ' },
        { id: '2', name: 'Голодный кодер', icon: ' ' },
        { id: '3', name: 'Кот Борис', icon: ' ' },
    ]);
    setDequeueMessage("Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
    e.preventDefault();
    if (!newCustomText.trim()) return;
    const item: Hypothesis = {
        id: Date.now().toString(),
        text: `Кандидат: "${newCustomText}"`,
        probability: Number(newCustomProb)
    };
    setHeap(prev => [...prev, item]);
    setNewCustomText('');
    setHeapMessage(`Добавлена гипотеза с вероятностью
});

const popHeap = () => {
    if (heap.length === 0) {
        setHeapMessage("⚠ Куча пуста! Нет кандидатов для
        return;
    }
    // Find highest probability
    let maxIdx = 0;
    for (let i = 1; i < heap.length; i++) {

```

```

        activeTab === 'stack'
          ? 'bg-blue-600/20 border-blue-500 text-blue-50'
          : 'bg-slate-800/60 border-slate-700/60 text-slate-50'
      }` }
    >
    <div className="flex items-center gap-3">
      <Layers className={`w-5 h-5 ${activeTab === 'stack' ? 'bg-blue-600/20 border-blue-500 text-blue-50' : 'bg-slate-800/60 border-slate-700/60 text-slate-50'} `}>
        <div className="text-left">
          <div className="font-bold text-sm text-white">
            <div className="text-xs opacity-80">LIFO
          </div>
        </div>
      </div>
      <span className="text-xs bg-blue-500/20 text-white">
        DFS
      </span>
    </button>

    <button
      onClick={() => setActiveTab('queue')}
      className={`flex items-center justify-between gap-3 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500 text-indigo-50' : 'bg-slate-800/60 border-slate-700/60 text-slate-50'} `}
    >
      <div className="flex items-center gap-3">
        <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500 text-indigo-50' : 'bg-slate-800/60 border-slate-700/60 text-slate-50'} `}>
          <div className="text-left">
            <div className="font-bold text-sm text-white">
              <div className="text-xs opacity-80">FIFO
            </div>
          </div>
        </div>
        <span className="text-xs bg-indigo-500/20 text-white">
          BFS
        </span>
      </button>

    <button

```

```

rounded-xl text-sm font-bold shadow-lg tra
>
  <Plus className="w-4 h-4" /> Поставить
</button>
<button
  onClick={popPlate}
  className="flex-1 md:flex-none flex item
border-blue-500/40 px-4 py-2.5 rounded-xl
>
  Помыть верхнюю
</button>
<button
  onClick={resetStack}
  title="Сбросить"
  className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
>
  <RotateCcw className="w-5 h-5" />
</button>
</div>
</div>

{popMessage && {
  <div className="bg-blue-950/60 border borde
imate-fade-in">
    <span className="inline-block w-2 h-2 roun
    {popMessage}
  </div>
}}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
>
  {stack.length === 0 ? {
    <div className="text-center py-12 text-sl
    <div className="text-5xl mb-3"> </div>
    <p className="text-base font-bold">Стор
    <p className="text-xs mt-1">Добавьте гр

```

```

        <div className="mt-6 text-xs text-slate-500" style={{
          position: 'relative',
          height: 100px,
          border: 1px solid #ccc,
          padding: 10px,
          background: 'linear-gradient(to bottom, #f0f0f0 49%, #fff 49%, #fff 51%, #f0f0f0 51%)',
          backgroundSize: '100% 100%',
          backgroundRepeat: 'repeat-y',
        }}>
          ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
        </div>
      </div>
    </div>
  )}

```

```

{ /* QUEUE TAB */}
{activeTab === 'queue' && {
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
      tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-whi
          <span> Симуляция очереди за супом</span>
          <span className="text-xs font-mono bg-i
            /span>
          </h3>
          <p className="text-slate-400 text-xs mt-1" style={{
            padding: 0 10px,
          }}>
            Кто первым пришел, тот первым получил суп
          </p>
        </div>
        <div className="flex flex-wrap items-center gap-2">
          <button
            onClick={addPerson}
            className="flex-1 md:flex-none flex item
              -2.5 rounded-xl text-sm font-bold shadow-lg" style={{
                padding: 5px 10px,
              }}>
            <Plus className="w-4 h-4" /> Встать в очередь
          </button>
          <button
            onClick={servePerson}
            className="flex-1 md:flex-none flex item
              er border-indigo-500/40 px-4 py-2.5 rounded
            >
            Налить суп первому
          </button>
          <button

```

```

    {queue.map((person, index) => {
      const isFirst = index === 0;
      return (
        <div
          key={person.id}
          className={`flex flex-col items-center
            ${isFirst
              ? 'bg-gradient-to-b from-indigo-500 to-white
                relative'
              : 'bg-slate-900/90 border-slate-800'}
          `}
        >
          {isFirst && (
            <span className="absolute -top-10 left-0
              se tracking-wider shadow">
                Первый (Head)
              </span>
            )}
          <span className="text-4xl mb-2">{
            <span className={`font-bold text-2xl
              {person.name}
            </span>
            <span className={`text-[10px] font-mono
              isFirst ? 'bg-indigo-500/30 text-white' : ''
            `}>
              {isFirst ? 'Получает сун' : `В ${person.age} лет`}
            </span>
            </div>
          )};
        )}}
    )}
  )}

```

```

    <div className="flex flex-col items-center justify-center
      -600 min-w-[120px] h-[120px]">
      <Plus className="w-6 h-6 mb-1" />
      <span className="text-xs font-mono uppercase">
        {person.name}
      </span>
    </div>
  </div>

```



```

        </button>
      </div>
    </div>

    {/* Add Form */}
    <form onSubmit={addHypothesis} className="bg-
      s-12 gap-4 items-end">
      <div className="md:col-span-6">
        <label className="block text-xs font-bold">
          <input
            type="text"
            value={newCustomText}
            onChange={e => setNewCustomText(e.target.value)}
            placeholder="например: "Привет, я лучший!"
            className="w-full bg-slate-900 border border-
              rder-emerald-500 transition-colors"
          />
        </div>
        <div className="md:col-span-3">
          <label className="block text-xs font-bold">
            Вероятность: <span className="text-emerald-500">
              {newCustomProb}
            </span>
          </label>
          <input
            type="range"
            min="0.01"
            max="0.99"
            step="0.01"
            value={newCustomProb}
            onChange={e => setNewCustomProb(Number(e.target.value))}
            className="w-full accent-emerald-500 cursor-pointer"
          />
        </div>
        <div className="md:col-span-3">
          <button
            type="submit"
            disabled={!newCustomText.trim()}
            className="w-full flex items-center justify-center gap-2
              bg-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
          >
            Добавить гипотезу
          </button>
        </div>
      </div>
    </form>
  )}
)

```

```

        isTop
        ? 'bg-gradient-to-r from-emerald-500 to-slate-900/10 scale-[1.01]'
        : 'bg-slate-900/80 border-slate-900/10'
      }` }
    }
  }
  >
  <div className="flex items-center justify-between" >
    <span className={`flex items-center justify-between`} >
      isTop ? 'bg-emerald-500 text-white' : 'bg-slate-900/80 text-white'
    </span>
    <div>
      <div className={`font-bold text-white`} >
        {item.text}
      </div>
      {isTop && (
        <span className="text-[10px] font-mono text-white" >
          Вершина Кучи (Root) • Будущее
        </span>
      )}
    </div>
  </div>

  <div className="flex items-center justify-between" >
    { /* Custom progress bar */ }
    <div className="hidden sm:block" >
      <div
        className={`h-full rounded-full`}
        style={{ width: `${percent}%` }}
      ></div>
    </div>
    <span className={`font-mono font-mono text-white`} >
      isTop ? 'text-emerald-400 text-white' : 'text-slate-400 text-white'
    </span>
  </div>

```

```
-----
export const SplayPlayground: React.FC<{ defaultTab?: Tab }> = ({ defaultTab }) => {
  const [tab, setTab] = useState<Tab>(defaultTab);
  const chapterId = useContext(VizChapterContext);

  // вкладка = подрежим страницы: панель Python подстав.
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, tab);
  }, [chapterId, tab]);

  return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border" >
      <div className="flex flex-wrap items-center justify-between" >
        <div className="flex items-center gap-3" >
          <div className="p-2.5 bg-indigo-600 text-white" >
            <span className="text-2xl" > </span>
          </div>
          <div>
            <h3 className="text-2xl font-bold text-white" >
              <p className="text-sm text-indigo-300" >Один
            </p>
          </div>
        </div>
      </div>

      <div className="flex flex-wrap gap-2 mb-5" >
        {[
          { id: "cases" as Tab, label: "1 · Механика: Z" },
          { id: "big" as Tab, label: "2 · Большое дерев" },
          { id: "play" as Tab, label: "3 · Собери сам" },
        ]}.map((t) => (
          <button
            key={t.id}
            onClick={() => setTab(t.id)}
            className={`px-4 py-2 rounded-lg text-xs font-medium
              ${tab === t.id ? "bg-indigo-600 text-white" : ""}
            `}
          >
            {t.label}
          </button>
        ))
      </div>
    </div>
  );
};
```

```

    left: TNode | null;
    right: TNode | null;
}

const leaf = (key: number): TNode => ({ key, left: null

const clone = (t: TNode | null): TNode | null =>
    t ? { key: t.key, left: clone(t.left), right: clone(t

/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
export const PRESETS: { name: string; hintCase: string;
    {
        name: "Линия влево",
        hintCase: "Zig-Zig",
        target: 1,
        build: () => {
            const n1 = leaf(1);
            const n2: TNode = { key: 2, left: n1, right: leaf
            const n4: TNode = { key: 4, left: n2, right: leaf
            return { key: 6, left: n4, right: leaf(7) };
        },
    },
    {
        name: "Змейка",
        hintCase: "Zig-Zag",
        target: 3,
        build: () => {
            const n3 = leaf(3);
            const n2: TNode = { key: 2, left: leaf(1), right:
            const n4: TNode = { key: 4, left: n2, right: leaf
            return { key: 6, left: n4, right: leaf(7) };
        },
    },
    {
        name: "Один уровень",
        hintCase: "Zig",
        target: 2,
        build: () => ({ key: 4, left: { key: 2, left: leaf(

```

```

        x.right = p;
    } else {
        p.right = x.left;
        x.left = p;
    }

    if (!gg) return x;
    if (gg.left === p) gg.left = x;
    else gg.right = x;
    return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number)
    const path = findPath(root, key);
    if (path.length < 2) return null;
    const x = path[path.length - 1];
    const p = path[path.length - 2];
    const g = path.length >= 3 ? path[path.length - 3] : null;

    if (!g) return { node: x.key, kind: "Zig" };

    const xLeft = p.left === x;
    const pLeft = g.left === p;
    if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
    return { node: x.key, kind: "Zig-Zag (сначала нижняя" };
}

/* ----- раскладка -----

interface Placed {
    key: number;
    x: number;
    y: number;
    depth: number;
}

function layout(root: TNode | null): { nodes: Placed[];

```

```

const solved = tree.key === target;

const reset = useCallback(
  (idx = presetIdx) => {
    setPresetIdx(idx);
    setTree(PRESETS[idx].build());
    setHistory([]);
    setMoves([]);
    setShowAnswer(false);
  },
  [presetIdx]
);

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, target))
const expected = useMemo(() => (solved ? null : canonicalNext(tree, target));

const clickNode = (key: number) => {
  if (solved) return;
  const p = findPath(tree, key);
  if (p.length < 2) return; // корень крутить не с кем

  const want = canonicalNext(tree, target);
  setHistory((h) => [...h, clone(tree) as TNode]);
  setMoves((m) => [...m, { node: key, correct: want?.key === key }]);
  setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {
  if (history.length === 0) return;
  setTree(history[history.length - 1]);
  setHistory((h) => h.slice(0, -1));
  setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build(), target));

```

```

    style={{ minWidth: Math.min(width * 1.5, 420)
    role="img"
  >
    {edges.map(([a, b], i) => {
      const na = nodes.find((n) => n.key === a);
      const nb = nodes.find((n) => n.key === b);
      if (!na || !nb) return null;
      const onPath = path.has(a) && path.has(b);
      return (
        <line
          key={i}
          x1={na.x}
          y1={na.y}
          x2={nb.x}
          y2={nb.y}
          stroke={onPath ? "#6366f1" : "#475569"}
          strokeWidth={onPath ? 3 : 1.8}
          style={{ transition: "all 700ms cubic-bezier(0.4, 0, 0.2, 1)" }}
        />
      );
    })}

    {nodes.map((n) => {
      const isTarget = n.key === target;
      const clickable = !solved && n.depth > 0;
      return (
        <g
          key={n.key}
          onClick={() => clickable && clickNode(n)}
          style={{
            transform: `translate(${n.x}px, ${n.y}px)`,
            transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
            cursor: clickable ? "pointer" : "default",
          }}
        >
          {isTarget && (
            <circle r={22} fill="none" stroke="#6366f1" stroke-width={2} />
          )}
        </g>
      );
    })}
  </div>
</pre>

```



```

*/

type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;

interface Frame {
  pos: Pos;
  edges: [NodeId, NodeId][];
  kind: "start" | "aim" | "result";
  title: string;
  text: string;
  /** Пара, которую крутим на этом шаге: кадр-«прицел». */
  focus?: [NodeId, NodeId];
  /** Кто переехал на этом кадре – рисуем призрак и стрелу. */
  moved?: NodeId[];
  ms: number;
}

interface Case {
  key: "zig" | "zig-zig" | "zig-zag";
  label: string;
  when: string;
  rotations: string;
  /** Номера узлов: роль → число. */
  keys: Partial<Record<NodeId, number>>;
  /** Обход слева направо – одинаковый на всех кадрах. */
  inorder: string[];
  /** Что означают поддеревья A..D. */
  ranges: string;
  frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----

```

```

    },
  ],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
  pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
  edges: [ ["g", "p"], ["g", "D"], ["p", "x"], ["p", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
  pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
  edges: [ ["p", "x"], ["p", "g"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
  key: "zig-zig",
  label: "Zig-Zig",
  when: "x и p – дети с ОДНОЙ стороны (оба слева или оба справа)",
  rotations: "2 поворота, первым – верхний",
  keys: { x: 20, p: 40, g: 60 },
  inorder: ["A", "20", "B", "40", "C", "60", "D"],
  ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
  frames: [
    {
      ...ZZ_S0,
      kind: "start",
      title: "Было: прямая линия 60 → 40 → 20",
      text: "Три узла выстроились в «бамбук»: каждый следующий узел  
зглаживает.",
      ms: RESULT_MS,
    },
    {
      ...ZZ_S0,
      kind: "aim",
      title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
      text: "Главная хитрость этого случая: первым крутит  
focus: ["g", "p"],

```

```

const ZG_S1 = {
  pos: { g: [258, 40], D: [328, 112], x: [148, 112], p: [148, 112] },
  edges: [ ["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"], ["p", "D"], ["p", "C"] ],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZAG: Case = {
  key: "zig-zag",
  label: "Zig-Zag",
  when: "x и p – дети с РАЗНЫХ сторон (змейка)",
  rotations: "2 поворота, первым – нижний",
  keys: { x: 40, p: 20, g: 60 },
  inorder: ["A", "20", "B", "40", "C", "60", "D"],
  ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
  frames: [
    {
      ...ZG_S0,
      kind: "start",
      title: "Было: 60 → влево на 20 → вправо на 40",
      text: "Узлы идут «змейкой»: сначала шаг влево, по",
      ms: RESULT_MS,
    },
    {
      ...ZG_S0,
      kind: "aim",
      title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
      text: "Дед (60) стоит на месте и в этом повороте",
      focus: ["p", "x"],
      ms: AIM_MS,
    },
    {
      ...ZG_S1,
      kind: "result",
      title: "Поворот 1: змейка выпрямилась",
      text: "40 заняло место 20 и забрало его к себе ле",
      moved: ["x", "p", "C"],
      ms: RESULT_MS,
    }
  ]
}

```

```

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
  const { pos, edges, focus, moved } = frame;
  const dim = (id: NodeId) => (focus ? !focus.includes(
  const ghosts = frame.kind === "result" ? (moved ?? [])

  return (
    <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
      <defs>
        <marker id="splay-arrow" viewBox="0 0 10 10" re
          <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
        </marker>
      </defs>

    {/* призраки: где узел стоял до поворота */}
    {ghosts.map((id) => {
      const from = prev[id]!;
      const to = pos[id]!;
      const r = isSubtree(id) ? 15 : 20;
      const dx = to[0] - from[0];
      const dy = to[1] - from[1];
      const len = Math.hypot(dx, dy) || 1;
      const pad = r + 8;
      return (
        <g key={`ghost-${id}`} opacity={0.55}>
          <circle cx={from[0]} cy={from[1]} r={r} fil
            {len > pad * 2 + 6 && (
              <line
                x1={from[0] + (dx / len) * pad}
                y1={from[1] + (dy / len) * pad}
                x2={to[0] - (dx / len) * pad}
                y2={to[1] - (dy / len) * pad}
                stroke="#a5b4fc"
                strokeWidth={1.5}
                strokeDasharray="4 3"
                markerEnd="url(#splay-arrow)"
              />
            )}
          </g>
        )
      )
    )}
  )

```

```

    }}
  >
    {hot && <circle r={r + 7} fill="none" stroke=
    <circle r={r} fill={c.fill} stroke={c.stroke}
    <text
      textAnchor="middle"
      dominantBaseline="central"
      fontSize={sub ? 12 : 15}
      fontWeight="700"
      fill={sub ? "#94a3b8" : "#fff"}
    >
      {sub ? id : keys[id]}
    </text>
    {!sub && (
      <text textAnchor="middle" y={-r - 7} font
        {id}
      </text>
    )}
  </g>
);
}}
</svg>
);
};

```

```

/** Имя случая из кода пользователя → вкладка демонстрация
const CASE_BY_NAME: Record<string, number> = {
  zig: 0, "zig-zig": 1, zigzig: 1, "zig zig": 1, "zig-zigzig": 1,
};

```

```

export const SplayRotationsViz: React.FC = () => {
  const [caseIdx, setCaseIdx] = useState(0);
  const [frame, setFrame] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [slow, setSlow] = useState(false);

```

```

/* — связь с Python-компилятором: case переключает в
const runtime = useVizRuntime();

```

```

useEffect(() => {
  if (!playing) return;
  const t = setTimeout(() => setFrame((f) => (f + 1)), 100);
  return () => clearTimeout(t);
}, [playing, frame, duration, total]);

return (
  <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
    <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar">
      {CASES.map((c, i) => (
        <button
          key={c.key}
          onClick={() => setCaseIdx(i)}
          className={`px-3 sm:px-4 py-2 rounded-lg text-white
            ${i === caseIdx ? "bg-indigo-600 text-white" : "bg-slate-800"}
          `}
        >
          {c.label}
        </button>
      ))}
    </div>

    <div className="mb-3 text-xs sm:text-[13px] text-slate-400">
      <span className="font-bold text-indigo-300">Корпус</span>
      <span className="text-slate-500">({active.rotations})</span>
    </div>

    {/* шаг + пояснение стоят НАД картинкой: сначала шаг, потом пояснение */}
    <div className="mb-3 rounded-xl border border-slate-800">
      <div className="flex items-center gap-2 mb-1 flex-wrap">
        <span
          className={`text-[10px] font-bold uppercase
            ${current.kind === "aim"
              ? "bg-amber-500/20 text-amber-300"
              : current.kind === "start"
              ? "bg-slate-700 text-slate-300"
              : "bg-indigo-500/20 text-indigo-300"}
          `}
        >

```

```

        className="h-full bg-indigo-500"
        style={
          playing
            ? { animation: `demoProgress ${duration}ms`
              : { width: "100%", background: "#334155"
            }
        }
      />
    </div>

    <div className="flex flex-wrap items-center gap-2"
      <button
        onClick={() => {
          setPlaying(false);
          setFrame((f) => (f - 1 + total) % total);
        }}
        disabled={idx === 0}
        className="px-3 py-2 rounded-lg bg-slate-800 text-white
          dark:hover:text-slate-300 text-xs font-bold transition-colors"
      >
        ← Назад
      </button>
      <button
        onClick={() => {
          setPlaying(false);
          setFrame((f) => (f + 1) % total);
        }}
        className="px-4 py-2 rounded-lg bg-indigo-600 text-white
          w-indigo-900/40"
      >
        {last ? "в Сначала" : "Дальше →"}
      </button>
      <button
        onClick={() => setPlaying((p) => !p)}
        className="flex items-center gap-1.5 px-3 py-2 text-white
          text-xs font-bold transition-colors"
      >
        {playing ? <Pause className="w-3.5 h-3.5" /> : "Автоматически"}
        {playing ? "Стоп" : "Авто"}
      </button>
    </div>
  )}
)

```

```

    </span>
    {liveCase || liveX !== null || liveStep !== null
    <>
      {liveCase && {
        <span className="text-[11px] font-mono bg-
          case = {liveCase}
        </span>
      }}
      {liveX !== null && <span className="text-[11px] font-mono bg-
        py-1">x = {liveX}</span>}
      {liveP !== null && <span className="text-[11px] font-mono bg-
        py-1">p = {liveP}</span>}
      {liveG !== null && <span className="text-[11px] font-mono bg-
        py-1">g = {liveG || "нет (Zig)"}</span>}
      {liveStep !== null && <span className="text-[11px] font-mono bg-
        -2 py-1">step = {liveStep}</span>}
    </>
  ) : (
    <span className="text-[11px] text-slate-500">
      запусти код из панели Python: переменная <code>case</code>
      случай,
      а <code>liveStep</code>
    </span>
  )}
</div>

<p className="mt-2 text-[11px] text-slate-500">
  Листай кнопкой «Дальше» в своём темпе – кадры не
</p>
</div>
);
};

```



```
export function bstInsert(root: SN | null, key: number) {
  if (!root) return { key, left: null, right: null, parent: null };
  const r = cloneWithParents(root)!;
  let cur: SN = r;
  for (;;) {
    if (key < cur.key) {
      if (!cur.left) {
        cur.left = { key, left: null, right: null, parent: cur };
        return r;
      }
      cur = cur.left;
    } else {
      if (!cur.right) {
        cur.right = { key, left: null, right: null, parent: cur };
        return r;
      }
      cur = cur.right;
    }
  }
}
```

/** Поднять child над его родителем (один атомарный поворот)

```
export function rotateUp(child: SN): void {
  const p = child.parent;
  if (!p) return;
  const g = p.parent;
  if (p.left === child) {
    p.left = child.right;
    if (child.right) child.right.parent = p;
    child.right = p;
    p.parent = child;
  } else {
    p.right = child.left;
    if (child.left) child.left.parent = p;
    child.left = p;
    p.parent = child;
  }
}
```

```

    cutEdge?: [number, number];
    cutLabel?: string;
    /** Куда  $\beta$  переедет: сторона у верхнего узла. */
    cutDest?: string;
    pathKeys?: number[];
    caseLabel?: string;
    note: string;
    rotCount: number;
}

/** Полный пошаговый прогон splay(key): спуск → (прицел
export function splaySteps(
    rootIn: SN | null,
    key: number,
    isInsert = false,
): { steps: SplayStep[]; found: boolean } {
    let root: SN | null = cloneWithParents(rootIn);
    const steps: SplayStep[] = [];
    if (!root)
        return { steps: [{ kind: "done", root: null, note:

// --- спуск ---
let cur: SN | null = root;
const pathKeys: number[] = [];
let found = false;
let last: SN = root;
while (cur) {
    last = cur;
    pathKeys.push(cur.key);
    if (key === cur.key) {
        found = true;
        break;
    }
    cur = key < cur.key ? cur.left : cur.right;
}
const x = found ? cur : last;
steps.push({
    kind: "search",

```

```

        : `Среднего поддерева нет (β пусто): ${child.key}
        rotCount,
    });
    const before = collectEdges(root);
    rotateUp(child);
    fixRoot();
    rotCount += 1;
    const diff = diffEdges(before, collectEdges(root));
    steps.push({
      kind: "rot",
      root: cloneWithParents(root)!,
      caseLabel: label,
      note: `Поворот: ${child.key} встал над ${p.key}.
      rotCount,
    });
  });
};

while (target.parent) {
  const p = target.parent;
  const g = p.parent;
  if (!g) {
    emitRotation(
      target,
      "Zig",
      [p.key, target.key],
      `Zig: у ${target.key} нет деда — хватит одного
    );
    continue;
  }
  const pIsLeft = g.left === p;
  const xIsLeft = p.left === target;
  if (pIsLeft === xIsLeft) {
    const side = pIsLeft ? "оба слева" : "оба справа"
    emitRotation(
      p,
      "Zig-Zig",
      [g.key, p.key],
      `Zig-Zig (${side}: линия ${g.key}—${p.key}—${ta

```

```

    }
  }

  steps.push({
    kind: "done",
    root: cloneWithParents(root)!,
    note: ` ${target.key} в корне за ${rotCount} ${rotCount} `,
    rotCount,
  });
  return { steps, found };
}

/* ----- служебное -----

const depthOf = (t: SN | null, key: number): number => {
  let d = 0;
  let cur = t;
  while (cur) {
    if (key === cur.key) return d;
    cur = key < cur.key ? cur.left : cur.right;
    d += 1;
  }
  return -1;
};

const heightOf = (t: SN | null): number => (t ? 1 + Math.max(
const inorderKeys = (t: SN | null): number[] => (t ? [
  ...inorderKeys(t.left),
  t.key,
  ...inorderKeys(t.right)
] : []);

export const INITIAL_KEYS = [50, 40, 35, 32, 30, 28, 12];

const buildInitial = (): SN => {
  let t: SN | null = null;
  for (const k of INITIAL_KEYS) t = bstInsert(t, k);
  return t!;
};

const CODE_LINES = [
  { line: 1, text: "while x has parent:" },
  { line: 2, text: "    p = parent(x); g = parent(p)" }
];

```

```
<text x="12" y="130" textAnchor="middle" fill=
<circle
    cx="48" cy="126" r="13"
    fill={isCut ? "#78350f" : "#0f172a"}
    stroke={isCut ? "#f59e0b" : "#64748b"}
    strokeWidth={isCut ? 2.5 : 1.5}
    strokeDasharray={isCut ? "4,3" : "none"}
/>
<text x="48" y="130" textAnchor="middle" fill=
<circle cx="80" cy="126" r="13" fill="#0f172a"
<text x="80" y="130" textAnchor="middle" fill=
<text x="98" y="92" fill={isCut ? "#f59e0b" :
    {isCut ? "β отсѣрхита" : "β при х"}}
</text>
<text x="16" y="160" fill="#64748b" fontSize=
</g>
<text x="163" y="96" fill="#10b981" fontSize="2
<g transform="translate(186,6)">
    <text y="10" fill="#94a3b8" fontSize="11" fon
    <line x1="70" y1="46" x2="45" y2="86" stroke=
    <line x1="70" y1="46" x2="95" y2="86" stroke=
    <line x1="45" y1="86" x2="27" y2="126" stroke=
    <line
        x1="45" y1="86" x2="63" y2="126"
        stroke={isRot ? "#10b981" : "#64748b"}
        strokeWidth={isRot ? 3 : 2}
    />
    <circle cx="70" cy="46" r="15" fill="#059669"
    <text x="70" y="50" textAnchor="middle" fill=
    <circle cx="45" cy="86" r="15" fill="#1e293b"
    <text x="45" y="90" textAnchor="middle" fill=
    <circle cx="27" cy="126" r="13" fill="#0f172a"
    <text x="27" y="130" textAnchor="middle" fill=
    <circle
        cx="63" cy="126" r="13"
        fill={isRot ? "#78350f" : "#0f172a"}
        stroke={isRot ? "#10b981" : "#64748b"}
        strokeWidth={isRot ? 2.5 : 1.5}
```

```

-----
const SplayWalkViz: React.FC = () => {
  const initial = useMemo(buildInitial, []);
  const [steps, setSteps] = useState<SplayStep[] | null>();
  const [idx, setIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [input, setInput] = useState("");
  const [treeVers, setTreeVers] = useState(0);
  const baseRef = useRef<SN>(initial);
  const timer = useRef<ReturnType<typeof setTimeout> | null>();

  const runKey = (key: number, isInsert = false) => {
    const { steps: st } = splaySteps(baseRef.current, key);
    setSteps(st);
    setIdx(0);
    setPlaying(true);
  };

  useEffect(() => {
    if (!playing || !steps) return;
    if (idx >= steps.length - 1) {
      setPlaying(false);
      baseRef.current = steps[steps.length - 1].root ?? null;
      setTreeVers((v) => v + 1);
      return;
    }
    timer.current = setTimeout(() => setIdx((v) => v + 1), 100);
    return () => {
      if (timer.current) clearTimeout(timer.current);
    };
  }, [playing, idx, steps]);

  const step = steps ? steps[Math.min(idx, steps.length - 1)] : null;
  const viewRoot = step ? step.root : baseRef.current;
  const pos = useMemo(() => layout(viewRoot), [viewRoot]);
  const nodes = useMemo(() => [...pos.entries()].map(([id, node]) => node), [pos]);
  const edges = useMemo(() => {
    const es: { a: number; b: number }[] = [];
    const walk = (n: SN | null) => {
      if (!n) return;
      es.push({ a: n.id, b: n.parent });
      walk(n.parent);
    };
    walk(viewRoot);
  }, [viewRoot]);

```

```

    runKey(v, true);
    setInput("");
  };

const stepColor = (k: number): string => {
  if (!step) return "#0f172a";
  if (step.kind === "search" && step.pathKeys?.includes(k)) return "#0f172a";
  if ((step.kind === "aim" || step.kind === "cut") &&
    const e = step.aimEdge ?? step.cutEdge!;
    if (e[0] === k) return "#1e293b";
    if (e[1] === k) return "#78350f";
  )
  if (step.kind === "rot" && step.pathKeys?.includes(k)) return "#0f172a";
  return "#0f172a";
};

const strokeColor = (k: number): string => {
  if (!step) return "#64748b";
  if (step.kind === "search" && step.pathKeys?.includes(k)) return "#64748b";
  if ((step.kind === "aim" || step.kind === "cut") &&
    const e = step.aimEdge ?? step.cutEdge!;
    if (e.includes(k)) return "#f59e0b";
  )
  if (step.pathKeys?.includes(k)) return "#38bdf8";
  return "#64748b";
};

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border" >
    {/* Wanka */}
    <div className="flex flex-wrap items-center justify-center" >
      <div className="flex items-center gap-3" >
        <div className="p-2.5 bg-indigo-600 text-white" >
          <span className="text-2xl" > </span>
        </div>
      </div>
      <div>
        <h3 className="text-2xl font-bold text-white" >
          <p className="text-sm text-indigo-300" >Клик
        </div>

```

```

        setSteps(null);
        setIdx(0);
        setPlaying(false);
        setTreeVers((v) => v + 1);
    }}
    className="px-3 py-2 rounded-lg text-xs font-
  >
    Перемешать
  </button>
</div>
</div>

{/* Плеер */}
{steps && (
  <div className="flex flex-wrap items-center gap-
    <button onClick={() => { setIdx(0); setPlaying(
      ld bg-slate-700 text-white disabled:opacity-
        В начало
    </button>
    <button onClick={() => { setPlaying(false); s
      -lg text-xs font-bold bg-slate-700 text-whi
        ← Шаг
    </button>
    <button onClick={() => setPlaying(!playing)} .
      er-600" : "bg-emerald-600"`}>
      {playing ? " Пауза" : "▶ Пуск"}
    </button>
    <button onClick={() => { setPlaying(false); s
      className="px-3 py-1.5 rounded-lg text-xs font-
        Шаг →
    </button>
    <span className="text-xs text-slate-400">
      Шаг {idx + 1} из {steps.length} • поворотов
    </span>
    {step?.caseLabel && (
      <span className="px-2.5 py-1 rounded-full text-
        00/60">
        {step.caseLabel}

```



```

        </g>
      );
    }}}
    {nodes.map((n) => {
      const aimE = step?.aimEdge ?? (step?.kind === "aim" ? step.aimEdge : null);
      const isAimNode = aimE?.includes(n.key);
      return {
        <g
          key={`n-${n.key}-${treeVers}`}
          transform={`translate(${n.px}, ${n.py})`}
          onClick={() => {
            if (playing) return;
            const has = inorderKeys(baseRef.current).includes(n.key);
            if (has) runKey(n.key);
          }}
          className="cursor-pointer"
        >
          <circle
            r={isAimNode ? 24 : 20}
            fill={stepColor(n.key)}
            stroke={strokeColor(n.key)}
            strokeWidth={isAimNode ? 4 : 2}
            className="transition-all duration-300"
          />
          <text y={4} fill="white" fontSize="13">
            {n.key}
          </text>
        </g>
      );
    }}}
  </svg>

```

```

  {/* Лог + механика */}
  <div className="w-full flex flex-col lg:flex-row" >
    <div className="flex-1 bg-slate-950/80 p-4 rounded" >
      <p className="font-semibold text-amber-400">
        {step?.kind === "aim"

```

```

<tr className="border-b border-slate-800">
  <td className="py-2 text-slate-400">В
  <td className="py-2 text-right text-slate-400">
</tr>
<tr>
  <td className="py-2 text-slate-400">п
  <td className="py-2 text-right text-slate-400">
</tr>
</tbody>
</table>
) : (
  <p className="text-xs text-slate-400">Запис
  <p className="text-xs text-slate-500 mt-3">Од
    щений дешевлеет.</p>
</div>
<div className="w-full lg:w-1/2 bg-slate-900/60">
  <h4 className="text-sm font-bold text-slate-200">
  <div className="bg-slate-950/80 rounded-lg p-4">
    {CODE_LINES.map((l) => {
      const zig = step?.caseLabel === "Zig" &&
      const zz1 = step?.caseLabel === "Zig-Zig"
      const zz2 = step?.caseLabel === "Zig-Zig"
      const za1 = step?.caseLabel === "Zig-Zag"
      const za2 = step?.caseLabel === "Zig-Zag"
      const active = zig || zz1 || zz2 || za1 || za2
      return (
        <div key={l.line} className={`py-1 px-2 border-400" : "text-slate-400"}`>
          <span className="text-slate-600 select-none">
            {l.text}
          </span>
        </div>
      )
    })}
  </div>
</div>
</div>
</div>

```

```

        ate: 'idle' },
    { id: 'p2', name: 'Тарелка из-под пиццы',    emoji:
      State: 'idle' },
    { id: 'p3', name: 'Блюдец от торта',          emoji: '
      tate: 'idle' },
  ]});

const runtime = useVizRuntime();
const liveStack = vizArray(runtime?.variables?.st) ??
const displayStack: Plate[] = liveStack
  ? liveStack.map((value, index) => ({
    id: `python-${index}`,
    name: String(value),
    emoji: " ",
    color: "from-indigo-500/20 to-indigo-600/20 border",
    isDirty: true,
    animState: "idle",
  })))
  : dirtyStack;

const [cleanRack, setCleanRack] = useState<Plate[]>([
const [isWashing, setIsWashing] = useState(false);
const [showSoap, setShowSoap]   = useState(false);
const [shakeEmpty, setShake]     = useState(false);
const [log, setLog]              = useState<string[]>([

const addLog = (msg: string) => setLog(prev => [msg,

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length >= 7) {
    addLog('⚠ Стопка слишком высокая, тарелки могут р
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }
}

```

```

// Step 2: Wash complete. Plate becomes clean and
const cleanPlate: Plate = {
  ...topPlate,
  isDirty: false,
  animState: 'clean',
};

setCleanRack(prev => [cleanPlate, ...prev].slice(0, 1));
setDirtyStack(prev => prev.slice(0, prev.length - 1));
setShowSoap(false);
setIsWashing(false);
addLog(` Готово! Чистая тарелка ${topPlate.emoji}`, 850);
}, [dirtyStack, isWashing]));

const reset = () => {
  counter = 0;
  setDirtyStack([
    { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', state: 'in' },
    { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', state: 'in' },
    { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', state: 'in' },
  ]);
  setCleanRack([]);
  setIsWashing(false);
  setShowSoap(false);
  setLog([' Стол сброшен. Посуда снова грязная!']);
};

const topIndex = displayStack.length - 1;

return (
  <div className="flex flex-col gap-6">
    {/* МНЕМОНИКА / ПРАВИЛО */}
    <div className="bg-blue-950/40 border border-blue-950/40 p-6">
      <div className="text-3xl"> </div>
    </div>
  </div>
);

```

```
    </button>
  </div>

  {/* ИНТЕРАКТИВНАЯ КУХНЯ */}
  <div className="grid grid-cols-1 md:grid-cols-12" >

    {/* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */}
    <div className="md:col-span-7 bg-slate-900 rounded-3xl
      [380px]">
      <div className="absolute top-4 left-4 text-[10px] font-mono">
        Грязная стопка (Стек вызовов / LIFO)
      </div>

      {/* Визуализация раковины */}
      <div className="absolute top-4 right-4 flex items-center
        r-slate-800/80 text-[10px] font-mono">
        <span className="w-1.5 h-1.5 rounded-full bg-white" />
        <span>РАКОВИНА </span>
      </div>

      {/* Мыльные пузыри при мытье */}
      {showSoap && (
        <div className="absolute inset-0 pointer-events-none">
          <div className="absolute w-6 h-6 rounded-full bg-white
            opacity-50"></div>
          <div className="absolute w-4 h-4 rounded-full bg-white
            opacity-30"></div>
          <div className="absolute w-5 h-5 rounded-full bg-white
            opacity-20"></div>
          <div className="absolute text-4xl anim-curve">
            <div></div>
          </div>
        </div>
      )}

      {displayStack.length === 0 ? (
        <div className="flex-1 flex flex-col items-center justify-center">
          <span className="text-6xl mb-3 animate-pulse">0</span>
          <p className="text-white font-black text-2xl">Пустая стопка</p>
          <p className="text-slate-500 text-xs mt-1">Нажмите на кнопку, чтобы добавить тарелку</p>
        </div>
      ) : (
        <div className="flex-1 flex flex-col items-center justify-center">
          <span className="text-6xl mb-3 animate-pulse">{displayStack.length}</span>
          <p className="text-white font-black text-2xl">Грязная тарелка</p>
          <p className="text-slate-500 text-xs mt-1">Нажмите на кнопку, чтобы мыть тарелку</p>
        </div>
      )}
    </div>

    {/* ПРАВАЯ КОЛОНКА: СТОПКА ЧИСТЫХ ТАРЕЛОК */}
    <div className="md:col-span-5 bg-slate-900 rounded-3xl">
      <div className="absolute top-4 left-4 text-[10px] font-mono">
        Чистая стопка (Очередь / FIFO)
      </div>

      {/* Визуализация раковины */}
      <div className="absolute top-4 right-4 flex items-center
        r-slate-800/80 text-[10px] font-mono">
        <span className="w-1.5 h-1.5 rounded-full bg-white" />
        <span>РАКОВИНА </span>
      </div>

      {/* Мыльные пузыри при мытье */}
      {showSoap && (
        <div className="absolute inset-0 pointer-events-none">
          <div className="absolute w-6 h-6 rounded-full bg-white
            opacity-50"></div>
          <div className="absolute w-4 h-4 rounded-full bg-white
            opacity-30"></div>
          <div className="absolute w-5 h-5 rounded-full bg-white
            opacity-20"></div>
          <div className="absolute text-4xl anim-curve">
            <div></div>
          </div>
        </div>
      )}

      {displayStack.length === 0 ? (
        <div className="flex-1 flex flex-col items-center justify-center">
          <span className="text-6xl mb-3 animate-pulse">0</span>
          <p className="text-white font-black text-2xl">Пустая стопка</p>
          <p className="text-slate-500 text-xs mt-1">Нажмите на кнопку, чтобы добавить тарелку</p>
        </div>
      ) : (
        <div className="flex-1 flex flex-col items-center justify-center">
          <span className="text-6xl mb-3 animate-pulse">{displayStack.length}</span>
          <p className="text-white font-black text-2xl">Чистая тарелка</p>
          <p className="text-slate-500 text-xs mt-1">Нажмите на кнопку, чтобы мыть тарелку</p>
        </div>
      )}
    </div>
  </div>
</div>
</div>
```

```

        </span>
      })
    </div>
  );
  }}}
</div>
})

  { /* Столешница раковины */
  <div className="w-full h-4 bg-gradient-to-r f
</div>

  { /* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК
  <div className="md:col-span-5 bg-slate-900 round
    <div className="absolute top-4 left-4 text-[1
      Сушилка для чистой посуды
    </div>

    <div className="absolute top-4 right-4 flex i
      order-emerald-800/80 text-[9px] font-mono">
      <Sparkles className="w-3.5 h-3.5 animate-pu
      <span>ЧИСТО </span>
    </div>

    <div className="flex-1 flex flex-col justify-f
      {cleanRack.length === 0 ? {
        <div className="flex-1 flex flex-col item
          <span className="text-5xl mb-2"> </span>
          <p className="text-xs font-bold font-mo
          <p className="text-[10px] mt-1">Здесь 6
        </div>
      } : {
        <div className="flex flex-col gap-2 w-fu
          {cleanRack.map(plate => {
            <div
              key={plate.id}
              className="w-full h-8 rounded-full
            items-center justify-center gap-2 opacity-

```

```
-----
import { useVizRuntime, vizArray, vizNumber, vizString } from 'viz.js';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'react-icons';

function generateKmpSteps(pattern: string, text: string) {
  if (!pattern) pattern = " ";
  const s = pattern + "#" + text;
  const n = s.length;
  const pi = new Array(n).fill(0);
  const steps: any[] = [];

  steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });

  for (let i = 1; i < n; i++) {
    let j = pi[i - 1];

    steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });

    while (j > 0 && s[i] !== s[j]) {
      steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
      j = pi[j - 1];
    }

    if (s[i] === s[j]) {
      steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
      j++;
    } else {
      steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадения нет` });
    }

    pi[i] = j;
    steps.push({ i, j, pi: [...pi], desc: `Записываем` });

    if (j === pattern.length) {
      steps.push({ i, j, pi: [...pi], found: true });
    }
  }
}
```

```

    }

    steps.push({ i, l, r, z: [...z], desc: `Фиксируем` });

    if (i + z[i] - 1 > r) {
      l = i;
      r = i + z[i] - 1;
      steps.push({ i, l, r, z: [...z], desc: `Обновляем` });
    }

    if (z[i] === pattern.length) {
      steps.push({ i, l, r, z: [...z], found: true });
    }
  }
  return { s, steps, patternLength: pattern.length };
}

export function StringAlgorithmsViz({ defaultMode = "kmp" }) {
  const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
  const [pattern, setPattern] = useState("aba");
  const [text, setText] = useState("abacaba");

  // Ensure inputs are valid dynamically
  const safePattern = pattern || " ";
  const safeText = text || " ";

  const { s: demoString, steps, patternLength: demoPatternLength } = generateKmpSteps(safePattern, safeText);
  if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
  else return generateZSteps(safePattern, safeText);
}, [mode, safePattern, safeText]);

const runtime = useVizRuntime();
const runtimeString = vizString(runtime?.variables?.runtimeString ?? demoString);
const patternLength = runtimeString ? runtimeString : demoPatternLength;

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

```



```

    j: liveJ ?? baseStep.j,
    l: liveL ?? baseStep.l,
    r: liveR ?? baseStep.r,
    [mode === "kmp" ? "pi" : "z"]: hasLiveValues
    desc: `Компилятор: i=${liveI ?? "-"}${mode ===
  }
  : baseStep;

return (
  <div className="space-y-4">
    <div className="flex items-center justify-b
      <div className="flex bg-slate-900 borde
        <button onClick={() => setMode("kmp
          className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
          КМП (π-ФУНКЦИЯ)
        </button>
        <button onClick={() => setMode("z")
          className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
          Z-ФУНКЦИЯ
        </button>
      </div>

    <div className="flex items-center gap-2
      <div className="bg-slate-800 p-1 fl
        <span className="text-[10px] te
        <input value={pattern} onChange
xt-white text-xs font-bold font-mono px-2 p
      </div>
      <div className="bg-slate-800 p-1 fl
        <span className="text-[10px] te
        <input value={text} onChange={e
te text-xs font-bold font-mono px-2 py-1 ou
      </div>
    </div>
  </div>

```

```

    })
    {mode == "z" && step.r
      <div className="abs
    })

    {/* Index */}
    <span className={`text-

    {/* Char Box */}
    <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
    {char}
    </div>

    {/* Value array box */}
    <div className="text-[1
slate-700/50 font-mono font-bold">
      {(mode == "kmp" ?
    </div>

    {/* KMP fingers */}
    {mode == "kmp" && step
      <div className="abs
z-20">
        <span classNam
      </div>
    })
    {mode == "kmp" && step
      <div className="abs
        <span classNam
      </div>
    })
    {mode == "kmp" && step
      <div className="abs
20">
        <span classNam
      </div>
    })

```

```

-400 hover:text-white hover:bg-slate-800 rounded
      <Undo strokeWidth={3} size={16} />
    </button>
    <button onClick={playPause} className=
x items-center justify-center min-w-[80px]"
      {autoplay ? <><Pause size={16} />
    </button>
    <button onClick={nextStep} disabled=
e="p-2 text-slate-400 hover:text-white hove
      <SkipForward strokeWidth={3} si
    </button>
    <button onClick={reset} aria-label=
: bg-slate-800 rounded-lg ml-2 border-l bord
      <RefreshCw strokeWidth={3} size
    </button>
  </div>

```

```

    {/* Description Log */}
    <div className="flex-1 bg-slate-900/50
erflow-hidden">
      <div className="absolute top-0 right
        <span className="font-mono text
      </div>
      <div className="text-[10px] text-sl
{steps.length}]</div>
      <div className="text-sm text-slate-:
        {step.desc}
      </div>
    </div>
  </div>

```

```

{/* Why this matters card */}
<div className="bg-slate-800 p-4 rounded-xl
  <span className="text-2xl mt-1"> </span>
  <p className="text-xs text-slate-400 le
    Обрати внимание на хитрость с решёт
    Мы пишем <b>Шаблон + # + Текст</b>.
    Когда значени {mode === 'kmp' ? "пр

```

```
import React, { useState } from "react";

interface TooltipProps {
  /** Содержимое подсказки (можно JSX). */
  content: React.ReactNode;
  /** Элемент, на который наводят курсор / фокус. */
  children: React.ReactNode;
  /** С какой стороны показывать. По умолчанию – сверху
  side?: "top" | "bottom";
  className?: string;
}

/**
 * Всплывающая подсказка (как title="", только красивая
 * Появляется по наведению курсора и по фокусу с клавиатуры
 */
export const Tooltip: React.FC<TooltipProps> = ({ content, children }) => {
  const [open, setOpen] = useState(false);

  const hiddenShift = side === "top" ? "translate-y-1" : "translate-y-0";

  return (
    <span
      className={`relative inline-flex ${className ?? ""}
        onMouseEnter={() => setOpen(true)}
        onMouseLeave={() => setOpen(false)}
        onFocus={() => setOpen(true)}
        onBlur={() => setOpen(false)}
      >
      {children}
      <span
        role="tooltip"
        className={`pointer-events-none absolute left-1 top-1
          bg-slate-900 px-3 py-2 text-left text-[11px]
          duration-150 ${
            side === "top" ? "bottom-full mb-2" : "top-full mb-2"
          }
        >
        {content}
      </span>
    </span>
  );
};
```

```

    v: number;
}

const dagNodes: TNode[] = [
  { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
  { id: 1, x: 280, y: 70, label: "1", name: "Дискретка" },
  { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра" },
  { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы" },
  { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение" },
];

const dagEdges: TEdge[] = [
  { u: 0, v: 1 },
  { u: 0, v: 2 },
  { u: 1, v: 3 },
  { u: 2, v: 3 },
  { u: 3, v: 4 },
];

interface TStep {
  desc: string;
  visited: Set<number>;
  fullyProcessed: Set<number>;
  currentNode: number | null;
  topoList: number[];
  dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
  const [steps, setSteps] = useState<TStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
  const v = vizNumber(vars.v);
  const orderLength = vizArray(vars.order)?.length;
  if (v === null && orderLength === undefined) return null;
  const candidates = steps
    .map((step, index) => ({ step, index }))
    .filter(({ step }) => (v === null || step.currentNode === v));

```

```

    );

    for (const to of adj[u]) {
        if (!visited.has(to)) {
            dfs(to);
        } else if (!fullyProcessed.has(to)) {
            // Explaining potential cycle detected (not in topological sort)
            recordStep(
                `⚠ Внимание: Рёбра в серую вершину ${to} от ${u}`
            );
        }
    }
}

fullyProcessed.add(u);
dfsStack.pop();
topoList.push(u); // prepend behavior: we write in reverse
recordStep(
    `DFS: закончили обработку '${dagNodes[u].name}'`
);
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
    if (!visited.has(i)) {
        recordStep(
            `Запускаем новый обход DFS от нетронутой вершины ${i}`
        );
        dfs(i);
    }
}

recordStep(
    "Топологическая сортировка завершена! Полученный топологический порядок:"
);

```

```

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Clock className="w-5 h-5 text-indigo-400" />
        Топологическая сортировка (Зависимости обучен
      </h3>

    <div className="flex items-center gap-2 bg-slate-
      <button
        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-
        500 text-white"
      >
        {isPlaying ? (
          <Pause className="w-4 h-4 ml-1" />
        ) : (
          <Play className="w-4 h-4 ml-1" />
        )}
        {isPlaying ? "Пауза " : "Старт "}
      </button>

      <button
        onClick={reset}
        aria-label="Сбросить сортировку"
        title="Сбросить сортировку"
        className="flex items-center justify-center
        ors"
      >
        <RotateCcw className="w-4 h-4" />
      </button>
    </div>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-12
    {/* Playfield SVG */}
    <div className="lg:col-span-8 bg-[#0B1120] rela

```

```

        x1={u.x}
        y1={u.y}
        x2={v.x}
        y2={v.y}
        stroke={isActive ? "#818cf8" : "#334155"}
        strokeWidth={isActive ? "3" : "2"}
        markerEnd={`url(#${isActive ? "dag-arrow" : "dag-arrow"})`}
        className="transition-all duration-300"
      />
    );
  })}

```

```

{/* Nodes */}
{dagNodes.map((node) => {
  const isCurrent = step.currentNode === node.id;
  const isVisited = step.visited.has(node.id);
  const isProcessed = step.fullyProcessed.has(node.id);

  let fill = "#1e293b";
  let stroke = "#334155";
  let textColor = "text-slate-400";
  void textColor;

  if (isCurrent) {
    fill = "#312e81";
    stroke = "#818cf8";
    textColor = "text-white font-bold";
  } else if (isProcessed) {
    fill = "#022c22";
    stroke = "#059669";
    textColor = "text-emerald-300";
  } else if (isVisited) {
    fill = "#3730a3";
    stroke = "#6366f1";
    textColor = "text-indigo-200";
  }

  return (

```



```

    { /* Console column */}
    <div className="lg:col-span-4 bg-slate-950 p-5 00 overflow-y-auto">
      <h4 className="text-xs font-bold text-slate-400">
        Статус сортировки
      </h4>

      <div className="bg-slate-900 border border-slate-800 p-2">
        <p className="text-xs text-indigo-300 min-h-12">
          {step.desc}
        </p>
      </div>

      <div className="flex-1 space-y-4 overflow-y-auto">
        { /* Topological List output */}
        <div>
          <span className="text-[10px] text-slate-500">
            РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):
          </span>
          <div className="bg-slate-900/60 p-2.5 rounded pr-1 text-slate-300">
            {step.topoList.length === 0 ? (
              <span className="text-slate-600 text-sm">
                ожидаем обработку узлов...
              </span>
            ) : (
              getTopoListString()
            )}
          </div>
        </div>
      </div>

      { /* Color key */}
      <div className="space-y-1">
        <span className="text-[10px] text-slate-500">
          ВЕРШИНЫ В DFS:
        </span>
        <div className="flex items-center gap-2">

```

```
        </div>
      </div>
    );
  };
};
```

ФАЙЛ 80/113: src/components/TreapBuildViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useContext, useEffect, useMemo, useState } from "react";
import { Terminal } from "lucide-react";
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from
```

```
/**
```

```
 * Интерактивный Treap — шесть режимов:
```

```
 *   build — сборка: ВИДИМАЯ сортировка точек (и чем сильнее, тем быстрее)
```

```
 *           координатная плоскость НАВЕРХУ (ассоциативная)
```

```
 *           чек-лист инвариантов (автопроверка, как у AVL)
```

```
 *   split — разрез по x с двумя корзинами (грамотная)
```

```
 *   merge — склейка двух деревьев по y.
```

```
 *   erase — удаление ПРОИЗВОЛЬНОЙ точки: найти по x, удалить
```

```
 *   layers — миф «дети в массиве по  $2k/2k+1$ »: работает
```

```
 *   search — «отсортирую бинарным поиском»: поиск не сортирует
```

```
 *           сортировка платит  $\geq n \cdot \log n$  сравнений, сортировка
```

```
 *
```

```
 * Канонический набор: все x и все y различны  $\Rightarrow$  дерево
```

```
 * Узел идентифицируется парой  $id = "x:y"$ .
```

```
 */
```

```
export interface Pair {
```

```
  key: number;
```

```
  grade: number;
```

```
}
```

```
type NodeId = string;
```

```
const pairId = (key: number, grade: number): NodeId =>
```

```

-----
export function insert(root: TNode | null, p: Pair): TNode | null {
  if (!root) return { id: pairId(p.key, p.grade), key: p.key, grade: p.grade };
  if (p.key <= root.key) root.left = insert(root.left, p);
  else root.right = insert(root.right, p);
  return root;
}

export function buildTree(ps: Pair[]): TNode | null {
  let root: TNode | null = null;
  for (const p of sortedByY(ps)) root = insert(root, p);
  return root;
}

export interface SplitResult {
  l: TNode | null;
  r: TNode | null;
  trace: { atId: NodeId; goLeft: boolean; note: string };
}

export function splitTree(root: TNode | null, x: number): SplitResult {
  const trace: SplitResult["trace"] = [];
  const go = (t: TNode | null): { l: TNode | null; r: TNode | null } | null {
    if (!t) return { l: null, r: null };
    if (t.key <= x) {
      const sub = go(t.right);
      trace.push({ atId: t.id, goLeft: true, note: `(${t.key} о правое поддерево.` });
      t.right = sub.l;
      return { l: t, r: sub.r };
    }
    const sub = go(t.left);
    trace.push({ atId: t.id, goLeft: false, note: `(${t.key} о левое поддерево.` });
    t.left = sub.r;
    return { l: sub.l, r: t };
  };
  const res = go(root);
  return { l: res.l, r: res.r, trace };
}

```

```

        t.right = go(t.right);
        return t;
    }
    trace.push({ atId: t.id, note: `Нашли (${t.key}, ${t.grade})` });
    const m = mergeTree(t.left, t.right);
    trace.push(...m.trace.map((s) => ({ atId: s.chosenId, note: s.note })));
    return m.root;
};
return { root: go(root), trace };
}

```

```

const cloneTree = (t: TNode | null): TNode | null =>
    t ? { id: t.id, key: t.key, grade: t.grade, left: cloneTree(t.left), right: cloneTree(t.right) } : null;

```

/* ----- валидация treap (для чек-листа и стрелочек) ----- */

```

export function checkTreap(root: TNode | null): { bst: boolean; heap: boolean } {
    let bst = true;
    let heap = true;
    let prev: number | null = null;
    const walk = (n: TNode | null, min: number, max: number, parentGrade: number | null): void => {
        if (!n) return;
        if (n.key <= min || n.key > max) bst = false;
        if (parentGrade !== null && n.grade > parentGrade) heap = false;
        walk(n.left, min, n.key, n.grade);
        if (prev !== null && n.key < prev) bst = false;
        prev = n.key;
        walk(n.right, n.key, max, n.grade);
    };
    walk(root, -Infinity, Infinity, null);
    return { bst, heap };
}

```

/* ----- раскладка сцены дерева ----- */

```

const layoutTree = (root: TNode | null) => {
    const pos = new Map<NodeId, { px: number; py: number }>();
    const layout = (node: TNode, x: number, y: number): void => {
        pos.set(node.id, { px: x, py: y });
        if (node.left) layout(node.left, x - 100, y + 100);
        if (node.right) layout(node.right, x + 100, y + 100);
    };
    layout(root, 0, 0);
    return pos;
}

```

```

const edges = useMemo(() => treeEdges(root), [root]);
const autoW = Math.max(420, ...nodes.map((n) => n.px);
const autoH = Math.max(260, ...nodes.map((n) => n.py);
const w = width ?? autoW;
const h = height ?? autoH;
const byId = (id: NodeId) => nodes.find((n) => n.id === id);
return (
  <div>
    <svg viewBox={`0 0 ${w} ${h}`} className="w-full h-full">
      {edges.map((e) => {
        const a = byId(e.from);
        const b = byId(e.to);
        if (!a || !b) return null;
        return (
          <line
            key={`te-${e.from}-${e.to}`}
            x1={a.px} y1={a.py} x2={b.px} y2={b.py}
            stroke={edgeColor ? edgeColor(e.from, e.to) : "#ccc"}
            strokeWidth={2.5}
            strokeDasharray={edgeDash && edgeDash(e.from, e.to)}
            className="transition-all duration-500"
          />
        );
      })}
      {nodes.map((n) => {
        const p = pairParts(n.id);
        const pulse = pulseId === n.id;
        return (
          <g key={`tn-${n.id}`} transform={`translate(${n.px}, ${n.py})`}
            <circle r={pulse ? 24 : 20} fill={nodeColor(n)}
              >strokeWidth={pulse ? 4 : 2} className="transition-all duration-500"
            <text y={-2} fill="#fff" fontSize="13" font-weight="bold">{n.id}</text>
            <text y={13} fill="#cbd5e1" fontSize="10">{p}</text>
          </g>
        );
      })}
    </svg>
    {subtitle && <p className="text-xs text-slate-400">{subtitle}</p>

```

```

<defs>
  <pattern id="pl-grid" width="14" height="14" patternUnits="userSpaceOnUse">
    <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1"/>
  </pattern>
</defs>
<rect x="34" y="16" width={PL_W - 46} height={PL_H - 16} fill="white" stroke="black" stroke-width="1"/>
<line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)} stroke="black" stroke-width="1"/>
<line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PY(0)} stroke="black" stroke-width="1"/>
{[0, 10, 20, 30, 40].map((k) => {
  <text key={`tx-${k}`} x={PX(k)} y={PY(0) + 16} fill="black" font-size="12"/>
})}
{[-4, 0, 4, 8].map((g) => {
  <text key={`ty-${g}`} x={PX(1) - 14} y={PY(g) + 16} fill="black" font-size="12"/>
})}
<text x={PL_W - 14} y={PY(0) - 8} fill="#94a3b8" font-size="12" font-weight="bold">
  <text x={PX(1) - 26} y="14" fill="#94a3b8" font-size="12" font-weight="bold">
    <g>
      {drawnEdges.map((e, i) => {
        const a = byPair(e.a);
        const b = byPair(e.b);
        const isBad = badEdge && ((badEdge.a.key === e.a.key && badEdge.b.key === e.b.key) || (badEdge.a.key === e.b.key && badEdge.b.key === e.a.key));
        return <line key={`pe-${i}`} x1={a.x} y1={a.y} x2={b.x} y2={b.y} stroke="black" stroke-width={isBad ? 4 : 2} strokeDasharray={isBad ? "4,3" : "none"}/>
      })}
    </g>
    <g>
      <rect x={PX(selected.key)} y={PY(selected.grade)} width={PL_W - 46} height={PL_H - 16} fill="white" stroke="black" stroke-width="1"/>
      <line x1={PX(selected.key)} y1={PY(selected.grade)} x2={PX(selected.key)} y2={PY(selected.grade) + 16} stroke="#818cf8" stroke-width={2.5} strokeDasharray="none"/>
    </g>
  </g>
  {points.map((p) => {
    const done = drawnEdges.some((e) => (e.a.key === p.key && e.b.key === p.key));
    const isSel = selected && selected.key === p.key;
    <rect x={PX(p.key)} y={PY(p.grade)} width={PL_W - 46} height={PL_H - 16} fill={isSel ? "#94a3b8" : "white"} stroke="black" stroke-width="1"/>
  })}

```

```

    if (n.right) { es.push({ a: { key: n.key, grade: n.
    });
    walk(buildTree(POINTS));
    return es;
  })();

```

```

const sameP = (p: Pair, q: Pair) => p.key === q.key &&
const gpid = (p: Pair) => `${p.key}:${p.grade}`;

```

```

/** Цепочка родителей от точки вверх. */
const parentOf = (edges: GEdge[], id: string): GEdge |
  edges.find((e) => gpid(e.b) === id);
const reaches = (edges: GEdge[], from: Pair, target: Pa
  let cur: Pair | undefined = from;
  const seen = new Set<string>();
  while (cur) {
    if (sameP(cur, target)) return true;
    if (seen.has(gpid(cur))) return false;
    seen.add(gpid(cur));
    cur = parentOf(edges, gpid(cur))?.a;
  }
  return false;
};

```

```

/** Проверка BST леса: у узла ≤1 ребёнка слева и ≤1 спр
export function forestIsBst(edges: GEdge[], all: Pair[])
  const kids = new Map<string, Pair[]>();
  for (const e of edges) kids.set(gpid(e.a), [...(kids.
  const hasParent = new Set(edges.map((e) => gpid(e.b))
  const roots = all.filter((p) => !hasParent.has(gpid(p
  const ok = (n: Pair | undefined, lo: number, hi: numb
    if (!n) return true;
    if (n.key <= lo || n.key > hi) return false;
    const ch = (kids.get(gpid(n)) ?? []).slice().sort((
    const left = ch.filter((c) => c.key < n.key);
    const right = ch.filter((c) => c.key > n.key);
    if (left.length > 1 || right.length > 1) return fal
    return ok(left[0], lo, n.key) && ok(right[0], n.key

```



```

const n = points.length;
if (n < 2) return { heapOk: true, singleParent: true,
const heapOk = edges.every((e) => e.a.grade > e.b.grade);
const parentCount = new Map<string, number>();
for (const e of edges) parentCount.set(pid(e.b), (parentCount.get(pid(e.b)) || 0) + 1);
const singleParent = [...parentCount.values()].every(v => v === 1);
// связность: компоненты через union-find
const comp = new Map<string, string>(points.map((p) => {
const find = (x: string): string => {
  while (comp.get(x) !== x) {
    comp.set(x, comp.get(x)!);
    x = comp.get(x)!;
  }
  return x;
});
for (const e of edges) {
  const ra = find(pid(e.a));
  const rb = find(pid(e.b));
  if (ra !== rb) comp.set(ra, rb);
}
const comps = new Set(points.map((p) => find(pid(p))));
const connected = comps.size === 1;
const countOk = edges.length === n - 1;
// BST: дети по x, inorder не убывает; корень один; э
const children = new Map<string, Pair[]>();
for (const e of edges) children.set(pid(e.a), [...(children.get(pid(e.b)) || []), e]);
const roots = points.filter((p) => !parentCount.has(pid(p)));
let bstOk = roots.length === 1;
if (bstOk) {
  const seen = new Set<string>();
  const inorder: number[] = [];
  const walk = (p: Pair): void => {
    if (seen.has(pid(p))) {
      bstOk = false; // цикл
      return;
    }
    seen.add(pid(p));
    const kids = children.get(pid(p)) ?? [];

```

```

const make = (p: Pair): TNode => ({
  id: pid(p),
  key: p.key,
  grade: p.grade,
  left: null,
  right: null,
});
const rootPair = roots.length ? roots[0].a : edges.length > 0 ? edges[0].a : null;
if (!rootPair) return null;
const build = (p: Pair): TNode => {
  const node = make(p);
  nodes.set(pid(p), node);
  const kids = edges.filter((e) => pid(e.a) === pid(p));
  if (kids.length > 2) return node; // невалидно – по
  kids.sort((u, v) => u.key - v.key);
  const left = kids.filter((k) => k.key <= p.key); //
  const right = kids.filter((k) => k.key > p.key);
  if (left.length > 1 || right.length > 1) return node;
  if (left[0]) node.left = build(left[0]);
  if (right[0]) node.right = build(right[0]);
  return node;
};
return build(rootPair);
}

```

```

const BuildMode: React.FC = () => {
  const [raw, setRaw] = useState(DEMO_RAW);
  const live = useTreapLive();

  // набор точек из Python-кода пересобирает плоскость
  useEffect(() => {
    if (!live.pairsText || live.pairsText !== raw) return;
    setRaw(live.pairsText);
    setPhase("connect");
    setOrder("y-desc");
    setSortIdx(3);
    setConnIdx(10000); // все рёбра проведены – ср
    setPlaying(false);
  }, [live.pairsText, raw]);
}

```

```

        const total = order === "y-desc" ? canonEdges.length
        if (connIdx < total) setConnIdx(connIdx + 1);
        else setPlaying(false);
    }
    }, 900);
    return () => clearTimeout(t);
}, [playing, phase, sortIdx, connIdx, algo, order, canonEdges]);

const startConnect = (ord: ConnectOrder) => {
    setOrder(ord);
    setPhase("connect");
    setConnIdx(0);
    setPlaying(true);
};

const comparisons = algo === "quicksort" ? (n > 1 ? Math.floor(n / 2) : 0) : 0;
const connEdgesAll = order === "y-desc" ? canonEdges : canonEdges.reverse();
const connEdges = connEdgesAll.slice(0, connIdx);
const connDone = phase === "connect" && n >= 1 && connIdx === connEdgesAll.length;

// --- игра «соедини сам» ---
const [gameEdges, setGameEdges] = useState<GameEdge[]>([]);
const [sel, setSel] = useState<Pair | null>(null);
const [cursor, setCursor] = useState<{ x: number; y: number}>({ x: 0, y: 0 });
const [gameMsg, setGameMsg] = useState<string | null>(null);
const [badEdge, setBadEdge] = useState<GameEdge | null>(null);
const [hintUsed, setHintUsed] = useState(false);

const resetGame = () => {
    setGameEdges([]);
    setSel(null);
    setCursor(null);
    setGameMsg(null);
    setBadEdge(null);
    setHintUsed(false);
};

const startGame = () => {
    resetGame();

```

```

        setCursor(null);
    };
    const planeMove = (e: React.PointerEvent<SVGSVGElement>) => {
        if (!sel) return;
        const rect = (e.currentTarget as SVGSVGElement).getBoundingClientRect();
        setCursor({ x: ((e.clientX - rect.left) / rect.width) * 100 });
    };

    const undoEdge = () => {
        setGameEdges((prev) => prev.slice(0, -1));
        setGameMsg(null);
    };

    const showSolution = () => {
        setGameEdges(canonEdges.map((e) => ({ a: e.a, b: e.b })));
        setHintUsed(true);
        setGameMsg("Решение показано – собери сам в следующем наборе");
    };

    return (
        <div className="space-y-5">
            {/* Произвольный список */}
            <div className="bg-slate-900/60 rounded-xl p-4 border border-slate-800">
                <div className="flex flex-wrap items-center justify-between">
                    <h4 className="text-sm font-bold text-slate-300">Задача 1
                    <button onClick={() => { setRaw(DEMO_RAW); setGameEdges(canonEdges); setGameMsg("Решение показано – собери сам в следующем наборе"); }}>
                        демо-набор
                    </button>
                </div>
                <textarea
                    value={raw}
                    onChange={(e) => { setRaw(e.target.value); setGameEdges(canonEdges); setGameMsg("Решение показано – собери сам в следующем наборе"); }}
                    rows={2}
                    className="w-full bg-slate-950 border border-slate-800 text-slate-300"
                    placeholder="3 5, 45 8, 1 -4 ..."
                />
            </div>
        </div>
    );

```

```

        <span className="text-xs text-slate-400">
        <button
          onClick={() => startConnect("y-desc")}
          className={`px-3 py-1.5 rounded-lg text-
bg-emerald-600 text-white" : "bg-slate-700
        >
          по y ↓ (алгоритм)
        </button>
        <button
          onClick={() => startConnect("left-right")}
          className={`px-3 py-1.5 rounded-lg text-
"bg-emerald-600 text-white" : "bg-slate-700
        >
          слева направо
        </button>
        <button
          onClick={startGame}
          disabled={n < 2}
          className={`px-3 py-1.5 rounded-lg text-
go-600 text-white" : "bg-indigo-500/80 text-
        >
          соединить сам
        </button>
      </>
    )}
  </div>
</div>
<Plane
  points={userPairs}
  drawnEdges={phase === "game" ? gameEdges : ph
  pending={null}
  interactive={phase === "game"}
  selected={sel}
  cursor={cursor}
  badEdge={badEdge}
  onPointDown={pointDown}
  onPointUp={pointUp}
  onMove={planeMove}

```

```

er-slate-700 bg-slate-900/40"}``}>
<h4 className="text-sm font-bold text-slate-300" >
<ul className="space-y-1.5 text-sm">
  {phase === "game" ? (
    <>
      <li>{verdict.heapOk ? " " : " "} каждое р
      <li>{verdict.singleParent ? " " : " "} у
      <li>{verdict.connected ? " " : " "} все т
      <li>{verdict.countOk ? " " : " "} линий р
      <li>{verdict.bstOk ? " " : " "} обход сле
    </>
  ) : (
    <>
      <li>{connDone ? " " : " "} каждое ребро в
      <li>{connDone ? " " : " "} обход слева-на
      <li>{connDone ? " " : " "} линий ровно <b
    </>
  )}
</ul>
{phase === "game" && (
  <div className="flex flex-wrap gap-2 mt-3">
    <button onClick={undoEdge} disabled={!gameE
    white disabled:opacity-30">↶ Убрать линию</b
    <button onClick={resetGame} disabled={!gameE
    ate-700 text-white disabled:opacity-30"> С
    <button onClick={showSolution} disabled={ga
    te disabled:opacity-30"> Показать решение<
  </div>
  )}
</div>

{/* Победное дерево в игре */}
{phase === "game" && gameTree && (
  <div className="bg-emerald-950/20 rounded-xl p-
    <h4 className="text-sm font-bold text-emerald
    <TreeView root={gameTree} subtitle="Слева-нап
    ществует." />
  </div>

```

```

const done = idx >= steps.length - 1;

const inL = new Set<NodeId>();
const inR = new Set<NodeId>();
steps.slice(0, idx + 1).forEach((s) => (s.goLeft ? inL.add(s.id) : inR.add(s.id)));

const lTree = split.l;
const rTree = split.r;

return (
  <div className="space-y-4">
    <div className="flex flex-wrap items-center gap-3">
      <span className="text-sm text-slate-300">Разрез
      <select value={x0} onChange={(e) => { setX0(Number(e.target.value)); }}>
        {steps[0].x0 === 2 ? <option value="2">2 : <option value="3">3 : <option value="6">6 : <option value="7">7 : <option value="9">9 : <option value="12">12 : <option value="45">45
      </select>
      <button onClick={() => setStepIdx(Math.max(0, idx - 1))} className="bg-slate-700 text-white disabled:opacity-30">←
      <button onClick={() => setStepIdx(Math.min(steps.length - 1, idx + 1))} className="font-bold bg-indigo-600 text-white disabled:opacity-30">→
      <span className="text-xs text-slate-400">шаг {idx + 1} из {steps.length}
    </div>

    <div className="bg-slate-900/40 rounded-xl p-4 border border-slate-700">
      <h4 className="text-sm font-bold text-slate-300">Шаг {idx + 1}
      , <span className="text-rose-400">розовые → R
      <TreeView
        root={fullTree}
        nodeColor={(id) => (inL.has(id) ? "#1e3a8a" : "#f43f00")}
        nodeStroke={(id) => (inL.has(id) ? "#60a5fa" : "#f43f00")}
        subtitle={done ? "Каждый узел покрашен ровно один раз" : ""}
      />
    </div>

    <div className="text-xs text-slate-300 bg-slate-900/40 rounded-xl p-4 border border-slate-700">
      {steps[idx]?.note}
    </div>
  </div>
)

```

```

const done = idx >= steps.length - 1;
const cur = steps[idx];

return (
  <div className="space-y-4">
    <div className="flex items-center gap-3">
      <button onClick={() => setStepIdx(Math.max(0, i
        bg-slate-700 text-white disabled:opacity-30">
      <button onClick={() => setStepIdx(Math.min(step
        font-bold bg-indigo-600 text-white disabled:op
      <span className="text-xs text-slate-400">mar {i
    </div>

    <div className="grid grid-cols-1 md:grid-cols-2 g
      <div className="bg-sky-950/20 rounded-xl p-3 bo
        <h5 className="text-sm font-bold text-sky-300
          <TreeView root={l} width={480} height={220} n
        </div>
      <div className="bg-rose-950/20 rounded-xl p-3 b
        <h5 className="text-sm font-bold text-rose-300
          <TreeView root={r} width={480} height={220} n
        </div>
    </div>

    <div className="text-xs text-slate-300 bg-slate-9
      {cur?.chosenId ? (
        <>⊗ {cur.note}</>
      ) : (
        <>Готово: {done ? "деревья склеены обратно в
          />
        </>
      )}
    </div>

    {done && (
      <div className="bg-emerald-950/20 rounded-xl p-3
        <h5 className="text-sm font-bold text-emerald
          <TreeView root={merged.root} subtitle="merge(
        </div>

```



```

    return ids;
  }, [steps, idx]));
const pathSet = new Set(pathIds);

return (
  <div className="space-y-4">
    <div className="flex flex-wrap items-center gap-3">
      <span className="text-sm text-slate-300">Удалить</span>
      <select value={key} onChange={(e) => { setKey(N
        te-700 rounded-lg px-3 py-1.5 text-sm font-mo
        {POINTS.map((p) => <option key={p.key} value=
      </select>
      <button onClick={() => setStepIdx(Math.max(0, i
        bg-slate-700 text-white disabled:opacity-30">
      <button onClick={() => setStepIdx(Math.min(step
        font-bold bg-indigo-600 text-white disabled:op
      <span className="text-xs text-slate-400">шаг {i
    </div>

    {!done ? (
      <div className="bg-slate-900/40 rounded-xl p-4 l
        <TreeView
          root={fullTree}
          nodeColor={(id) => (pathSet.has(id) ? "#783
          nodeStroke={(id) => (pathSet.has(id) ? "#f5
          subtitle="Куча сама умеет удалять только ве
        />
      </div>
    ) : (
      <div className="bg-slate-900/40 rounded-xl p-4 l
        <h4 className="text-sm font-bold text-slate-3
        <TreeView root={res.root} subtitle="Валидность
        <ul className="mt-2 space-y-1 text-sm">
          <li> обход слева-направо по-прежнему сорти
          <li> каждое ребро вниз по y (куча целая) –
        </ul>
      </div>
    ))
  )
}

```



```

    let lo = 0, hi = sortedByKey.length - 1;
    while (lo <= hi) {
      const mid = Math.floor((lo + hi) / 2);
      if (sortedByKey[mid].key === target) { steps.push(
        нашли за ${steps.length + 1} сравнение(й).` });
      if (sortedByKey[mid].key < target) { steps.push({
        } — вся левая половина выброшена.` }); lo = mid
      else { steps.push({ lo, hi, mid, found: false, no
        ` }); hi = mid - 1; }
    }
    return steps;
  }, [sortedByKey]);

```

```

const wrongMid = Math.floor((shuffled.length - 1) / 2)
const comparisons = stage === 2 ? bidx + 1 : 0;

```

```

return (
  <div className="space-y-4">
    <div className="flex gap-2">
      {stage < 2 && (
        <button onClick={() => { setStage(stage === 0
          digo-600 text-white">
            {stage === 0 ? "Найти y для x=7 бинарным по
          </button>
        )}
        {stage === 2 && (
          <button onClick={() => setBidx(Math.min(bsSte
            -1.5 rounded-lg text-xs font-bold bg-indigo
            Шаг поиска →
          </button>
        )}
        <button onClick={() => { setStage(0); setBidx(0
          Сброс</button>
      </div>

```

```

    <div className="flex gap-1.5 overflow-x-auto">
      {(stage === 2 ? sortedByKey : shuffled).map((p,
        const highlight = stage === 1 && i === wrongM

```

```

-----
const GameMode: React.FC = () => {
  const [edges, setEdges] = useState<GEdge[]>([]);
  const [sel, setSel] = useState<Pair | null>(null);
  const [hover, setHover] = useState<Pair | null>(null);
  const [msg, setMsg] = useState<string>("Кликни точку-1");

  const v = gameVerdict(edges, POINTS);

  const click = (p: Pair) => {
    if (!sel) {
      setSel(p);
      setMsg(`Выбрана (${p.key}; ${p.grade}). Теперь кликай!`);
      return;
    }
    if (sameP(sel, p)) {
      setSel(null);
      return;
    }
    const res = tryConnect(edges, sel, p);
    if (res.ok) {
      const parent = sel.grade > p.grade ? sel : p;
      const child = sel.grade > p.grade ? p : sel;
      setEdges([...edges, { a: parent, b: child }]);
      setMsg(`Линия (${parent.key}; ${parent.grade})`);
      setSel(null);
    } else {
      setMsg("x " + res.reason);
      setSel(null);
    }
  };

  const win = v.win;

  return (
    <div className="space-y-4">
      <div className="flex flex-wrap items-center gap-2">
        <button onClick={() => { setEdges(edges.slice(0, edges.length - 1)); }}>

```

```

        "#a5b4fc" : isHover ? "#94a3b8" : "#38bdf8"
        transition-all duration-200" />
        <text x={PX(p.key)} y={PY(p.grade) + 3.5}
        space">{p.key}</text>
        <text x={PX(p.key)} y={PY(p.grade) + 24}
        </text>
      </g>
    );
  }}}
</svg>
</div>

{/* Судья – условия, как у Ахо–Корасика */}
<div className={`rounded-xl p-4 border transition
  ate-900/40`} `>
  <h4 className="text-sm font-bold text-slate-300">
  <ul className="space-y-1.5 text-sm">
    <li>{v.hear ? " " : " "} каждое ребро – вниз
    <li>{v.oneParent ? " " : " "} у каждой точки
    <li>{v.acyclic ? " " : " "} циклов нет</li>
    <li>{v.bst ? " " : " "} обход слева-направо –
    <li>{v.complete ? " " : " "} линий  $n-1 = \{POI$ 
  </ul>
  {win && (
    <p className="mt-3 text-sm font-bold text-eme
    ения просто не существует.</p>
  )}
</div>

  <div className="text-xs text-slate-300 bg-slate-9
</div>
);
};

/* ===== СВЯЗЬ С КОМПИЛЯТОРОМ =====

/**
 * Живые переменные Python-кода страницы treap:

```

```
-----
type Tab = "build" | "split" | "merge" | "erase" | "lay
```

```
const TABS: { id: Tab; label: string }[] = [
  { id: "build", label: "Собрать" },
  { id: "split", label: "Split ≈ " },
  { id: "merge", label: "Merge  " },
  { id: "erase", label: "Erase   " },
  { id: "layers", label: "Миф: 2k/2k+1" },
  { id: "search", label: "Поиск ≠ сортировка" },
  { id: "game", label: "Игра    " },
];
```

```
export const TreapBuildViz = () => {
  const [tab, setTab] = useState<Tab>("build");
  const chapterId = useContext(VizChapterContext);
  const live = useTreapLive();
```

```
// вкладка = подрежим страницы: панель Python подстав.
useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, tab);
}, [chapterId, tab]);
```

```
return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border">
    <div className="flex flex-wrap items-center justify">
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-emerald-600 text-white">
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white">
            <p className="text-sm text-emerald-300">Пар
          </div>
        </div>
      </div>
    </div>
  </div>
```

```
  <div className="flex flex-wrap gap-2 mb-5">
    {TABS.map((t) => {
```

```

</div>

    {tab === "build" && <BuildMode />}
    {tab === "split" && <SplitMode />}
    {tab === "merge" && <MergeMode />}
    {tab === "erase" && <EraseMode />}
    {tab === "layers" && <LayersMode />}
    {tab === "search" && <SearchMode />}
    {tab === "game" && <GameMode />}
  </div>
);
};

```

ФАЙЛ 81/113: src/components/VizLiveStatus.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { liveMatrix, pickLiveVar, useVizRuntime } from
import { VizSourceBadge } from "../VizSourceBadge";

interface VizLiveStatusProps {
  /** Название демонстрации – подписывается в режиме вс
  demoTitle?: string;
  className?: string;
}

/** Имена, по которым визуализации обычно читают матрицу
const MATRIX_NAMES = ["d", "dist", "dp", "D", "S", "P",

/**
 * Строка состояния связи «компилятор → демонстрация» д
 *
 * Раньше компонент молча проигрывал собственный встроен
 * редакторе Python был пустой код, код с другими именами
 * размерности, – выглядело так, будто компилятор исполн
 * картинка живёт отдельной жизнью. Теперь источник дан

```

Универсальное пособие • полный исходный код

```
-----
import { ArticulationPointsViz } from "./ArticulationPo
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimula
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphComponentsViz } from "./GraphComponentsVi
import { GraphTraversalViz } from "./GraphTraversalViz"
import { ComplexityViz } from "./ComplexityViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import { PlanarColoringViz } from "./PlanarColoringViz"
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomato
import { SegmentTreeVisualizer } from "./SegmentTreeVis
import SplayPlayground from "./SplayPlayground";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithms'
import { TopologicalSortViz } from "./TopologicalSortVi
import { WaterfallAnimationWidget } from "./WaterfallAn
import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumVi
import { SparseTableViz } from "./visualizers/SparseTab
import { TreapBuildViz } from "./TreapBuildViz";
import { CheatSheetViz } from "./CheatSheetViz";

export interface VizEntry {
  /** Заголовок панели/модалки. */
  title: string;
  /** Короткое пояснение, что здесь можно потыкать. */
  hint?: string;
  Component: React.FC;
}
```



```

    hint: "7 вкладок: Собрать (сортировка + соединение +  

        собери дерево мышкой, судья-инварианты объясняет к  

    Component: TreapBuildViz,
},
"prefix-sums-2d": {
    title: "Префиксные суммы (1D и 2D)",
    hint: "Наведите на число – увидите, из чего оно сло  

    Component: PrefixSumViz,
},
"dynamic-programming": {
    title: "Динамическое программирование",
    hint: "Таблица подзадач заполняется на глазах.",
    Component: DPVisualizer,
},
"splay-tree": {
    title: "Splay: механика → большое дерево → собери с  

    hint: "Три вкладки: покадровая механика поворотов в  

        песочница «подними узел сам» с проверкой порядка.  

    Component: SplayPlayground,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component: GraphDfsBfsViz,
"top-sort": { title: "Топологическая сортировка", Component: TopSortViz,
"scc-kosaraju": { title: "Компоненты сильной связности", Component: SccKosarajuViz,
"graph-articulation": { title: "Точки сочленения", Component: GraphArticulationViz,
"bridges-code": { title: "Мосты: DFS + tin/low", Component: BridgesCodeViz,
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: EulerPathVsCycleViz,
"planarity-euler-formula": {
    title: "Планарность и покраска:  $K_5/K_{3,3}$ ,  $\chi \leq \Delta+1$ , д  

    hint: "Четыре вкладки: непланарные  $K_5$  и  $K_{3,3}$ , жадна  

        ашего кода красит вершины напрямую.",
    Component: PlanarColoringViz,
},
dijkstra: {
    title: "Дейкстра",
    hint: "Жадно забираем ближайшую вершину и релаксиру  

    Component: DijkstraViz,
},
"bellman-ford": {

```

```
-----
    title: "Интерактивный симулятор базовых структур: с
    hint: "Проверь LIFO, FIFO и приоритетную очередь на
    Component: Simulator,
  },
  "graph-cheatsheet": {
    title: "Быстрая шпаргалка: BFS – Прим – Декстер",
    hint: "Один граф, один формат edges[x] = [(y, w), .
    ,
    Component: CheatSheetViz,
  },
  "everyday-basics": {
    title: "Бытовой тренажёр: стек, очередь, куча",
    hint: "Тарелки, очередь в столовой и мешок гипотез
    Component: Simulator,
  },
};
```

```
export const getViz = (id?: string): VizEntry | undefined
```

ФАЙЛ 83/113: src/components/VizSourceBadge.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
-----
import { Cpu, Film } from "lucide-react";
import { Tooltip } from "../Tooltip";
```

```
interface VizSourceBadgeProps {
  /**
   * true – картинку рисуют переменные из выполненного I
   * false – компонент показывает собственную встроенную
   */
  linked: boolean;
  /**
   * Компилятор уже выполнил код и прислал шаг, но пере
   * (отладчик стоит ДО первой строки или в коде другие
   * подпись должна объяснять, почему картинка ещё не и
```

```

        ? "border-emerald-500/50 bg-emerald-500/10"
        : awaiting
        ? "border-sky-500/50 bg-sky-500/10 text-slate-500"
        : "border-slate-600 bg-slate-800/80 text-slate-500"
      }`}
    >
    {linked ? <Cpu className="h-3 w-3" /> : awaiting}
    {linked ? "данные из компилятора" : awaiting}
    {linked && detail ? <span className="font-mono" /> : ""}
  </span>
</Tooltip>

{!linked && !awaiting && demoTitle ? (
  <span className="rounded-full border border-slate-500" />
  сценарий: {demoTitle}
</span>
) : null}

{mismatch ? (
  <Tooltip side="bottom" content={mismatch}>
    <span
      tabIndex={0}
      className="cursor-help rounded-full border border-slate-500"
    />
    <div>
      размерность не совпадает
    </div>
  </Tooltip>
) : null}
</div>
);
}

```

```

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
  0: { H: 1, S: 7 },
  1: { E: 2, I: 5 },
  2: { R: 3 },
  3: { S: 4 },
  4: {},
  5: { S: 6 },
  6: {},
  7: { H: 8 },
  8: { E: 9 },
  9: {},
};

export const WaterfallAnimationWidget: React.FC = () => {
  // Переключатель режимов: 'training' (Полный BFS обход)
  const [activeMode, setActiveMode] = useState<'training'>('training');

  /* — связь с Python-компилятором (PAGE_SYNC["aho-corasick"])
     text — сканируемый текст, state — вершина автомата
     matches — найденные вхождения (индекс конца, слово)
  */
  const chapterId = useContext(VizChapterContext);
  const runtime = useVizRuntime();
  const vars = runtime?.variables;

  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, "aho");
  }, [chapterId]);

  const liveText = vars ? vizString(vars.text) ?? vizString('') : vizString('');
  const liveState = vars ? vizNumber(vars.state) ?? vizNumber(0) : vizNumber(0);
  const liveI = vars ? vizNumber(vars.i) ?? vizNumber(0) : vizNumber(0);
  const liveMode = vars ? vizString(vars.mode) : null;
  const liveMatchesRaw = vars ? vizArray(vars.matches) : vizArray([]);
  const liveMatchesSig = useMemo(() => (liveMatchesRaw.map(m => m[0])).join('')) : '';
  const liveFail = vars ? vars.fail : undefined;

  useEffect(() => {

```

```

const compilerLinked = Boolean(liveText || (liveState

// === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
const [textToScan, setTextToScan] = useState<string>(
const [currentIndex, setCurrentIndex] = useState<number>(
const [currentNode, setCurrentNode] = useState<number>(
const [isSearchDone, setIsSearchDone] = useState<boolean>(
const [searchLog, setSearchLog] = useState<string>(
  'Режим поиска: Введите текст и жмите «Следующий шаг»
);
const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>(
const [foundMatches, setFoundMatches] = useState<{ index: number; text: string }[]>(
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и
const [trainStep, setTrainStep] = useState<number>(0);

const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const clean = e.target.value.toUpperCase().replace(/ /g, '');
  setTextToScan(clean);
  resetSearchSimulation();
};

const resetSearchSimulation = () => {
  setCurrentIndex(0);
  setCurrentNode(0);
  setIsSearchDone(false);
  setSearchLog('Симуляция сброшена. Лосось вернулся на базу');
  setJumpPath(null);
  setFoundMatches([]);
  setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
  if (currentIndex >= textToScan.length) {
    setIsSearchDone(true);
    setSearchLog('Мы проплыли весь текст! Лосось доволен');
  }
};

```

```

const currentMatches: { index: number; word: string }[] = [];
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
    if (WATERFALL_NODES[temp].is_terminal) {
        WATERFALL_NODES[temp].words.forEach((word) => {
            currentMatches.push({ index: currentIndex, word: word });
            matchedWordsForLog.push(word);
        });
    }
    temp = WATERFALL_NODES[temp].term_link;
}

if (matchedWordsForLog.length > 0) {
    logMsg += `    БИНГО! Найдено слово: ${matchedWordsForLog.join(' ')}\n`;
    setActiveSearchCodeLine(4);
}

setCurrentNode(curr);
setCurrentIndex(currentIndex + 1);
setSearchLog(logMsg);
if (currentMatches.length > 0) {
    setFoundMatches((prev) => [...prev, ...currentMatches]);
}
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
    {
        targetNodeId: 0,
        title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
        desc: 'Начало алгоритма BFS. Корень (ROOT) не обрабатывается. Другие вершины сразу инициализируются с fail = ROOT',
        codeLine: 1,
        scoutPos: 0,
        inspectedParentFail: 0,
    },

```

```

    title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
    desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
        H" и "S" не подходят. Ветки нет, fail[HI] = ROOT',
    codeLine: 3,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'I',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 8,
    title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТАТЬ
    desc: '    ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
        ). Рыба плавает в ROOT и ищет ветку "H". У корня
        суффикс "H"!',
    codeLine: 4,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'H',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 3,
    title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
    desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
        е подходят. fail[HER] = ROOT.',
    codeLine: 3,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'R',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 6,
    title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
    desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель :
        но ветка "S" → #7 (S) совпадает! fail[HIS] = #7
    codeLine: 4,

```

```

    <span className="text-3xl"> </span>
  <div>
    <h4 className="text-2xl font-bold text-white">
      Анимация водопада: Полное обучение автомата
    </h4>
    <p className="text-sm text-slate-400">
      Визуализация пошагового построения всех ф
    </p>
  </div>
</div>

{/* Переключатель режимов */}
<div className="bg-slate-900 p-1.5 rounded-xl b
  <button
    onClick={() => setActiveMode('training')}
    className={`flex items-center space-x-1.5 p-
      activeMode === 'training'
        ? 'bg-indigo-600 text-white shadow-lg s
        : 'text-slate-400 hover:text-white'
    `}
  >
    <GitBranch className="w-4 h-4" />
    <span>Режим 1: Полное обучение (BFS от корн
  </button>
  <button
    onClick={() => setActiveMode('search')}
    className={`flex items-center space-x-1.5 p-
      activeMode === 'search'
        ? 'bg-blue-600 text-white shadow-lg sha
        : 'text-slate-400 hover:text-white'
    `}
  >
    <Play className="w-4 h-4" />
    <span>Режим 2: Поиск в тексте</span>
  </button>
</div>
</div>

```



```

        ion-all"
        >
        Назад
    </button>
    <button
        onClick={() => setTrainStep((prev) => M
        disabled={trainStep === TRAINING_STEPS_
        className="flex-1 bg-indigo-600 hover:b
    ition-all shadow-lg shadow-indigo-900/50"
        >
        {trainStep === TRAINING_STEPS_INFO.leng
    </button>
</div>
</div>

{/* Блок с кодом построения fail-ссылки и опи
<div className="lg:col-span-2 bg-slate-900/90
    >
    <div>
        <h5 className="text-white font-bold mb-3
            <span className="flex items-center gap-
                <span>❌</span> Код BFS построения fail
            </span>
            <span className="text-xs bg-indigo-950 l
                Вершина: #{targetTrainingNode.id} ({t
            </span>
        </h5>
        <div className="bg-slate-950 p-4 rounded-
            <div className={`p-1.5 rounded flex ite
            -l-4 border-indigo-500 text-indigo-300 font
                <span>1. let u = trie[r][char]; // Те
                {currentTrainInfo.codeLine === 1 && <
            тут</span>}
            </div>
            <div className={`p-1.5 rounded flex ite
            -l-4 border-indigo-400 text-indigo-200 font
                <span>2. let v = fail[r]; // Подъем к
            ainInfo.inspectedParentFail}.label}}</span>

```

```

<div className="grid grid-cols-1 lg:grid-cols-3"
  <div className="bg-slate-900/60 p-5 rounded-x
    <div>
      <h5 className="text-white font-bold mb-2"
        <span>» </span> Текст для сканирования
      </h5>
      <p className="text-xs text-slate-400 mb-4"
        Введи текст, чтобы лосось просканировал
      </p>
      <input
        type="text"
        value={textToScan}
        onChange={handleTextChange}
        maxLength={15}
        placeholder="ВВЕДИТЕ ТЕКСТ..."
        className="w-full bg-slate-800 border b
        cus:outline-none focus:border-blue-500"
      />
    </div>

    <div>
      <div className="mb-4 flex flex-wrap gap-1"
        <span className="text-xs text-slate-400"
          {textToScan.split('').map((char, idx) =>
            <span
              key={idx}
              className={`w-7 h-8 flex items-cent
                idx === currentIndex
                  ? 'bg-blue-600 text-white shadow
                  : idx < currentIndex
                  ? 'bg-emerald-950/60 text-emera
                  : 'bg-slate-800 text-slate-500'
                }`}
            >
              {char}
            </span>
          )}}
        </div>
      </div>
    </div>
  </div>
  {currentIndex >= textToScan.length && (

```

```

        ружок по fail!</span>}
      </div>
      <div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold` :
        <span>3. curr = trie[curr].get(char)
        {activeSearchCodeLine === 3 && <span>
ем по течению</span>}
      </div>
      <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 font
        <span>4. if (is_terminal[curr]) report
        {activeSearchCodeLine === 4 && <span>
овпадение!</span>}
      </div>
    </div>
  </div>

  <div>
    <h5 className="text-slate-300 font-bold m
    <AlertCircle className="w-3.5 h-3.5 tex
    </h5>
    <div className="bg-slate-800 p-3.5 rounded
tems-center">
      {searchLog}
    </div>
  </div>
</div>
</div>
))

```

```

{ /* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 vi
-hidden shadow-2xl">
  { /* Водопадные фоновые эффекты */}
  <div className="absolute inset-0 opacity-10 bg-
o-900 to-transparent pointer-events-none"></d
  <div className="absolute top-0 left-1/2 -transl
    <div className="w-8 bg-blue-400 h-full animat

```

```

        strokeDasharray="4,4"
        opacity="0.4"
      />
      <rect x="25" y={level.y - 12} width="14" height="24" />
    />
    <text x="32" y={level.y + 4} fill="#94a0b4" />
      {level.label}
    </text>
  </g>
)}}
```

```

{/* Рендер прямых переходов (потоков воды) */}
Object.entries(TRIE_TRANSITIONS).map(([fromId, toId]) => {
  const fromNode = WATERFALL_NODES[Number(fromId)];
  return Object.entries(children).map(([className, node]) => {
    const toNode = WATERFALL_NODES[toId];

    // Подсветка в режиме поиска
    let isHighlighted = activeMode === 'search' ?
    type === 'normal';

    // Подсветка в режиме обучения (когда ищем ответ)
    let isTrainingExamined = activeMode === 'train' ?
    let isMatchBranch = isTrainingExamined && node.id === toId;

    return (
      <g key={`edge-${fromNode.id}-${toNode.id}`} />
      {
        /* Линия течения воды */
        <line
          x1={fromNode.x}
          y1={fromNode.y}
          x2={toNode.x}
          y2={toNode.y}
          stroke={isHighlighted ? '#3b82f6' : '#ccc'}
          strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
          className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
        />
        {
          /* Символ перехода */

```

```

    let isHighlighted = activeMode === 'search' || activeMode === 'fail';
    let isJustEstablished = activeMode === 'transition' || activeMode === 'target-link';

    // Вычисляем изогнутую кривую для красоты
    const dx = targetNode.x - node.x;
    const dy = targetNode.y - node.y;
    const cx = node.x + dx / 2 + (node.id % 2 === 0 ? -1 : 1) * 30;
    const cy = node.y + dy / 2 - 30;

    return (
      <g key={`fail-${node.id}`}>
        <path
          d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${targetNode.x} ${targetNode.y}`}
          fill="none"
          stroke={isHighlighted ? '#f43f5e' : '#10b98f'}
          strokeWidth={isHighlighted ? 6 : 2}
          strokeDasharray="6,6"
          className={isHighlighted ? 'highlighted' : ''}
          opacity={isHighlighted ? 1 : 0.5}
        />
        {isJustEstablished && (
          <text x={cx} y={cy - 10} fill="#10b98f">
            fail[{node.label}] = {targetNode.label}
          </text>
        )}
      </g>
    );
  }
}
}

```

```

/* Рендер вершин (бассейнов водопада) */
Object.values(WATERFALL_NODES).map((node) => {
  const isCurrent = activeFishPosNode.id === node.id;
  const isTargetChild = activeMode === 'transition' || activeMode === 'target-link';
  const hasWords = node.words.length > 0;

```

```

        <text x="0" y="2" fontSize="10" f
            ИЩЕМ FAIL
        </text>
    </g>
}}

{/* Индикатор словарных слов */}
{hasWords && {
    <g transform={`translate(${node.x +
        <circle cx="0" cy="0" r="9" fill=
        <text x="0" y="3" fontSize="10" f
            *
        </text>
    </g>
}}
</g>
});
}}}
```

```

{/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */}
<g>
    {/* Круг-подложка под рыбу */}
    <circle
        cx={activeFishPosNode.x + 15}
        cy={activeFishPosNode.y - 25}
        r="18"
        fill={activeMode === 'training' ? '#8b5
        stroke="#ffffff"
        strokeWidth="2"
        style={{ transition: 'cx 0.8s cubic-bez
        className="shadow-lg"
    />
    {/* Сама рыба */}
    <text
        x={activeFishPosNode.x + 15}
        y={activeFishPosNode.y - 19}
        fontSize="18"
        textAnchor="middle"
```

```

    </div>
  })

  { /* что сейчас сообщил компилятор */ }
  <div className="mt-6 flex flex-wrap items-center">
    <span className="text-[10px] uppercase tracking-wider">
      <Terminal className="w-3.5 h-3.5" /> из компилятора
    </span>
    { compilerLinked ? (
      <>
        { liveText && <span className="text-[11px] font-mono">
          >text = { liveText } </span> }
        { liveI !== null && <span className="text-[11px] font-mono bg-cyan-300 py-1">
          i = { liveI } { liveText ? ` (${ liveText }` : '' }
          { liveState !== null && (
            <span className="text-[11px] font-mono bg-cyan-300">
              state = { liveState } { WATERFALL_NODES[ liveI ] }
            </span>
          ) }
        { liveMatchesSig && <span className="text-[11px] font-mono bg-cyan-300 py-1">
          matches: { foundMatches.map( (m) => m ) }
          { liveFail !== undefined && <span className="text-[11px] font-mono bg-cyan-300 py-1">
            fail = { JSON.stringify( liveFail ) }
          }
        </>
      ) : (
        <span className="text-[11px] text-slate-500">
          открой панель Python и запусти Ахо–Корасик:
          "font-mono text-cyan-300">state</code>, <code>state</code>
          "font-mono text-cyan-300">matches</code> поведут лосося по этому пути
        </span>
      ) }
    </div>
  </div>
);
};

```

=====

```

const cellBase =
  "flex items-center justify-center rounded-md font-monospace text-sm
  w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";

export const PrefixSumViz: React.FC = () => {
  const [mode, setMode] = useState<"1d" | "2d">("1d");
  const chapterId = useContext(VizChapterContext);

  // Вкладка 2D = своё демо (свой скелет): компилятор prefix-sum
  useEffect(() => {
    if (chapterId) emitVizDemo(chapterId, mode === "2d" ? "2d" : "1d"), [chapterId, mode]);
  }, [chapterId, mode]);

  return (
    <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
      <div className="flex gap-2 mb-4 overflow-x-auto">
        {
          [
            ["1d", "1. Одномерные суммы"],
            ["2d", "2. Двумерные суммы"],
          ] as const
        }.map(([key, label]) => (
          <button
            key={key}
            onClick={() => setMode(key)}
            className={`px-3 py-2 rounded-lg text-xs sm:text-sm
              mode === key ? "bg-indigo-600 text-white" : "text-slate-400"
            }`)
          >
            {label}
          </button>
        ))
      </div>
      {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
    </div>
  );
};

```



```

<div className="flex gap-1.5 mb-1 pl-[52px]">
  {A1.map(({_, i} => {
    <div key={i} className={` ${cellBase} !h-5
      {i}
    </div>
  }}}
</div>

{/* массив a */}
<div className="flex gap-1.5 items-center mb-1"
  <div className="w-[46px] shrink-0 text-right"
    {A1.map(({v, i} => {
      const inCover = covered && i >= covered.f
      const inRange = i >= range.l && i <= range.r
      const isDrivenAdd = shownDriven !== null
      return (
        <div
          key={i}
          onMouseEnter={() => setHover({ kind:
          onMouseLeave={() => setHover(null)}
          onClick={() => setRange((r) => (i < r
          className={` ${cellBase} cursor-pointe
            isDrivenAdd
              ? "bg-amber-500 text-black ring-2
              : inCover
                ? "bg-indigo-500 text-white rin
                : inRange
                  ? "bg-indigo-900/70 text-indi
                  : "bg-slate-800 text-slate-300
            }`
          title={`a[${i}] = ${v}`}
        >
          {v}
        </div>
      );
    })}
  </div>

```

```

        </div>
    </div>
</div>

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border
    {shownDriven !== null && !hover ? (
        <p className="text-slate-300">
            <b className="text-emerald-400">Компилятор
            <span className="font-mono text-white">i =
            <span className="font-mono text-emerald-300
            {hasLiveP && <> = <span className="font-mono
            <span className="ml-1 text-slate-500">Пустой
        </p>
    ) : hover?.kind === "pref" ? (
        hover.i === 0 ? (
            <p className="text-slate-300">
                <b className="text-emerald-400">P[0] = 0<
                отрезка, начинающегося с нуля.
            </p>
        ) : (
            <p className="text-slate-300">
                <b className="text-emerald-400">
                    P[{hover.i}] = {pref[hover.i]}
                </b>{" "}
                – это сумма всего, что набежало от начала
            <span className="font-mono text-indigo-300
                a[0..{hover.i - 1}] = {A1.slice(0, hove
            </span>
            </p>
        )
    ) : hover?.kind === "a" ? (
        <p className="text-slate-300">
            <b className="text-indigo-400">
                a[{hover.i}] = {A1[hover.i]}
            </b>{" "}
            входит во все префиксы, начиная с{" "}
            <span className="font-mono text-emerald-300

```

```
const P = useMemo(() => {
  const p = Array.from({ length: n + 1 }, () => Array(
    for (let i = 1; i <= n; i++) {
      for (let j = 1; j <= m; j++) {
        p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i]
      }
    }
  );
  return p;
}, [n, m]);
```

```
const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });
```

```
/**
```

```
 * Матрицы из кода пользователя рисуются В СВОЕЙ размерности.
 *
```

```
 * Раньше значения S[i][j] протискивались в демо-сетку,
 * размере матрицы часть ячеек становилась «пустыми»,
 * шаги компилятора не совпадали с картинкой.
```

```
 */
```

```
const livePrefSource = pickLiveVar(vars, ["S", "P", "g"]);
const liveS = liveMatrix(livePrefSource?.value);
const hasLiveS = liveS !== null;
```

```
const liveArrSource = pickLiveVar(vars, ["a", "A", "g"]);
const liveA = liveMatrix(liveArrSource?.value);
```

```
const prefRows = liveS?.rows ?? P.length;
const prefCols = liveS?.cols ?? (P[0]?.length ?? 0);
const grid: (number | null)[][] = liveS
  ? Array.from({ length: prefRows }, (_, i) =>
    Array.from({ length: prefCols }, (_, j) => liveS[i][j])
  )
  : P;
```

```
const srcRows = liveA?.rows ?? n;
const srcCols = liveA?.cols ?? m;
const srcGrid: (number | null)[][] = liveA
```

```

const cornerOf = (i: number, j: number) => {
  if (i === r2 + 1 && j === c2 + 1) return "A";
  if (i === r1 && j === c2 + 1) return "B";
  if (i === r2 + 1 && j === c1) return "C";
  if (i === r1 && j === c1) return "D";
  return null;
};

return (
  <div className="space-y-5">
    {sizeMismatch ? (
      <p className="rounded-lg border border-amber-500">
        {sizeMismatch}
      </p>
    ) : null}
    <div className="grid gap-5 lg:grid-cols-2">
      {/* Исходная матрица */}
      <div className="overflow-x-auto">
        <div className="text-xs text-slate-400 mb-2">
          Исходная матрица {liveA ? `${liveArrSource?}` : ''}
          чтобы задать прямоугольник
        </div>
        <div className="inline-block">
          {srcGrid.map((row, i) => (
            <div key={i} className="flex gap-1.5 mb-1">
              {row.map((v, j) => {
                const inRect = i >= r1 && i <= r2 &&
                return (
                  <div
                    key={j}
                    onClick={() =>
                      setRect((r) =>
                        i < r.r1 || j < r.c1
                          ? { r1: i, c1: j, r2: Math.max(r.r2, i), c2: Math.max(r.c2, j) }
                          : { ...r, r2: i, c2: j }
                      )
                    }
                  </div>
                )
              })}
            </div>
          ))}
        </div>
      </div>
    </div>
  )
);

```

```

        onMouseEnter={() => setHover({ i,
        onMouseLeave={() => setHover(null
        className={` ${cellBase} cursor-he
        isCompilerCell
            ? "bg-emerald-600 text-white
            : isHover
            ? "bg-amber-500 text-white
            : blank
            ? "bg-slate-950 text-slate-400
            : cornerColor
        }`}
        title={hasLiveS ? `${livePrefSource}
    >
    {blank ? "." : String(shownValue)}
    {corner && !isHover && (
        <span className="absolute -top-
ht">
        {corner}
        </span>
    )}
    </div>
    );
    }}}}
    </div>
    )}}
    </div>
    </div>
    </div>

    <div className="min-h-[54px] bg-slate-900 border
    {hover ? (
        <p className="text-slate-300">
            <b className="text-amber-400">
                {liveS ? livePrefSource?.name ?? "S" : "P
            </b>{" "}
            – сумма всего прямоугольника от левого верх
        </p>
    ) : (

```

```

-----
import { motion, AnimatePresence } from "motion/react";
import {
  Play,
  RotateCcw,
  ChevronRight,
  ChevronLeft,
  Lock,
  ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () =>
  for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
  for (let j = 1; j < LOG; j++) {
    for (let i = 0; i + (1 << j) <= N; i++) {
      st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j)]);
    }
  }
);

// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
  [45, 12, 88, 34, 11, 76, 23, 90],
  [67, 19, 44, 55, 33, 21, 65, 87],
  [14, 51, 99, 13, 22, 64, 43, 76],
  [89, 32, 54, 71, 15, 88, 29, 60],
  [25, 41, 16, 92, 9, 17, 56, 31],
  [59, 18, 77, 24, 61, 82, 35, 12],
  [73, 8, 38, 85, 47, 95, 19, 58],
  [39, 81, 62, 28, 51, 42, 85, 14]
];

const ST2D: number[][][][] = Array.from({length: 8}, () =>
  Array.from({length: 8}, () =>
    Array.from({length: 4}, () =>

```

```

// Каждая вкладка = своё демо с собственным скелетом:
useEffect(() => {
  if (chapterId) emitVizDemo(chapterId, tab === "2d_build", [chapterId, tab]);
}, [chapterId, tab]);

return (
  <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-5" >
    <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800" >
      <button
        onClick={() => setTab("1d")}
        className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white" : "bg-slate-800 text-slate-400`}
      >
        1. Сворачивание 1D
      </button>
      <button
        onClick={() => setTab("2d_build")}
        className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white" : "bg-slate-800 text-slate-400`}
      >
        2. Сворачивание 2D (Построение)
      </button>
      <button
        onClick={() => setTab("2d_query")}
        className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white" : "bg-slate-800 text-slate-400`}
      >
        3. Запрос 2D (Формула)
      </button>
    </div>

    <AnimatePresence mode="wait">
      {tab === "1d" ? (
        <Viz1D key="1d" />
      ) : tab === "2d_build" ? (
        <Viz2DBuild key="2d_build" />
      ) : (
        <Viz2DQuery key="2d_query" />
      )}
    </AnimatePresence>
  </div>
)

```

```

        ся демо-размера  $\{N\} \times \{LOG\}$ .`
        : null;

// Total steps: we animate computing each element for
const computeSteps: { i: number; j: number }[] = [];
for (let j = 1; j < LOG; j++) {
  for (let i = 0; i + (1 << j) <= N; i++) {
    computeSteps.push({ i, j });
  }
}

const maxStep = computeSteps.length;
// Эта таблица читает i/j/st напрямую. Не двигаем её
// строки произвольного Python-кода.
const pointedCell = hover ?? liveTarget;
const covered = pointedCell ? { from: pointedCell.i,

return (
  <div className="flex flex-col gap-5">
    <div className="flex items-center justify-between">
      <div>
        <h3 className="text-lg font-bold text-indigo-500">
          Построение Sparse Table (Min)
        </h3>
        <p className="text-sm text-slate-400">
          Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$ 
        </p>
        {dimMismatch ? (
          <p className="mt-2 max-w-[560px] rounded-lg border border-slate-200 p-2">
            {dimMismatch}
          </p>
        ) : null}
      </div>
      <div className="flex gap-2">
        <button
          onClick={() => setStep(Math.max(0, step - 1))}
          className="p-2 bg-slate-800 hover:bg-slate-700"
          disabled={step === 0}
        >

```



```

        </div>
        <span className="text-[9px] text-slate-400">
        </div>
    );
    }}}
</div>
<div className="mt-2 text-xs min-h-[36px]">
    {hover && covered ? (
        <p className="text-slate-300">
            <b className="text-sky-400">
                ST[{hover.i}][{hover.j}] = {compilerLin
            j]}
            </b>{" "}
            – это минимум на отрезке{" "}
            <span className="font-mono text-sky-300">
                [{covered.from}, {covered.to}]
            </span>{" "}
            длины  $2^{\text{hover.j}}$  = {1 << hover.j}:{ " "}
            <span className="font-mono text-slate-400">
                min({arr1D.slice(covered.from, covered.to)}
            </span>
        </p>
    ) : (
        <p className="text-slate-500">
            Наведите курсор (или тапните) на любое чи
        </p>
    )}
</div>
</div>

<div className="overflow-x-auto pb-2">
    <div className="inline-block min-w-full bg-slate-100">
        <div className="grid grid-cols-[auto_repeat(4,1fr)]">
            {/* Headers */}
            <div className="text-slate-500 font-mono text-sm">
                i \ j
            </div>
            <div className="text-center font-bold text-sm">

```

```

        let isSrc2 = false;

        if (compilerLinked && liveTarget && liveTarget.i > 0) {
            isSrc1 = i === liveTarget.i && j === liveTarget.j;
            isSrc2 = i === liveTarget.i + (1 << liveTarget.j);
        } else if (!compilerLinked && step > 0) {
            const currentStep = computeSteps[step];
            isActive = currentStep.i === i && currentStep.j === j;
            if (currentStep.j - 1 === j) {
                if (currentStep.i === i) isSrc1 = true;
                if (currentStep.i + (1 << (currentStep.j - 1)) === i) isSrc2 = true;
            }
        }

        // Готовые вычисленные уровни демо не нужны
        // уровень j=0 всегда берём из уже существующих
        const shownValue = compilerLinked ? compilerLinked : st;
        const displayValue =
            shownValue === undefined || shownValue === null
            ? "."
            : typeof shownValue === "object"
            ? JSON.stringify(shownValue)
            : String(shownValue);

        return (
            <div key={j} className="relative flex flex-col items-center justify-center" style={style}
                {!isValid ? (
                    <div className="w-[clamp(30px,90px,100px)] h-[clamp(30px,90px,100px)]" style={style}
                        : (
                            <motion.div
                                onMouseEnter={() => (compilerLinked ? setCompilerLinked(shownValue) : setCompilerLinked(st))}
                                onMouseLeave={() => setHover(false)}
                                onClick={() => (compilerLinked ? setCompilerLinked(st) : setCompilerLinked(st))}
                                style={style}
                                initial={{ opacity: 0, scale: 1 }}
                                animate={{
                                    // При st = [] каркас уже с opacity: compilerLinked ? 1

```

```

        <path
          d={`M 0 0 C 30 0, 30 ${isSrc1 ? "10b981" : "white"}
          fill="none"
          stroke={isSrc1 ? "#10b981" : "white"}
          strokeWidth="3"
        />
      </motion.svg>
    )}
  </div>
);
}}}
</React.Fragment>
)}}
</div>
</div>
</div>

```

```

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-800 p-4">
  {compilerLinked && liveTarget ? (
    <p className="text-base text-slate-300">
      <span className="text-emerald-400 font-bold">ST</span>
      <span className="font-mono text-white">i =
      — активна ячейка <span className="font-mono text-white">
        {(hasLiveTable || hasLiveSource) && <> = <span>
          ?.[liveTarget.j] ?? (liveTarget.j === 0 ? liveTarget.i : liveTarget.j)
        </span>
      </p>
    ) : step > 0 && step <= maxStep ? (
      (() => {
        const c = computeSteps[step - 1];
        const half = 1 << (c.j - 1);
        return (
          <p className="text-lg text-slate-300">
            <span className="text-white font-bold">
              ST[{c.i}][{c.j}]
            </span>{" "}
            = min(
              <span className="text-emerald-400 font-bold">

```

```

-----
* клик есть, а матрица не меняется. Закрепление живё
*/
const [pinnedK, setPinnedK] = useState<number | null>
const prevLiveK = useRef<number | null>(liveK);
useEffect(() => {
  if (liveK !== prevLiveK.current) {
    prevLiveK.current = liveK;
    setPinnedK(null);
  }
}, [liveK]);
const liveR = vizNumber(vars?.r);
const liveC = vizNumber(vars?.c);
const liveTable = vizRecord(vars?.st2);
const hasLiveSource = !!vars && Object.prototype.hasOwnProperty.call(vars, 'source');
const hasLiveTable = !!vars && Object.prototype.hasOwnProperty.call(vars, 'table');
const liveTableEmpty = hasLiveTable && Object.keys(liveTable).length === 0;
const codeCell = liveR !== null && liveC !== null ? {
  const compilerLinked = codeCell !== null || liveK !== null;
  const shownK = pinnedK !== null
    ? pinnedK
    : liveK !== null
      ? Math.max(0, Math.min(3, Math.trunc(liveK)))
      : hasLiveSource
        ? 0
        : k;

  /** 2D-демонстрация нарисована на фиксированной сетке
const liveSource2D = liveMatrix(vars?.A);
const dimMismatch2D = liveSource2D
  ? liveSource2D.rows !== 8 || liveSource2D.cols !== 8
  : `Демо 2D построено на матрице 8x8, а в вашем ко
    не её не отрисовываются, значения вашей матрицы
    : null
    : null;

// Уровень и клетка берутся прямо из k/r/c; номер строки
useEffect(() => {
  setHover(null);

```

```

    </p>
  })

  <div className="flex flex-col items-center gap-1
    scrollbar">
    {[...Array(shownK + 1)].map((_, i) => shownK
      const stepL = 1 << step;
      const stepSize = 8 - stepL + 1;
      const isCurr = step === shownK;

      return (
        <React.Fragment key={step}>
          {idx > 0 && <div className="text-slate
            <div className={`flex flex-col items-c
              <h4 className={`font-bold mb-3 flex
            }>
              <span>Матрица блоков {stepL}x{step
                <span className={`text-[10px] font
                  rder-indigo-800/40 text-indigo-300' : 'bg-s
                </h4>
                <div className="max-w-full overflow-
                  <div className={`grid gap-[2px] p-
                    go-900/40 shadow-[0_0_25px_rgba(99,102,241,
                    eColumns: `repeat(${stepSize}, max-content)
                  {Array.from({length: stepSize * st
                    const r = Math.floor(idx / step
                    const c = idx % stepSize;

                    const liveValue = liveTable?.[`
                    // Базовый слой (блоки 1x1) – э
                    // одного элемента равен ему сам
                    // показываем из A сразу, как A
                    // цикла построения st2. Уровни
                    const sourceValue = step === 0
                    const shownValue = compilerLink
                    const isBlank = shownValue ===
                    let highlightClass = isBlank
                      ? 'bg-slate-950 text-slate-600

```

```

        }
      }

      return (
        <div
          key={idx}
          onMouseEnter={() => { if(is
          onMouseLeave={() => { if(is
          onClick={() => {
            if(isCurr) {
              if(locked && locked.
              else setLocked({r, c
            }
          }}
          className={`flex items-cent
sCurr ? 'w-[clamp(24px,6.5vw,38px)] h-[clamp
5.2vw,30px)] h-[clamp(20px,5.2vw,30px)] min
          title={hasLiveTable ? `st2[
        >
        {isBlank ? "." : String(sho
      </div>
    );
  }}}
  </div>
</div>
</div>
</React.Fragment>
)
  }}}
</div>
</div>

<div className="flex-1 min-w-0 flex flex-col gap-
  <div className="bg-slate-900 p-4 sm:p-6 rounded
    <h3 className="text-lg font-bold text-slate-3
      {shownK === 0 ? {
        <div className="text-slate-400 text-sm lea
          {liveTableEmpty ? {

```



```

    </div>
  );
};

const Viz2DQuery: React.FC = () => {
  const [r1, setR1] = useState(1);
  const [c1, setC1] = useState(1);
  const [r2, setR2] = useState(5);
  const [c2, setC2] = useState(6);
  const runtime = useVizRuntime();

  // Координаты запроса – те же захардкоженные имена,
  useEffect(() => {
    const vars = runtime?.variables;
    const nr1 = vizNumber(vars?.r1);
    const nc1 = vizNumber(vars?.c1);
    const nr2 = vizNumber(vars?.r2);
    const nc2 = vizNumber(vars?.c2);
    if (nr1 !== null) setR1(Math.max(0, Math.min(7, nr1)));
    if (nc1 !== null) setC1(Math.max(0, Math.min(7, nc1)));
    if (nr2 !== null) setR2(Math.max(0, Math.min(7, nr2)));
    if (nc2 !== null) setC2(Math.max(0, Math.min(7, nc2)));
  }, [runtime]);

  const queryVars = runtime?.variables;
  const liveCoords = ["r1", "c1", "r2", "c2"].map((name, i) => {
    const queryMismatch = liveCoords.some((v) => v !== queryVars?.[name]);
    return queryMismatch
      ? `Координаты запроса в вашем коде выходят за дем
        ] ?? "–"}, r2=${liveCoords[2] ?? "–"}, c2=${liveCoords[3] ?? "–"}
      : null;
  });

  const H = Math.max(1, r2 - r1 + 1);
  const W = Math.max(1, c2 - c1 + 1);
  const kx = Math.floor(Math.log2(H));
  const ky = Math.floor(Math.log2(W));
  const Lx = 1 << kx;
  const Ly = 1 << ky;

```



```

        marginTop: '8px',
        marginLeft: '8px',
        height: 'calc(' + (H/8*100) +
        width: 'calc(' + (W/8*100) +
    }}
  />

  {/* Показываем 4 угла в самой матрице */}
  <div className="absolute pointer-events-none shadow-[0_0_10px_#f43f5e]" style={{ top: `calc(${(Ly / 8) * 100}% - 16px)`, height: `calc(${(Ly / 8) * 100}% - 16px)`, width: `calc(${(Ly / 8) * 100}% - 16px)`, left: `calc(${(Lx / 8) * 100}% - 16px)`}}>
    <div className="absolute pointer-events-none shadow-[0_0_10px_#3b82f6]" style={{ top: `calc(${(Ly / 8) * 100}% - 16px)`, height: `calc(${(Ly / 8) * 100}% - 16px)`, width: `calc(${(Ly / 8) * 100}% - 16px)`, left: `calc(${(Lx / 8) * 100}% - 16px)`}}>
      <div className="absolute pointer-events-none shadow-[0_0_10px_#10b981]" style={{ top: `calc(${(Ly / 8) * 100}% - 16px)`, height: `calc(${(Ly / 8) * 100}% - 16px)`, width: `calc(${(Ly / 8) * 100}% - 16px)`, left: `calc(${(Lx / 8) * 100}% - 16px)`}}>
        <div className="absolute pointer-events-none shadow-[0_0_10px_#f59e0b]" style={{ top: `calc(${(Ly / 8) * 100}% - 16px)`, height: `calc(${(Ly / 8) * 100}% - 16px)`, width: `calc(${(Ly / 8) * 100}% - 16px)`, left: `calc(${(Lx / 8) * 100}% - 16px)`}}>
          </div>
        </div>
      </div>
    </div>
  </div>

  <div className="bg-slate-950 p-4 rounded-2xl shadow-[inset_0_2px_10px_rgba(0,0,0,0.5)]" style={{ width: '100%', height: '100%' }}>
    <label className="flex items-center justify-between" style={{ padding: '10px 0' }}>
      <span className="text-slate-400 font-medium">Настройка</span>
      <div className="flex gap-2">
        <input type="number" min="0" max="100" value="100" style={{ width: 50px; border: 1px solid #333; border-radius: 4px; padding: 2px 10px; background-color: #f0f0f0; }} />
        <input type="number" min="0" max="100" value="100" style={{ width: 50px; border: 1px solid #333; border-radius: 4px; padding: 2px 10px; background-color: #f0f0f0; }} />
      </div>
    </label>
    <label className="flex items-center justify-between" style={{ padding: '10px 0' }}>
      <span className="text-slate-400 font-medium">Настройка</span>
      <div className="flex gap-2">
        <input type="number" min="0" max="100" value="100" style={{ width: 50px; border: 1px solid #333; border-radius: 4px; padding: 2px 10px; background-color: #f0f0f0; }} />
        <input type="number" min="0" max="100" value="100" style={{ width: 50px; border: 1px solid #333; border-radius: 4px; padding: 2px 10px; background-color: #f0f0f0; }} />
      </div>
    </label>
  </div>

```

```

      &nbsp;&nbsp;&nbsp;<span className="text-rose-800">r1</span>]<span className="text-white bg-rose-800" style="border: 1px solid black; padding: 2px 5px; display: inline-block; min-width: 100px; text-align: center;">

```

```

      &nbsp;&nbsp;&nbsp;<span className="text-slate-800">во, чтобы правый край уперся в c2)</span><br/>
      &nbsp;&nbsp;&nbsp;<span className="text-blue-800">r1</span>]<span className="text-white bg-blue-800" style="border: 1px solid black; padding: 2px 5px; display: inline-block; min-width: 100px; text-align: center;">

```

```

      &nbsp;&nbsp;&nbsp;<span className="text-slate-800">рх, чтобы нижний край уперся в r2)</span><br/>
      &nbsp;&nbsp;&nbsp;<span className="text-emerald-800" style="border: 1px solid black; padding: 2px 5px; display: inline-block; min-width: 100px; text-align: center; border-radius: 5px;">r2 - Lx + 1</span>]<span className="text-white bg-emerald-800" style="border: 1px solid black; padding: 2px 5px; display: inline-block; min-width: 100px; text-align: center;">

```

```

      &nbsp;&nbsp;&nbsp;<span className="text-slate-800">и влево от c2)</span><br/>
      &nbsp;&nbsp;&nbsp;<span className="text-amber-800" style="border: 1px solid black; padding: 2px 5px; display: inline-block; min-width: 100px; text-align: center;">r2 - Lx + 1</span>]<span className="text-white bg-amber-800" style="border: 1px solid black; padding: 2px 5px; display: inline-block; min-width: 100px; text-align: center;">
        {'}}''));
    </div>

```

```

    <div className="w-full flex justify-center items-center gap-10">
      <div className="flex flex-col items-center gap-10">
        <h4 className="text-slate-400 font-bold">
          Откуда берутся эти 4 числа: матрица ST
        </h4>

```

```

        <div className="grid gap-[2px] p-2.5 border-slate-700 shadow-sm">ST[][][kx]][ky]]</div>
        Columns: `repeat(${matCols}, 1fr)`}}>
          {Array.from({length: matRows * matCols}, (v, idx) => {
            const r = Math.floor(idx / matCols);
            const c = idx % matCols;
            let isTL = r === r1 && c === c1;
            let isTR = r === r1 && c === c2;
            let isBL = r === r2 - Lx + 1 && c === c1;
            let isBR = r === r2 - Lx + 1 && c === c2;

            let color = "bg-slate-800 text-white";
            let txt = ST2D[r][c][kx][ky].toLocaleString();

```

```
        </div>
      </div>

      </div>
    </div>
  )
}
```

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

ФАЙЛ 87/113: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 137 | РАЗМЕР

```
        </div>
      </main>
    </div>

    <script>
      function setMode(mode) {
        document.getElementById('btn-mode-normal').cla
          ? 'px-3 py-1 rounded text-sm font-bold l
          : 'px-3 py-1 rounded text-sm font-bold

        document.getElementById('btn-mode-wtf').cla
          ? 'px-3 py-1 rounded text-sm font-bold l
          : 'px-3 py-1 rounded text-sm font-bold

        const aside = document.querySelector('aside
        const headerMain = document.querySelector('
        const questionsContainer = document.getElem
        const wtfContainer = document.getElementById
```

```

        ext-white transition-colors' : 'px-3 py-1 r
        ion-colors';
        document.getElementById('btn-theme-notebook
        -600 text-white transition-colors' : 'px-3 p
        ransition-colors';
    }

    // Initialize theme and mode
    setTheme('dark');

    // Setup intersection observer for TOC highlight
    document.addEventListener('DOMContentLoaded', () {
        const mobileMenuBtn = document.getElementById('mobile-menu-btn');
        const mobileDrawer = document.getElementById('mobile-drawer');
        const closeDrawerBtn = document.getElementById('close-drawer');
        const drawerOverlay = document.getElementById('drawer-overlay');

        function toggleDrawer() {
            const isClosed = mobileDrawer.classList.contains('closed');
            if (isClosed) {
                mobileDrawer.classList.remove('transl
                if (drawerOverlay) {
                    drawerOverlay.classList.remove('op
                    // Timeout allows display:block
                    setTimeout(() => drawerOverlay.style.display = 'block', 10);
                }
                document.body.style.overflow = "hidden";
            } else {
                mobileDrawer.classList.add("transl
                if (drawerOverlay) {
                    drawerOverlay.classList.add("op
                    setTimeout(() => drawerOverlay.style.display = 'block', 10);
                }
                document.body.style.overflow = "";
            }
        }

        if (mobileMenuBtn && mobileDrawer) {

```

```
        }, observerOptions);

        document.querySelectorAll('section[id^="que
            observer.observe(section);
        });
    });
</script>
</body>
</html>
```

ФАЙЛ 88/113: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР: 10000

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  <title>Пособие по алгебре: От пиццы к предикатам</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://polyfill.io/v3/polyfill.min.js"></script>
  <script id="MathJax-script" async src="https://cdn.
  <script>
    window.MathJax = {
      tex: {
        inlineMath: [['$', '$'], ['\\(', '\\)']],
        displayMath: [['$$', '$$'], ['\\[', '\\]']],
      },
    };
  </script>
  <style>
    @import url('https://fonts.googleapis.com/css2?
    @reference "../src/index.css";
    .math-font { font-family: 'Fira Code', monospace;
    .uno-card {
```

```

    }
    mjsx-container[display="true"]::-webkit-scrollbar {
      display: none;
    }

    @media (max-width: 640px) {
      html { font-size: 13px; }

      .uno-card { width: 40px; height: 60px; font-size: 12px; }
      .uno-card::before, .uno-card::after { font-size: 12px; }
      .uno-card::before { top: 1px; left: 2px; }
      .uno-card::after { bottom: 1px; right: 2px; }

      .uno-card.mini { width: 28px !important; height: 40px; }
      .uno-card.mini::before, .uno-card.mini::after { font-size: 10px; }

      .uno-equation .text-2xl { font-size: 1.2rem; }
      .uno-equation.gap-4 { gap: 0.5rem !important; }

      /* Allow horizontal scroll for equations in mobile view
      .formula-scroll { flex-wrap: nowrap !important; }
      .formula-scroll { width: 100%; }
    }

    .uno-inner {
      position: absolute; width: 110%; height: 75%;
      border-radius: 50%; transform: rotate(-30deg);
      box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
    }

    .uno-val { position: relative; z-index: 2; }
    .card-red { background-color: #e52521; } .card-blue { background-color: #004dcf; } .card-green { background-color: #339e35; } .card-yellow { background-color: #ffc400; } .card-black { background-color: #222; }
    .card-black::before, .card-black::after { text-align: center; }
    .info-block { @apply bg-slate-800/80 border-l-4 border-r-4 border-t-4 border-b-4; }
    .info-title { @apply font-bold mb-2 flex items-center; }

```

```
body.theme-notebook .text-slate-200, body.theme-
  ant; }
body.theme-notebook .text-slate-400 { color: #5
body.theme-notebook .border-slate-600, body.the
  : #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "
body.theme-notebook #top-controls { background-

/* CREEPY BLINKING */
@keyframes creepy-blink {
  0%, 95%, 98%, 100% { transform: scaleY(1);
  96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
  transform-origin: center;
  transform-box: fill-box;
  animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
  transform-origin: center;
  transform-box: fill-box;
  animation: creepy-blink 5.2s infinite;
  animation-delay: 1.5s;
}
.creepy-emoji {
  display: inline-block;
  transform-origin: center;
  animation: creepy-blink 3.8s infinite;
}
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead
  <div class="sticky top-0 z-50 w-full bg-slate-950/9
    center items-center gap-4 sm:gap-8 transition-col
      <div class="flex items-center gap-3">
        <span class="text-sm font-bold uppercase tr
```

```

<marker id="arrow-lime" markerWidth="6" mar
    <path d="M0,0 L6,3 L0,6 Z" fill="#a3e633"
</marker>
<marker id="arrow-green" markerWidth="6" ma
    <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80"
</marker>

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
    <rect x="0" y="0" width="40" height="20"
    <rect x="6" y="-5" width="8" height="6"
    <rect x="26" y="-5" width="8" height="6"
    <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-red">
    <rect x="0" y="0" width="40" height="20"
    <rect x="6" y="-5" width="8" height="6"
    <rect x="26" y="-5" width="8" height="6"
    <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-rem">
    <rect x="0" y="0" width="40" height="10"
    <rect x="6" y="-5" width="8" height="6"
    <rect x="26" y="-5" width="8" height="6"
    <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<!-- МЯСОРУБКА -->
<g id="meat-grinder">
    <path d="M 10 40 L 15 20 Q 25 15 35 20
    <rect x="5" y="40" width="40" height="20"
    <!-- Ручка -->
    <path d="M 45 50 Q 60 50 60 30 L 75 30"
    <circle cx="75" cy="30" r="4" fill="#333"
    <!-- Выход -->
    <path d="M 5 45 L -5 45 L -5 55 L 5 55
    <circle cx="25" cy="50" r="15" fill="#fff"
</g>
<g id="lego-green">

```



```
        <svg xmlns="http://www.w3.org/2000/svg">
          <path stroke-linecap="round" stroke="black" stroke-width="2" d="M10 9 49 49" />
        </svg>
      </button>
</div>
<nav class="space-y-3 text-sm font-medium">
  <details class="group" open>
    <summary class="list-none cursor-pointer">
      <a href="#question-1" class="text-indigo-500 pl-3 font-bold text-indigo-300">
        1. Алгебраические структуры
      </a>
    </summary>
    <div class="pl-6 space-y-2 text-sm">
      <a href="#section-1-1" class="text-indigo-500">1.1. Группы</a>
      <a href="#section-1-2" class="text-indigo-500">1.2. Кольца</a>
      <a href="#section-1-3" class="text-indigo-500">1.3. Поля</a>
      <a href="#section-1-4" class="text-indigo-500">1.4. Модули</a>
    </div>
  </details>

  <details class="group">
    <summary class="list-none cursor-pointer">
      <a href="#question-2" class="text-indigo-500 pl-3 font-bold text-indigo-300">
        2. Комплексные числа
      </a>
    </summary>
    <div class="pl-6 space-y-2 text-sm">
      <a href="#section-2-1" class="text-indigo-500">2.1. Комплексные числа</a>
      <a href="#section-2-2" class="text-indigo-500">2.2. Комплексные функции</a>
      <a href="#section-2-3" class="text-indigo-500">2.3. Комплексные интегралы</a>
    </div>
  </details>

  <details class="group">
    <summary class="list-none cursor-pointer">
      <a href="#question-3" class="text-indigo-500 pl-3 font-bold text-indigo-300">
        3. Дифференциальное исчисление
      </a>
    </summary>
    <div class="pl-6 space-y-2 text-sm">
      <a href="#section-3-1" class="text-indigo-500">3.1. Дифференциальное исчисление</a>
      <a href="#section-3-2" class="text-indigo-500">3.2. Дифференциальные уравнения</a>
      <a href="#section-3-3" class="text-indigo-500">3.3. Дифференциальная геометрия</a>
    </div>
  </details>

```

```

        <a href="#q5-prim" class="block
        <a href="#q5-crit" class="block
        <a href="#q5-cyclic" class="blo
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-6" class=
er-lime-500 pl-3 font-bold text-lime-300">
        6. Делимость, НОД и Евклид
    </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q6-1" class="block ho
        <a href="#q6-2" class="block ho
        <a href="#q6-3" class="block ho
        <a href="#q6-4" class="block ho
        <a href="#q6-5" class="block ho
        <a href="#q6-6" class="block ho
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-7" class=
er-pink-500 pl-3 font-bold text-pink-300">
        7. Взаимно простые числ
    </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q7-1" class="bloc
        <a href="#q7-2" class="bloc
        <a href="#q7-3" class="bloc
        <a href="#q7-4" class="bloc
        <a href="#q7-5" class="bloc
    </div>
```

```
der-rose-500 pl-3 font-bold text-rose-400">
```

```
10. Делимость и Схема Г
```

```
</a>
```

```
</summary>
```

```
<div class="pl-6 space-y-2 text
```

```
<a href="#q10-1" class="blo
```

```
<a href="#q10-2" class="blo
```

```
<a href="#q10-3" class="blo
```

```
a>
```

```
<a href="#q10-4" class="blo
```

```
<a href="#q10-5" class="blo
```

```
</div>
```

```
</details>
```

```
<details class="group">
```

```
<summary class="list-none curso
```

```
<a href="#question-11" clas
```

```
der-cyan-500 pl-3 font-bold text-cyan-400">
```

```
11. НОД и НОК многочлен
```

```
</a>
```

```
</summary>
```

```
<div class="pl-6 space-y-2 text
```

```
<a href="#q11-1" class="blo
```

```
<a href="#q11-2" class="blo
```

```
нственности НОД</a>
```

```
<a href="#q11-3" class="blo
```

```
ПИРАЛЬ)</a>
```

```
<a href="#q11-4" class="blo
```

```
вращается!)</a>
```

```
</div>
```

```
</details>
```

```
<details class="group">
```

```
<summary class="list-none curso
```

```
<a href="#question-12" clas
```

```
der-blue-500 pl-3 font-bold text-blue-400">
```

```
12. Взаимно простые мно
```

```
</a>
```

```
</details>

<details class="group">
  <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
    15. Производная. Критерии
  </a>
</summary>
<div class="pl-6 mt-2 space-y-2">
  <a href="#q15-1" class="block">
  <a href="#q15-2" class="block">
  <a href="#q15-3" class="block">
  <a href="#q15-4" class="block">
</div>
</details>

<details class="group">
  <summary class="list-none cursor-pointer text-red-500 pl-3 font-bold text-red-400">
    16. Неприводимые многочлены
  </a>
</summary>
<div class="pl-6 mt-2 space-y-2">
  <a href="#q16-1" class="block">
  <a href="#q16-2" class="block">
  <a href="#q16-3" class="block">
</div>
</details>

<div class="mt-4 mb-2 text-xs font-sans">
  <a href="#question-17" class="block text-slate-500 pl-3 text-slate-400">
    <span class="font-bold text-teal-500">
  </a>
  <a href="#question-18" class="block text-slate-500 pl-3 text-slate-400">
```

```

        // Scroll spy logic
        document.addEventListener('DOMContentLoaded', () => {
            const sections = document.querySelectorAll(
                'h2[id^="q5-"], div[id^="q6-"], div[id^="q7-"], div[id^="q8-"], div[id^="q9-"], div[id^="q10-"], div[id^="q11-"], div[id^="q12-"], div[id^="q13-"], div[id^="q14-"], div[id^="q15-"]'
            );
            let isScrolling = false;

            document.querySelector('.group-links').addEventListener('click', (e) => {
                isScrolling = true;
                setTimeout(() => isScrolling = false, 200);
            });

            const observer = new IntersectionObserver((entries) => {
                if (isScrolling) return;

                let visibleId = null;
                entries.forEach(entry => {
                    if (entry.isIntersecting) {
                        visibleId = entry.target.id;
                    }
                });

                // Highlight active link
                document.querySelectorAll('a').forEach(a => {
                    a.style.fontWeight = 'normal';
                    if (a.href.endsWith(visibleId)) {
                        a.style.fontWeight = 'bold';
                    }
                });

                // Open the parent details
                const correspondingLink = document.querySelector(`a[href="#${visibleId}"]`);
                if (correspondingLink) {
                    const details = correspondingLink.closest('details');
                    if (details && !details.open) {
                        details.open();
                        document.querySelector(`div[id="${visibleId}"]`);
                    }
                }
            });
        });
    
```

```
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('div');
header.innerText = 'Экспорт в ' + format;
header.style.marginBottom = '10px';

const subHeader = document.createElement('div');
subHeader.innerText = 'Из-за особенностей веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#fff';
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
    if (navigator.clipboard) {
        navigator.clipboard.writeText(contentRaw);
    }
}
```

```

clone.querySelectorAll('h2').fo
clone.querySelectorAll('h3').fo
clone.querySelectorAll('.analog
clone.querySelectorAll('li').fo
clone.querySelectorAll('svg').fo
    let caption = '';
    if (el.nextElementSibling &
        caption = el.nextElement
    }
    el.textContent = '\n\n[ Илл
});
clone.querySelectorAll('.img-ca

let text = clone.innerText;
text = text.replace(/\n{3,}/g,

if (window !== window.top) {
    fallbackModal(text, 'Markdo
    return;
}
const blob = new Blob([text], {
const url = URL.createObjectURL
const link = document.createEle
link.href = url;
link.download = 'vyisshaya_algel
document.body.appendChild(link)
link.click();
document.body.removeChild(link)
}

```

```
function setTheme(theme) {
```

```

    const body = document.body;
    body.classList.remove('bg-slate

```

```

    );

```

```

    document.getElementById('theme-
    document.getElementById('theme-

```

```

        /* Tweak card blocks */
        body.bg-white .card-red {
            background-color: #f08080;
        }

        body.bg-white aside {
            background-color: #334155 !important;
        }

        body.bg-white .text-white {
            color: white;
        }

        /* Ensure active links are visible */
        body.bg-white aside a.active {
            background-color: #334155;
            color: white;
        }
        body.bg-white aside a.active {
            background-color: #334155;
            color: white;
        }
        body.bg-white aside a.active {
            background-color: #334155;
            color: white;
        }
        body.bg-white aside a.active {
            background-color: #334155;
            color: white;
        }
        body.bg-white aside a.active {
            background-color: #334155;
            color: white;
        }
        body.bg-white aside a:active {
            background-color: #334155;
            color: white;
        }

        body.bg-white table tr {
            background-color: #f0f0f0;
        }
        body.bg-white table tr {
            background-color: #f0f0f0;
        }
        body.bg-white .text-teal {
            color: #20b2aa;
        }
        body.bg-white .text-rose {
            color: #e91e63;
        }
        body.bg-white .text-ember {
            color: #f44336;
        }
        body.bg-white .text-cyan {
            color: #00bcd4;
        }
        body.bg-white .text-amber {
            color: #ffc107;
        }
        body.bg-white .text-fuchsia {
            color: #e91e63;
        }
        body.bg-white .text-lime {
            color: #c6efce;
        }
        body.bg-white .text-blue {
            color: #2196f3;
        }
        body.bg-white .text-green {
            color: #4caf50;
        }
        body.bg-white .text-indigo {
            color: #3f51b5;
        }

        `;
    } else if (theme === 'terminal') {
        body.classList.add('bg-black');
        document.getElementById('theme').innerHTML = `
            body.bg-black .bg-slate-950 {
                background-color: #1a202c;
            }
        `;
    }
}

```



```

        t-sm font-bold flex items-center justify-center
            <span> </span> HTML
        </button>
    </div>

    <!-- Смена темы -->
    <div>
        <div class="text-xs text-slate-500">
            <div class="flex items-center gap-2">
                <button id="theme-dark" onclick="toggleTheme('dark')"
                    justify-center text-amber-300 hover:text-white
                    line-amber-500" title="Темная тема"> </button>
                <button id="theme-light" onclick="toggleTheme('light')"
                    justify-center text-amber-50 hover:text-white
                    ">* </button>
                <button id="theme-terminal" onclick="toggleTheme('terminal')"
                    justify-center justify-center text-green-400 hover:text-green-500
                    терминала"> </button>
            </div>
        </div>
    </div>
</aside>

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16"
    -base min-w-0">
    <header class="text-center mb-12 border-b border-slate-700">
        <h1 class="text-4xl md:text-5xl font-bl
            4">
            ВЫСШАЯ АЛГЕБРА
        </h1>
        <p class="text-slate-400 italic text-lg">
            <!-- Print Table of Contents -->
            <div class="hidden print:block mt-8 text-center">
                <p>Печать содержания
            </div>
            </div>
    </main>

```

```

<div class="bg-cyan-900/40 border-4 border-yellow-500 hover:bg-slate-700 transition" style="padding: 10px;">
    сные числа</div>

```

```

<!-- Row 1: База -->
<a href="#question-1" onclick="setMainQuestion(1)" class="block w-full text-center text-yellow-500 border-4 border-yellow-500 hover:bg-slate-700 transition" style="padding: 10px;">
    <div class="text-xs text-yellow-500" style="margin-bottom: 5px;">Вопрос 1
```

```

-4 border-cyan-500 hover:bg-slate-700 trans
    <div class="text-xs text-cyan-500">
    <div class="font-bold text-slate-700">
</a>

<!-- Row 6: Кратность -->
<div class="hidden md:block"></div>
<a href="#question-14" onclick="setQuestion(14)">
-l-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange-500">
    <div class="font-bold text-slate-700">
</a>
<div class="hidden md:block"></div>

<!-- Row 7: Производная -->
<div class="hidden md:block"></div>
<a href="#question-15" onclick="setQuestion(15)">
-l-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange-500">
    <div class="font-bold text-slate-700">
</a>
<div class="hidden md:block"></div>

<!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
<div class="col-span-1 md:col-span-3">
b-2 border-b border-emerald-500/30 pb-2">Кл

    <a href="#question-17" onclick="setQuestion(17)">
-l-4 border-teal-500 hover:bg-slate-700 tra
    <div class="text-xs text-teal-500">
    <div class="font-bold text-slate-700">
</a>
    <a href="#question-18" onclick="setQuestion(18)">
-l-4 border-blue-500 hover:bg-slate-700 tra
    <div class="text-xs text-blue-500">
    <div class="font-bold text-slate-700">
</a>
    <a href="#question-19" onclick="setQuestion(19)">

```

```
rg/2000/svg">
```

```
      <path stroke-linecap="round" stroke-width="4" data-bbox="110 400 490 600" style="stroke:#f00;stroke-width:4px;"/>
    </svg>
```

```
  </div>
```

```
  <h2 class="text-3xl pr-4 border" style="border: 1px solid #ccc; padding: 5px; margin-top: 10px;">Пример
```

```
</div>
```

```
<div class="prose prose-invert max-w-4xl">
```

```
  <p class="text-lg">
```

```
    Хватит тупить. Доказательств нет. Просто жать на кнопки. Вот жесть.
  </p>
```

```
  <h2 class="text-2xl font-bold border" style="border: 1px solid #ccc; padding: 5px; margin-top: 10px;">Меметика
```

```
</h2>
```

```
<!-- Прямое -->
```

```
<div class="bg-slate-800/80 p-6">
```

```
  <h3 class="text-xl text-blue-400">Прямое
```

```
  <p class="text-sm text-slate-400">Прямое
```

```
  <ul class="list-disc pl-5">
```

```
    <li><span class="text-white">Прямое
    </li>
```

```
    <li><span class="text-white">Прямое
```

```
    <li><span class="text-white">Прямое
    </li>
```

```
  </ul>
```

```
</div>
```

```
<!-- От противного -->
```

```
<div class="bg-slate-800/80 p-6">
```

```
  <h3 class="text-xl text-fuchsia">От противного
```

```
  <p class="text-sm text-slate-400">От противного
```

```
  <ul class="list-disc pl-5">
```

```
    <li><span class="text-white">От противного
    </li>
```

```
    <li><span class="text-white">От противного
```

```
        <li><span class="text-w
вверх по уравнениям).</li>
        <li><span class="text-w
рху вниз, показывая, что  $\Delta \vdots r_i$ 
        <li><span class="text-w
х. Твой  $d$  больше.</li>
    </ul>
</div>
```

```
    <h2 class="text-2xl font-bold b
оритмы экзекуции)</h2>
```

```
    <div class="grid grid-cols-1 md
    <div class="bg-slate-800 p-
        <h4 class="text-red-400
        <p class="text-sm text-
        <p class="text-xs font-
со следующим. Если в конце 0 – пациент мертв
    </div>
```

```
    <div class="bg-slate-800 p-
        <h4 class="text-red-400
        <p class="text-sm text-
        <p class="text-xs font-
 $c$ . Пока получается 0 – корень жив. Как то
корня минус один.</p>
    </div>
```

```
    <div class="bg-slate-800 p-
        <h4 class="text-red-400
        <p class="text-sm text-
        <p class="text-xs font-
ициентов (они делятся на  $p$ ), кроме старше
зложить.</p>
    </div>
```

```
    <div class="bg-slate-800 p-
```

```

        <p class="text-sm text-gray-500">
            есть сложение — есть ноль. Если есть умножение, то
            <code class="text-x"
        </div>
    </li>

    <li class="bg-slate-800/50 p-4">
        <span class="text-2xl">
        <div>
            <h4 class="text-cyan">
            <p class="text-sm text-gray-500">
                авь их драться. Умножение всегда врывается и
                <code class="text-x"
            </div>
        </li>
    </ul>
</div>
</section>
</div>

<div id="questions-container">
<!-- ===== ВОПРОС 1 =====>
```

РАЗДЕЛ: УТИЛИТЫ

ФАЙЛ 89/113: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
```

```
export function cn(...inputs: ClassValue[]) {
```

```

    } catch {
      /* чужой домен – пропускаем */
    }
  }
}
return parts.join("\n");
}

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
  border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem;
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid #
body { font-size: 15px; line-height: 1.6; }
.export-shell p, .export-shell li { line-height: 1.65;
.export-shell pre { font-size: 12px; line-height: 1.5;
.export-figs { margin: 0 0 1.6rem; }
.export-figs-title { color: #e2e8f0; font-size: .95rem;
.export-fig { margin: 0 0 .9rem; }
.export-fig img { width: 100%; border: 1px solid #1e293b;
.export-fig figcaption { color: #94a3b8; font-size: .75rem;
@media print {
  body { background: #fff; color: #0f172a; }
  .export-note, details { break-inside: avoid; }
  details { display: block; }

```

```

        /* ссылки – улучшение, а не обязательная часть файла
    }
}

const ids: string[] = [];
host.querySelectorAll<HTMLElement>(".term-hint").forEach((el) => {
    const id = el.dataset.termId;
    if (!id) return;
    if (!ids.includes(id)) ids.push(id);
    const link = document.createElement("a");
    link.className = "term-hint";
    link.href = `#term-${id}`;
    link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
    link.textContent = el.textContent ?? "";
    el.replaceWith(link);
});

// <details> в файле лучше раскрыть: иначе код и разбросан
host.querySelectorAll("details").forEach((d) => d.setOpen());

return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[], book = false) {
    if (!termIds.length) return "";
    const items = termIds
        .map((id) => {
            const e = GLOSSARY_BY_ID.get(id);
            if (!e) return "";
            return `<div class="export-term" id="term-${escapeHtml(id)}">
                <h3>${escapeHtml(e.title)}</h3>
                <p>${escapeHtml(e.short)}</p>
                ${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>` : ""}
                ${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
            </div>`;
        })
        .join("\n");
}

```



```

    <footer class="export-foot">Универсальное пособие по
</div>
</body>
</html>`;
}

```

```
/**
```

```

 * Картинки встраиваются в файл data-URL: автономная ко
 * иллюстрации офлайн (раньше src вел на ассет приложен
 * картинки исчезали – оставались только самые первые,
 */

```

```

async function inlineImages(html: string): Promise<string> {
  const doc = new DOMParser().parseFromString(`<body>${html}</body>`);
  const imgs = Array.from(doc.querySelectorAll("img[src]"));
  await Promise.all(
    imgs.map(async (img) => {
      const src = img.getAttribute("src") ?? "";
      if (!src || src.startsWith("data:")) return;
      try {
        const res = await fetch(src);
        if (!res.ok) return;
        const blob = await res.blob();
        if (blob.size > 4_000_000) return;
        const data = await new Promise<string>((resolve, reject) => {
          const fr = new FileReader();
          fr.onload = () => resolve(String(fr.result));
          fr.onerror = () => reject(fr.error);
          fr.readAsDataURL(blob);
        });
        img.setAttribute("src", data);
      } catch {
        /* картинка недоступна – глава останется со ссылкой */
      }
    })
  );
  return doc.body.innerHTML;
}

```

```

}

function triggerDownload(html: string, filename: string) {
  const blob = new Blob([html], { type: "text/html;charset=utf-8" });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = filename;
  document.body.appendChild(a);
  a.click();
  a.remove();
  setTimeout(() => URL.revokeObjectURL(url), 2000);
}

/** Имя файла: «04-splay-derevo.html» – без кириллицы и пробелов */
function safeName(title: string, id: string): string {
  const num = title.match(/^s*(\d+)/)?.[1];
  const base = id.replace(/[^a-z0-9-]+/gi, "-").toLowerCase();
  return `${num ? String(num).padStart(2, "0") + "-" : ""}${base}.html`;
}

/** Собрать автономный HTML одной главы. */
export async function buildChapterHtml(chapter: Chapter) {
  const css = await collectCss();
  const { body: rawBody, termIds } = prepareBody(chapter);
  const body = (await vizFigures(chapter.id)) + (await vizFigures(chapter.id));
  return wrap({
    title: chapter.title,
    subtitle: chapter.category || "Универсальное пособие",
    css,
    body,
    gloss: glossarySection(termIds),
    origin: window.location.origin,
  });
}

/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter) {
  const html = await buildChapterHtml(chapter);
  triggerDownload(html, safeName(chapter.title, chapter.id));
}

```

```
        origin: window.location.origin,
    });
    return html;
}

/** Выгрузить всё пособие одним файлом. */
export async function downloadBookHtml(chapters: Chapter[] | null) {
    triggerDownload(await buildBookHtml(chapters), "posobie.html");
}
```

ФАЙЛ 91/113: src/utils/ticketLinks.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 165 | РАЗМЕР: 7.4 KB

```
import { useEffect } from "react";
import { chapterTopics } from "../data/content";

/**
 * Перекрёстные ссылки между билетами.
 *
 * В тексте страниц десятки упоминаний вида «...берите Формулу
 * или «сердце Краскала (билет 18)». Раньше это был просто номер,
 * но попасть на него не мог — ссылка была «битой» и вела в никуда.
 * Здесь такие упоминания превращаются в настоящие ссылки.
 *
 * • в приложении — ``, который
 *   `ChapterView` и переключает страницу без перезагрузки;
 * • в текстовой версии — ``,
 \*   работающий URL, который понимают парсеры и ридеры;
 \* • в PDF — остаётся текст \(ссылки там всё равно не работают\).
 \*
 \* Номер билета → страница берётся из `chapterTopics`, например,
 \* «билет 19» корректно ведёт на общую страницу остовных билетов
 \* \(она закрывает 18, 19 и 20 сразу\).
 \*/
```

```

-----
/**
 * Строковая версия – для текстовой (`?lite=1`) версии,
 * через `dangerouslySetInnerHTML`, а эффекты разметки
 * HTML режется на «тег» и «текст», меняются только тек
 * вне `pre/code/a`.
 */
export function linkifyTicketsHtml(html: string, opts:
  let inCode = 0;
  let inLink = 0;
  return html
    .split(/(<[>]*)/g)
    .map((part) => {
      if (!part) return part;
      if (part.startsWith("<")) {
        const closing = part.startsWith("</");
        const tag = part.slice(closing ? 2 : 1).split(/
        if (tag === "pre" || tag === "code" || tag ===
          inCode = Math.max(0, inCode + (closing ? -1 :
        } else if (tag === "a") {
          inLink = Math.max(0, inLink + (closing ? -1 :
        }
        return part;
      }
      if (inCode > 0 || inLink > 0) return part;
      return part.replace(TICKET_RE, (whole, a: string,
        const from = Number(a);
        const to = b ? Number(b) : from;
        const id = chapterForRange(from, to);
        if (!id || id === opts.selfId) return whole;
        return `<a class="ticket-link" data-goto="${id}
      });
    });
    .join("");
}

```

```

/**
 * DOM-версия – для интерактивной страницы. Проходит то
 * (как `annotateTerms`), поэтому атрибуты и разметка н

```

```

        const a = document.createElement("a");
        a.className = "ticket-link";
        a.dataset.goto = id;
        a.setAttribute("href", hrefFor(id, opts.lite ?? false));
        a.textContent = m[0];
        frag.appendChild(a);
        last = m.index + m[0].length;
        created += 1;
    }
    if (last === 0) continue;
    if (last < text.length) frag.appendChild(document.createTextNode(
        textNode.parentNode?.replaceChild(frag, textNode);
    ));
    return created;
}

/** Размечает ссылки при смене главы и подстраховывается
export function useTicketLinks(
  ref: React.RefObject<HTMLElement | null>,
  deps: unknown[],
  opts: { selfId?: string; lite?: boolean } = {}
) {
  useEffect(() => {
    const el = ref.current;
    if (el) annotateTicketLinks(el, opts);
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, deps);
}

```

РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

```

    return {
      id: c.id,
      title: c.title,
      num: Number.parseInt(c.title.match(/^(\d+)\./)?.[1]),
      analogies,
      hasAnalogy: analogies > 0,
      inlineSvg: /<svg[\s>]/i.test(html),
      image: /<img[\s>]/i.test(html),
      interactive: vizIds.has(c.id),
      chars: html.length,
    };
  });
});

const has2D = (r) => r.interactive || r.inlineSvg || r.image;
const gapBoth = rows.filter((r) => !r.hasAnalogy && !has2D(r));
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && has2D(r));
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D(r));
const orphanViz = [...vizIds].filter((id) => !chapters[id]);

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) => i + 1).filter(n => !nums.includes(n));

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
  console.log("| # | Глава | Аналогия | 2D-виз. | SVG |");
  console.log("| --- | --- | --- | --- | --- |");
  for (const r of rows) {
    console.log(
      `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy ? "да" : "нет"} |`
    );
  }
} else {
  console.log(`Глав в пособии: ${rows.length}\n`);
  const dump = (title, list) => {
    console.log(`${title} (${list.length}):`);
    list.forEach((r) => console.log(`  * ${r.title} [${r.num || "-"}]`));
  };
  dump("Главы с аналогиями", gapBoth);
  dump("Главы без аналогий", gapAnalogy);
  dump("Главы без визуализации", gapViz);
  dump("Остаток глав", orphanViz);
}

```

```

    "giu"
  );
  const map = new Map(GLOSSARY_LABELS.map((l) => [l.label, l]));

  // те же лимиты, что и в src/hooks/useTermHints.ts
  const MAX_PER_SURFACE = 2;
  const MAX_PER_TERM = 8;

  const bad = [];
  const used = new Set();
  console.log("тем | подсв. | глава");
  for (const c of chapters) {
    const text = c.content
      .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi, "")
      .replace(/<[^>]+>/g, " ");

    const perTerm = new Map();
    const perSurface = new Map();
    let highlights = 0;
    const ids = new Set();

    for (const m of text.matchAll(re)) {
      const surface = m[1].toLowerCase();
      const id = map.get(surface);
      if (!id) continue;
      const ut = perTerm.get(id) ?? 0;
      const us = perSurface.get(surface) ?? 0;
      if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
      perTerm.set(id, ut + 1);
      perSurface.set(surface, us + 1);
      highlights += 1;
      ids.add(id);
      used.add(id);
    }

    if (ids.size === 0) bad.push(c.title);
    console.log(
      String(ids.size).padStart(3), "|", String(highlights)
    );
  }
}

```

```
-----
const SHOTS_DIR = path.join(ROOT, "tmp", "viz-shots");
const ENTRY = path.join(ROOT, "tmp", "guide-entry.ts");
```

/** Векторные схемы графов и таблицы состояний для PDF

```
const VECTOR_VISUALIZERS = {
  dijkstra: {
    title: "Векторный граф релаксации (Дейкстра)",
    nodes: [
      { id: "A", label: "A", x: 40, y: 60, dist: "0", s
      { id: "B", label: "B", x: 130, y: 30, dist: "3",
      { id: "C", label: "C", x: 130, y: 90, dist: "2",
      { id: "G", label: "G", x: 220, y: 90, dist: "5",
      { id: "D", label: "D", x: 220, y: 30, dist: "6",
      { id: "E", label: "E", x: 310, y: 90, dist: "7",
      { id: "F", label: "F", x: 400, y: 60, dist: "8",
    ],
    edges: [
      { u: "A", v: "B", w: 4, highlighted: true },
      { u: "A", v: "C", w: 2, highlighted: true },
      { u: "C", v: "B", w: 1, highlighted: true },
      { u: "B", v: "D", w: 5 },
      { u: "C", v: "G", w: 3, highlighted: true },
      { u: "C", v: "E", w: 8 },
      { u: "G", v: "D", w: 1, highlighted: true },
      { u: "G", v: "E", w: 2, highlighted: true },
      { u: "D", v: "F", w: 2 },
      { u: "E", v: "F", w: 1, highlighted: true },
    ],
    table: [
      ["Вершина", "Старт", "Кратчайшее dist", "Предшест
      ["Пиццерия A", "s = A", "0", "-", "зафиксирована"
      ["Парк C", "из A", "2", "A", "зафиксирована"],
      ["Дом 1 B", "из C", "3", "C", "зафиксирована"],
      ["Метро G", "из C", "5", "C", "зафиксирована"],
      ["Офис D", "из G", "6", "G", "зафиксирована"],
      ["Дом 2 E", "из G", "7", "G", "зафиксирована"],
```



```

[ "из \\ в", "S (0)", "A (1)", "B (2)", "C (3)", "D (4)",
  ["S (0)", "0", "4", "5", "3", "6"],
  ["A (1)", "∞", "0", "∞", "1", "4"],
  ["B (2)", "∞", "∞", "0", "-2", "1"],
  ["C (3)", "∞", "∞", "∞", "0", "3"],
  ["D (4)", "∞", "∞", "∞", "∞", "0"],
],
},
mst: {
  title: "Минимальный остов (Краскал / Прим / Борувка)",
  nodes: [
    { id: "1", label: "1", x: 50, y: 30, status: "done" },
    { id: "2", label: "2", x: 150, y: 30, status: "done" },
    { id: "3", label: "3", x: 250, y: 30, status: "done" },
    { id: "4", label: "4", x: 50, y: 90, status: "done" },
    { id: "5", label: "5", x: 150, y: 90, status: "done" },
    { id: "6", label: "6", x: 250, y: 90, status: "done" },
    { id: "7", label: "7", x: 350, y: 60, status: "done" },
  ],
  edges: [
    { u: "1", v: "2", w: 2, highlighted: true },
    { u: "2", v: "3", w: 3, highlighted: true },
    { u: "1", v: "4", w: 1, highlighted: true },
    { u: "4", v: "5", w: 4, highlighted: true },
    { u: "2", v: "5", w: 5 },
    { u: "3", v: "6", w: 2, highlighted: true },
    { u: "5", v: "6", w: 6 },
    { u: "6", v: "7", w: 3, highlighted: true },
  ],
  table: [
    ["Ребро", "Вес w", "Краскал (DSU)", "Прим (Куча)", "Борувка"],
    ["(1, 4)", "1", "добавлено (comp 1-4)", "добавлено", "добавлено"],
    ["(1, 2)", "2", "добавлено (comp 1-2-4)", "добавлено", "добавлено"],
    ["(3, 6)", "2", "добавлено (comp 3-6)", "добавлено", "добавлено"],
    ["(2, 3)", "3", "добавлено (объединение)", "добавлено", "добавлено"],
    ["(6, 7)", "3", "добавлено (финиш)", "добавлено", "добавлено"],
  ],
},

```

```
/**
 * Убираем символы, которых нет в DeJaVu (эмодзи и пиктограммы)
 * на их месте появляется «пустой» глиф. Стрелки, математические
 * геометрические фигуры DeJaVu знает — их оставляем.
 */
const UNSUPPORTED = /[\u{1F000}-\u{1FAFF}\u{2700}-\u{2700}]/gu;
const plural = (n, one, few, many) => {
  const mod10 = Math.abs(n) % 10;
  const mod100 = Math.abs(n) % 100;
  if (mod10 === 1 && mod100 !== 11) return one;
  if (mod10 >= 2 && mod10 <= 4 && (mod100 < 10 || mod100 > 10)) return many;
};

const plain = (text) =>
  String(text ?? "")
    .replace(UNSUPPORTED, "")
    .replace(/\s{2,}/g, " ")
    .replace(/\u0000/g, "")
    .trim();

/* ----- данные приложения (через esbuild) ----- */

async function loadAppData() {
  ensureDir(path.dirname(ENTRY));
  fs.writeFileSync(
    ENTRY,
    [
      'export { chapters, chapterTopics } from "../src/data/chapters";',
      'export { PAGE_SYNC } from "../src/data/vizSync";',
      'export { VIZ_REGISTRY } from "../src/components/vizRegistry";',
      'export { quizzes } from "../src/data/quizzes";',
    ].join("\n")
  );
};

const res = await build({
  entryPoints: [ENTRY],
});
```

```

    this.doc = doc;
    this.fonts = fonts;
    this.footerTitle = "";
    this.pages = 0;
    this.cy = TOP;
}

/**
 * Колонтитул текущей страницы (печатаем до перехода к следующей)
 *
 * Важно: на время печати нижнее поле страницы уменьшаем,
 * решает, что текст не влез, и молча добавляет пустые строки
 */
stampFooter() {
    if (this.pages === 0 || !this.doc.page) return;
    const prevBottom = this.doc.page.margins.bottom;
    this.doc.page.margins.bottom = 16;
    const y = PAGE.height - PAGE.margin + 4;
    this.doc.font(this.fonts.regular).fontSize(7.6).fill('black');
    this.doc.text("Универсальное пособие • алгоритмы и структуры данных",
        width: CONTENT_W * 0.62,
        align: "left",
        lineBreak: false,
        ellipsis: true,
    ));
    this.doc.text(`${this.footerTitle} • ${this.pages}`,
        width: CONTENT_W * 0.38,
        align: "right",
        lineBreak: false,
        ellipsis: true,
    ));
    this.doc.page.margins.bottom = prevBottom;
    this.doc.y = this.cy;
}

newPage(footerTitle) {
    this.stampFooter();
    if (footerTitle !== undefined) this.footerTitle = footerTitle;
}

```

```

    };
    for (const sentence of source.split(/(?<=[.!?...;:])\s/)) {
      const withSentence = current ? `${current} ${sentence}` : sentence;
      if (measure(withSentence) <= max) {
        current = withSentence;
        continue;
      }
      flush();
      if (measure(sentence) <= max) {
        current = sentence;
        continue;
      }
      for (const word of sentence.split(/\s+/)) {
        const withWord = current ? `${current} ${word}` : word;
        if (measure(withWord) <= max) current = withWord;
        else {
          flush();
          current = word;
        }
      }
    }
    flush();
    return chunks.length ? chunks : [source];
  }

```

```

rule(color = "#c7d2fe", width = 1) {
  const y = this.cy;
  this.doc.moveTo(PAGE.margin, y).lineTo(PAGE.margin, y);
  this.cy = y + 8;
}

```

```

h1(rawText, { size = 20, color = "#111827", rule = true }) {
  const text = plain(rawText);
  if (!text) return;
  const h = this.height(text, { font: this.fonts.bold });
  this.ensure(h + (rule ? 14 : 6) + gapBefore, 22);
  this.cy += gapBefore;
  this.doc.font(this.fonts.bold).fontSize(size).fillColor(

```

```

        this.cy = Math.max(y0 + h, this.doc.y) + (last ? 0 : 2.5);
        if (this.cy > BOTTOM) this.newPage();
    });
}

```

```

bullet(rawText, { ordinal = null, depth = 0, size = 9 }) {
    const text = plain(rawText);
    if (!text) return;
    const indent = 16 + depth * 14;
    const marker = ordinal ? `${ordinal}.` : "*";
    const width = CONTENT_W - indent - 4;
    const opts = { font: this.fonts.regular, size, width };
    const chunks = this.splitToFit(text, opts, BOTTOM);
    chunks.forEach((chunk, index) => {
        const h = this.height(chunk, opts);
        this.ensure(h + 2.5, 6);
        const y0 = this.cy;
        if (index === 0) {
            this.doc.font(this.fonts.regular).fontSize(size)
                .text(marker, { align: "right", lineGap: 1 });
        }
        this.doc.font(this.fonts.regular).fontSize(size)
            .text(chunk, { align: "left", lineGap: 1 });
        this.cy = Math.max(y0 + h, this.doc.y) + 2.5;
        if (this.cy > BOTTOM) this.newPage();
    });
}

```

```

/** Код: режим на фрагменты по высоте страницы, каждый фрагмент
code(rawText, { size = 8.0 } = {}) {
    const font = this.fonts.mono;
    const inner = CONTENT_W - 22;
    const lines = String(rawText ?? "").replace(UNSUPPORTED_CHARS, " ");
    // собираем фрагменты, которые влезают в одну рамку
    let i = 0;
    while (i < lines.length) {
        // Если у низа страницы осталось меньше минимальной высоты
        // СРАЗУ: иначе рамка уедет за BOTTOM, и pdfkit с ошибкой вылетит
        // в обход рендерера (без колонтитула и нумерации)
        const h = font.height(lines.slice(i, i + 1), { size, width: inner });
        if (this.doc.y + h > BOTTOM) break;
        this.doc.font(font).text(lines.slice(i, i + 1), { align: "left", lineGap: 1 });
        i++;
    }
    this.newPage();
}

```

```

        .text(title, PAGE.margin + 12, startY + 8);
    }

    // Рёбра графа
    for (const e of edges) {
        const fromNode = nodes.find((n) => n.id === e.u);
        const toNode = nodes.find((n) => n.id === e.v);
        if (!fromNode || !toNode) continue;

        const x1 = PAGE.margin + fromNode.x;
        const y1 = startY + fromNode.y;
        const x2 = PAGE.margin + toNode.x;
        const y2 = startY + toNode.y;

        const strokeColor = e.highlighted ? "#059669" : "#000000";
        const strokeWidth = e.highlighted ? 2 : 1;

        this.doc
            .moveTo(x1, y1)
            .lineTo(x2, y2)
            .lineWidth(strokeWidth)
            .strokeColor(strokeColor)
            .stroke();

        if (e.w !== undefined) {
            const mx = (x1 + x2) / 2;
            const my = (y1 + y2) / 2;
            this.doc
                .circle(mx, my, 7)
                .fillAndStroke("#ffffff", strokeColor);
            this.doc
                .font(this.fonts.bold)
                .fontSize(7)
                .fillColor("#0f172a")
                .text(String(e.w), mx - 7, my - 3.5, { width: 14 });
        }
    }
}

```

```

const pad = 5;
const header = rows[0];

const rowHeight = (cells) => {
  let max = 0;
  for (let i = 0; i < cols; i++) {
    const h = this.height(cells[i] ?? "", { font: this.fonts.bold });
    max = Math.max(max, h);
  }
  return max + pad * 2;
};

const drawRow = (cells, y, isHeader, zebra) => {
  const h = rowHeight(cells);
  this.doc.rect(PAGE.margin, y, CONTENT_W, h).fill({ color: zebra ? "#f2f2f2" : "#fff" });
  for (let i = 0; i < cols; i++) {
    const x = PAGE.margin + widths.slice(0, i).reduce((a, b) => a + b, 0);
    this.doc
      .font(isHeader ? this.fonts.bold : this.fonts.regular)
      .fontSize(size)
      .fillColor("#1f2937")
      .text(String(cells[i] ?? ""), x, y + pad, { width: widths[i] });
  }
  this.doc.rect(PAGE.margin, y, CONTENT_W, h).lineWidth(1);
  return h;
};

this.ensure(rowHeight(header) + 8, 14);
let y = this.cy;
rows.forEach((cells, idx) => {
  const row = [];
  for (let i = 0; i < cols; i++) row.push(cells[i]);
  const h = rowHeight(row);
  if (y + h > BOTTOM) {
    this.cy = y;
    this.newPage();
    y = this.cy;
  }
  if (idx > 0) y += drawRow(header, y, true, false);
  drawRow(row, y, false, idx % 2 === 0);
  y += h;
});

```

```

        this.ensure(h + capH + 10, 20);
        const y0 = this.cy;
        const x0 = PAGE.margin + (CONTENT_W - w) / 2;
        this.doc.image(file, x0, y0, { width: w, height: h });
        this.doc.rect(x0, y0, w, h).lineWidth(0.6).strokeColor('black');
        this.cy = y0 + h + 4;
        if (caption) {
            this.doc.font(this.fonts.regular).fontSize(8).fill('black').text(
                caption, x0, y0, { align: 'center' });
            this.cy = Math.max(this.cy, this.doc.y) + 6;
        }
        return true;
    }
}

/** Бокс с заголовком и строками; длинные режутся на две строки
 * @param {string} rawTitle заголовок
 * @param {string[]} rawLines строки
 * @param {object} options опции
 * @param {string} options.color цвет фона
 * @param {boolean} options.border наличие границы
 */
function box(rawTitle, rawLines, { color = "#eef2ff", border = true }) {
    const title = plain(rawTitle);
    const lines = rawLines.map((l) => plain(l)).filter((l) => l.length > 0);
    const inner = CONTENT_W - 20;
    let rest = lines.slice();
    let first = true;
    while (rest.length || first) {
        first = false;
        const headH = this.height(title, { font: this.fonts.regular });
        this.ensure(headH + 24, 16);
        const y0 = this.cy;
        // сколько строк влезет на эту страницу
        const avail = BOTTOM - y0 - headH - 16;
        const taken = [];
        let h = 0;
        while (rest.length) {
            const ch = this.height(rest[0], { font: this.fonts.regular });
            if (taken.length && h + ch > avail) break;
            taken.push(rest.shift());
            h += ch;
        }
        const boxH = headH + h + 12;
        this.doc.roundedRect(PAGE.margin, y0, CONTENT_W, boxH).fill(color).stroke(
            border ? 'black' : 'none', 1);
    }
}

```



```

    },
  });
  const stream = fs.createWriteStream(GUIDE_PDF_PATH);
  doc.pipe(stream);

  const R = new Renderer(doc, fonts);
  const today = new Date().toISOString().slice(0, 10);
  const totalTopics = new Set(Object.values(chapterTopics));
  const vizCount = chapters.filter((c) => VIZ_REGISTRY[c]);

  /* — обложка — */
  R.firstPage("Обложка");
  R.cy = TOP + 90;
  R.h1("Универсальное пособие", { size: 26, rule: false });
  R.h1("Алгоритмы и структуры данных", { size: 17, color: "#4b5563" });
  R.para(
    `Билеты 1-${totalTopics} · ${chapters.length} ${plus} ${vizCount}, "интерактивная демонстрация", "интерактивная демонстрация"`,
    { size: 10.5, color: "#4b5563" }
  );
  R.para(
    "Автоматический компактный сборник: весь текст тематической визуализации. Навигация приложения и код панели Python Pages.",
    { size: 9.6, color: "#374151" }
  );
  R.para(`Дата сборки: ${today}`, { size: 9, color: "#6b728b" });

  const missingViz = chapters.filter((c) => !VIZ_REGISTRY[c]);
  R.box("Как устроено пособие", [
    "• Страница = один или несколько билетов: текст, анимация, интерактивная демонстрация.",
    "• У каждой страницы есть интерактивная демонстрация, которую можно посмотреть напрямую.",
    "• В этом PDF демонстрации показаны снимками (когда анимация работает).",
    missingViz.length
      ? `• Страницы без демонстрации: ${missingViz.map((c) => c).join(", ")}`
      : "• Демонстрация и синхронизация с компилятором"
  ]);

```

```

    } else if (b.type === "li") {
      R.bullet(b.text, { ordinal: b.ordinal, depth: b
    } else if (b.type === "code") {
      R.code(b.text);
    } else if (b.type === "table") {
      R.table(b.rows);
    } else if (b.type === "quote") {
      R.quote(b.text);
    } else if (b.type === "caption") {
      R.para(b.text, { size: 8.3, color: "#6b7280", i
    } else {
      R.para(b.text);
    }
  }
}

```

```

/* демонстрация: интерактивная схема (векторные гра
const viz = VIZ_REGISTRY[chapter.id];
const vecViz = VECTOR_VISUALIZERS[chapter.id];
const syncKeys = Object.keys(PAGE_SYNC).filter((k) :

```

```

if (viz || syncKeys.length || vecViz) {
  R.h2("Демонстрация и интерактивная схема");
  if (viz) {
    R.para(viz.title, { size: 10.2, color: "#1e293b
    if (viz.hint) R.para(viz.hint, { size: 9.1, col
  }

```

```

  if (vecViz) {
    if (vecViz.nodes && vecViz.edges) {
      R.vectorGraph(vecViz.nodes, vecViz.edges, { h
    }
    if (vecViz.table) {
      R.table(vecViz.table);
    }
  }
}

```

```

for (const key of syncKeys) {
  const sync = PAGE_SYNC[key];

```

```
-----
// в файл без нижнего колонтитула и без нумерации.
const pdfLib = await import("pdf-lib");
const physical = (await pdfLib.PDFDocument.load(fs.re
console.log(`          страниц: ${R.pages} (рендер) / $
if (R.pages > 100) console.warn(`          ВНИМАНИЕ: ${
if (physical !== R.pages) {
    console.error(`          ОШИБКА: pdftkit добавил ${phy
    process.exitCode = 1;
}
}

main().catch((err) => {
    console.error("Ошибка сборки PDF пособия:", err.messa
    process.exit(1);
});
-----
```

ФАЙЛ 95/113: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМ

```
-----
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pa
 * в единый документ public/export/full_code_all_pages.p
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir,

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
    if (!fs.existsSync(PAGES_DIR)) {
-----
```

```
-----
    console.log("[3/3] png -> pdf");
    console.log(`      страниц: ${pages.length}`);
    console.log(`      выход:   ${path.relative(ROOT, PDF)}`);
  }

main().catch((err) => {
  console.error("Ошибка сборки PDF:", err.message);
  process.exit(1);
});
-----
```

ФАЙЛ 96/113: scripts/export/capture-viz-shots.mjs
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 122 | РАЗМЕР: 1.5 КБ

```
-----
/**
 * Снимки визуализаций для PDF-конспекта: tmp/viz-shots.
 *
 * Headless-браузер открывает каждую страницу с демонстрацией
 * блок #chapter-viz. У страниц с вкладками (sparse-table)
 * снимается каждая вкладка — файл получает имя `<chapter>_<tab>`
 * и PDF подписывает его как «режим «вкладка»».
 *
 *   npm run export:shots           # сервер уже поднят
 *   SHOTS_BASE_URL=http://127.0.0.1:4173 npm run export:shots
 *
 * В CI это делает rebuild-pdf.yml: сборка → vite preview →
 * npm run export:guide. Локально без браузера скрипт просто
 * честно пишет, что снимки добавляются при сборке в CI.
 */
import fs from "node:fs";
import path from "node:path";
import { build } from "esbuild";
import puppeteer from "puppeteer";
import { ROOT } from "../common.mjs";

const OUT_DIR = path.join(ROOT, "tmp", "viz-shots");
```

```

const browser = await puppeteer.launch({
  headless: true,
  args: ["--no-sandbox", "--disable-setuid-sandbox", "--no-zygote"],
});
const page = await browser.newPage();
await page.setViewport({ width: 1400, height: 1000, deviceScaleFactor: 1 });

let done = 0;
let failed = 0;
for (const id of ids) {
  try {
    await page.goto(`${BASE_URL}/?topic=${id}`, { waitUntil: 'load' });
    await page.waitForSelector("#chapter-viz", { timeout: 10000 });
    // демо-анимации: даём кадру собраться до осмысленного состояния
    await new Promise((r) => setTimeout(r, 1600));
    const viz = await page.$("#chapter-viz");
    if (!viz) throw new Error("нет #chapter-viz");
    await viz.screenshot({ path: path.join(OUT_DIR, `${id}.png`), type: 'png' });
    done += 1;

    for (const tab of TABS[id] ?? []) {
      try {
        const clicked = await page.evaluate((label) => {
          const root = document.querySelector("#chapter-viz");
          if (!root) return false;
          const btn = Array.from(root.querySelectorAll("button")).find(
            (b) => b.textContent ?? "" === label.trim()
          );
          if (!btn) return false;
          btn.dispatchEvent(new MouseEvent("click", { bubbles: true }));
          return true;
        }, tab);
        if (!clicked) continue;
        await new Promise((r) => setTimeout(r, 1200));
        const el = await page.$("#chapter-viz");
        await el.screenshot({ path: path.join(OUT_DIR, `${id}_${tab}.png`), type: 'png' });
        done += 1;
      } catch (e) {
        failed += 1;
      }
    }
  } catch (e) {
    failed += 1;
  }
}

```

```
*/
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
  ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
  ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человекочитаемые категории для оглавления. */
const CATEGORIES = [
  [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
  [/^src\/data\/$/, "Контент и данные пособия"],
  [/^src\/components\/visualizers\/$/, "Визуализаторы (в
  [/^src\/components\/$/, "Интерактивные компоненты и си
  [/^src\/layout\/$/, "HTML-разметка (layout)"],
  [/^src\/utils\/$/, "Утилиты"],
  [/^src\/$/, "Ядро приложения"],
  [/^scripts\/$/, "Скрипты сборки и экспорта"],
  [/^\.github\/$/, "CI / автоматизация"],
  [/^[^\/]+\$/ , "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re

/** Порядок разделов в документе. */
const ORDER = [
  "Конфигурация проекта",
  "Ядро приложения",
  "Контент и данные пособия",
  "Билеты (учебный контент)",
```

```

        const ia = ORDER.indexOf(categoryOf(a));
        const ib = ORDER.indexOf(categoryOf(b));
        return ia !== ib ? ia - ib : a.localeCompare(b);
    });
}

/** Достаём заголовки глав из данных пособия (без запус
function extractChapters(files) {
    const chapters = [];
    const seen = new Set();
    for (const rel of files.filter((f) => f.startsWith("s
        const src = fs.readFileSync(path.join(ROOT, rel), "
        const re = /id:\s*"([^"]+)\s*,\s*\n?\s*title:\s*"
        let m;
        while ((m = re.exec(src)) !== null) {
            const [, id, title] = m;
            if (seen.has(id)) continue;
            seen.add(id);
            chapters.push({ id, title, file: rel });
        }
    }
    return chapters.sort((a, b) => {
        const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
        const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
        return na - nb;
    }));
}

function main() {
    const files = listFiles();
    const chapters = extractChapters(files);
    ensureDir(OUT_DIR);

    const out = [];
    const push = (s = "") => out.push(s);

    const stamp = new Date().toLocaleString("ru-RU", { ti
    const totalBytes = files.reduce((s, f) => s + fs.stat

```

```
    const cat = categoryOf(rel);
    if (cat !== current) {
        current = cat;
        push(HR);
        push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
        push(HR);
        push();
    }
    const content = fs.readFileSync(path.join(ROOT, rel));
    const lines = content.split("\n");
    push(SUBHR);
    push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
    push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} | `);
    push(SUBHR);
    push();
    push(content.replace(/\n+$/, ""));
    push();
    push();
  });

  push(HR);
  push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
  push(HR);

  const txt = out.join("\n") + "\n";
  fs.writeFileSync(TXT_PATH, txt, "utf8");

  console.log("[1/3] код -> txt");
  console.log(`    файлов:  ${files.length}`);
  console.log(`    глав:    ${chapters.length}`);
  console.log(`    строк:   ${txt.split("\n").length}`);
  console.log(`    выход:   ${path.relative(ROOT, TXT_PATH)}`);
}

main();
```

```
    /** Строк содержимого на странице (+2 служебные: шапка  
    rows: 76,  
};  
  
export const HR = "=".repeat(PAGE.cols);  
export const SUBHR = "-".repeat(PAGE.cols);  
  
export function ensureDir(dir) {  
  fs.mkdirSync(dir, { recursive: true });  
}  
  
export function humanSize(bytes) {  
  if (bytes < 1024) return `${bytes} B`;  
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;  
  return `${(bytes / 1024 / 1024).toFixed(2)} MB`;  
}  
  
/** ImageMagick 7 (`magick`) или 6 (`convert`). */  
export function imageMagickCmd() {  
  for (const cmd of ["magick", "convert"]) {  
    try {  
      execFileSync(cmd, ["-version"], { stdio: "ignore" });  
      return cmd;  
    } catch {}  
    /* пробуем следующий */  
  }  
  throw new Error(  
    "ImageMagick не найден. Установите его: sudo apt-get install imagemagick  
  );  
}
```

```

const stack = [root];
const re = /<!--[\s\S]*?-->|<\/([a-zA-Z0-9]+)\s*>|<([a-zA-Z0-9]+)\s*/;
let m;
while ((m = re.exec(html)) !== null) {
  const [full, closeTag, openTag, attrsStr, selfClose] = m;
  if (full.startsWith("<!--")) continue;

  if (closeTag) {
    const tag = closeTag.toLowerCase();
    // закрываем до совпадения (прощает незакрытые <br> и т.п.)
    for (let i = stack.length - 1; i >= 1; i--) {
      if (stack[i].tag === tag) {
        stack.length = i;
        break;
      }
    }
    continue;
  }

  if (openTag) {
    const tag = openTag.toLowerCase();
    const attrs = {};
    const attrRe = /([a-zA-Z-_:]+)\s*=\s*"([^"]*)"|'([^']*)'|([a-zA-Z-_:]+)\s*/;
    let a;
    while ((a = attrRe.exec(attrsStr || "")) !== null) {
      const node = { tag, attrs, children: [] };
      stack[stack.length - 1].children.push(node);
      if (!VOID.has(tag) && !selfClose) stack.push(node);
      continue;
    }

    if (textChunk !== undefined && textChunk.length) {
      stack[stack.length - 1].children.push({ tag: "#text", text: textChunk });
    }
  }
  return root;
}

```

```

    } else if (BLOCK.has(c.tag)) {
      continue;
    } else if (c.tag === "br") {
      out += "\n";
    } else {
      out += inlineText(c);
    }
  }
  return clean(out);
}

/** Весь текст поддерева (для ячеек таблиц и подписей).
function collectText(node) {
  if (node.tag === "#text") return clean(node.text);
  if (SKIP.has(node.tag)) return "";
  if (node.tag === "table") return tableRows(node).map(
    return clean(node.children.map(collectText).join(" "))
  )
}

/** Сырой текст внутри <pre> – переносы строк важны. */
function collectRawPre(node) {
  if (node.tag === "#text") return node.text;
  if (node.tag === "br") return "\n";
  if (SKIP.has(node.tag) && node.tag !== "img") return
  return node.children.map(collectRawPre).join("");
}

/** Таблица → массив строк из массивов ячеек. */
function tableRows(node) {
  const rows = [];
  const visit = (n) => {
    if (n.tag === "tr") {
      const cells = n.children.filter((c) => c.tag ===
      if (cells.some((c) => c.length)) rows.push(cells)
      return;
    }
    n.children.forEach(visit);
  };
};

```

```

    if (node.tag === "table") {
      const rows = tableRows(node);
      if (rows.length) push({ type: "table", rows });
      return;
    }
    if (node.tag === "pre") {
      const text = decodeEntities(collectRawPre(node));
      if (text.trim()) push({ type: "code", text: text });
      return;
    }

    // есть блочные дети: идём по порядку, инлайн-пробе
    if (hasBlockChild(node)) {
      let buf = "";
      const flush = () => {
        const t = clean(buf);
        if (t) push({ type: "p", text: t });
        buf = "";
      };
      for (const c of node.children) {
        if (c.tag === "#text") {
          buf += c.text;
          continue;
        }
        if (SKIP.has(c.tag)) continue;
        if (!BLOCK.has(c.tag)) {
          buf += inlineText(c) + " ";
          continue;
        }
        if (c.tag === "ul" || c.tag === "ol") {
          flush();
          emitList(c, listDepth);
          continue;
        }
        flush();
        walk(c, listDepth);
      }
    }
  }

```

ФАЙЛ 100/113: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМ: 1.5 КБ

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
  ROOT,
  PAGES_DIR,
  TXT_PATH,
  PAGE,
  ensureDir,
  humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
 * function wrap(line) {
 *   const expanded = line.replace(/\t/g, "    ").replace(
 *   if (expanded.length <= PAGE.cols) return [expanded];
 *   const parts = [];
 *   const indent = "    ".repeat(Math.min((expanded.match(/^
 *   let rest = expanded;
 *   let first = true;
 *   while (rest.length > 0) {
 *     const width = first ? PAGE.cols : PAGE.cols - indent;
 *     parts.push((first ? "" : indent) + rest.slice(0, width));
 *     rest = rest.slice(width);
 *     first = false;
 *   }
 *   return parts;
 * }
```

```

    "-density", String(PAGE.density),
    "-page", PAGE.size,
    "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : null,
    "-pointsize", String(PAGE.pointsized),
    `text:${job.txtFile}`,
    "-background", "white",
    "-flatten",
    "-colorspace", "Gray",
    ...(PAGE.monochrome ? ["-monochrome"] : []),
    "-strip",
    "-define", "png:compression-level=9",
    job.pngFile,
  ];

```

```

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

```

```

async function worker() {
  while (cursor < jobs.length) {
    const job = jobs[cursor++];
    await execFileAsync(im, args(job));
    fs.rmSync(job.txtFile, { force: true });
    done += 1;
    if (done % 25 === 0 || done === total) {
      process.stdout.write(`      отрендерено ${done}\n`);
    }
  }
}

```

```

await Promise.all(Array.from({ length: concurrency }, () => worker()));
process.stdout.write("\n");

```

```

const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
if (missing.length) throw new Error(`Не отрендерены следующие файлы: ${missing.map(j => j.pngFile).join(", ")}`);

```

```

const bytes = jobs.reduce((s, j) => s + fs.statSync(j.pngFile).size, 0);
console.log("[2/3] txt -> png");

```

```
    ms: number;
    detail: string;
    consoleErrors: string[];
}

const results: TestResult[] = [];
let consoleErrors: string[] = [];
const realConsoleError = console.error;

/* -----
/*  утилиты
/* -----

const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));

async function settle(ms = 40) {
    await act(async () => {
        await sleep(ms);
    });
}

function bodyText(): string {
    return (document.body.textContent ?? "").replace(/\s+/g, " ");
}

function allButtons(): string[] {
    return Array.from(document.querySelectorAll("button"))
        .filter(b => (b.getAttribute("aria-label") || b.getAttribute("title")));
}

/** Кнопки без доступного имени: ни aria-label, ни title
function namelessButtons(scope: ParentNode = document): string[] {
    return Array.from(scope.querySelectorAll("button"))
        .filter(b => {
            const name = [b.getAttribute("aria-label"), b.getAttribute("title")]
                .filter(Boolean)
                .join(" ");
            return !name;
        });
}
```

```

-----
    if (!setter) throw new Error("нет нативного setter'a");
    act(() => {
        setter.call(ta, value);
        ta.dispatchEvent(new window.Event("input", { bubble: true }));
    });
}

async function key(k: string, init: KeyboardEventInit = {}): Promise<void> {
    await act(async () => {
        document.dispatchEvent(new window.KeyboardEvent("keydown", init));
        await sleep(20);
    });
}

/** Ждём появления признака в DOM (текст/предикат), перепроверяем каждые 20мс
 *  @param pred - предикат, возвращающий true, если признак появился
 *  @param timeoutMs - время ожидания в мс
 */
async function waitFor(pred: () => boolean, timeoutMs: number = 1000): Promise<boolean> {
    const deadline = Date.now() + timeoutMs;
    while (Date.now() < deadline) {
        if (pred()) return true;
        await settle(step);
    }
    return pred();
}

const RUN_DONE = [
    "Трасса готова автоматически",
    "Нет шагов трассы",
    "Пустой код",
    "Код не выполнен",
    "Выполнение остановлено после",
    "Не удалось выполнить код",
    "Ошибка:",
];

let root: Root | null = null;
let host: HTMLElement | null = null;

function freshHost(): HTMLElement {

```



```

    root = createRoot(host);
    await act(async () => {
      root!.render(
        <VizChapterContext.Provider value={vizId}>
          <VizLiveStatus demoTitle={entry.title} />
          <Comp />
        </VizChapterContext.Provider>
      );
    });
    await settle(60);
    return host;
  }

const BAD_DOM = [
  { re: /\bNaN\b/, why: "NaN в разметке" },
  { re: /\bundefined\b/, why: "undefined в разметке" },
  { re: /\[object Object\]/, why: "[object Object] в ра
];

/**
 * Битые подстановки видны в КОРОТКИХ текстовых узлах (
 * вершины): «undefined», «NaN», «[object Object]». В д
 * псевдокода те же слова законны – их не трогаем.
 */
function domDefects(scope: HTMLElement | null): string[] {
  if (!scope) return [];
  const found: string[] = [];
  const walker = document.createTreeWalker(scope, 4 /*
  let node = walker.nextNode() as Text | null;
  while (node) {
    const text = (node.nodeValue ?? "").replace(/\s+/g,
    if (text.length > 0 && text.length < 60) {
      for (const bad of BAD_DOM) {
        if (bad.re.test(text)) found.push(`${bad.why}:
      }
    }
    node = walker.nextNode() as Text | null;
  }
}

```

```

-----
const onlySync = new Set((process.env.HAR_SYNC || "").split(","));

export async function runSuite({ py }: { py: Py }): Promise<void> {
  const chapterList = onlyChapters.length ? chapters.filter(chapter =>
  const want = (group: string) => !skipGroups.has(group);
  // перехват console.error приложения (React пишет сюда)
  console.error = (...args: unknown[]) => {
    consoleErrors.push(args.map(a => (a instanceof Error ? a : String(a))));
  };

  /* — 1. Панель Python: реальный запуск на каждой главе */
  for (const chapter of chapterList) {
    const group = "A. страницы";
    await test(group, `${chapter.id}: монтируется, бейджик`);
    await mountApp(chapter.id);
    const txt = bodyText();
    if (!txt.includes(chapter.title.slice(0, 18))) throw new Error("некорректная загрузка");
    if (!VIZ_REGISTRY[chapter.id]) return "визуализация не найдена";
    if (!/встроенное демо|данные из компилятора/.test(txt)) return "некорректная загрузка";
    const defects = domDefects(host);
    if (defects.length) throw new Error(`артефакты в DOM: ${defects.join(", ")}`);
    const nameless = namelessButtons(host!);
    if (nameless.length) throw new Error(`кнопок без имени: ${nameless.join(", ")}`);
    return `бейдж «встроенное демо» на месте, кнопок: ${nameless.length}`;
  });

  await test(group, `${chapter.id}: панель открывается`);
  await mountApp(chapter.id);
  const toggle = findButton("Боковая панель Python");
  await click(toggle, "кнопка панели Python в навбаре");
  const ta = textarea();
  if (!ta) throw new Error("редактор (textarea) не найден");
  if (!ta.value.includes("# «")) return `редактор не настроен`;
  const eye = findButton(/эта?лонный код/i);
  if (!eye) throw new Error("нет кнопки-«глаза» (встроенный редактор)");
  const run = findButton(/Запустить код|автопроход/);
  if (!run) throw new Error("нет кнопки запуска ▶");
  return `кнопок в панели: ${allButtons().length}`;
}

```



```

const ch = "dijkstra";

await test("F. редактор", "только комментарии: редактор", async () => {
  await mountApp(ch);
  await click(findButton("Боковая панель Python"), "python");
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  typeInto(ta, "# просто комментариев\n# и ещё один\n");
  const done = await waitFor(() => bodyText().include("#"));
  if (!done) throw new Error(`не дождались сообщения`);
  if (!textarea()) throw new Error("редактор исчез по таймауту");
  if (!/встроенное демо/.test(bodyText())) throw new Error("не встроенное демо");
  return "редактор на месте, бейдж «встроенное демо»";
});

await test("F. редактор", "синтаксическая ошибка: редактор", async () => {
  await mountApp(ch);
  await click(findButton("Боковая панель Python"), "python");
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  typeInto(ta, "def broken(:\n    pass\n");
  const done = await waitFor(() => /Код не выполнен/);
  if (!done) throw new Error(`ошибка синтаксиса не появилась`);
  if (!textarea()) throw new Error("редактор исчез на таймауте");
  if (!/invalid syntax|SyntaxError/i.test(bodyText())) throw new Error("ошибка не показана");
  return "ошибка показана, редактор остался";
});

await test("F. редактор", "гигантский print обрезается", async () => {
  await mountApp(ch);
  await click(findButton("Боковая панель Python"), "python");
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  typeInto(ta, 'dist = {0: 0}\nprint("x" * 200000)\n');
  const ready = await waitFor(() => bodyText().include("x"));
  if (!ready) throw new Error(`программа с огромным print не завершилась`);
  const block = document.querySelector('[data-testid="result"]');
  if (!block) throw new Error("блок «Результат» не отображается");
  return "гигантский print обрезается";
});

```

```

if (!trace) throw new Error(`эталон не запустился.
if (!textarea()) throw new Error("редактор исчез во
if (/Ошибка:/.test(panelText())) throw new Error(`э
const ta2 = textarea();
const firstRefLine = ref.split("\n").find((l) => l.
if (!ta2?.value.includes(firstRefLine)) {
  throw new Error(`выполняется не тот текст, что ле
    0, 100)}`);
}
const mustBe = [
  { what: "пуск/автопроход", ge: /Запустить код|Про
  { what: "копирование кода", ge: /Скопировать код|
  { what: "заккрыть панель", ge: /Заккрыть компилятор
  { what: "шаг вперед", ge: /Шаг вперед|вперед/i },
  { what: "шаг назад", ge: /Шаг назад|назад/i },
  { what: "свернуть листинг", ge: /Свернуть листинг
];
const btns = allButtons().join(" | ");
for (const m of mustBe) {
  if (!m.re.test(btns)) throw new Error(`в панели н
}
// На первом шаге переменных ещё нет (отладчик вста
// поэтому связь с демонстрацией проверяем, дойдя д
await click(findButton("В конец трассы"), "в конец
const linked = await waitFor(() => /данные из компи
if (!linked) throw new Error(`эталон выполнен до ко

// — глаз ВЫКЛ: кода нет вовсе, ничего не выполняе
await click(findButton("Убрать эталонный код"), "гл
const cleared = await waitFor(() => bodyText().incl
if (!cleared) throw new Error(`после выключения гла
const ta3 = textarea();
if (!ta3) throw new Error("редактор пропал после вы
if (ta3.value.trim() !== "") throw new Error(`глаз
if (!/встроенное демо/.test(bodyText())) throw new
if (/данные из компилятора/.test(bodyText())) throw
const afterButtons = allButtons();
const lost = editorButtons.filter((b) => b && !afte

```

```

// Листинг сворачивается (освобождая место редактору
// и редактор остаются на месте.
const codeLines = () => panel()?.querySelectorAll("
if (codeLines() === 0) throw new Error("в раскрытой
await click(findButton("Свернуть листинг трассы"),
await settle(80);
if (!textarea()) throw new Error("после сворачивани
if (codeLines() !== 0) throw new Error(`листинг не
if (!findButton("В конец трассы")) throw new Error(
await click(findButton("Показать листинг трассы"),
await settle(80);
if (codeLines() === 0) throw new Error("листинг не
// Esc: прогон остановлен, трасса убрана, редактор
await key("Escape");
await settle(150);
if (!textarea()) throw new Error("Esc убрал редактор
if (findButton(/Остановить прогон трассы/)) throw n
if (!/встроенное демо/.test(bodyText())) throw new
return "редактор всегда на месте: трасса рядом, сво
));

await test("F. редактор", "закрытие панели во время пр
await mountApp("floyd"); // 869 шагов: прогон долги
await click(findButton("Боковая панель Python"), "п
await waitFor(() => panel() !== null, 4000);
await click(findButton("Заккрыть компилятор"), "крес
await settle(150);
if (panel()) throw new Error("панель не закрылась")
const backToDemo = await waitFor(() => /встроенное
if (!backToDemo) throw new Error(`после закрытия па
await settle(3000); // ждём, пока догоняющий прогон
if (/данные из компилятора/.test(bodyText())) {
    throw new Error("закрытая панель всё-таки опублик
}
if (!/встроенное демо/.test(bodyText())) throw new
return "панель закрыта во время выполнения – демонс
));

```

```

    await key("Escape");
    await settle(80);
    const ta = textarea();
    if (!ta) throw new Error("редактор не вернулся по E
    typeInto(ta, "");
    const cleared = await waitFor(() => /встроенное дем
    if (!cleared) throw new Error("после очистки кода б
    if (/переменных пока нет/.test(bodyText())) throw n
    return "демо → выполнено/переменных нет → данные из
  });

```

/* — 7. Текстовый режим для PDF/парсеров —————

/* — 9. Ссылки между билетами, разделы содержания, б.

```

await test("Н. доступность", "содержание сжимается в
  await mountApp("bridges-code");
  const root = () => document.querySelector<HTMLElemen
  const toggle = () => findButton(/Сжать содержание|С
  if (!root()) throw new Error("нет сайдбара");
  if (!/lg:w-56/.test(root()!.className)) throw new E
  await click(toggle(), "сжать");
  if (!/lg:w-44/.test(root()!.className)) throw new E
  const items = Array.from(root()!.querySelectorAll("
  if (items.length < chapters.length) throw new Error
  if (root()!.querySelector("input")) throw new Error
  await click(toggle(), "скрыть");
  if (!/lg:w-0/.test(root()!.className)) throw new Er
  await click(toggle(), "развернуть");
  if (!/lg:w-56/.test(root()!.className)) throw new E
  return "широкое → узкое (темы на месте, поиска нет)
});

```

```

await test("J. шапка", "кнопки загрузки не прячутся и
  await mountApp("segment-trees");
  const menu = document.querySelector("#download-menu
  if (!menu) throw new Error("нет меню загрузки в шап
  const want = [
    { id: "download-guide-pdf-btn", file: "export/gui

```



```

await test("I. ссылки", "содержание: разделы по поряд
  await mountApp("bridges-code");
  const sidebar = document.querySelector("#sidebar-ro
  if (!sidebar) throw new Error("нет сайдбара содержа
  const headers = Array.from(sidebar.querySelectorAll
    .map((el) => (el.textContent ?? "").trim()))
    .filter((t) => SECTIONS.some((sec) => sec.title === t))
  if (headers.length !== SECTIONS.length) {
    throw new Error(`в содержании ${headers.length} раз
  }
  const expected = SECTIONS.map((sec) => sec.title).join(" | ")
  if (headers.join(" | ") !== expected) throw new Error("не совпало")
  // 11 «Мосты» и 12 «Точки сочленения» читаются парой
  const items = Array.from(sidebar.querySelectorAll("li"))
  const iBridges = items.findIndex((t) => t.includes("Мосты"))
  const iArtic = items.findIndex((t) => t.includes("Точки сочленения"))
  if (iBridges < 0 || iArtic < 0) throw new Error("не найдены")
  if (Math.abs(iArtic - iBridges) !== 1) throw new Error("не рядом")
  // 2 «разреженная» и 5 «префиксные» – в одном разделе
  if (sectionOf("sparse-table").id !== sectionOf("prefixes").id)
    throw new Error("билеты 2 и 5 попали в разные разделы")
  }
  if (sectionOf("bridges-code").id !== sectionOf("graphs").id)
    throw new Error("билеты 11 и 12 попали в разные разделы")
  }
  // порядок чтения: номера билетов сверху вниз не убывающий
  const nums = Array.from(sidebar.querySelectorAll("b"))
    .map((b) => (b.querySelector("span")?.textContent ?? ""))
    .filter((t) => /\d/.test(t))
    .map((t) => Number.parseInt(t, 10));
  const ticketed = chapters.filter((c) => (chapterTopic === c.title));
  if (nums.length !== ticketed) {
    throw new Error(`номера билетов видны у ${nums.length} билетов`)
  }
  for (let i = 1; i < nums.length; i += 1) {
    if (nums[i] < nums[i - 1]) throw new Error(`порядок чтения не убывающий`)
  }
  if (!sidebar.textContent?.includes("Порядок чтения"))

```

```

    }
    const wrong = list[0].options.findIndex((_, i) => i
    await click(ask(list[0].options[wrong].slice(0, 20)
    await settle(80);
    if (!/Неверно/.test(shown())) throw new Error("на не
    if (!shown().includes(list[0].explanation.slice(0,
        throw new Error("после ответа нет разбора – польз
    }
    const nextBtn = ask("Следующий вопрос");
    if (!nextBtn) throw new Error("нет кнопки «Следующий
    await click(nextBtn, "дальше");
    await settle(80);
    if (!shown().includes(list[1].question.slice(0, 24)
    // верный ответ
    await click(ask(list[1].options[list[1].correctIndex
    await settle(80);
    if (!/Верно/.test(shown())) throw new Error("на вер
    if (!/вопрос 2\\//.test(shown())) throw new Error(`с
    return `блиц из ${list.length} вопросов отвечает, р
  });

```

```

await test("I. факты", "память: разреженная жирная, п
  const memoryRow = async (topic: string) => {
    await mountApp(topic);
    const rows = Array.from(document.querySelectorAll
    const row = rows.find((r) => /^\\s*Память/.test((r
    if (!row) throw new Error(`на странице «${topic}»
    return Array.from(row.querySelectorAll("td")).map
  };
  const sparse = await memoryRow("sparse-table");
  const prefix = await memoryRow("prefix-sums-2d");
  // билет 2: у префиксной  $N \times M$ , у разреженной – с л
  if (!/log/.test(sparse.join(" "))) throw new Error(
    ;
  const prefixCell = sparse.find((c) => !/log/.test(c
  if (!prefixCell) throw new Error(`в сравнении билет
  // билет 5: зеркально – у префиксной без логарифмов
  if (!/log/.test(prefix.join(" "))) throw new Error(

```

 /* — 10. Сквозной путь: свой код чужой размерности, 1

```

await test("К. свой код", "Флойд 4x4 из редактора вед
  await mountApp("floyd");
  await click(findButton("Боковая панель Python"), "п
  const ta = textarea();
  if (!ta) throw new Error("нет редактора");
  typeInto(
    ta,
    [
      "n = 4",
      "INF = 999",
      "d = [[0, 3, INF, 7], [8, 0, 2, INF], [5, INF,
      "k = 0",
      "i = 0",
      "j = 0",
      "for k in range(n):",
      "    for i in range(n):",
      "        for j in range(n):",
      "            if d[i][k] + d[k][j] < d[i][j]:",
      "                d[i][j] = d[i][k] + d[k][j]",
      "print(d[0], d[1][0])",
    ].join("\n")
  );
  const ready = await waitFor(() => bodyText().includ
  if (!ready) throw new Error(`своя программа не выпол
  if (!bodyText().includes("[0, 3, 5, 6]")) {
    throw new Error(`Флойд на своей матрице 4x4 посчи
  }
  if (!textarea()) throw new Error("редактор исчез, п
  await click(findButton("В конец трассы"), "в конец
  const linked = await waitFor(() => /данные из компи.
  if (!linked) throw new Error("демонстрация не приня
  const page = bodyText();
  if (!/4x4/.test(page)) throw new Error(`демонстраци
  const defects = domDefects(document.querySelector("
  if (defects.length) throw new Error(`артефакты в де
  return "набрал 4x4 с INF=999 → d[0] = [0, 3, 5, 6]

```

```

    await click(findButton("Вставить эталонный код"), "
    const ready = await waitFor(() => bodyText().includes("
    if (!ready) throw new Error(`эталон Косарайю не вып
    if (/Ошибка:/.test(panelText())) throw new Error(`э
    const out = panelText();
    if (!/порядок выхода:/.test(out)) throw new Error(`
    // граф на экране: 0→1→2→0, 2→3, 3→4 – ровно две SC
    if (!/компонент: 2/.test(out)) throw new Error(`в гр
    if (!textarea()) throw new Error("редактор исчез, п
    await click(findButton("В конец трассы"), "в конец
    const linked = await waitFor(() => /данные из компи
    if (!linked) throw new Error("демонстрация не приня
    const defects = domDefects(document.querySelector("
    if (defects.length) throw new Error(`артефакты в де
    return "эталон Косарайю: order считается DFS-ом, на
  ));

```

```

await test("G. lite-режим", "?lite=1 отдаёт текст без
  if (root) {
    await act(async () => {
      root?.unmount();
    });
    root = null;
  }
  if (host) host.remove();
  host = freshHost();
  window.history.replaceState({}, "", "/?lite=1&topic
  root = createRoot(host);
  await act(async () => {
    root!.render(<App />);
  });
  await settle(120);
  const txt = bodyText();
  if (txt.length < 2000) throw new Error(`lite-режим
  if (/main\.py|Pyodide/.test(txt)) throw new Error("
  if (!/разреженн/i.test(txt)) throw new Error("в lite
  if (findButton("Боковая панель Python")) throw new
  // текстовая версия: ссылки на другие билеты ведут

```

```
-----  
    return { total: results.length, passed: results.filter  
      .fromEntries(byGroup) };  
}
```

```
/** Инициализация страницы из vizSync – нужна для проверки  
export const initTemplateOf = (id: string) => buildInit
```

ФАЙЛ 102/113: scripts/verify/check-links.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 199 | РАЗМЕР: 10.5 КБ

```
-----  
/**  
 * Аудит внутренних ссылок пособия: ничего не должно вешаться  
 *  
 * npm run verify:links  
 *  
 * Проверяет (по настоящим модулям приложения, собранным из  
 * 1. билеты 1–24 закрыты страницами, id в chapterTopics  
 * 2. разделы содержания: каждая страница в своём разделе  
 * 3. упоминания «билет N» в тексте ведут на существующие  
 * 4. все data-goto / href="?topic=..." указывают на существующие  
 * 5. якоря href="#..." есть в той же главе;  
 * 6. glossary.simulator существует в реестре и (кроме того)  
 *    имеет свою страницу – иначе модалка не получит данных  
 * 7. glossary.demo – работающая мини-демо;  
 * 8. у каждого ключа PAGE_SYNC есть запись в реестре  
 * 9. у каждой страницы есть визуализация;  
 * 10. страницы связаны перекрёстными ссылками (темы не исключение)  
 */
```

```
import * as esbuild from "esbuild";  
import { mkdirSync, writeFileSync } from "fs";
```

```
const OUT = "tmp/verify/links-entry.mjs";  
const FAILS = [];  
const NOTES = [];  
const ok = (cond, message) => (cond ? true : (FAILS.push(message), false));
```

```

-----
/** Демо без собственной страницы: открываются из подсказки
const EXTRA_DEMOS = new Set(["stack-dfs", "queue-bfs",

/* 1. билеты 1–24 */
const covered = new Set();
for (const [id, tickets] of Object.entries(chapterTopics)) {
  ok(chapterIds.has(id), `chapterTopics ссылается на не-
  tickets.forEach((n) => covered.add(n));
}
for (let n = 1; n <= 25; n += 1) {
  ok(covered.has(n), `билет ${n} не закрыт ни одной стр.
}

/* 2. разделы содержания */
for (const chapter of chapters) {
  const sec = sectionOf(chapter.id);
  ok(!sec, `страница «${chapter.id}» не попала ни в од
  const tickets = chapterTopics[chapter.id] ?? [];
  ok(
    tickets.every((n) => n >= sec.from && n <= sec.to),
    `билеты страницы «${chapter.id}» (${tickets.join(",
  });
}
{
  const bySection = SECTIONS.map((s) => ({
    s,
    nums: chapters.flatMap((c) => (sectionOf(c.id).id =
  }));
  for (let i = 1; i < bySection.length; i += 1) {
    const prevMax = Math.max(...bySection[i - 1].nums);
    const curMin = Math.min(...bySection[i].nums);
    ok(curMin > prevMax, `разделы пересекаются: «${bySe
  }
  const all = bySection.flatMap((x) => x.nums);
  for (let n = all[0]; n <= all[all.length - 1]; n += 1)
    ok(all.includes(n), `в содержании дыра: билет ${n}
  }
}

```

```

    }

    if (mentionsHere === 0 && !/data-goto=/.test(html)) w
}

/* 6–7. глоссарий */
const demoKinds = new Set(MINI_DEMO_KINDS);
for (const term of GLOSSARY) {
  if (term.simulator) {
    ok(registryIds.includes(term.simulator), `глоссарий
    ok(
      chapterIds.has(term.simulator) || EXTRA_DEMOS.has
      `глоссарий «${term.id}»: simulator "${term.simula
      ые компилятора`
    );
  }
  if (term.demo) ok(demoKinds.has(term.demo), `глоссарий
}

/* 8–9. реестр и синхронизация */
for (const key of Object.keys(PAGE_SYNC)) {
  const base = key.split("#")[0];
  ok(registryIds.includes(base), `PAGE_SYNC["${key}"]:
}
for (const id of registryIds) {
  if (!chapterIds.has(id) && !EXTRA_DEMOS.has(id)) {
    FAILS.push(`реестр: визуализация "${id}" не является
  }
}
for (const chapter of chapters) {
  ok(registryIds.includes(chapter.id), `страница «${cha
}

/* 96. файлы, на которые ведут кнопки шапки, действител
{
  const { existsSync, statSync } = await import("fs");
  for (const file of ["public/export/guide_lite.pdf", "
    ok(existsSync(file), `шапка ссылается на ${file}, н

```

```
* 1. код не падает и не обрывается по лимиту шагов;  
* 2. трасса не пустая (иначе панель показала бы пустоту);  
* 3. каждая переменная, обещанная странице, реально есть;  
*    иначе визуализация не может за ним следовать;  
* 4. в снимках нет «мусорных» значений, которые уронят картинку;  
*/
```

```
import * as esbuild from "esbuild";  
import { writeFileSync, mkdirSync } from "fs";  
import { loadPyodide } from "pyodide";
```

```
const OUT_DIR = "tmp/verify";  
const t0 = Date.now();  
mkdirSync(OUT_DIR, { recursive: true });
```

```
writeFileSync(  
  `${OUT_DIR}/contract-entry.ts`,  
  `import { buildDebugRunner } from "../../src/data/debugRunner";  
import { PAGE_SYNC } from "../../src/data/vizSync";  
import { chapters } from "../../src/data/content";  
export const runnerOf = (src: string) => buildDebugRunner(src);  
export const sync = Object.fromEntries(Object.entries(PAGE_SYNC).map(  
  ([key, value]) => [key, value.sync]));  
export const declared = Object.fromEntries(Object.entries(DECLARED).map(  
  ([key, value]) => [key, value.declared]));  
export const chapterIds = chapters.map((c) => c.id);  
`  
);
```

```
const built = await esbuild.build({  
  entryPoints: [`${OUT_DIR}/contract-entry.ts`],  
  bundle: true,  
  outfile: `${OUT_DIR}/contract-bundle.mjs`,  
  format: "esm",  
  platform: "node",  
  jsx: "automatic",  
  sourcemap: false,  
  loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl" },  
  logLevel: "warning",  
});  
void built;
```



```

if (missing.length) issues.push(`код не создаёт обеща
/*
 * Обещанная переменная не должна быть пустой на ВСЕХ
 * объявил и не наполнил – визуализации нечего показы
 * «защитый» результат вместо работы алгоритма. Именн
 * Косарайю: order был дан готовым списком, а comp та
 */
const isEmpty = (v) =>
  Array.isArray(v) ? v.length === 0
  : typeof v === "string" ? v.length === 0
  : typeof v === "object" && v !== null ? Object.keys
  : false;
const neverFilled = (mod.declared[key] ?? [])
  .map(alts)
  .filter((a) => a.some(ident))
  .map((a) => a.find((x) => ident(x) && seen.has(x)))
  .filter(Boolean)
  .filter((name) => {
    let sawValue = false;
    let sawNonEmpty = false;
    for (const step of [...result.steps, { values: re
      const v = step.values?.[name];
      if (v === undefined || v === null) continue;
      sawValue = true;
      if (!isEmpty(v)) sawNonEmpty = true;
    })
    return sawValue && !sawNonEmpty;
  });
if (neverFilled.length) issues.push(`объявлены, но ни

const stderr = printed.filter((p) => p.startsWith("[s
if (stderr.length) issues.push(`предупреждения: ${std

if (issues.length) problems++;
rows.push({ key, steps: result.steps.length, vars: se
}

rows.sort((a, b) => (a.status === "ок") - (b.status ===

```

```

    onError?.(error);
  });
  // console.error/warn приложения собираем отдельно —
  const consoleErrors = [];
  virtualConsole.on("error", (...args) => consoleErrors.push(...args));
  virtualConsole.on("warn", () => {});

  const dom = new JSDOM(
    `<!doctype html><html lang="ru"><head><meta charset="utf-8"></head><body><div id="root"></div></body></html>`,
    { url, pretendToBeVisual: true, runScripts: "outside-only" },
  );
  const { window } = dom;

  window.ResizeObserver = window.ResizeObserver ?? OBSERVER;
  window.IntersectionObserver = window.IntersectionObserver ?? OBSERVER;
  window.matchMedia =
    window.matchMedia ??
    ((media) => ({
      matches: false,
      media,
      onchange: null,
      addListener() {},
      removeListener() {},
      addEventListener() {},
      removeEventListener() {},
      dispatchEvent: () => false,
    }));
  window.scrollTo = () => {};
  window.HTMLElement.prototype.scrollIntoView = () => {};
  window.HTMLElement.prototype.focus = window.HTMLElement.prototype.focus;
  window.URL.createObjectURL = window.URL.createObjectURL;
  window.URL.revokeObjectURL = window.URL.revokeObjectURL;
  Object.defineProperty(window, "innerWidth", { value: 400 });
  Object.defineProperty(window, "innerHeight", { value: 400 });
  Object.defineProperty(window, "devicePixelRatio", { value: 1 });
  window.Element.prototype.getBoundingClientRect = function() {
    return { x: 0, y: 0, top: 0, left: 0, right: 400, bottom: 400 };
  };

```

```
    try {
      Object.defineProperty(globalThis, key, { value: w
    } catch {
      /* read-only в Node – не критично */
    }
  }
  globalThis.IS_REACT_ACT_ENVIRONMENT = true;

  return { dom, window, document: window.document, cons
}
```

ФАЙЛ 105/113: scripts/verify/live-matrix.test.mjs
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 103 | РАЗМ

```
/**
 * Модульные проверки разбора «живых» данных компилятор
 *
 * npm run verify:unit
 *
 * liveMatrix() – то, благодаря чему визуализации матриц
 * суммы 2D, разреженная таблица) принимают данные ЛЮБО
 * пользователя вместо захардкоженной демо-сетки.
 */
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";

const OUT_DIR = "tmp/verify";
mkdirSync(OUT_DIR, { recursive: true });
writeFileSync(
  `${OUT_DIR}/unit-entry.ts`,
  `export { liveMatrix, pickLiveVar, clampIndex } from
`);
await esbuild.build({
  entryPoints: [`${OUT_DIR}/unit-entry.ts`],
  bundle: true,
```

```

check("словарь словарей", cells({ 0: { 0: 7, 1: 8 }, 1:
check('ключи-пары "{i, j}" после снапшота tuple', cells
check('ключи-пары без скобок "0, 1"', cells({ "0, 0": 5

```

```

/* — liveMatrix: что матрицей НЕ является
check("плоский список чисел → null", m([1, 2, 3, 4, 5])
check("словарь по именам → null", m({ dist: 1, prev: 2
check("пустой список → null", m([]), null);
check("пустой словарь → null", m({}), null);
check("undefined → null", m(undefined), null);
check("null → null", m(null), null);
check("число → null", m(42), null);
check("строка → null", m("abc"), null);
check("массив строк-мусора → null", m(["a", "b"], ["c"

```

```

/* — liveMatrix: особые значения
check('"inf" → +Infinity', cells(["inf", 1], [2, "-inf
check('числа-строки "42" → 42', cells(["1", "2"], ["3"
check("булевы значения ячейкой не считаются", m([[true,
check("обрывок до 64×64", shape(Array.from({ length: 80
    4", numbers: true }));
check("hasNumbers=false невозможна для живой матрицы",

```

```

/* — pickLiveVar
check("первое найденное имя по приоритету", pickLiveVar
check("берёт следующее имя, если первого нет", pickLive
check("ничего не найдено → null", pickLiveVar({ x: 1 },
check("нет снимка → null", pickLiveVar(undefined, ["d"]
check("ключ со значением undefined всё равно считается"

```

```

/* — clampIndex
check("null остаётся null", clampIndex(null, 5), null);
check("размер 0 → null", clampIndex(2, 0), null);
check("обрезка сверху", clampIndex(99, 5), 4);
check("обрезка снизу", clampIndex(-7, 5), 0);
check("дробный индекс усекается", clampIndex(2.9, 5), 2
check("NaN → null", clampIndex(Number.NaN, 5), null);
check("Infinity → null", clampIndex(Number.POSITIVE INF

```

```

function state(tag: string) {
  const p = panel();
  const ed = editor();
  const txt = (p?.textContent ?? "").replace(/\s+/g, " ");
  const step = txt.match(/шаг[^\s]*\s*\d+\s*\s*\s*\d+/i);
  process.stdout.write(
    `  [${tag.padEnd(9)}] редактор: ${ed ? "ДА" : "нет"}
    step ? step[0] : "-" | футер: ${footer()}\n`
  );
  process.stdout.write(`                бейдж страницы: ${
    /встроенное демо/.test(document.body.textContent ?? "")
  }`);
}

function typeInto(ta: HTMLTextAreaElement, value: string) {
  const setter = Object.getOwnPropertyDescriptor(window, "type").value;
  act(() => {
    setter.call(ta, value);
    ta.dispatchEvent(new window.Event("input", { bubble: true }));
  });
}

async function click(el: Element | null | undefined, label: string) {
  if (!el) throw new Error(`нет кнопки: ${label}`);
  await act(async () => {
    el.dispatchEvent(new window.MouseEvent("click", { bubble: true }));
    await sleep(20);
  });
}

function findButton(match: RegExp): HTMLButtonElement | undefined {
  return Array.from(document.querySelectorAll("button")).find(
    b => match.test([b.getAttribute("aria-label"), b.getAttribute("value")]
  ) as HTMLButtonElement | undefined;
}

export async function probe() {
  const host = document.createElement("div");
  document.body.appendChild(host);
  window.history.replaceState({}, "", "/?topic=dijkstra");
  const root = createRoot(host);
  await act(async () => {

```

```

    state(`t+${t}`);
  }

process.stdout.write("\n=== 4. очищаем редактор полностью\n");
const ed4 = editor();
if (ed4) {
  typeInto(ed4, "");
  state("очистили");
  for (const t of [200, 800, 2000]) {
    await settle(t);
    state(`t+${t}`);
  }
} else {
  process.stdout.write("  редактора нет (листинг) — ждём\n");
  await act(async () => {
    document.dispatchEvent(new window.KeyboardEvent("keydown", {
      key: "Esc",
    }));
    await sleep(30);
  });
  state("после Esc");
}
}

```

ФАЙЛ 107/113: scripts/verify/README.md

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМЕР: 1.5 КБ

Проверки приложения (`npm run verify`)

Три уровня проверок, которые можно гонять локально и в CI. Для запуска нужен: DOM даёт `jsdom`, Python — `pyodide` из `node\_modules` 3.12, что грузится в браузере пользователя, только без `node\_modules`.

```
```bash
```

```

npm run verify # всё сразу: unit → эталонные коды
npm run verify:unit # 38 проверок разбора живых данных
npm run verify:code # 46 эталонных кодов страниц по

```

й список, 80×80 (обрезка), пустая матрица, чужое имя  
| С. матричные страницы (6) | префиксные суммы 2D и раз  
| D. графы путей (2) | Дейкстра/Форд-Беллман с вершинами  
гравания гаснут и оживают обратно |  
| Е. эталонный код (46) | прогон кода в CPython → финал  
е показать мусор |  
| F. редактор (6) | код из одних комментариев, синтаксис  
ляет эталон и возвращает инициализацию; код не перете  
| G. lite-режим (1) | `?lite=1` отдаёт текст без навига  
| H. доступность (2) | в панели подписаны все кнопки в  
очника |

Фильтры: `HAR\_CHAPTERS=floyd,dijkstra`, `HAR\_SKIP=A,E`,

## Как это заведено в приложение

- \* `window.\_\_algv0PythonRuntime` — точка подмены интерпретатора  
`PythonCompiler.tsx`. В браузере пользователя поле не  
грузит Pyodide с CDN как раньше; харнес кладёт туда л
- \* Состояние визуализаций передаётся через `emitVizState`  
(`src/data/vizStepBus.ts`) — те же события, что шлёт  
идёт по боевому пути, а не через внутренние API компо

---

ФАЙЛ 108/113: scripts/verify/run-browser-checks.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 96 | РАЗМ

---

/\*\*

\* DOM-проверки приложения: jsdom + настоящий Pyodide и

\*

\* npm run verify:dom

\*

\* Фильтры:

\* HAR\_CHAPTERS=floyd,dijkstra только эти страницы

\* HAR\_SKIP=A,E пропустить группы (A

\*

С матрицы, D графы,

```

 "import.meta.env.BASE_URL": '"/',
 "import.meta.env.MODE": '"development"',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 "import.meta.env.SSR": "false",
 },
 loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl", ".svg": "dataurl" },
 logLevel: "warning",
});
const mod = await import(`${process.cwd()}/${OUT_DIR}/bundle.js`);

process.on("unhandledRejection", (reason) => {
 jsdomErrors.push(`unhandledRejection: ${reason instanceof Error ? reason.message : reason}`);
});

log("прогоняю сценарии...\n");
const results = await mod.runSuite({ py, window, document });
const sum = mod.summary();

writeFileSync(`${OUT_DIR}/report.json`, JSON.stringify(results));

const failed = results.filter((r) => !r.ok);
console.log("\n" + "=".repeat(96));
console.log("СВОДКА");
console.log("=".repeat(96));
for (const [group, g] of Object.entries(sum.byGroup)) {
 console.log(`  ${group.padEnd(26)} ok: ${String(g.ok)}`);
}
console.log(`\n  ВСЕГО ${sum.total}:    прошло ${sum.passed},    не прошло ${sum.failed}`);
if (failed.length) {
 console.log("\nПРОВАЛЫ:");
 for (const f of failed) console.log(`  ✖  [${f.group}]`);
}
if (jsdomErrors.length) {
 console.log("\nОШИБКИ ОКРУЖЕНИЯ (jsdom/неперехваченные ошибки)");
 for (const e of [...new Set(jsdomErrors)].slice(0, 10)) console.log(`  ✖  ${e}`);
}
const stray = consoleErrors.filter((e) => !/not wrapped/.test(e));

```



```
await esbuild.build({
 entryPoints: ["scripts/verify/probe-compiler.tsx"],
 bundle: true,
 outfile: `${OUT_DIR}/probe-bundle.mjs`,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
 sourcemap: true,
 define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/'',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 },
 loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl" },
 logLevel: "warning",
});
const mod = await import(`${process.cwd()}/${OUT_DIR}/probe-bundle.mjs`);
log("зондирую панель...");
await mod.probe();
process.exit(0);
```

---

## РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

---

---

ФАЙЛ 110/113: [.github/workflows/deploy-pages.yml](#)  
КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9 КБ

---

name: Deploy app to GitHub Pages

# ДВЕ настройки репозитория нужно сделать один раз ВРУЧНУЮ  
# (у бота-интеграции нет admin-прав на них – API отвечает за них)

- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Build for the repository subpath  
env:  
  VITE\_BASE: /algv0/  
run: npm run build
- name: Make GitHub Pages fallback  
run: cp dist/index.html dist/404.html
- name: Upload Pages artifact  
uses: actions/upload-pages-artifact@v5  
with:  
  path: ./dist

#### deploy:

```
name: Publish to GitHub Pages
needs: build
runs-on: ubuntu-latest
environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
steps:
 # Ранняя и понятная ошибка вместо невнятного «Fail
 # когда в репозитории Pages не переключён на сбор
 - name: Preflight – Pages собирается через GitHub
 env:
 GH_TOKEN: ${ secrets.GITHUB_TOKEN }
 run: |
 bt=$(gh api "repos/${ github.repository }/pages" |
 echo "Pages build_type: ${bt}"
 if ["${bt}" != "workflow"]; then
 echo "::error title=Pages не настроен::Смотрите этот запуск. См. комментарий в начале деплоя"
 exit 1
 fi
```

```
concurrency:
```

```
 group: rebuild-pdf-${{ github.ref }}
 cancel-in-progress: true
```

```
permissions:
```

```
 contents: write
```

```
jobs:
```

```
 rebuild-pdf:
```

```
 name: код → txt → png → pdf
```

```
 runs-on: ubuntu-latest
```

```
 # не реагируем на собственный коммит бота
```

```
 if: "!contains(github.event.head_commit.message, '[
```

```
 steps:
```

```
 - name: Checkout
```

```
 uses: actions/checkout@v6
```

```
 with:
```

```
 fetch-depth: 0
```

```
 persist-credentials: true
```

```
 - name: Setup Node.js
```

```
 uses: actions/setup-node@v7
```

```
 with:
```

```
 node-version: "24"
```

```
 cache: npm
```

```
 - name: Install ImageMagick + DejaVu fonts
```

```
 run: |
```

```
 sudo apt-get update -qq
```

```
 sudo apt-get install -y --no-install-recommen
```

```
 # На Ubuntu политика ImageMagick иногда запре
```

```
 # разрешаем то, что нужно пайплайну (text -> p
```

```
 for policy in /etc/ImageMagick-6/policy.xml /
```

```
 if [-f "$policy"]; then
```

```
 sudo sed -i 's/rights="none" pattern="\{P
```

```
 fi
```

```
 done
```

```
mkdir -p public/export/viz-shots
cp tmp/viz-shots/*.png tmp/viz-shots/manifest
```

- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \`public/export/full\_code\_all\_pages`"  
 echo "| \`public/export/full\_code\_all\_pages`"  
 echo "| \`public/export/guide\_lite.pdf\` |"  
 echo "| Снимков визуализаций | \${ls tmp/viz-shots/\*} |"  
 echo "| PNG-страниц | \${ls tmp/export-pages/\*} |"  
 } >> "\$GITHUB\_STEP\_SUMMARY"
- name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v7  
with:  
 name: full-code-export  
 path: |  
 public/export/full\_code\_all\_pages.pdf  
 public/export/full\_code\_all\_pages.txt  
 if-no-files-found: error  
 retention-days: 30
- name: Upload artifacts (PNG-страницы)  
uses: actions/upload-artifact@v7  
with:  
 name: full-code-pages-png  
 path: tmp/export-pages/\*.png  
 if-no-files-found: error  
 retention-days: 7

```
 - "**/*.md"
 pull_request:
 workflow_dispatch:

concurrency:
 group: verify-${{ github.ref }}
 cancel-in-progress: true

permissions:
 contents: read

jobs:
 verify:
 name: unit + эталонный код + DOM
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4

 - uses: actions/setup-node@v4
 with:
 node-version: 22
 cache: npm

 # puppeteer здесь не нужен (он для растеризации Р
 # тянет Chromium – пропускаем, чтобы установка не
 - name: Зависимости
 run: npm ci
 env:
 PUPPETEER_SKIP_DOWNLOAD: 1
 PUPPETEER_SKIP_CHROMIUM_DOWNLOAD: 1

 - name: Типы и сборка
 run: |
 npx tsc --noEmit -p tsconfig.json
 npm run build

 - name: Модульные проверки (liveMatrix)
 run: npm run verify:unit
```



```

Текст с акцентом и инлайн-кодом
dist[v]
</p>
```
```

Инлайн-«пилюля» для имени переменной/константы:

```
```html
<span class="bg-slate-950 px-2 py-0.5 rounded text-xs f
```
```

3.3 Сетка ключевых чисел (3 ячейки)

```
```html
<div class="grid grid-cols-1 sm:grid-cols-3 gap-3 my-6"
 <div class="bg-slate-900 border border-slate-700 rounded
 <p class="text-[10px] uppercase tracking-wider text
 <p class="font-mono text-emerald-300 text-sm">0{(V+H
 </div>
 <!-- 2-я и 3-я ячейки — тот же markup -->
</div>
```
```

3.4 Сетка аналогий — ****обязательна**** на каждой странице

Слева — неформальная аналогия, справа — техническая аль

```
```html
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border border
 <div class="absolute -top-3 left-4 bg-slate-700 text
 ald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">...</p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2 bord
```

```

<details class="bg-slate-900/40 rounded-lg border border-
 <summary class="p-4 font-bold text-slate-400 outline-l
 e-700/50">
 ▶ Код: Дейкстра на куче (Python)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <pre class="bg-slate-950 p-4 rounded overflow-x-aut
...</code></pre>
 <p class="text-slate-300 text-sm">Комментарий после
 </div>
</details>
```
```

- Вложенные списки внутри спойлера: `<ul class="list-di
- Подзаголовок внутри блока: `<h4 class="text-lg font-b
- или `<p class="text-lg font-bold text-white mt-8 mb-3

3.7 Ловушки / частые ошибки

```
```html
<div class="bg-rose-950/30 p-5 rounded-lg border border
 <p class="text-sm font-bold text-rose-300 mb-3">△ Гд
 <ul class="list-disc pl-5 space-y-2 text-sm text-slat
 Отрицательные веса. Дейкстра их не перев
 Переполнение: <span class="font-mono text-emera

</div>
```
```

4. Порядок разделов на странице

1. Шапка: бейдж билета + `h2`.
2. «Одной фразой» / «Суть алгоритма» + центрированное о
3. 1–3 абзаца объяснения (3.2), затем сетка ключевых чи
4. Аналогии (3.4) – обязательный блок.
5. Механика: пошаговое описание, таблица состояний/сравн

- код — `rge`; определения — обычный `r` (см. 3.1);
- ****смысл не должен держаться на эмодзи****: в PDF используется эмодзи-глифы — пиктограммы вырезаются, остаётся « Операции на отрезках» доедет как «Операции на отрезках», а вот « значит да / значит нет» потеряет смысл, пиктограммы
- спойлеры в PDF раскрываются полностью — не прячьте ту же информацию минимум, но и не дублируйте его в основном тексте;
- картинки страниц берутся из `CHAPTER\_IMAGES` (`script` снимки демонстраций — из `tmp/viz-shots/\*.png` (их перенести в `npm run export:shots` в CI); локально без браузера в `PAGE\_SYNC`.

Цель по объёму: 40–60 страниц, жёсткий предел — 100.